APPUNTI DI STUDIO

Machine Learning e Deep Learning

Dispensa di teoria

MLP - Multilayer Perceptron
CNN - Reti convoluzionali
RNN - Reti ricorrenti
TCN - Temporal Convolutional Networks
Transformer

Indice degli argomenti

1 MLP - Multilayer Perceptron 1

2 CNN - Reti neurali convoluzionali 43

3 RNN - Reti neurali ricorrenti 111

4 TCN - Temporal Convolutional Networks 145

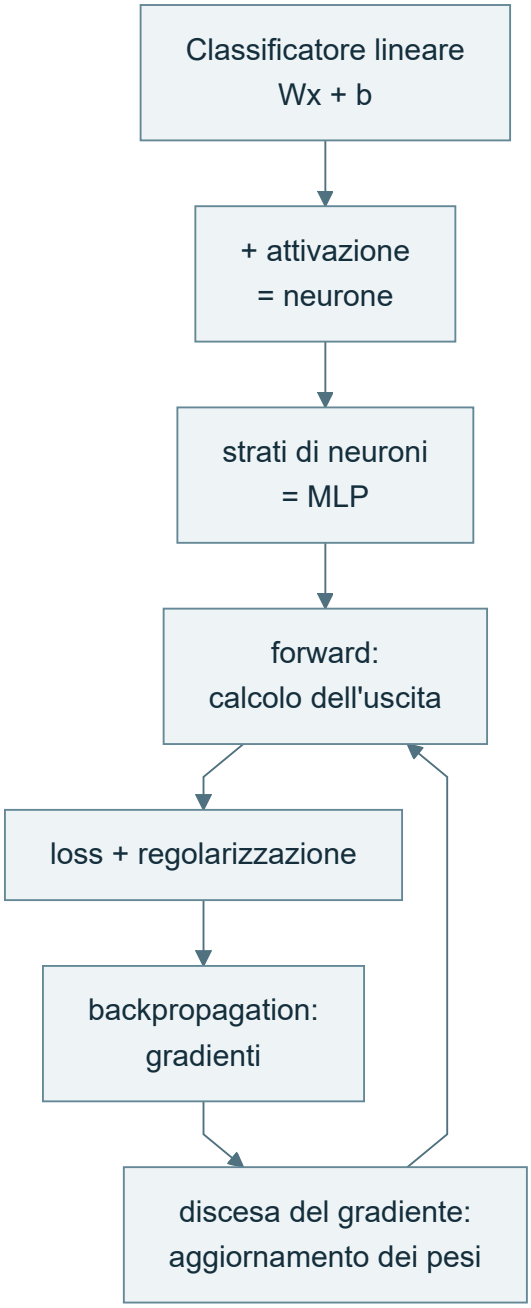
5 Transformer 157

Argomento 1

MLP - Multilayer Perceptron

Contenuti dell'argomento

- Introduzione
- Un passo indietro: i classificatori lineari
- Modellare un neurone
- Reti neurali: il Multilayer Perceptron
- Potere rappresentazionale
- Iperparametri e overfitting
- Addestrare una rete: funzione obiettivo e loss
- Backpropagation (slide 39–50)
- Inizializzazione dei pesi (slide 51)
- Regolarizzazione nelle reti profonde (slide 52–56)
- Riepilogo



1.1 Introduzione

1.1.1 Perché studiare il Deep Learning (slide 4)

Gran parte dei risultati dell'IA moderna si fonda su sistemi di deep learning:

- IA generativa e *large language models*;
- IA per la chimica (progettazione di proteine e farmaci);
- la parte di **percezione** della guida autonoma;
- in robotica, i modelli **Vision-Language-Action (VLA)**, che uniscono interpretazione del linguaggio, percezione e decisione per muovere il robot.

Questi sistemi sono in larga parte **indipendenti dall'applicazione**. Come in tutto il machine learning, una volta che si sa come sono fatti i modelli, come combinarli e come addestrarli, l'applicazione diventa secondaria: si prendono i dati del dominio e, se il modello è ben progettato, funziona.

Molte "novità" sono in realtà un **rebranding** di tecniche di 10–15 anni fa: i *world models* sono in sostanza autoencoder (li vedremo nel corso) e gli LLM come GPT sono il *decoder* del Transformer. Quello che va padroneggiato è il machine learning che sta sotto, non l'etichetta applicativa.



Figura 1.1: Slide 4 – Here is the HYPE

1.1.2 Le reti neurali non sono nuove: tre ondate (slide 5)

Periodo	Cosa succede
1940–1960	Teorie di apprendimento ispirate alla biologia; primi modelli pionieristici come il **perceptron**. Per la prima volta si addestra un singolo neurone.
1980–1990	Reti con un paio di strati nascosti addestrate con **backpropagation** (1986).
2006–oggi	Ondata attuale di ricerca.

Negli anni '80 le reti si scontrarono con due limiti: **mancava la potenza di calcolo** e **mancavano i dati**, e le reti neurali sono modelli molto *data hungry*.

1.1.3 Perché proprio ora? (slide 6–9)

Le idee di base esistevano da tempo, ma le prestazioni sono esplose solo di recente grazie a fattori complementari:

- grandi dataset etichettati;
- potenza di calcolo;

- addestramento su **GPU** (eventualmente distribuito).

La crescita della scala dei dati è stata più che lineare, quasi esponenziale:

- **2003** (Fergus, Perona, Zisserman): un dataset "grande" contava **fino a 400 immagini** (slide 7);
- **2017** (YouTube-BoundingBoxes, Real et al.): **240.000 video**, 5,6 milioni di bounding box e 10,5 milioni di annotazioni umane (slide 8).

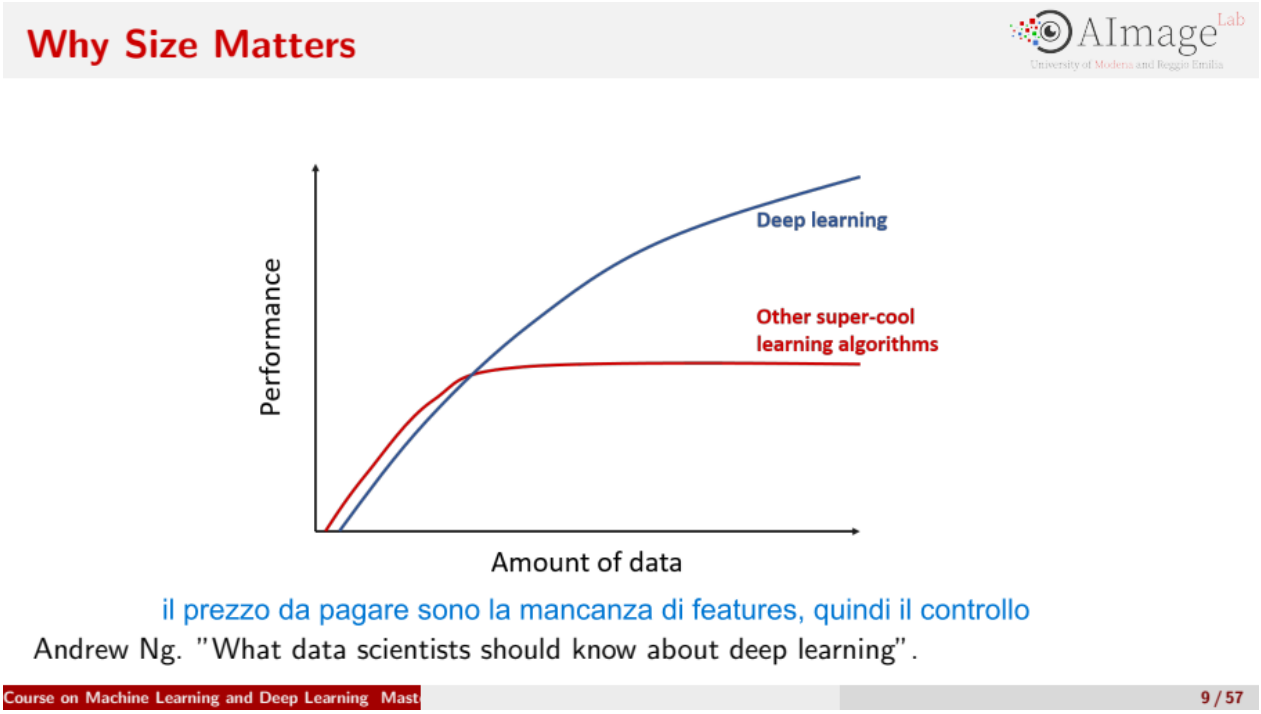


Figura 1.2: Slide 9 – Why Size Matters (Andrew Ng)

Il grafico di Andrew Ng (slide 9) riassume il punto:

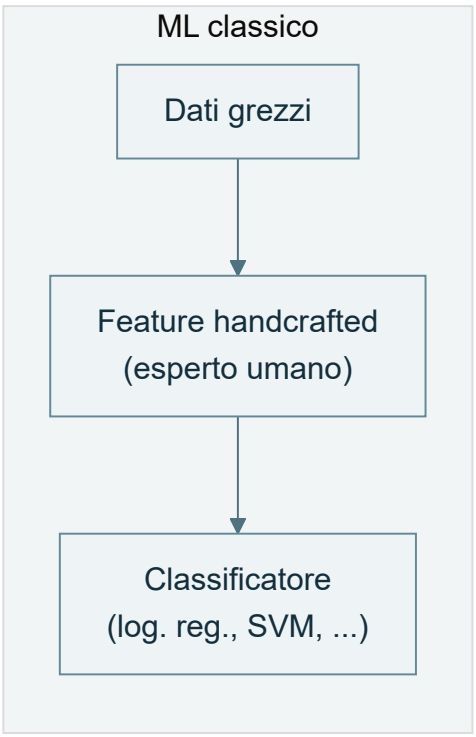
- con pochi dati, gli algoritmi classici e il deep learning si equivalgono;
- aumentando i dati, gli algoritmi classici raggiungono un **plateau**, mentre il deep learning **continua a migliorare**.

Per questo la "magia" di sistemi come GPT non è stata tecnica: si è presa un'architettura già esistente (il Transformer, 2017) e la si è riempita con un'enorme quantità di dati.

1.1.4 Il problema della rappresentazione dei dati (slide 10)

Le prestazioni di un algoritmo di ML dipendono fortemente da **come rappresentiamo i dati**. Le **feature** sono i pezzi discreti d'informazione che usiamo come *proxy* per modellare la complessità del mondo. Per molto tempo sono state **progettate a mano** (*handcrafted*), in modo diverso per ogni compito. Il deep learning invece **impara la rappresentazione direttamente dai dati**.





Perché le feature fissate impongono un tetto. Supponiamo di classificare immagini usando come unica feature il **colore medio**. Non si raggiungerà mai il 100% di accuratezza, qualunque classificatore e qualunque quantità di dati si usi: la feature è troppo povera. Fissare le feature significa fissare implicitamente anche un **tetto alle prestazioni** (*performance cap*): le feature sono il "carburante" del sistema di ML.

Cosa cambia con il deep learning. Una rete neurale risolve **contemporaneamente due problemi**:

1. partendo dal dato il più grezzo possibile, **estrae una rappresentazione** adatta al problema;
2. su questa rappresentazione, **costruisce il classificatore**.

I due problemi sono accoppiati: la rappresentazione appresa è quella più utile per minimizzare la loss. Se le feature possono cambiare, all'aumentare dei dati la rete impara feature sempre più espressive e può migliorare le prestazioni, senza garantirne la convergenza al 100%. È questa la spiegazione della curva blu della slide 9.

Il prezzo da pagare: la perdita di controllo

La rete rappresenta il mondo con feature scelte **per risolvere la loss**, non secondo le nostre categorie. Per queste feature **non abbiamo un nome**, e ciò che non si può nominare non si può davvero capire. Si possono estrarre dal modello spiegazioni o regole, ma il "carburante" resta privo di significato per un essere umano. L'idea che si possano comprendere fino in fondo le rappresentazioni interne di un modello profondo è quindi falsa. (Annotazione slide 9: *"il prezzo da pagare è la mancanza di feature [interpretabili], quindi il controllo"*.)

1.2 Un passo indietro: i classificatori lineari

Un **classificatore lineare** è un classificatore in cui la mappa è affine nell'input a parametri fissati e affine nei parametri a input fissato, ma non congiuntamente lineare nelle due variabili: calcola una **somma pesata** degli input. In 2D separa gli oggetti con una retta; in più dimensioni con un piano o un iperpiano. Conviene ripartire da qui perché **una rete neurale è una collezione di classificatori lineari**, ed è sorprendente che basti combinarli per ottenere modelli così potenti.

1.2.1 Formalizzazione (slide 12–14)

Ci concentriamo sulla **classificazione di immagini**. Il training set contiene N immagini:

$$x_i \in \mathbb{R}^D, \quad i = 1, \dots, N, \qquad (x_i, y_i),\ y_i \in \{1, \dots, K\}$$

L'obiettivo è una funzione $f : \mathbb{R}^D \to \mathbb{R}^K$ che associa a ogni immagine **un punteggio (score) per ciascuna classe**; la classe predetta è quella con lo score più alto. La mappa lineare è:

$$\boxed{f(x_i, W, b) = Wx_i + b}$$

Esempio CIFAR-10 (slide 13–14). Il dataset contiene $N = 60\,000$ immagini RGB 32×32 divise in $K = 10$ classi.



Figura 1.3: Slide 13 – CIFAR-10

Ogni immagine ha 3 canali (R, G, B) e va "srotolata" in un vettore colonna:

$$D = 32 \times 32 \times 3 = 3072 \;\Rightarrow\; x_i \in \mathbb{R}^{3072}$$

- $W \in \mathbb{R}^{10\times 3072}$ è la **matrice dei pesi**: una riga per classe, cioè 10 righe da 3072 coefficienti. Ogni riga contiene i parametri che producono lo score di una classe.
- $b \in \mathbb{R}^{10}$ è il **vettore dei bias**.
- $Wx_i + b \in \mathbb{R}^{10}$ contiene i 10 score.

Si vuole imparare W, b dal training set in modo che, per una nuova immagine x^{test}, **lo score della classe corretta sia più alto degli score delle altre classi**.

Il bias. Se i dati non sono centrati nell'origine, il bias permette di **traslare** l'iperpiano di separazione. È importante, ma non cambia la sostanza matematica: si può sempre **assorbire nei dati**. Basta aggiungere a x una componente costante pari a 1 e a W una colonna con i bias:

$$Wx + b = \begin{bmatrix} W & b \end{bmatrix} \begin{bmatrix} x \\ 1 \end{bmatrix} = W'x'$$

1.2.2 Esempio numerico (slide 15)

È la stessa formula $f(x) = Wx + b$ del caso CIFAR-10, ma in miniatura, così i conti si fanno a mano: un'immagine di **4 pixel in scala di grigi** ($D = 4$) e **3 classi**, gatto, cane e nave ($K = 3$).

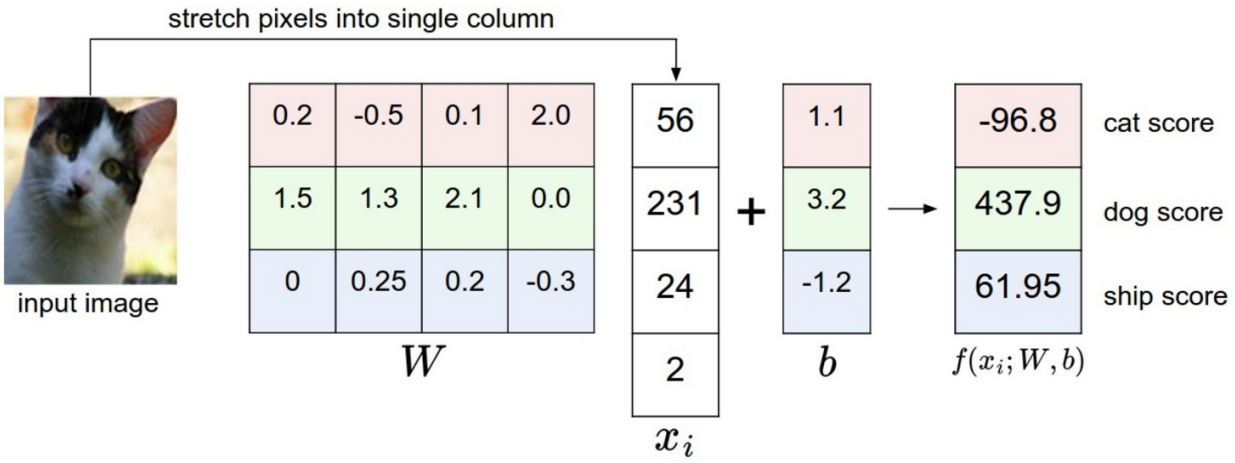


Figura 1.4: Slide 15 – Mappare un'immagine sugli score

Passo 1: l'immagine diventa un vettore. L'immagine è 2×2 e ogni pixel è un'intensità di grigio tra 0 (nero) e 255 (bianco). La si "stira" in una colonna (*stretch pixels into single column*):

$$\begin{bmatrix} 56 & 231 \\ 24 & 2 \end{bmatrix} \longrightarrow x = \begin{bmatrix} 56 \\ 231 \\ 24 \\ 2 \end{bmatrix} \in \mathbb{R}^4$$

Passo 2: i parametri. W ha **una riga per classe** e **una colonna per pixel**, quindi è 3×4. La riga k dice quanto conta ciascun pixel per la classe k:

- un peso positivo significa "se questo pixel è chiaro, lo score della classe sale";
- un peso negativo significa il contrario.

Il bias b aggiunge un valore fisso per ogni classe.

$$W = \begin{bmatrix} 0.2 & -0.5 & 0.1 & 2.0 \\ 1.5 & 1.3 & 2.1 & 0.0 \\ 0 & 0.25 & 0.2 & -0.3 \end{bmatrix} \begin{matrix} \leftarrow \text{gatto} \\ \leftarrow \text{cane} \\ \leftarrow \text{nave} \end{matrix} \qquad b = \begin{bmatrix} 1.1 \\ 3.2 \\ -1.2 \end{bmatrix}$$

Passo 3: uno score per classe. Ogni score è il **prodotto scalare** tra la riga della classe e x (pixel per peso, poi somma), più il bias della classe:

$$s_k = \sum_{j=1}^{4} W_{kj}\, x_j + b_k$$

Classe	pixel 1 (56)	pixel 2 (231)	pixel 3 (24)	pixel 4 (2)	somma	+ bias	**score**
gatto	$0.2 \cdot 56 = 11.2$	$-0.5 \cdot 231 = -115.5$	$0.1 \cdot 24 = 2.4$	$2.0 \cdot 2 = 4.0$	-97.9	$+1.1$	$\mathbf{-96.8}$
cane	$1.5 \cdot 56 = 84.0$	$1.3 \cdot 231 = 300.3$	$2.1 \cdot 24 = 50.4$	$0.0 \cdot 2 = 0$	434.7	$+3.2$	**437.9**
nave	$0 \cdot 56 = 0$	$0.25 \cdot 231 = 57.75$	$0.2 \cdot 24 = 4.8$	$-0.3 \cdot 2 = -0.6$	61.95	-1.2	**60.75**

In forma matriciale sono le tre righe dello stesso prodotto: $Wx + b = [-96.8,\ 437.9,\ 60.75]^T$.

Passo 4: la predizione. Vince la classe con lo score più alto, cioè **cane**. L'immagine però è un gatto: la predizione è **sbagliata**, perché i pesi sono arbitrari e non addestrati. Addestrare il classificatore significa proprio **modificare** W **e** b finché, per le immagini di gatto, è la riga "gatto" a dare lo score più alto.

Il pixel 2 (231) è molto chiaro: la riga "cane" gli assegna un peso positivo alto (1.3), la riga "gatto" un peso negativo (-0.5). Da solo, questo pixel sposta gli score di centinaia di punti.

Errata slide 15

La slide riporta $s_{ship} = 61.95$, che è il valore di $w_3^T x$ **senza** il bias -1.2. Il valore corretto è **60.75**. Gli altri due score sono corretti.

Cosa c'è di sbagliato in questi score? Non sono normalizzati. 437.9 è un valore alto o basso? Non si può dire: gli score vivono in $(-\infty, +\infty)$, non sommano a 1 e non hanno una scala di riferimento. Serve una funzione che li trasformi in quantità confrontabili, come nella regressione logistica.

1.2.3 Regressione logistica e binary cross-entropy (slide 16–17)

Aggiungendo una non linearità sigmoide alla mappa lineare si ottiene la **regressione logistica**:

$$f(x_i, W, b) = \sigma(Wx_i + b), \qquad \sigma(x) = \frac{1}{1 + e^{-x}}$$

La si addestra minimizzando la **binary cross-entropy** su m esempi, dove f_{x_i} indica $f(x_i, W, b)$:

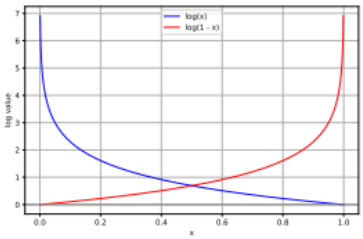
$$\mathcal{L}(W, b) = -\frac{1}{m} \sum_{i=1}^{m} \Big(y_i \log f_{x_i} + (1 - y_i) \log(1 - f_{x_i}) \Big), \qquad y_i \in \{0, 1\}$$

Logistic Regression

AImage Lab — University of Modena and Reggio Emilia

Let's try to understand better **binary cross-entropy loss**:

$$L(W,b) = -\frac{1}{m}\sum_{i=1}^{m}\left(y_i \log(f_{x_i}) + (1-y_i)\log(1-f_{x_i})\right) \tag{2}$$



Note that $y_i \in \{0,1\}$ (binary classification task)

The label y_i act as a selector. For positive examples we penalize predictions close to zero. For negative examples we penalize predictions close to one.

Course on Machine Learning and Deep Learning Mast — 17 / 57

Figura 1.5: Slide 17 – Le due componenti della BCE

L'etichetta y_i fa da **selettore**:

- se $y_i = 1$, il secondo termine si annulla $(1 - y_i = 0)$ e resta $-\log f_{x_i}$. Vale 0 se $f_{x_i} = 1$ e tende a $+\infty$ se $f_{x_i} \to 0$: **penalizza le predizioni vicine a 0**;
- se $y_i = 0$, il primo termine si annulla e resta $-\log(1 - f_{x_i})$, che **penalizza le predizioni vicine a 1**.

In pratica si prende lo score che il modello assegna alla classe giusta e lo si vuole **alzare**, cioè si vuole abbassare il suo $-\log$.

1.2.4 Classificatore Softmax (slide 18–19)

Il **classificatore Softmax** generalizza la regressione logistica al caso multiclasse. La funzione softmax prende un vettore z di score reali arbitrari e lo "schiaccia" in valori tra 0 e 1 che sommano a 1:

$$\boxed{\text{softmax}_j(z) = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}}}$$

Calcolo passo passo dell'esempio della slide 18, con $z = [1.2,\ 5.1,\ 2.7]$:

	z_j	e^{z_j}	$e^{z_j} / \sum_k e^{z_k}$
classe 1	1.2	3.32	**0.018**
classe 2	5.1	164.02	**0.900**
classe 3	2.7	14.88	**0.082**
somma		182.22	1

Rispetto alla regressione logistica multiclasse vista nel corso precedente, che modellava $K-1$ classi e ricavava la K-esima per complemento, qui si tratta **allo stesso modo tutte le K classi**: esponenziale di ogni score, diviso la somma di tutti gli esponenziali. Si usa un insieme di parametri in più, ma la formulazione è più semplice.

La softmax produce una distribuzione di probabilità del modello: ogni componente è positiva e la somma è 1. Questo non garantisce che i valori coincidano con le frequenze reali delle classi. Non è una soglia rigida e non spinge necessariamente il massimo vicino a 1: score tutti uguali producono $1/K$ per classe. Le differenze fra score, non i loro rapporti, determinano i rapporti tra probabilità.

Il motivo sta nell'esponenziale: il rapporto tra due uscite dipende dalla **differenza** degli score, non dal loro rapporto:

$$\frac{\text{softmax}_2}{\text{softmax}_3} = \frac{e^{5.1}}{e^{2.7}} = e^{5.1-2.7} = e^{2.4} \approx 11$$

L'annotazione «stiamo semplicemente applicando una soglia» va intesa come analogia con una decisione morbida. La decisione discreta avviene dopo, scegliendo $\arg\max_j p_j$ (oppure confrontando una probabilità binaria con una soglia). La calibrazione riguarda l'attendibilità delle probabilità, non la loro validità matematica.

Nel classificatore softmax gli score $f(x_i, W) = Wx_i$ sono interpretati come **log-probabilità non normalizzate**, e lo si addestra con la **cross-entropy**:

$$\mathcal{L} = -\sum_k y_k^{true} \log\left(y_k^{pred}\right)$$

dove y^{true} è la distribuzione vera (un vettore **one-hot** che seleziona la classe corretta) e $y^{pred} = \text{softmax}(f)$. La forma esplicita per il singolo esempio è derivata in §7.2.

Errata slide 18–19

Nella slide 18 il denominatore è scritto $\sum_k e^{z_j}$: l'indice corretto è $\sum_k e^{z_k}$. Nella slide 19 manca il segno **meno** davanti alla cross-entropy.

Riepilogo:

$$\underbrace{Wx+b}_{\text{classificatore lineare}} + \text{softmax} + \text{cross-entropy} + \text{discesa del gradiente} = \textbf{regressione logistica}$$

1.3 Modellare un neurone

1.3.1 Il neurone biologico (slide 21)

Le reti neurali sono un modello matematico **ispirato in modo grossolano** (*coarsely*) al funzionamento del cervello. Il neurone ne è l'unità computazionale di base: ne abbiamo circa **86 miliardi**, collegati da circa 10^{14}–10^{15} **sinapsi**. Ogni neurone riceve segnali dai **dendriti** e produce l'uscita lungo il suo (unico) **assone**, che si ramifica e si collega tramite sinapsi ai dendriti di altri neuroni.

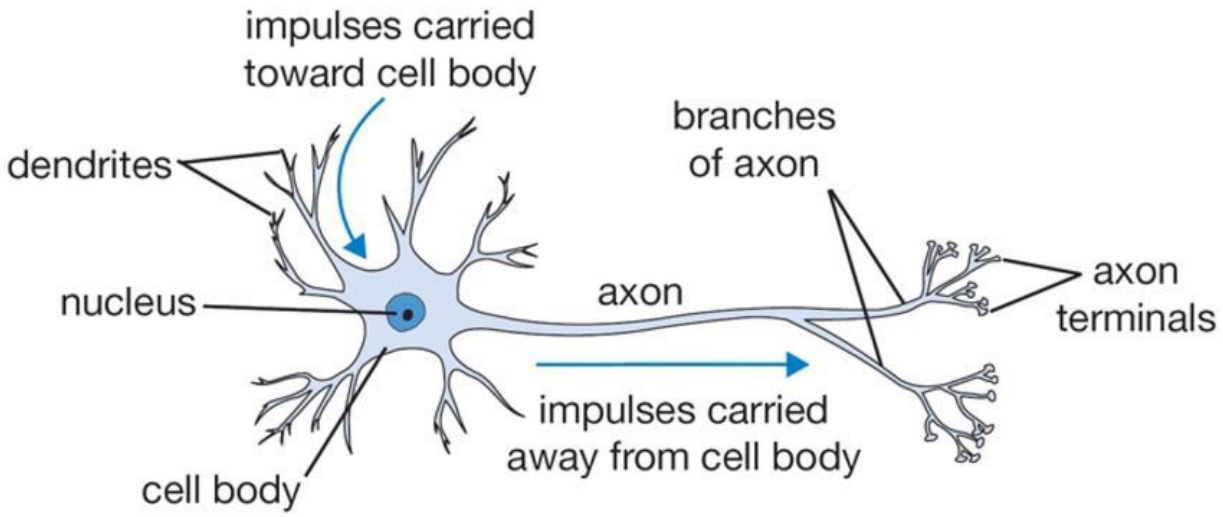


Figura 1.6: Slide 21 – Neurone biologico

L'analogia utile associa la forza delle connessioni ai pesi, la combinazione dei segnali alla somma pesata e la risposta del neurone all'attivazione. Non è una descrizione fisiologica letterale: in particolare l'immagine di una sinapsi «dura o morbida» è soltanto una metafora della modulazione del segnale. Il neurone biologico ha una dinamica temporale che questa mappa statica non rappresenta.

1.3.2 Il neurone artificiale (slide 22–23)

La traduzione in matematica è uno a uno:

Biologia	Matematica
Dendriti (ingressi)	input x_i
Sinapsi (amplifica/attenua)	moltiplicazione per un peso w_i (grande o piccolo)
Cellula (mette insieme i segnali)	**somma**: $\sum_i w_i x_i + b$
Soglia di accensione	**funzione di attivazione** $f(\cdot)$
Assone (uscita)	$y = f\big(\sum_i w_i x_i + b\big)$

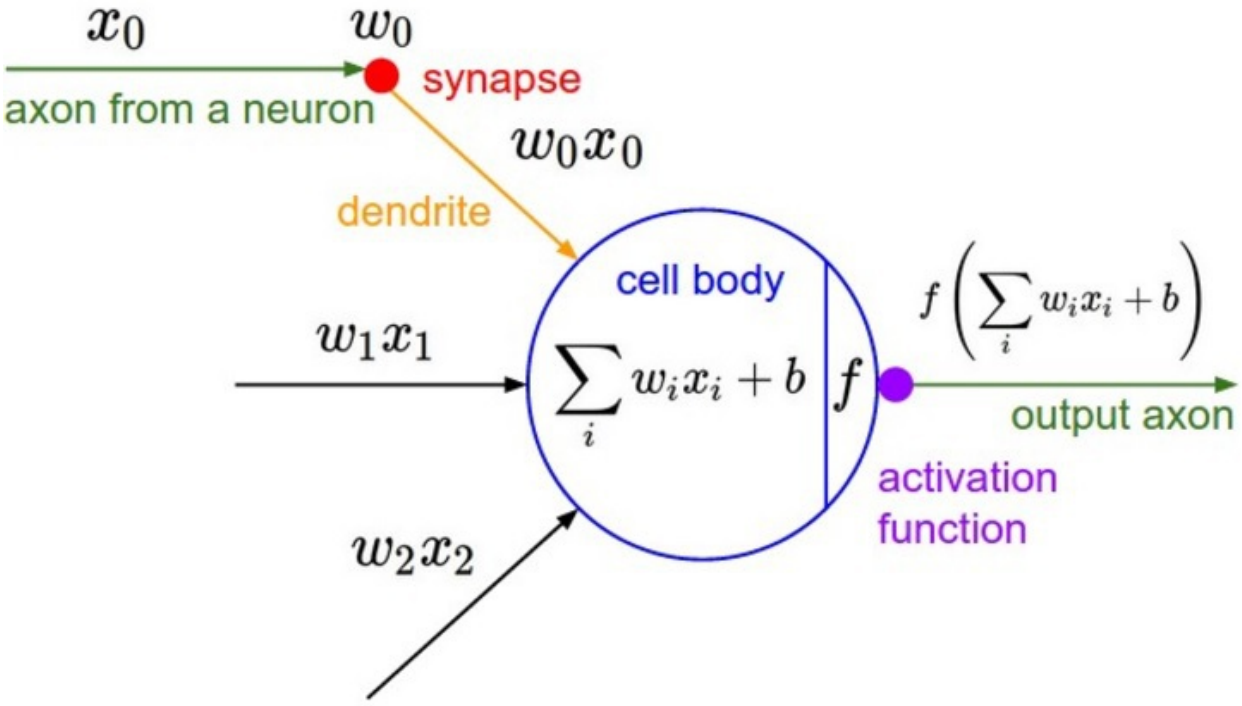


Figura 1.7: Slide 22 – Modello del singolo neurone

Un neurone può avere più ingressi. La sua uscita è il **prodotto scalare tra ingressi e pesi, più il bias, a cui si applica una non linearità**:

$$\boxed{y = f\left(w^T x + b\right)}$$

È il **perceptron** di McCulloch e Pitts (anni '40).

Neurone = regressione logistica (slide 23)

Una combinazione affine seguita dalla **sigmoide** definisce il modello di regressione logistica binaria. Se il neurone usa la sigmoide ed è addestrato con la binary cross-entropy tramite discesa del gradiente, gli ingredienti sono gli stessi, al 100%: stesse equazioni, quindi stesso modello.

Il grado di libertà in più: nulla impone che la funzione di sogliatura sia la sigmoide o la softmax, se ne può usare qualunque. Prende il nome di **funzione di attivazione** perché stabilisce se il neurone è attivo (acceso) oppure no.

1.3.3 Funzioni di attivazione (slide 24–26)

Le funzioni di attivazione sono **funzioni non lineari** calcolate sull'uscita di ogni neurone. Le più usate storicamente sono **sigmoide, tanh e ReLU**; ne esistono anche di più complesse.

Nel deep learning il successo dipende dai **dettagli**. Progettare una rete si fa in un pomeriggio, farla addestrare è un'altra storia. Molto spesso il problema non è di progetto ma **tecnico**:

- la saturazione del gradiente;
- **neuroni che non si attivano mai**;
- il **gradiente che svanisce** perché la rete è troppo profonda.

Le attivazioni sono il primo di questi dettagli.

1.3.3.1 Sigmoide

$$\boxed{\sigma(x) = \frac{1}{1+e^{-x}} = \frac{e^{x}}{1+e^{x}} \in (0,1)}$$

La seconda forma si ottiene moltiplicando numeratore e denominatore per e^x ed è quella della regressione logistica. La curva ha la forma a "S" di una soglia morbida: sopra la soglia spinge verso 1, sotto verso 0.

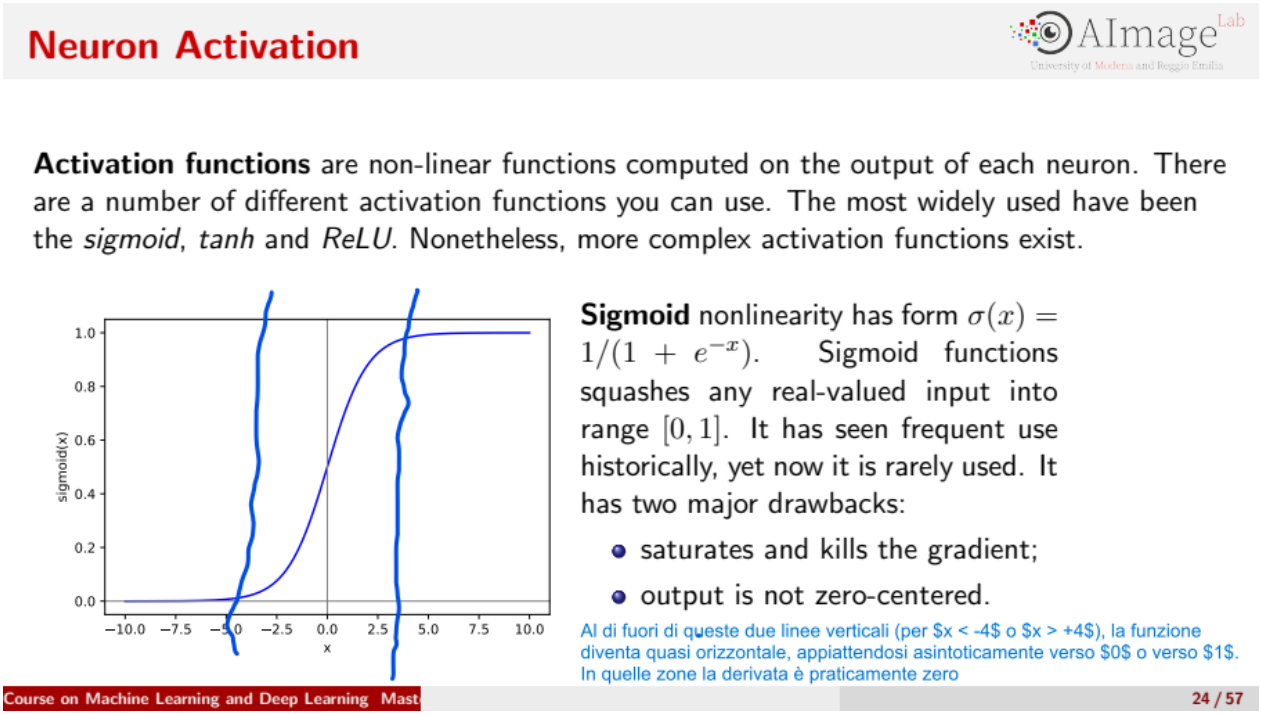


Figura 1.8: Slide 24 – Sigmoide e zone di saturazione

La derivata, passo passo. Serve per capire il secondo difetto della sigmoide.

1. Si riscrive $\sigma(x) = (1+e^{-x})^{-1}$.
2. Si deriva con la regola della catena, dove la funzione esterna è u^{-1} e quella interna è $u = 1+e^{-x}$:

$$\sigma'(x) = -(1+e^{-x})^{-2} \cdot \frac{d}{dx}(1+e^{-x}) = -(1+e^{-x})^{-2} \cdot (-e^{-x}) = \frac{e^{-x}}{(1+e^{-x})^{2}}$$

3. Si spezza la frazione in due fattori:

$$\sigma'(x) = \frac{1}{1+e^{-x}} \cdot \frac{e^{-x}}{1+e^{-x}}$$

4. Si riconosce che il secondo fattore è $1-\sigma(x)$:

$$\frac{e^{-x}}{1+e^{-x}} = \frac{(1+e^{-x})-1}{1+e^{-x}} = 1 - \frac{1}{1+e^{-x}} = 1-\sigma(x)$$

5. Quindi:

$$\boxed{\sigma'(x) = \sigma(x)\big(1-\sigma(x)\big)}$$

x	0	2	4	5	10	100
$\sigma(x)$	0.5	0.881	0.982	0.993	0.99995	≈ 1
$\sigma'(x)$	**0.25**	0.105	0.018	0.0066	0.00005	≈ 0

Il massimo della derivata è 0.25, in $x = 0$. Già per $|x| \gtrsim 4$ la derivata è praticamente nulla: sono le zone esterne alle due linee verticali disegnate sulla slide 24.

La sigmoide è meno usata negli strati nascosti profondi, per **due difetti** (slide 24).

1. Non è centrata sullo zero. Le uscite della sigmoide sono positive e valgono 0.5 quando la pre-attivazione è zero. Questa asimmetria può influenzare la dinamica dell'ottimizzazione; non è una contraddizione biologica del modello matematico.

2. Saturazione. Negli strati nascosti una derivata piccola può attenuare il segnale di apprendimento.

Con la BCE e $p = \sigma(z)$, la regola della catena dà

$$\begin{aligned}\frac{\partial L}{\partial z} &= \left(-\frac{y}{p} + \frac{1-y}{1-p}\right) p(1-p) \\ &= -y(1-p) + (1-y)p = p - y.\end{aligned}$$

Per $z = 5$ e $y = 1$ vale circa -0.0067; per $z = 100$ è estremamente vicino a zero perché la predizione è corretta e molto sicura. Con lo stesso z ma $y = 0$ il gradiente è invece vicino a 1. Non esiste una soglia finita oltre la quale il gradiente della BCE sia sempre nullo: l'arrotondamento numerico va distinto dal limite matematico.

Negli strati nascosti ogni sigmoide introduce il fattore $\sigma'(z) \leq 1/4$, ma compaiono anche le matrici dei pesi. Il contributo delle sole attivazioni lungo dieci strati è al più $4^{-10} \approx 9.54 \cdot 10^{-7}$; non è un limite sull'intero gradiente senza ulteriori ipotesi sui pesi.

La saturazione rende difficile trasmettere gradienti attraverso molte attivazioni sigmoidi. È una difficoltà di ottimizzazione e va distinta dall'errore piccolo di una predizione già corretta.

1.3.3.2 Tangente iperbolica (tanh)

Per risolvere la mancata centratura si riscalano l'argomento e l'uscita della sigmoide e si sottrae 1, così che in 0 valga 0:

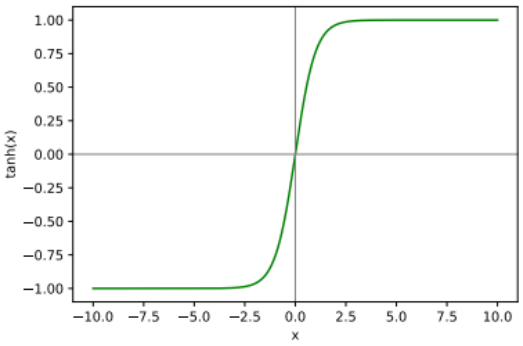
$$\boxed{\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2x) - 1 \in (-1, 1)}, \qquad \tanh(0) = 0$$

Verifica dell'identità, passo passo:

$$2\sigma(2x) - 1 = \frac{2}{1 + e^{-2x}} - 1 = \frac{2 - (1 + e^{-2x})}{1 + e^{-2x}} = \frac{1 - e^{-2x}}{1 + e^{-2x}} \overset{\cdot e^x/e^x}{=} \frac{e^x - e^{-x}}{e^x + e^{-x}} = \tanh(x)$$

Neuron Activation

AImage Lab
University of Modena and Reggio Emilia



Tanh is a scaled sigmoid neuron: $\tanh(x) = 2\sigma(x) - 1$. Differently to sigmoid, input is squashed in range $[-1, 1]$, so the output is 0-centered. However, activation can still saturate and kill the gradient.

we dont' solve the problem of saturation

Figura 1.9: Slide 25 – Tanh

L'uscita è **centrata sullo zero**, ma la funzione **satura ancora**: la sua derivata $1 - \tanh^2(x)$ vale al massimo 1 in $x = 0$ e tende a 0 per $|x|$ grande. La tanh risolve il primo difetto della sigmoide, non il secondo.

Errata slide 25

La slide scrive $\tanh(x) = 2\sigma(x) - 1$. L'identità esatta è $\tanh(x) = 2\sigma(2x) - 1$ (con $2\sigma(x) - 1$ si ottiene $\tanh(x/2)$; per $x = 0.7$: $\tanh(0.7) = 0.604$, mentre $2\sigma(0.7) - 1 = 0.336$). Il concetto non cambia: è una sigmoide riscalata e centrata.

1.3.3.3 ReLU (Rectified Linear Unit)

$$\boxed{\text{ReLU}(x) = \max(0, x)}, \qquad \frac{d}{dx}\text{ReLU}(x) = \begin{cases} 1 & x > 0 \\ 0 & x < 0 \end{cases}$$

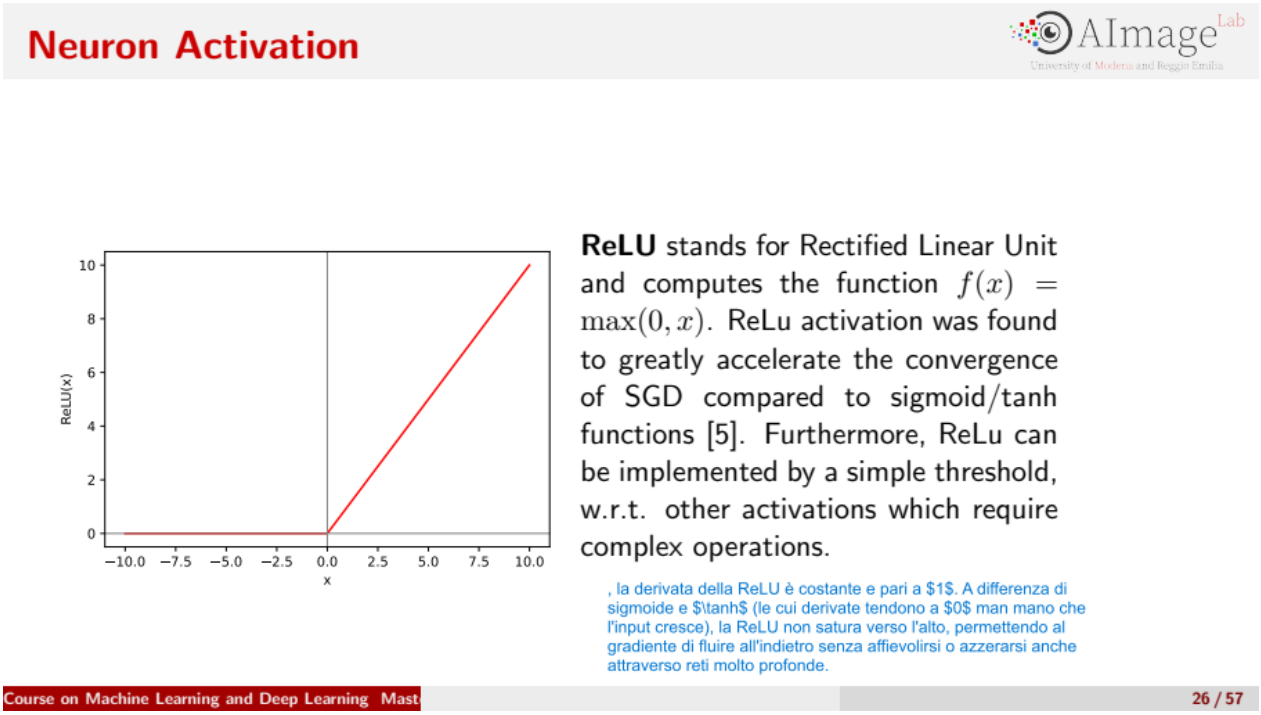


Figura 1.10: Slide 26 – ReLU

La ReLU evita la saturazione sul semiasse positivo:

- nel **semipiano sinistro** il neurone è **spento** (uscita 0);
- nel **semipiano destro** il neurone **copia** l'ingresso.

Proprietà principali:

- **Non satura verso l'alto.** Per $x > 0$ la derivata è 1: questo fattore non attenua il gradiente. I pesi degli altri strati possono comunque amplificarlo o ridurlo, quindi non è una garanzia contro ogni problema di gradiente.

Precisazione

La ReLU è **continua** in 0 ma **non derivabile** in 0 (spigolo). La non linearità deriva dallo spigolo, non da una discontinuità. In pratica in $x = 0$ si usa per convenzione derivata 0.

1.3.4 Perché servono le non linearità (slide 27)

Viene spontaneo chiedersi perché non togliere l'attivazione e usare solo Wx: non satura ed è centrata sullo zero. Il problema è che **la composizione di funzioni lineari è lineare**.

1. Con due strati e attivazioni la rete è $f(x) = \phi\big(W_2\,\phi(W_1 x)\big)$.
2. Si tolgono le attivazioni: $f(x) = W_2(W_1 x)$.
3. Per l'associatività del prodotto matriciale, $W_2(W_1 x) = (W_2 W_1)x$.
4. $W_2 W_1$ è **un'unica matrice**: con $W_1 \in \mathbb{R}^{H\times D}$ e $W_2 \in \mathbb{R}^{K\times H}$, si ha $W = W_2 W_1 \in \mathbb{R}^{K\times D}$. Quindi $f(x) = Wx$.

Qualunque sia il numero di strati, senza attivazioni tutta la rete **collassa in un singolo strato lineare**, cioè in una mappa affine se sono presenti bias. Si ottiene regressione logistica solo conservando una sigmoide o softmax finale. Le attivazioni, essendo non lineari (ReLU compresa), **impediscono questa fattorizzazione**: sono loro a dare senso alla profondità.

1.4 Reti neurali: il Multilayer Perceptron

1.4.1 Struttura a strati (slide 29)

Connettendo un insieme di neuroni in un grafo si ottiene un **Multilayer Perceptron** (**MLP**). I neuroni sono organizzati in **strati** (*layer*):

- **input layer**: un cerchio per ogni feature d'ingresso. Sono **cerchi fittizi**: nessun peso, nessuna attivazione, **nessun parametro**;
- **N hidden layer** nel mezzo;
- **output layer**: **un cerchio per ogni classe**, ciascuno dà lo score della sua classe.

Le connessioni sono **fully connected**:

- **nessuna connessione tra neuroni dello stesso strato**;
- ogni neurone di uno strato è connesso a **tutti** i neuroni dello strato successivo;
- ogni connessione è una **sinapsi**, cioè un peso w.

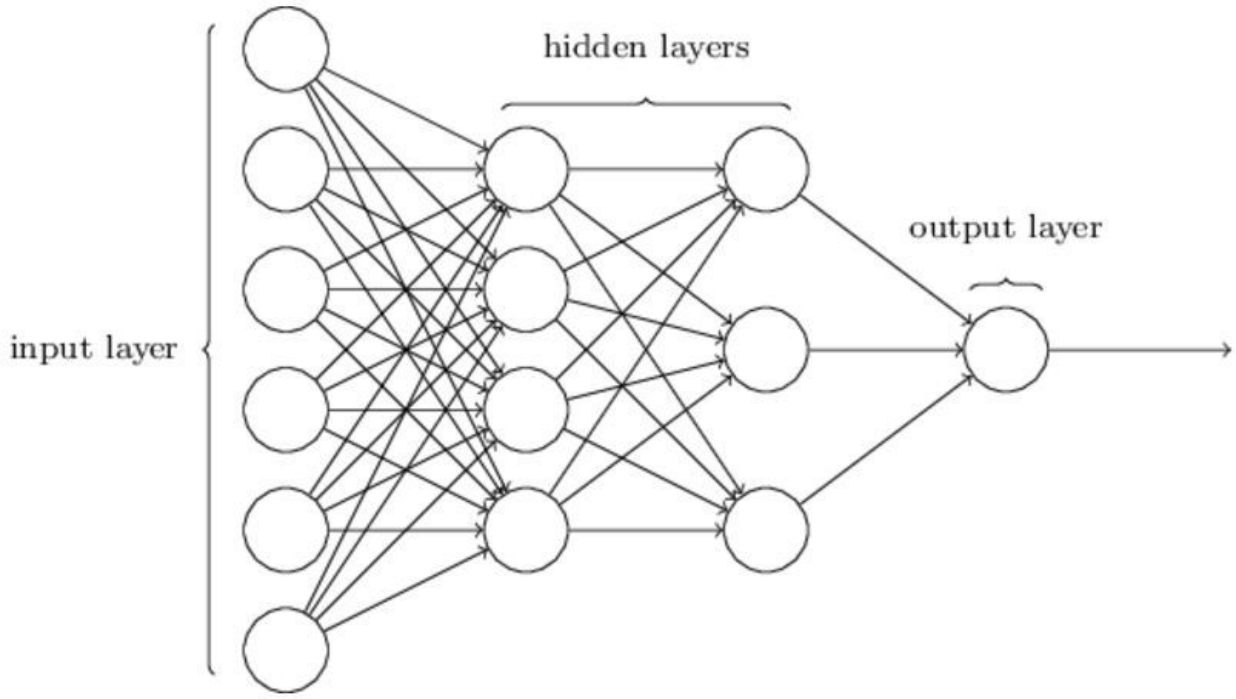


Figura 1.11: Slide 29 – MLP con 47 parametri

L'equazione di un layer è esattamente il classificatore lineare di §2, seguito dall'attivazione. La matrice W ha:

- **numero di colonne = numero di feature in ingresso** (*inbound*);
- **numero di righe = numero di feature in uscita** (*outbound*, cioè i neuroni dello strato).

Ogni neurone ha la sua funzione di attivazione. Nella figura ogni neurone del primo hidden layer riceve 6 pesi e produce un valore, quindi W_1 ha 6 colonne e 4 righe.

1.4.2 Esercizio: da dove vengono i 47 parametri?

Ogni cerchio rappresenta una combinazione affine $w^T x + b$ seguita da un'attivazione: ogni neurone non di input ha **un peso per ogni ingresso più un bias**. Il conteggio $6 \cdot 4 + 4 \cdot 3 + 3 \cdot 1 = 39$ dimentica proprio i bias.

Strato	Pesi	Bias	Totale
Input (6)	–	–	0
Hidden 1 (4)	$4 \times 6 = 24$	4	28
Hidden 2 (3)	$3 \times 4 = 12$	3	15
Output (1)	$1 \times 3 = 3$	1	4
Totale	**39**	**8**	**47**

In generale, uno strato con d ingressi e k neuroni ha $k \cdot d + k = k(d+1)$ parametri. Questi conteggi vanno avuti perfettamente chiari, perché le architetture cresceranno molto rapidamente.

1.4.3 Forward propagation (slide 30–31)

La rete a 4 strati della figura si scrive così, con i bias incorporati nelle matrici W per alleggerire la notazione:

$$\boxed{\text{out} = \phi\Big(W_3\,\phi\big(W_2\,\phi(W_1 x)\big)\Big)}$$

con $x \in \mathbb{R}^6$, $W_1 \in \mathbb{R}^{4\times 6}$, $W_2 \in \mathbb{R}^{3\times 4}$, $W_3 \in \mathbb{R}^{1\times 3}$. Esplicitando i bias, strato per strato:

Passo	Calcolo	Forma
1	$h_1 = \phi(W_1 x + b_1)$	$(4 \times 6)(6) + (4) \to 4$
2	$h_2 = \phi(W_2 h_1 + b_2)$	$(3 \times 4)(4) + (3) \to 3$
3	$\text{out} = \phi(W_3 h_2 + b_3)$	$(1 \times 3)(3) + (1) \to 1$

Errata slide 30

La slide chiama tutte le matrici W_1 e indica $W_2 \in \mathbb{R}^{4\times 3}$. Seguendo la regola "righe = outbound, colonne = inbound", le dimensioni corrette sono $W_2 \in \mathbb{R}^{3\times 4}$ e $W_3 \in \mathbb{R}^{1\times 3}$.

La **forward propagation** è il calcolo dell'uscita della rete dato l'ingresso. Si chiama *forward* perché segue il flusso dell'informazione **dall'input verso la risposta**, ad esempio dall'immagine alla sua classe.

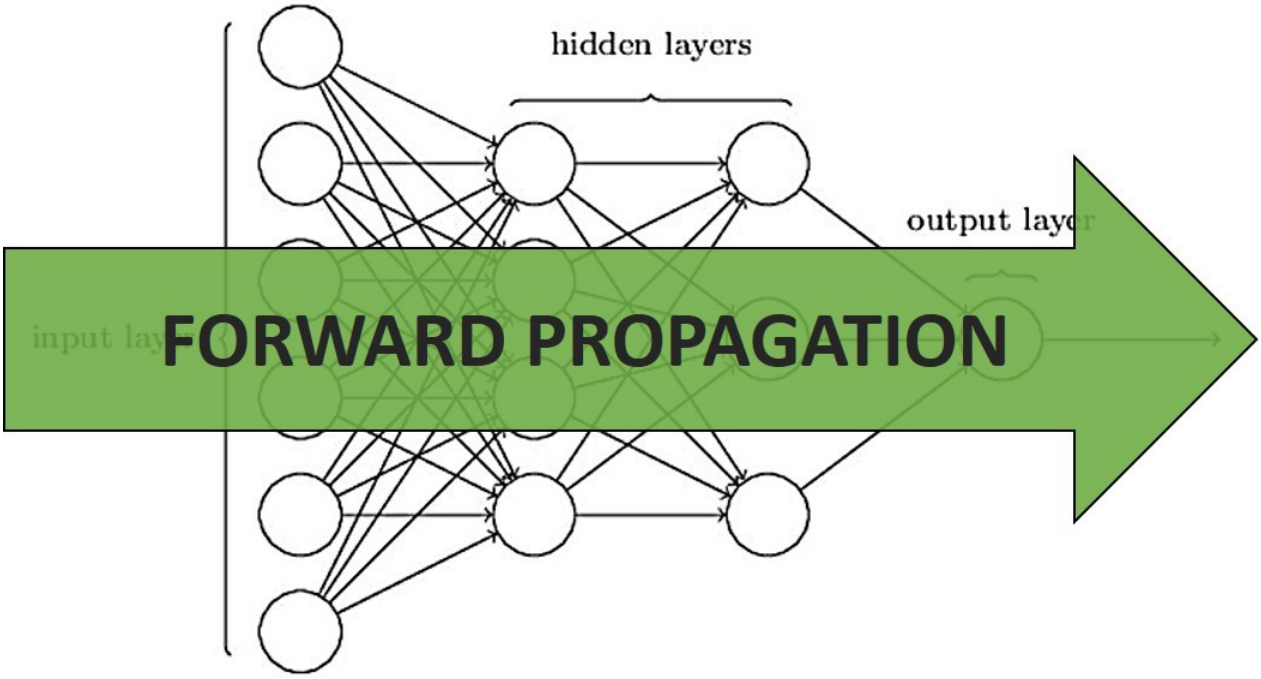


Figura 1.12: Slide 31 – Forward propagation

Una rete neurale è quindi una **composizione non lineare di tante unità lineari**, cioè una grande composizione di funzioni.

Le equazioni rendono esplicito il calcolo della rete. Questo non rende automaticamente interpretabili le rappresentazioni apprese: conoscere le operazioni e attribuire un significato semantico a ogni feature sono problemi distinti.

1.5 Potere rappresentazionale

Nella formula compatta senza bias questi sono **omessi**, non incorporati nelle matrici delle dimensioni riportate. Per incorporarli bisogna aumentare l'ingresso di una componente costante e aggiungere una colonna a ogni matrice: $[W\ b][h^T\ 1]^T = Wh + b$.

1.5.1 Il teorema di approssimazione universale (slide 32)

Una rete è fatta solo di prodotti scalari, somme e soglie. Che percentuale di "tutte le funzioni del mondo", anche quelle non ancora scoperte, può rappresentare?

Teorema di approssimazione universale (slide 32)

Siano $K \subset \mathbb{R}^d$ compatto e $f : K \to \mathbb{R}$ continua. Con un'attivazione adatta, per esempio sigmoide, per ogni $\epsilon > 0$ esiste una rete con uno strato nascosto sufficientemente ampio tale che

$$\boxed{\sup_{x \in K} |f(x) - g(x)| < \epsilon.}$$

Una forma della rete è $g(x) = \sum_{j=1}^{m} a_j \sigma(w_j^T x + b_j) + c$, con $w_j \in \mathbb{R}^d$ e gli altri coefficienti scalari.

La precisione è uniforme su K: vale per ogni suo punto, non soltanto in media sui campioni. Il numero m può dipendere dalla funzione e dalla precisione richiesta. L'enunciato non promette un'approssimazione uniforme sull'intero spazio non limitato, né dice che qualunque attivazione o qualunque rete piccola siano sufficienti. Non permette di dedurre la prevedibilità di fenomeni casuali o l'esistenza di informazioni assenti dall'input.

Per esempio, approssimare una funzione su $[0, 1]$ con errore inferiore a 0.01 significa mantenere tutta la curva della rete nella fascia verticale $f(x) \pm 0.01$. È una proprietà di rappresentazione, non un algoritmo per trovare i pesi.

Rappresentare non significa imparare bene. Il teorema dice che lo strumento *può* rappresentare la funzione, non che la imparerà. È come chiedersi "sei fisicamente in grado di giocare a calcio?": sì. "Sei un buon giocatore?": dipende dall'allenamento. Il teorema riguarda la **rappresentazione**, non la **performance**.

1.5.2 Dalla teoria alla pratica

Passando dalla matematica al computer si introducono approssimazioni:

1. nel computer funzioni e numeri **non sono continui ma discreti** (precisione finita);
2. anche le **misure** degli strumenti sono discrete (risoluzione finita).

Una capacità sufficiente permette di rappresentare relazioni complesse; più profondità può rappresentarle in modo più efficiente. Aumentare la dimensione, però, non garantisce prestazioni migliori: contano dati, ottimizzazione e regolarizzazione. La precisione finita non dimostra che una rete più grande sia sempre preferibile.

1.6 Iperparametri e overfitting

1.6.1 Iperparametri dell'architettura (slide 33)

Numero di layer e numero di neuroni per layer sono **iperparametri** dell'architettura:

- aumentarli fa **crescere la capacità** della rete, cioè lo spazio delle funzioni rappresentabili (**bene**);
- reti più grandi **senza regolarizzazione** sono molto più inclini all'**overfitting** (**male**).

1.6.2 Perché le reti tendono a fare overfitting

Durante l'addestramento si dà alla rete solo una **loss** da ottimizzare sul training set, chiedendole di risolverla al meglio. Con molti parametri la rete ha **molti "posti" in cui memorizzare il training set**: questo è l'overfitting.

La sovraparametrizzazione rende possibile memorizzare il training set, ma non implica necessariamente overfitting. Il problema è una discrepanza tra buone prestazioni di training e prestazioni peggiori su dati nuovi: si valuta con un insieme di validazione indipendente.

L'overfitting è una causa di scarsa generalizzazione anche quando il training è ben ottimizzato e va tenuto sempre presente.

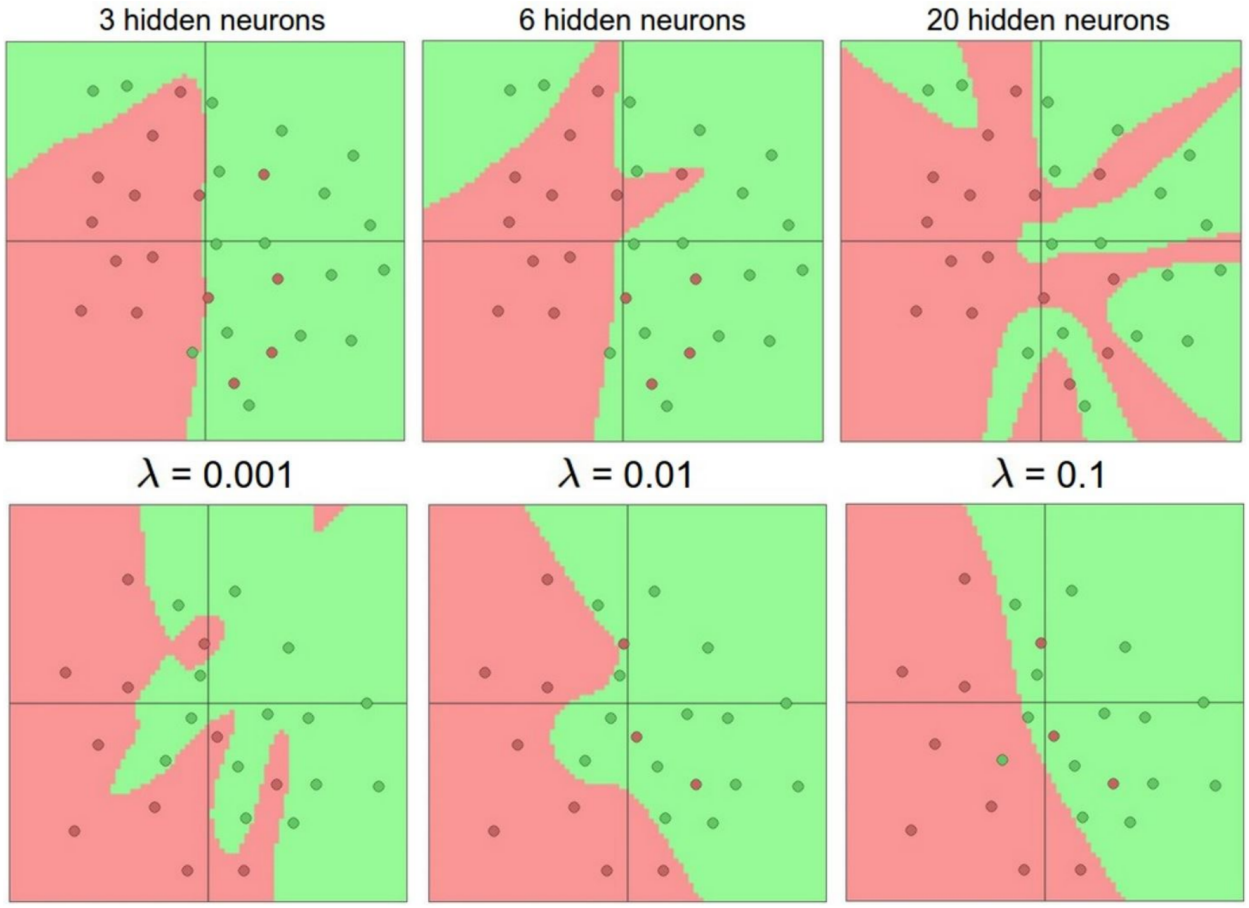


Figura 1.13: Slide 34 – Superfici di decisione al variare dei neuroni e di λ

Riga superiore (slide 34). Rete a 3 layer che separa punti rossi e verdi, al variare del numero di neuroni nascosti:

- **3 neuroni**: superficie non lineare ma regolare;
- **6 neuroni**: compaiono forme irregolari;
- **20 neuroni**: superficie molto irregolare, con zone rosse attorno a singoli punti che probabilmente sarebbero verdi. Aumentando ancora, la rete finirebbe per **disegnare un cerchio attorno a ogni punto**.

Una frontiera molto irregolare può suggerire overfitting, che va verificato su dati indipendenti. E 20 neuroni sono pochissimi.

Regola pratica (slide 34). Le reti si fanno **grandi quanto consente il budget computazionale**, e poi si previene l'overfitting con opportune **tecniche di regolarizzazione** (dropout, weight decay, input noise…), viste in §7.4 e §10.

1.7 Addestrare una rete: funzione obiettivo e loss

1.7.1 La funzione obiettivo (slide 36)

Con l'architettura si è definita una **funzione parametrica**, la forward propagation. Ora serve una procedura per **imparare i parametri**, esattamente come per qualunque modello parametrico di ML: si definisce una loss e la si minimizza.

La loss deve essere un **surrogato continuo** del problema da risolvere: cross-entropy per la classificazione, MSE per la regressione, ecc. Valgono tutte le considerazioni di ML già viste: **l'ipotesi i.i.d.**, il modo di campionare i dati, la **partizione training/test**, la variabilità del training set.

La funzione obiettivo misura la **qualità della mappatura** input → output e ha due termini:

$$\mathcal{L}(\theta; x, y) = \underbrace{\frac{1}{N}\sum_{i=1}^{N} L_i(\theta; x_i, y_i)}_{\text{data term}} + \underbrace{\lambda\,\Omega(\theta)}_{\text{regularization term}}$$

Simbolo	Significato
N	numero di esempi di training
θ	insieme di **tutti** i parametri della rete: $\{W_1, b_1, W_2, b_2, \dots\}$
L_i	loss del singolo esempio i
$\Omega(\theta)$	regolarizzatore
$\lambda \geq 0$	**forza** della regolarizzazione

- **Data term.** Dipende dai **parametri**, dagli **input** e dalla **ground truth**. È una media sugli esempi e misura quanto sono buone le predizioni rispetto alle etichette; dipende dal problema (classificazione o regressione).
- **Regularization term.** Dipende **solo dai parametri**: non dall'input, non dall'output, non dalla bontà delle predizioni. Impone un **vincolo strutturale** sui pesi: chiede di risolvere il problema **usando i parametri il meno possibile**, e così riduce il rischio di overfitting.

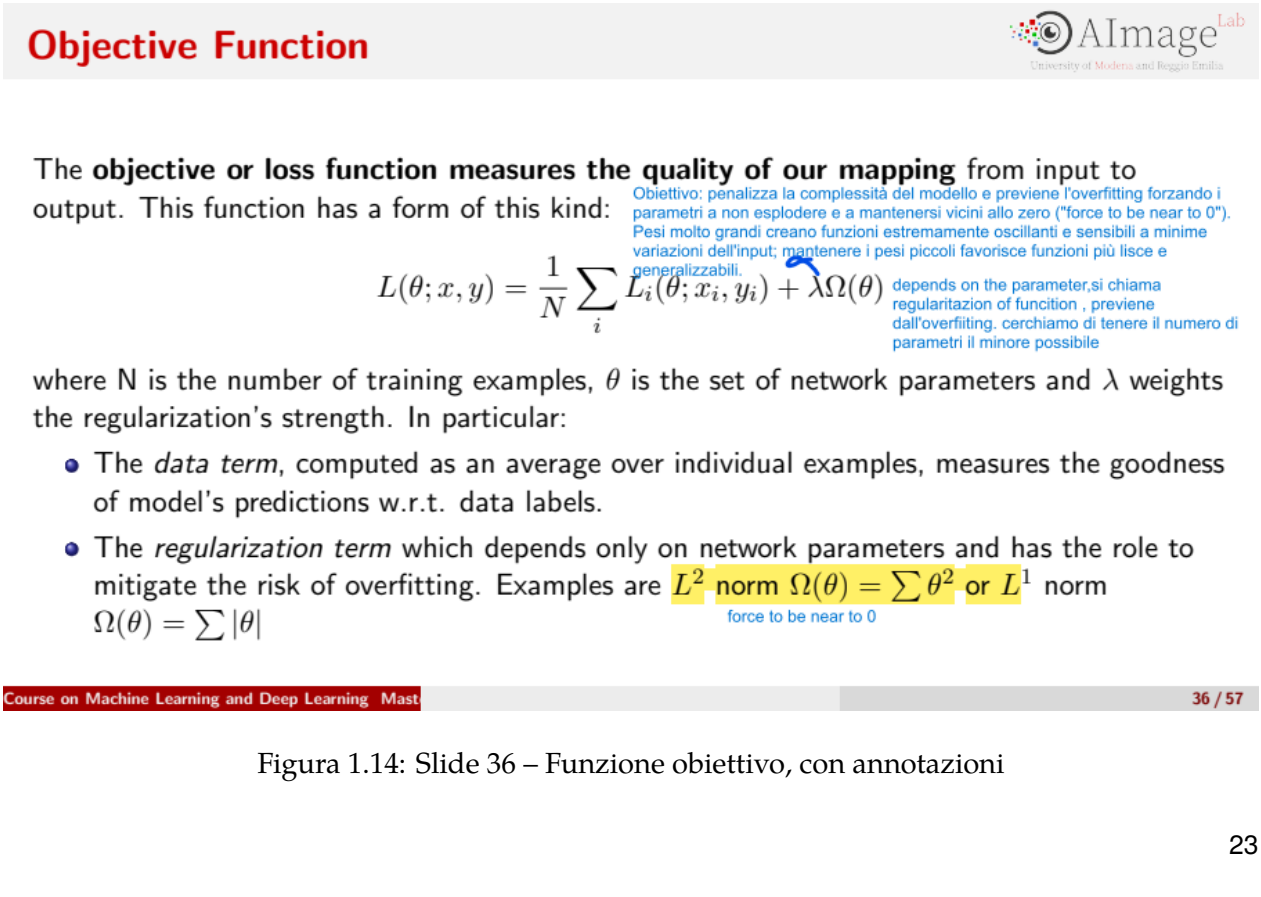


Figura 1.14: Slide 36 – Funzione obiettivo, con annotazioni

1.7.2 Le loss più usate (slide 37)

Le funzioni di loss sono moltissime, ma circa il 90% dei problemi usa quelle della slide 37. La maggior parte delle loss moderne sono loro **piccole modifiche**.

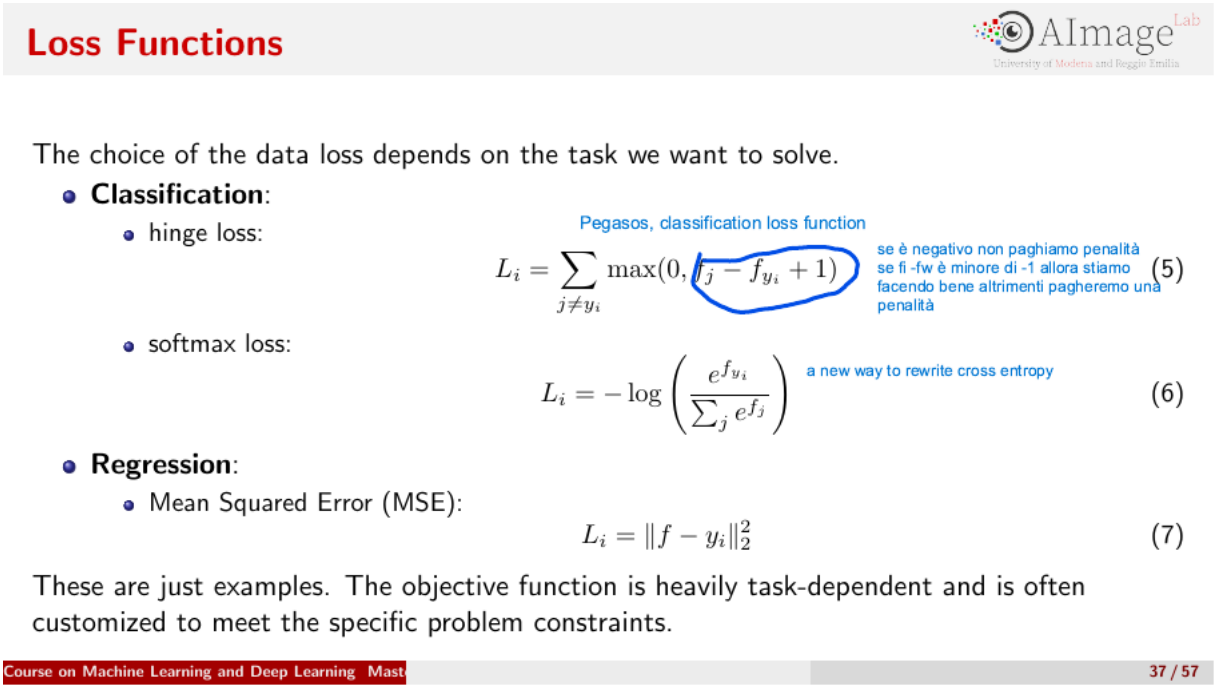


Figura 1.15: Slide 37 – Loss functions, con annotazioni

Notazione: $f = f(x_i; \theta) \in \mathbb{R}^K$ è il vettore degli score prodotti dalla rete per l'esempio i, f_j lo score della classe j, y_i l'indice della classe corretta.

1.7.2.1 Hinge loss (classificazione)

$$\boxed{L_i = \sum_{j \neq y_i} \max\left(0,\ f_j - f_{y_i} + 1\right)}$$

È la loss della **SVM lineare** (algoritmo *Pegasos*). Si chiede al classificatore non solo di classificare correttamente, ma di farlo con un **margine**: lo score della classe corretta deve superare quello di **ogni** altra classe di almeno 1.

Come funziona, passo passo. Per ogni classe sbagliata j:

1. si calcola $f_j - f_{y_i} + 1$;
2. se $f_{y_i} \geq f_j + 1$, cioè la classe giusta batte la j con margine almeno 1, la quantità è ≤ 0 e il max restituisce **0: nessuna penalità**;
3. altrimenti la quantità è positiva e si **paga una penalità proporzionale** a quanto manca per raggiungere il margine;
4. si sommano i contributi di tutte le classi sbagliate.

(Annotazione slide 37: *"se è negativo non paghiamo penalità; se $f_j - f_{y_i}$ è minore di -1 allora stiamo facendo bene, altrimenti pagheremo una penalità"*.)

Esempio con gli score della slide 15, $f = [-96.8,\ 437.9,\ 60.75]$ (gatto, cane, nave):

Classe vera	Termini della somma	L_i
gatto	$\max(0,\ 437.9 + 96.8 + 1) +$ $\max(0,\ 60.75 + 96.8 + 1) =$ $535.7 + 158.55$	**694.25**
cane	$\max(0,\ -96.8 - 437.9 + 1) +$ $\max(0,\ 60.75 - 437.9 + 1) = 0 + 0$	**0**

Se l'immagine è un gatto il classificatore paga molto. Se fosse un cane la loss sarebbe nulla, perché "cane" batte le altre classi con ampio margine.

È la stessa idea del margine della SVM, dove si chiede una distanza minima tra l'iperplano di separazione e le due classi. Qui però il margine non è imposto con un **vincolo**, ma tramite una sua versione "morbida" e linearizzata, il $\max(0, \cdot)$.

Perché proprio 1? La riscalatura del margine richiede anche di adattare il peso relativo della regolarizzazione per mantenere lo stesso problema. Qualunque altro margine $\Delta > 0$ (10, 20, 300...) sarebbe equivalente, perché può essere **assorbito nella norma dei parametri**:

1. moltiplicando per $c > 0$ i pesi e il bias dello strato di uscita (in un classificatore lineare, tutti i parametri), tutti gli score si moltiplicano per c: $f \to cf$;
2. quindi anche le differenze si moltiplicano per c: $f_{y_i} - f_j \to c\,(f_{y_i} - f_j)$;
3. se dei parametri garantiscono il margine 1, gli stessi parametri con lo strato di uscita moltiplicato per Δ garantiscono il margine Δ, e viceversa.

Il valore del margine si ottiene quindi semplicemente cambiando la scala dei pesi. È lo stesso passaggio della SVM, dove il margine 2ρ veniva riportato a 1 assorbendolo in w.

1.7.2.2 Softmax loss / cross-entropy (classificazione)

$$\boxed{L_i = -\log\left(\frac{e^{f_{y_i}}}{\sum_j e^{f_j}}\right)}$$

È la loss più usata, ed è **un altro modo di scrivere la cross-entropy** (annotazione slide 37: *"a new way to rewrite cross entropy"*). Prende infatti i nomi di *cross-entropy* o *softmax loss*. Anche GPT usa questa loss, in una forma leggermente diversa.

Derivazione dalla cross-entropy, passo passo:

1. Si parte dalla cross-entropy di §2.4: $L_i = -\sum_k y_k^{true} \log y_k^{pred}$.
2. y^{true} è one-hot: vale 1 solo per $k = y_i$ e 0 altrove. La somma fa quindi da **selettore**, come nella BCE, e sopravvive un solo termine: $L_i = -\log y_{y_i}^{pred}$.
3. y^{pred} è la softmax degli score, quindi $y_{y_i}^{pred} = \dfrac{e^{f_{y_i}}}{\sum_j e^{f_j}}$, da cui la formula nel riquadro.
4. Usando $\log(a/b) = \log a - \log b$ e $\log e^a = a$ si ottiene una forma equivalente, utile per i calcoli:

$$L_i = -f_{y_i} + \log \sum_j e^{f_j}$$

Minimizzare $-\log(\cdot)$ equivale a **massimizzare** la probabilità (normalizzata) della classe corretta: si prende lo score della risposta giusta e lo si vuole alzare.

Esempio con gli score della slide 18, $z = [1.2,\ 5.1,\ 2.7]$ e softmax $[0.018,\ 0.900,\ 0.082]$:

Classe vera	$L_i = -\log(\text{softmax}_{y_i})$
classe 2 (score più alto)	$-\log 0.900 = 0.105$
classe 3	$-\log 0.082 = 2.51$
classe 1 (score più basso)	$-\log 0.018 = 4.01$

Il gradiente della softmax loss. È il termine $\partial L/\partial o$ da cui parte la backpropagation (§8.4). Derivando la forma del passo 4 rispetto al generico score f_k:

1. il termine $-f_{y_i}$ ha derivata -1 se $k = y_i$ e 0 altrimenti; si scrive $-\mathbb{1}[k = y_i]$;
2. il termine $\log \sum_j e^{f_j}$ ha derivata $\frac{1}{\sum_j e^{f_j}} \cdot e^{f_k}$ (regola della catena: derivata del log per derivata dell'argomento), cioè proprio $\text{softmax}_k(f)$;
3. quindi:

$$\boxed{\frac{\partial L_i}{\partial f_k} = \text{softmax}_k(f) - \mathbb{1}[k = y_i]}$$

Il gradiente è "probabilità predetta meno target": per l'esempio con classe vera 2 vale $[0.018,\ 0.900 - 1,\ 0.082] = [0.018,\ -0.100,\ 0.082]$. È la stessa forma "predizione − etichetta" della regressione logistica.

1.7.2.3 Mean Squared Error (regressione)

$$\boxed{L_i = \|f - y_i\|_2^2 = \sum_k \left(f_k - y_{i,k}\right)^2}$$

Nella regressione si conoscono coppie di input e output di una funzione, ma non la sua forma né la sua equazione, e la si vuole "indovinare" a meno di una certa approssimazione. Nella regressione lineare la funzione cercata era lineare; con una rete neurale non lo è. La loss più usata, nella sua forma base, è l'errore quadratico: si prende la differenza tra predizione e target e la si eleva al quadrato.

Ad esempio, con predizione 2.5 e target 2 si ha $L_i = 0.25$. Il gradiente rispetto alla predizione è $2(f - y_i)$, cioè di nuovo proporzionale all'errore.

Le loss dipendono dal compito

Queste sono le famiglie principali. La funzione obiettivo dipende molto dal compito e spesso la si personalizza, con termini aggiuntivi o una forma adattata alle particolarità del problema.

1.7.3 Imparare i parametri (slide 38)

Durante il training si cerca l'insieme di parametri che **minimizza la funzione obiettivo** (data term + regolarizzazione) sul training set:

$$\boxed{\theta^* = \arg\min_\theta \left(\frac{1}{N} \sum_i L_i(\theta; x_i, y_i) + \lambda\, \Omega(\theta) \right)}$$

Lo si fa con la discesa del gradiente, $\theta \leftarrow \theta - \eta\, \nabla_\theta \mathcal{L}$. Serve quindi la derivata della loss rispetto a **ogni** parametro di **ogni** strato, ed è questo il compito della backpropagation (§8).

1.7.4 Regolarizzazione esplicita: L_2 e L_1 (slide 36)

Quadrato della norma L_2 (equivalente a decadimento dei pesi nella discesa del gradiente semplice):

$$\Omega(\theta) = \sum_j \theta_j^2, \qquad \frac{\partial \Omega}{\partial \theta_j} = 2\theta_j$$

È minima quando i parametri sono **nulli o piccoli**. Essendo una funzione liscia, **non forza i parametri a essere esattamente zero, ma li rende piccoli** (*"force to be near to 0"*). Pesi molto grandi producono funzioni oscillanti e sensibili a minime variazioni dell'input; pesi piccoli favoriscono funzioni **più lisce e generalizzabili** (annotazione slide 36).

Norma L_1**:**

$$\Omega(\theta) = \sum_j |\theta_j|, \qquad \frac{\partial \Omega}{\partial \theta_j} = \text{sign}(\theta_j) \quad (\theta_j \neq 0)$$

La derivata ha modulo **1 per ogni parametro non nullo**, indipendentemente da quanto è piccolo. La L_1 è un regolarizzatore di **sparsità**: favorisce soluzioni con alcuni parametri **esattamente zero**, ma penalizza la somma dei moduli, non il numero dei coefficienti non nulli.

Da dove viene la differenza: l'aggiornamento passo passo. Si guarda solo l'effetto del regolarizzatore su un singolo peso θ, con learning rate η.

1. L_2**:**

   $$\theta \leftarrow \theta - \eta\lambda \cdot 2\theta = (1 - 2\eta\lambda)\,\theta$$

   A ogni passo il peso viene **moltiplicato** per un fattore tra 0 e 1 quando $0 < 2\eta\lambda < 1$: decade in proporzione a quanto è grande (da qui il nome *weight decay*). Più è piccolo, più piccola è la spinta.
2. L_1**:**

   $$\theta \leftarrow \theta - \eta\lambda \ \text{sign}(\theta)$$

   A ogni passo al peso si **sottrae** una quantità **costante**, qualunque sia il suo valore. La spinta non si attenua, ma il metodo a passo costante può oltrepassare zero e oscillare. Per esempio, con passo 0.1, $0.05 \to -0.05 \to 0.05$. La sparsità dell'ottimo non garantisce che un aggiornamento elementare raggiunga esattamente zero. In zero si usa un sottogradiente in $[-1, 1]$.

Esempio: $\theta = 1$ con $2\eta\lambda = 0.2$ per L_2 e $\eta\lambda = 0.1$ per L_1, ignorando il data term.

passo	0	1	2	3	...	10
L_2: $\theta \cdot 0.8$	1	0.8	0.64	0.512	...	0.107 (mai 0)
L_1: $\theta - 0.1$	1	0.9	0.8	0.7	...	**0**

L_2 **vs** L_1

- L_2 rende i pesi **piccoli**.

- L_1 rende una parte dei pesi **esattamente zero** (sparsità).

Entrambe penalizzano i valori dei parametri della rete, quindi entrambe regolarizzano. Si chiamano **regolarizzatori espliciti** perché sono scritti esplicitamente nella loss e agiscono attraverso i gradienti.

Effetto di λ (slide 34, riga inferiore). La rete con **20 neuroni**, quella in overfitting, viene addestrata con regolarizzazione L_2 di forza crescente:

- $\lambda = 0.001$: superficie ancora frastagliata;
- $\lambda = 0.01$: forme più regolari;
- $\lambda = 0.1$: il problema viene risolto **in modo quasi lineare**.

Il risultato si ottiene solo **costringendo la rete a usare meno parametri possibile**, senza cambiarne l'architettura.

1.8 Backpropagation (slide 39–50)

1.8.1 La regola della catena (slide 39)

L'algoritmo che calcola i gradienti della loss rispetto ai parametri si chiama **backpropagation**. Si basa sulla **regola della catena**, la regola per derivare una **composizione di funzioni**. Una rete è proprio una composizione di funzioni, in cui ogni funzione è l'equazione di uno strato e a valle c'è la loss: prima o poi bisogna calcolare la derivata di $f(g(x))$.

$$\boxed{\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}}, \qquad z = f(g(x)), \quad y = g(x)$$

Si introduce una variabile ausiliaria y (l'argomento della funzione esterna) e si scrive la derivata come prodotto di due derivate più semplici.

Controllo rapido. Per verificare di aver scritto bene la catena si "semplificano" formalmente i termini uguali al numeratore e al denominatore (il dy, barrato nell'annotazione della slide 39). Se si ottiene un'identità, $\frac{dz}{dx} = \frac{dz}{dx}$, la catena è scritta correttamente; altrimenti c'è un errore. È solo un controllo mnemonico, non un'operazione algebrica vera.

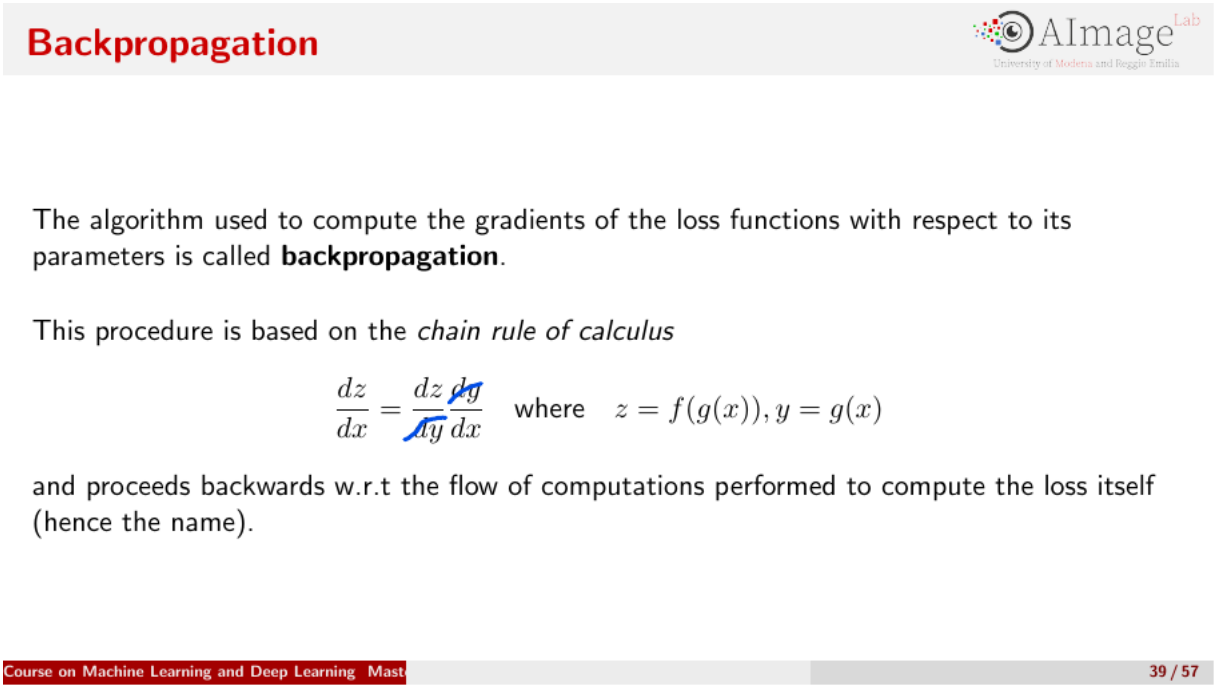


Figura 1.16: Slide 39 – Regola della catena, con annotazioni

Esempio: $z = (3x + 1)^2$.

1. Si pone $y = 3x + 1$, quindi $z = y^2$.
2. $\frac{dz}{dy} = 2y$ e $\frac{dy}{dx} = 3$.
3. $\frac{dz}{dx} = 2y \cdot 3 = 6(3x + 1)$.

La procedura procede **all'indietro** rispetto al flusso di calcoli fatto per ottenere la loss, da qui il nome.

1.8.2 Le due derivate locali di uno strato (slide 40)

La backpropagation è un processo **locale**: un neurone (o uno strato) non conosce la topologia della rete in cui si trova. Perché l'algoritmo funzioni, ogni strato deve saper calcolare **solo due cose**.

Si consideri lo strato i, con ingresso $h_i \in \mathbb{R}^d$, parametri $W_{i+1} \in \mathbb{R}^{k \times d}$ (k righe = uscite, d colonne = ingressi) e uscita $h_{i+1} \in \mathbb{R}^k$. Con attivazione ReLU (bias incorporato in W):

$$h_{i+1} = \max\left(0,\ W_{i+1} h_i\right)$$

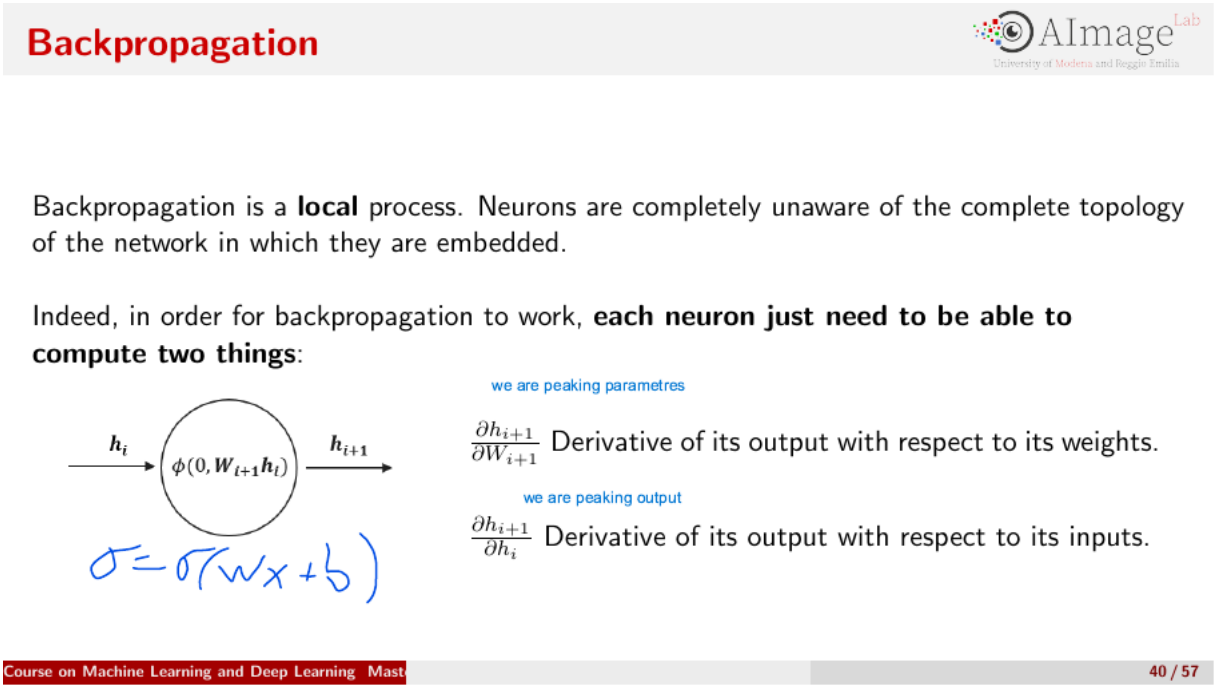


Figura 1.17: Slide 40 – Le due derivate locali, con annotazioni

Le due derivate da saper calcolare sono:

1. **la derivata dell'uscita rispetto ai pesi**, $\frac{\partial h_{i+1}}{\partial W_{i+1}}$, che serve ad **aggiornare** i pesi dello strato;
2. **la derivata dell'uscita rispetto all'ingresso**, $\frac{\partial h_{i+1}}{\partial h_i}$, che serve a **passare il gradiente** allo strato precedente.

Calcolo nel caso scalare. Un neurone con un ingresso: $h_{i+1} = \max(0,\ w\, h_i)$. Si considera il caso "acceso" $(wh_i > 0)$, in cui la ReLU è l'identità e lo strato è lineare, $h_{i+1} = w\, h_i$. Come per $a \cdot x$, la cui derivata rispetto a x è a:

- $\frac{\partial h_{i+1}}{\partial w} = h_i$: la derivata rispetto al peso **"preleva" l'ingresso**;
- $\frac{\partial h_{i+1}}{\partial h_i} = w$: la derivata rispetto all'ingresso **"preleva" il peso**.

Se il neurone è spento $(wh_i < 0)$, l'uscita è costantemente 0 ed entrambe le derivate valgono **0**.

Calcolo nel caso vettoriale, passo passo. Si chiama $z = W_{i+1}h_i$ (il "pre-attivazione", k componenti).

1. La componente k è $z_k = \sum_{j=1}^{d} W_{kj}\, h_{i,j}$, quindi $\frac{\partial z_k}{\partial W_{kj}} = h_{i,j}$ e $\frac{\partial z_k}{\partial h_{i,j}} = W_{kj}$.
2. La ReLU agisce componente per componente: $h_{i+1,k} = \max(0, z_k)$, con derivata $m_k = \mathbb{1}[z_k > 0]$ (1 se il neurone è acceso, 0 se spento).
3. Per la catena: $\frac{\partial h_{i+1,k}}{\partial W_{kj}} = m_k\, h_{i,j}$ e $\frac{\partial h_{i+1,k}}{\partial h_{i,j}} = m_k\, W_{kj}$.
4. Nella backpropagation arriva da valle il vettore $g = \partial \mathcal{L}/\partial h_{i+1}$ (k componenti). Si calcola prima $\delta = g \odot m$, cioè si azzera il gradiente dei neuroni spenti. Allora:

$$\boxed{\frac{\partial \mathcal{L}}{\partial W_{i+1}} = \delta\, h_i^T \in \mathbb{R}^{k \times d}} \qquad \boxed{\frac{\partial \mathcal{L}}{\partial h_i} = W_{i+1}^T\, \delta \in \mathbb{R}^d}$$

Le forme tornano: il gradiente rispetto a W ha la stessa forma di W $(k \times d)$, quello rispetto a h_i la stessa forma di h_i (d). Le due formule sono la versione matriciale di "preleva l'ingresso" (h_i^T) e "preleva il peso" (W^T).

Derivate locali e addestramento

Per propagare il gradiente bastano le derivate locali necessarie alla regola della catena, calcolate esplicitamente o mediante differenziazione automatica. Ai punti non differenziabili occorre una convenzione appropriata. Questo rende calcolabile il backward, ma non garantisce convergenza o buona generalizzazione.

Con un bias $b \in \mathbb{R}^k$, $z = Wh + b$ e $\partial z_j / \partial b_i = \delta_{ij}$: pertanto $\boxed{\nabla_b \mathcal{L} = \delta \in \mathbb{R}^k}$.

1.8.3 Il gradiente che si propaga (slide 41)

Durante la backpropagation lo strato riceve da valle $\dfrac{\partial \mathcal{L}}{\partial h_{i+1}}$, cioè l'influenza della sua uscita sull'uscita dell'intera rete. Questo gradiente viene **concatenato** con i due gradienti locali:

$$\frac{\partial \mathcal{L}}{\partial W_{i+1}} = \frac{\partial \mathcal{L}}{\partial h_{i+1}} \frac{\partial h_{i+1}}{\partial W_{i+1}} \quad \text{(per aggiornare i pesi)}, \qquad \frac{\partial \mathcal{L}}{\partial h_i} = \frac{\partial \mathcal{L}}{\partial h_{i+1}} \frac{\partial h_{i+1}}{\partial h_i} \quad \text{(passato allo strato precedente)}$$

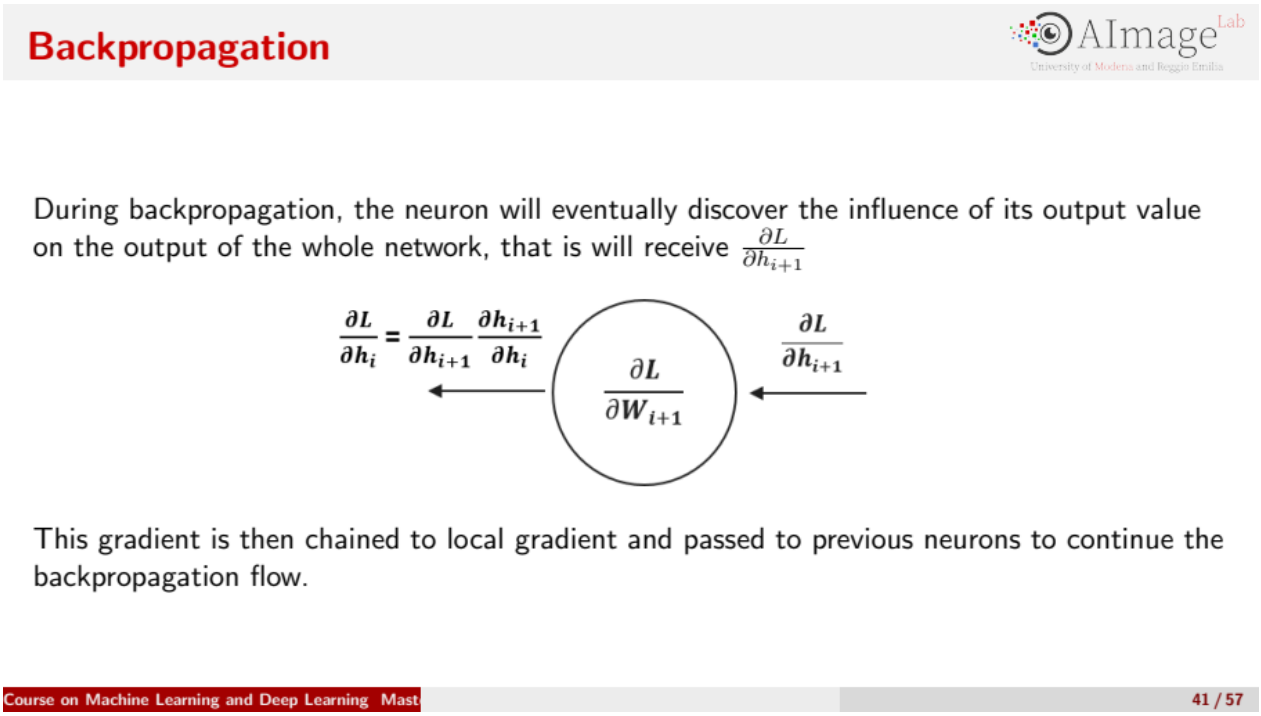


Figura 1.18: Slide 41 – Concatenazione del gradiente

1.8.4 Esempio: MLP a 3 strati con ReLU (slide 42–50)

Si consideri un MLP a tre strati, in cui ogni cerchio è uno strato con attivazione ReLU, e un dataset su cui addestrarlo.

1. Forward (slide 43–47). Si passa un esempio x attraverso i tre strati e si calcola la penalità della loss sull'uscita:

$$h_1 = \max(0, W_1 x), \qquad h_2 = \max(0, W_2 h_1), \qquad o = \max(0, W_3 h_2), \qquad \mathcal{L} = \text{Loss}(o, y)$$

Dato che o e y sono numeri, la penalità si calcola con l'equazione della loss scelta. Per applicare la discesa del gradiente servono ora le derivate parziali della loss rispetto a **ogni** insieme di parametri. Le si calcola **strato per strato, a partire dall'ultimo**: prima W_3, poi W_2, poi W_1.

2. Gradiente rispetto a W_3 (slide 47–48). Tra $\mathcal{L}$ e W_3 non c'è una dipendenza diretta: in mezzo c'è l'equazione della loss (un quadrato, un esponenziale, ...), che è a sua volta una funzione di o. Si applica la regola della catena una prima volta:

$$\boxed{\frac{\partial \mathcal{L}}{\partial W_3} = \frac{\partial \mathcal{L}}{\partial o} \cdot \frac{\partial o}{\partial W_3}}$$

- $\dfrac{\partial o}{\partial W_3}$ è la derivata locale di §8.2: vale h_2 se il neurone è acceso, 0 se è spento.
- $\dfrac{\partial \mathcal{L}}{\partial o}$ dipende dall'equazione della loss: ad esempio $2(o-y)$ per la MSE, oppure $\text{softmax}(o) - y$ per la cross-entropy (§7.2).

Una volta calcolato il valore numerico (su un esempio, su un batch o su tutto il dataset, come nella discesa del gradiente classica), si prepara l'aggiornamento del parametro. **Tutti i gradienti si calcolano con gli stessi pesi del forward; gli aggiornamenti si applicano solo dopo avere completato il backward:**

$$W_3^{new} = W_3 - \alpha \frac{\partial \mathcal{L}}{\partial W_3}$$

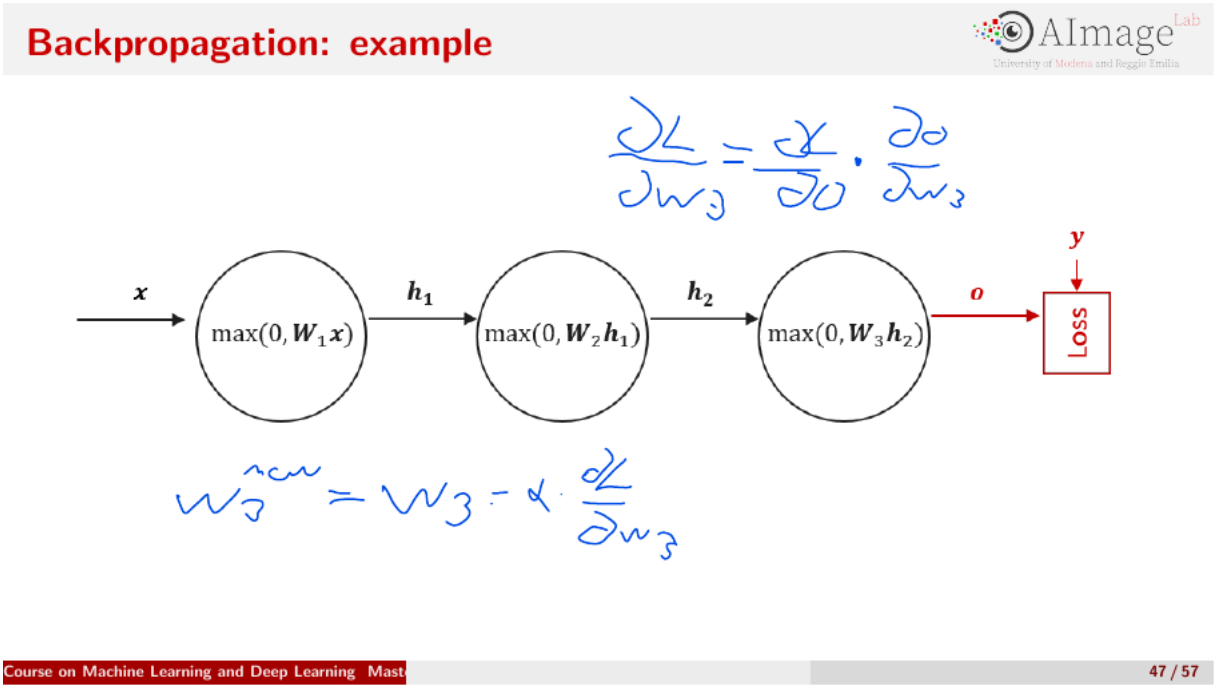


Figura 1.19: Slide 47 – Gradiente e aggiornamento di W3, con annotazioni

3. Gradiente rispetto a W_2 (slide 49). Anche qui non c'è dipendenza diretta, perché tra la loss e W_2 c'è un altro strato.

Un primo tentativo naturale ma **sbagliato** è $\frac{\partial \mathcal{L}}{\partial o} \cdot \frac{\partial o}{\partial W_3} \cdot \frac{\partial W_3}{\partial W_2}$. Non esiste alcuna equazione che leghi W_3 a W_2: sono parametri **indipendenti**, quindi $\partial W_3 / \partial W_2 = 0$: quel percorso non esprime la dipendenza della loss da W_2.

La strada giusta è passare attraverso le **attivazioni**, che sono legate tra loro dalle equazioni degli strati:

- o dipende da h_2 tramite $o = \max(0, W_3 h_2)$;
- h_2 dipende da W_2 tramite $h_2 = \max(0, W_2 h_1)$.

$$\boxed{\frac{\partial \mathcal{L}}{\partial W_2} = \frac{\partial \mathcal{L}}{\partial o} \cdot \frac{\partial o}{\partial h_2} \cdot \frac{\partial h_2}{\partial W_2}}$$

Per §8.2, nel caso acceso $\frac{\partial o}{\partial h_2} = W_3$ ("preleva il peso") e $\frac{\partial h_2}{\partial W_2} = h_1$ ("preleva l'ingresso"). I primi due fattori insieme sono $\frac{\partial \mathcal{L}}{\partial h_2}$, la freccia rossa che torna indietro nella slide 49.

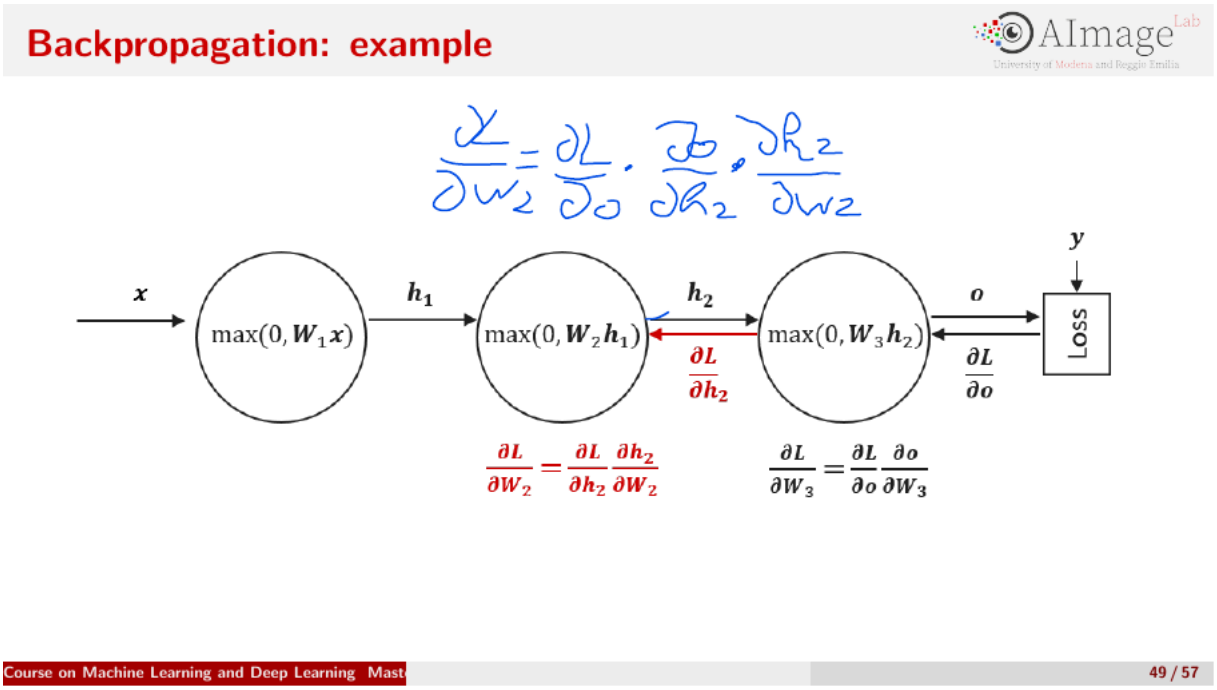


Figura 1.20: Slide 49 – Gradiente di W2, con annotazioni

4. Gradiente rispetto a W_1 (slide 50). Stesso ragionamento, con un anello in più nella catena:

$$\boxed{\frac{\partial \mathcal{L}}{\partial W_1} = \frac{\partial \mathcal{L}}{\partial o} \cdot \frac{\partial o}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial W_1}}$$

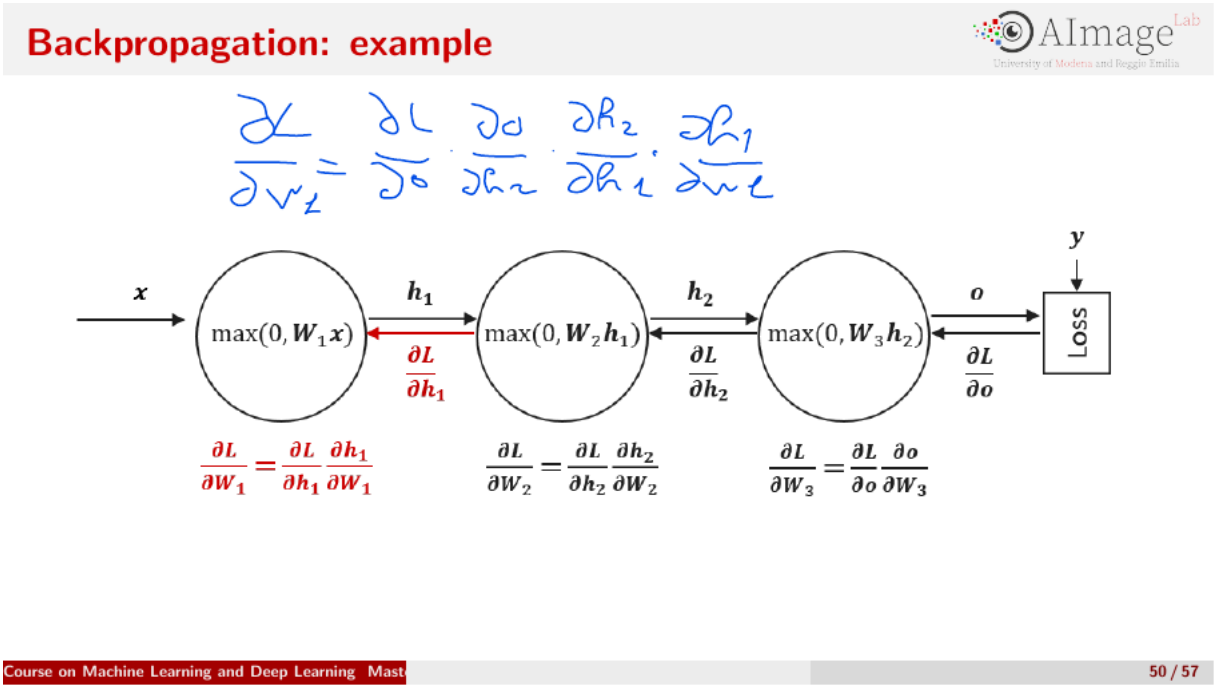


Figura 1.21: Slide 50 – Backpropagation completa, con annotazioni

La forma generale. Per ogni strato, il gradiente dei pesi è sempre **"gradiente della loss rispetto all'uscita dello strato"** $\times$ **"derivata dell'uscita rispetto ai pesi"**:

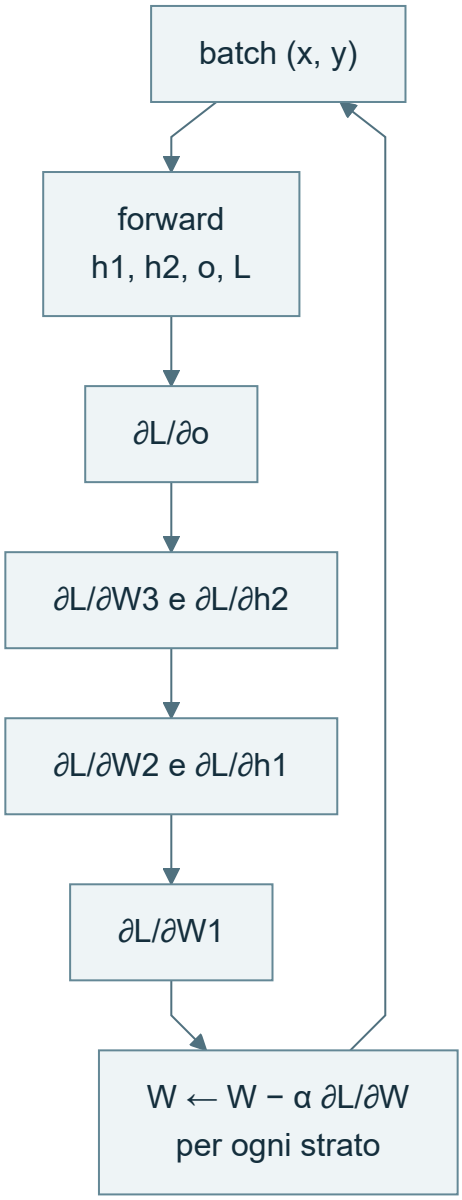
$$\frac{\partial \mathcal{L}}{\partial W_3} = \frac{\partial \mathcal{L}}{\partial o}\frac{\partial o}{\partial W_3}, \qquad \frac{\partial \mathcal{L}}{\partial W_2} = \frac{\partial \mathcal{L}}{\partial h_2}\frac{\partial h_2}{\partial W_2}, \qquad \frac{\partial \mathcal{L}}{\partial W_1} = \frac{\partial \mathcal{L}}{\partial h_1}\frac{\partial h_1}{\partial W_1}$$

E i gradienti rispetto alle uscite si ottengono uno dall'altro, **riusando quello appena calcolato**:

$$\frac{\partial \mathcal{L}}{\partial h_2} = \frac{\partial \mathcal{L}}{\partial o} \cdot \frac{\partial o}{\partial h_2}, \qquad \frac{\partial \mathcal{L}}{\partial h_1} = \frac{\partial \mathcal{L}}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_1}$$

Per W_1 non si ricalcola tutta la catena da zero: si prende $\partial \mathcal{L}/\partial h_2$, già calcolato per W_2, e lo si moltiplica per un solo fattore locale. Per questo si procede **dall'ultimo strato al primo**: ogni strato riceve il gradiente da valle, lo usa e lo passa a monte. È ciò che rende l'algoritmo efficiente.

5. Il ciclo completo. Aggiornati i parametri, si prende un altro esempio o un altro batch, si ripete forward, backward e aggiornamento, e si itera fino a convergenza: è la discesa del gradiente stocastica.



Esempio numerico (rete scalare, un neurone per strato)

Con $x = 1$, $y = 2$, $w_1 = 0.5$, $w_2 = 2$, $w_3 = 1.5$, ReLU e loss $\mathcal{L} = (o - y)^2$.

Forward: - $h_1 = \max(0,\ 0.5 \cdot 1) = 0.5$ - $h_2 = \max(0,\ 2 \cdot 0.5) = 1$ - $o = \max(0,\ 1.5 \cdot 1) = 1.5$ - $\mathcal{L} = (1.5 - 2)^2 = 0.25$

Tutti i neuroni sono accesi, quindi le ReLU si comportano come identità.

Backward, dall'uscita verso l'ingresso:

Passo	Formula	Valore
$\partial\mathcal{L}/\partial o$	$2(o - y)$	$2(1.5 - 2) = -1$
$\partial\mathcal{L}/\partial w_3$	$\partial\mathcal{L}/\partial o \cdot h_2$	$-1 \cdot 1 = -1$
$\partial\mathcal{L}/\partial h_2$	$\partial\mathcal{L}/\partial o \cdot w_3$	$-1 \cdot 1.5 = -1.5$
$\partial\mathcal{L}/\partial w_2$	$\partial\mathcal{L}/\partial h_2 \cdot h_1$	$-1.5 \cdot 0.5 = -0.75$
$\partial\mathcal{L}/\partial h_1$	$\partial\mathcal{L}/\partial h_2 \cdot w_2$	$-1.5 \cdot 2 = -3$
$\partial\mathcal{L}/\partial w_1$	$\partial\mathcal{L}/\partial h_1 \cdot x$	$-3 \cdot 1 = -3$

Aggiornamento con $\alpha = 0.05$: - $w_1 = 0.5 + 0.15 = 0.65$ - $w_2 = 2 + 0.0375 = 2.0375$ - $w_3 = 1.5 + 0.05 = 1.55$

Nuovo forward: $o = 2.053$ e $\mathcal{L} = 0.0028$, molto più vicino al target. I gradienti sono stati verificati numericamente con le differenze finite.

1.8.5 Perché la backpropagation è così importante

- **Vale per qualunque strato.** Nelle formule di §8.4 non si specifica mai l'equazione degli strati: si scrive solo $\partial h_2/\partial h_1$ o $\partial h_1/\partial W_1$. L'algoritmo vale quindi per qualunque strato di cui si sappiano calcolare le due derivate locali, ed è questo che dà la libertà di costruire qualunque architettura.
- **È lo stesso algoritmo dal 1986.** Ciò che è cambiato è l'**ottimizzatore**, cioè la regola di aggiornamento (SGD, Adam, ...), ma il modo di calcolare i gradienti è rimasto identico. In oltre 30 anni nessuno ha trovato un metodo migliore per efficienza, scalabilità e tempo di calcolo; ci sono stati tentativi, alcuni buoni e altri meno.
- **La usa ogni rete.** Chi ha addestrato una rete, ad esempio per la tesi triennale, l'ha usata senza saperlo, che fosse un Transformer, una rete convoluzionale o altro. In PyTorch è ciò che esegue `loss.backward()` (vedi 01 Introduzione PyTorch e 02 LogReg torch optim).

1.9 Inizializzazione dei pesi (slide 51)

La discesa del gradiente parte da una **inizializzazione casuale** dei pesi. Con 10 o 20 pesi la cosa sembra banale; con milioni, miliardi o migliaia di miliardi di pesi anche l'inizializzazione diventa un problema. I requisiti sono due.

1. Pesi piccoli, per evitare l'esplosione dei valori. Si consideri uno strato con $n = 10\,000$ ingressi. Ogni neurone calcola una somma di 10 000 termini $w_j x_j$.

- Se tutti i pesi valessero 1 e gli ingressi fossero circa 100, l'uscita sarebbe circa $10\,000 \cdot 100 = 10^6$.
- Strato dopo strato, numeri così crescono senza controllo.

Si vuole quindi evitare il più possibile che la grandezza delle uscite degli strati cresca, e per questo all'inizio servono pesi **molto piccoli**, ma non trascurabili.

2. Pesi diversi tra loro, per rompere la simmetria. I pesi non possono essere tutti uguali. Se neuroni permutabili sono inizializzati simmetricamente sia nelle connessioni entranti sia in quelle uscenti, ricevono gli stessi aggiornamenti:

1. con le stesse righe in W, tutti i neuroni calcolano la **stessa uscita**;
2. ricevono quindi lo **stesso gradiente** (δh^T ha righe uguali);
3. subiscono lo **stesso aggiornamento**;
4. restano uguali per sempre, e lo strato si comporta come un solo neurone.

Ogni neurone riesce a imparare qualcosa di diverso solo perché parte da pesi diversi.

La prassi è campionare valori **piccoli, attorno allo zero, positivi o negativi**, ad esempio da una gaussiana a media zero, e mettere i **bias a zero**. Il difficile è trovare per ogni neurone pesi abbastanza piccoli da evitare problemi numerici e, allo stesso tempo, abbastanza diversi da quelli degli altri neuroni dello strato.

Scalare con il fan-in (inizializzazione di Glorot). Un trucco molto diffuso è **scalare i pesi in base al fan-in** del neurone, cioè al numero dei suoi ingressi. Per controllare la varianza della somma, la deviazione standard dei pesi si scala in proporzione a $1/\sqrt{n}$; dividere i pesi per n ridurrebbe eccessivamente la varianza, come mostra il calcolo seguente.

Perché funziona, passo passo. Si calcola la varianza dell'uscita $z = \sum_{j=1}^{n} w_j x_j$ di un neurone con n ingressi.

1. Si assume che i pesi w_j siano indipendenti, a media zero, con varianza σ_w^2, e per semplicità ingressi fissati con $x_j^2 = 1$.
2. La varianza di una somma di termini indipendenti è la somma delle varianze: $\mathrm{Var}(z) = \sum_j \mathrm{Var}(w_j x_j)$.
3. Ogni termine vale $\mathrm{Var}(w_j x_j) = x_j^2\,\sigma_w^2 = \sigma_w^2$.
4. Quindi $\mathrm{Var}(z) = n\,\sigma_w^2$: la deviazione standard dell'uscita è $\sqrt{n}\,\sigma_w$ e **cresce con il numero di ingressi**.
5. Per mantenerla dell'ordine di 1 serve $\sigma_w^2 = 1/n$, cioè pesi con deviazione standard $1/\sqrt{n}$.

Pesi ($n = 10\,000$, $x_j = 1$)	Deviazione standard di z
$\mathcal{N}(0, 1)$	≈ 100
$\mathcal{N}(0, 1/n)$, cioè std $1/\sqrt{n}$	≈ 1
$\mathcal{N}(0, 1)/n$	≈ 0.01 (troppo piccola)

Precisazione sulle formule esatte

Dividere per n dà l'idea, ma per tenere la varianza costante si scala la **varianza** per $1/\text{fan-in}$, cioè la deviazione standard per $1/\sqrt{\text{fan-in}}$. Le varianti note sono: - **Glorot/Xavier**: $\mathrm{Var}(w) = 2/(\text{fan-in} + \text{fan-out})$; - **He**: $2/\text{fan-in}$, pensata per la ReLU.

Tutte queste strategie sono già implementate in **PyTorch**, non vanno scritte a mano. Quando si crea uno strato lineare (`nn.Linear`), è già inizializzato; si può scegliere un'altra inizializzazione (gaussiana, uniforme, Glorot, ...) con le funzioni di `torch.nn.init`.

1.10 Regolarizzazione nelle reti profonde (slide 52–56)

1.10.1 Regolarizzazione implicita (slide 53)

Le reti profonde hanno una **capacità rappresentazionale molto alta**: è un bene, perché possono apprendere funzioni complesse. Per lo stesso motivo, però, sono soggette a overfitting se non vengono regolarizzate.

- È facilissimo aggiungere milioni di parametri, mentre il dataset contiene magari migliaia di esempi: quasi sempre ci sono più parametri che dati.
- Oltre ai **regolarizzatori espliciti** (L_1, L_2 di §7.4), che sono scritti nella loss e agiscono tramite i gradienti, esistono tecniche **implicite**.
- Si chiamano implicite perché non compaiono nell'equazione da ottimizzare: sono **accorgimenti applicati durante l'apprendimento** che costringono la rete a regolarizzarsi.

1.10.2 Dropout (slide 54)

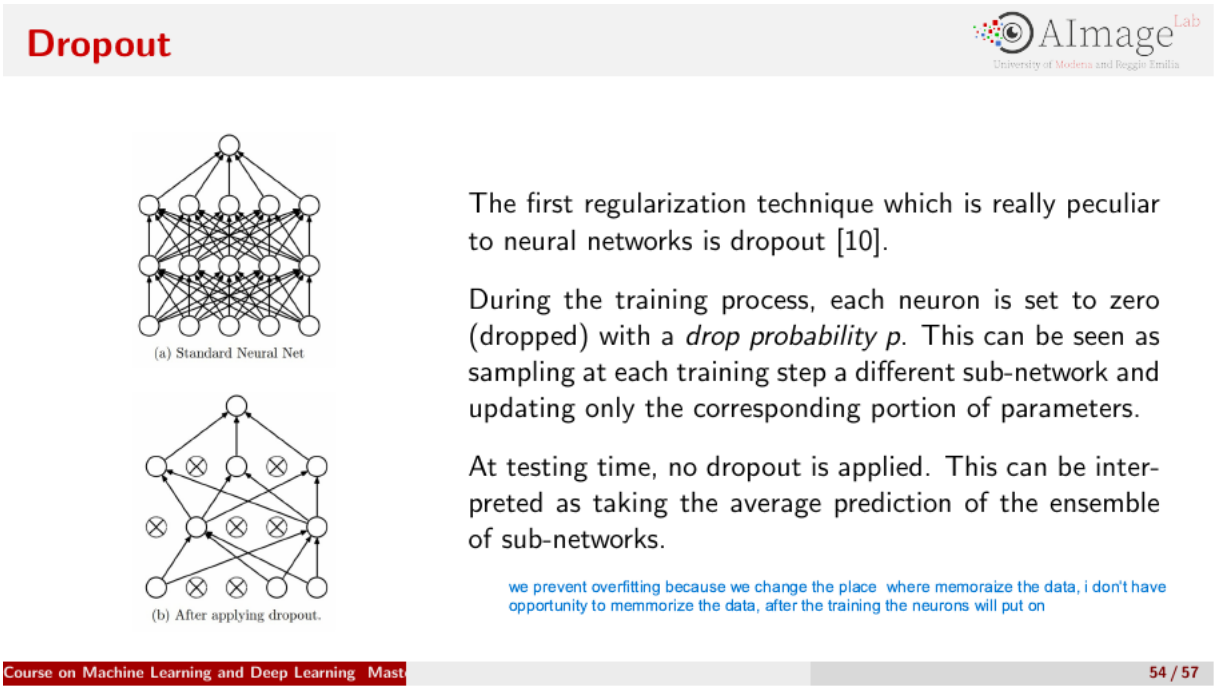


Figura 1.22: Slide 54 – Dropout, con annotazioni

Il **dropout** è la tecnica implicita più popolare. Ha come iperparametro una **probabilità di drop** p **per ogni strato**, tipicamente tra il 10% e il 50%.

Come funziona in training:

1. per ogni neurone dello strato, prima del forward, si "lancia una moneta" che con probabilità p dice di spegnerlo;
2. il neurone spento ha uscita **0**, sia nel forward sia nel backward;
3. a ogni passo il neurone spento è diverso.

Formalmente l'uscita dello strato diventa $\tilde{h} = r \odot h$, dove ogni r_k vale 0 con probabilità p e 1 con probabilità $1-p$. Nel backward $\partial \tilde{h}_k / \partial h_k = r_k$: un neurone spento non riceve gradiente e il suo contributo al gradiente dei pesi in ingresso è nullo in quel passo. Regolarizzazione o memoria dell'ottimizzatore possono comunque modificarli. La maschera campionata nel forward viene mantenuta nel relativo backward.

Il risultato è che si tagliano **a caso** alcuni percorsi dentro la rete: la rete non può più usare quei neuroni e deve trovare altre strade. Equivale a **campionare a ogni passo una sotto-rete diversa** e ad aggiornare solo la sua porzione di parametri.

Perché può ridurre l'overfitting. Il dropout rende meno affidabile l'uso congiunto di specifiche unità e incoraggia rappresentazioni utili anche quando altre unità mancano. L'immagine dei posti di memoria che cambiano è un'analogia: non dimostra che la memorizzazione diventi impossibile.

In test. Nessun dropout: dopo il training tutti i neuroni sono attivi. Lo si può interpretare come un'approssimazione della **predizione di un ensemble** di sotto-reti; per una rete non lineare il riscalamento non è generalmente la loro media esatta.

Precisazione: il riscalamento

In training ogni neurone è attivo solo con probabilità $1-p$, quindi in media la sua uscita vale $(1-p)\,h_k$. In test, con tutti i neuroni accesi, varrebbe h_k. Per rendere coerenti le due fasi: - si moltiplicano le uscite per $(1-p)$ in test; - oppure, come fa PyTorch (*inverted dropout*), si dividono per $(1-p)$ in training.

Perché spegnere neuroni a caso, in modo uniforme.

- In training non si sa a priori quali neuroni stiano imparando cosa, e l'addestramento è dinamico: un neurone può imparare una cosa all'inizio e un'altra dopo.
- Si campiona quindi in modo **uniforme**. Spegnere un neurone che è già spento non cambia nulla: il dropout ha effetto solo sui neuroni effettivamente attivi.

Una probabilità per strato. p è la stessa per tutti i neuroni di uno strato, ma di solito cambia da strato a strato, in base al numero di neuroni e alla posizione nella rete.

La probabilità può variare fra strati. Eliminare il 90% delle unità di uno strato molto stretto può lasciare troppo poca informazione; non esiste una regola generale che vieti la regolarizzazione vicino all'uscita.

Le probabilità di dropout sono iperparametri da valutare sul validation set. La sensibilità alla loro scelta dipende dal modello e dai dati: non è garantito che le differenze rimangano entro pochi punti percentuali.

1.10.3 Data augmentation (slide 55)

Il modo migliore per generalizzare meglio sarebbe **allenarsi su più dati**, che però non sempre sono disponibili. La **data augmentation** genera **nuovi esempi di training applicando trasformazioni agli input** del training set, trasformazioni che devono **preservare l'etichetta** dell'esempio originale.

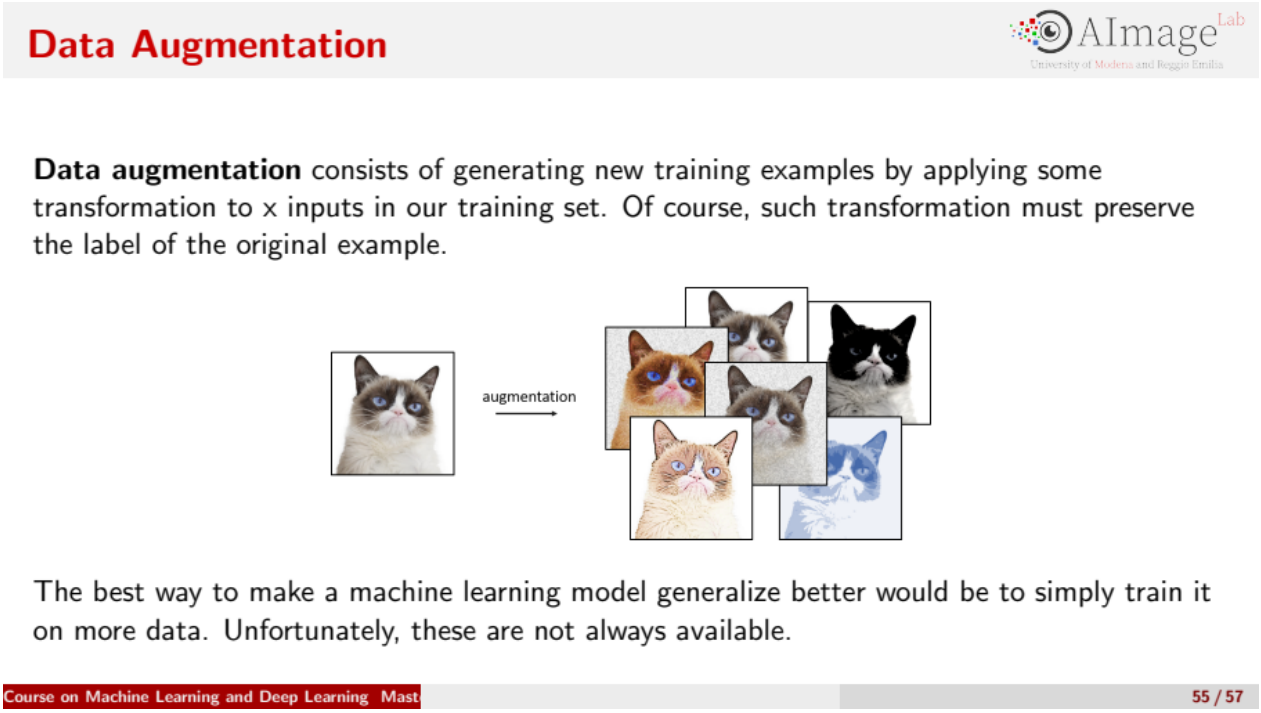


Figura 1.23: Slide 55 – Data augmentation

Per un'immagine si applicano trasformazioni **non distruttive**:

- una leggera rotazione;
- un cambio di scala;
- un ribaltamento (*flip*);
- un cambio di colori, contrasto o luminosità;
- la conversione in bianco e nero.

Si applicano solo le trasformazioni che mantengono valida l'etichetta per il compito scelto.

Due vantaggi contro l'overfitting:

1. **Più dati senza nuove annotazioni.** Con 10 trasformazioni per esempio il dataset diventa 10 volte più grande, e più il dataset è grande, più è difficile memorizzarlo.
2. **Memorizzare diventa inefficiente.** Si creano varianti dello stesso input che devono dare **la stessa uscita**. Memorizzando, la rete dovrebbe imparare a parte ogni variante: un lavoro moltiplicato per il numero di augmentation. Le conviene invece imparare la **regola** che porta tutte le varianti alla stessa risposta. In un certo senso si "inganna" la rete, presentandole un problema in cui generalizzare costa meno che memorizzare.

La data augmentation aumenta la varietà delle osservazioni disponibili e può limitare la memorizzazione. Ogni trasformazione deve conservare il significato del target: una rotazione adatta a una categoria di immagini può cambiare l'etichetta in un altro compito.

Non sempre è facile. Per le immagini esiste un ampio insieme di trasformazioni non distruttive; per altri dati no.

- **Testo.** Una frase non si può ribaltare, ricolorare o ruotare. Servono trasformazioni lessicali, come i sinonimi, che sono più complicate.
- **Serie temporali** (temperatura, pressione...). Ribaltarle inverte il trend e distrugge l'informazione.

1.10.4 Early stopping (slide 56)

Prolungare il training può peggiorare la generalizzazione: l'early stopping usa la validazione per riconoscere quando ulteriori aggiornamenti non migliorano il risultato.

Come si riconosce. Dopo ogni **epoca** (un passaggio completo sul dataset) si calcolano due loss:

- quella sul **training set**, per vedere se gli aggiornamenti la fanno scendere;
- quella su un piccolo **validation set**, non usato per l'addestramento.

Se la loss di training continua a scendere mentre quella di validazione **inizia a salire**, la rete sta entrando nella fase di **memorizzazione**: sui dati nuovi comincerà a funzionare peggio.

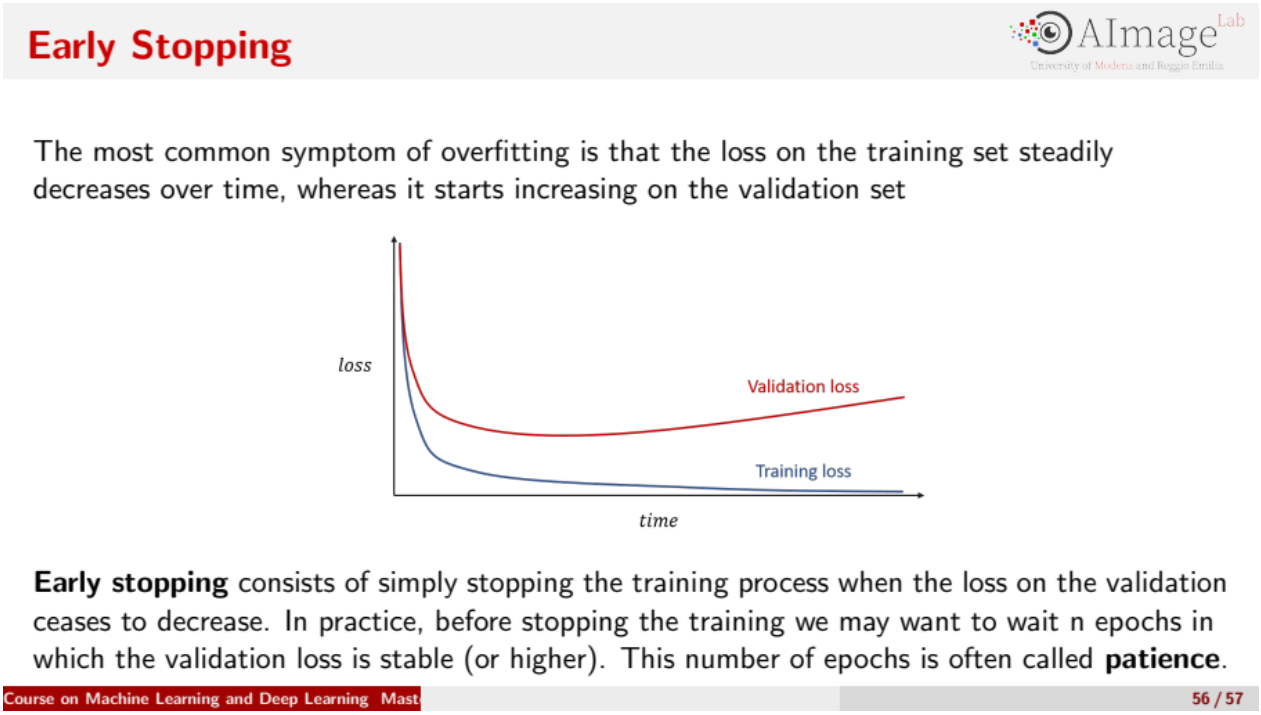


Figura 1.24: Slide 56 – Training vs validation loss

Early stopping: si interrompe l'addestramento secondo il criterio di pazienza e si recupera il checkpoint con la migliore metrica di validazione, che può precedere l'ultima epoca. Il test set resta escluso da questa scelta.

La patience. Non si decide in base a una sola misura, che potrebbe essere una fluttuazione. Si fissa un numero n di valutazioni consecutive in cui la loss di validazione resta stabile o peggiora prima di fermarsi. Questo numero si chiama **patience**.

1.11 Riepilogo

1. Il deep learning funziona oggi grazie a **dati + GPU** e perché **impara le feature** invece di fissarle. Il prezzo è la loro **non interpretabilità**.
2. **Classificatore lineare** $Wx+b$ + **softmax** + **cross-entropy** + discesa del gradiente = **regressione logistica**. La softmax definisce probabilità del modello, senza garantirne la calibrazione.
3. **Neurone artificiale** = mappa affine seguita da un'attivazione; il caso sigmoide corrisponde al modello logistico binario.
   - La sigmoide satura ($\sigma' = \sigma(1-\sigma) \leq 0.25$) e non è centrata.
   - La tanh ($= 2\sigma(2x) - 1$) è centrata ma satura.
   - La **ReLU** non satura per $x > 0$ ed è economica.
4. **Senza non linearità** qualunque rete collassa in un singolo strato lineare ($W = W_2 W_1$).
5. **MLP**: strati fully connected, $W \in \mathbb{R}^{\text{out}\times\text{in}}$, e **ogni neurone ha un bias** (6-4-3-1 $\rightarrow$ 47 parametri). La **forward propagation** è una composizione di funzioni: la rete è un'equazione esplicita.
6. **Approssimazione universale**: in teoria basta un hidden layer per qualunque funzione continua; in pratica, per la discretizzazione, reti più grandi e profonde sono più espressive. Rappresentare non significa saper imparare.
7. **Funzione obiettivo** $= \frac{1}{N}\sum_i L_i + \lambda\Omega(\theta)$.

- **Loss:** hinge (margine 1, assorbibile nella scala dei pesi), softmax/cross-entropy (gradiente = softmax − one-hot), MSE.
- **Regolarizzatori:** L_2 (pesi piccoli, *weight decay*) e L_1 (pesi nulli).

8. **Backpropagation**:
   - regola della catena applicata **dall'uscita verso l'ingresso**;
   - ogni strato calcola solo $\partial h_{i+1}/\partial W_{i+1}$ ("preleva l'ingresso") e $\partial h_{i+1}/\partial h_i$ ("preleva il peso");
   - il gradiente rispetto alle attivazioni viene **riusato** strato dopo strato;
   - funziona per qualunque strato di cui si sappiano scrivere le due derivate ed è invariata dal 1986.
9. **Inizializzazione**: pesi piccoli, a media zero e **diversi** (rottura della simmetria), scalati con il fan-in; bias a zero.
10. **Regolarizzazione implicita**:
    - **dropout**: spegne neuroni a caso in training, mai in test;
    - **data augmentation**: varianti con la stessa etichetta;
    - **early stopping**: si ferma quando la loss di validazione sale, con una *patience*.

Riepilogo errata delle slide

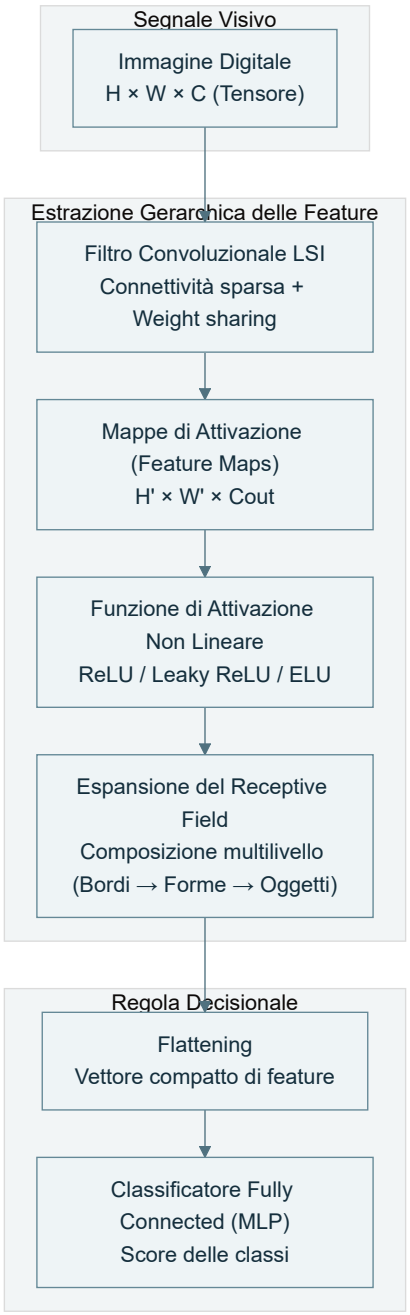
- **Slide 15**: $s_{ship} = 60.75$, non 61.95 (manca il bias).
- **Slide 18**: denominatore della softmax $\sum_k e^{z_k}$.
- **Slide 19**: manca il segno meno nella cross-entropy.
- **Slide 25**: $\tanh(x) = 2\sigma(2x) - 1$.
- **Slide 30**: $W_2 \in \mathbb{R}^{3\times4}$, $W_3 \in \mathbb{R}^{1\times3}$.

Argomento 2

CNN - Reti neurali convoluzionali

Contenuti dell'argomento

- Rappresentazione Digitale delle Immagini nel Calcolatore (slide 3–6)
- Paradigmi di Computer Vision ed Evoluzione Storica (slide 7–11)
- Limiti del Multilayer Perceptron per Dati Visivi (slide 12–18)
- Principi Architetturali e Ispirazione Biologica (slide 19–23)
- Operatori su Immagini e Sistemi Lineari Traslazione-Invarianti (LSI) (slide 24–34)
- Il Nucleo della Convoluzione nel Deep Learning (slide 35–39)
- Convoluzione Multicanale e Feature Maps (slide 40–42)
- Iperparametri del Livello Convoluzionale: Kernel, Stride e Padding (slide 43–46)
- Calcolo dei Parametri Apprendibili (slide 47)
- Visualizzazione dei Filtri e Gerarchia delle Rappresentazioni (slide 48–51)
- Funzioni di Attivazione e Superamento della ReLU Standard (slide 52–54)
- Livelli di pooling: aggregazione locale e riduzione spaziale (slide 55–58)
- Dalle feature al classificatore: pooling adattivo (slide 59)
- LeNet e AlexNet: evoluzione delle CNN (slide 60–62)
- VGG: distribuire la capacità lungo la profondità (slide 63–68)
- Profondità e residual learning: ResNet (slide 69–75)
- Batch Normalization: statistiche e trasformazione appresa (slide 76–86)
- Riepilogo Formule Operative
- Errata e precisazioni principali



2.1 Rappresentazione Digitale delle Immagini nel Calcolatore (slide 3–6)

Un'immagine reale è la manifestazione visibile di un fenomeno fisico continuo: la luce riflessa dagli elementi di una scena tridimensionale attraversa il sistema ottico di una fotocamera e incide su una superficie fotosensibile bidimensionale. Il calcolatore, tuttavia, è un dispositivo discreto sia nello spazio sia nei valori numerici che può memorizzare. Il passaggio da un segnale ottico continuo alla sua rappresentazione binaria interna esige pertanto due trasformazioni fondamentali, formalizzate nella Slide 4:

1. **Campionamento spaziale (Sampling)**: la scena continua viene discretizzata spazialmente proiettandola su una griglia bidimensionale uniforme composta da $H \times W$ campioni discreti chiamati **pixel** (*picture elements*), dove H denota l'altezza (numero di righe) e W la larghezza (numero di colonne).
2. **Quantizzazione (Quantization)**: l'intensità luminosa continua misurata da ciascun fotosensore in corrispondenza del singolo pixel viene approssimata e mappata su un insieme discreto di 2^m livelli numerici, corrispondenti a una codifica su m bit per campione.

Nella quasi totalità delle applicazioni digitali standard si adotta una profondità di quantizzazione pari a $m = 8$ bit per canale. Lo spazio dei valori discreti associabili a ogni campione comprende dunque $2^8 = 256$ intensità distinte, convenzionalmente indicizzate nell'intervallo chiuso:

$$\mathcal{I} \in \{0, 1, 2, \ldots, 255\}$$

dove il valore 0 corrisponde all'assenza totale di segnale luminoso (nero assoluto) e il valore 255 corrisponde alla massima intensità di saturazione del sensore (bianco assoluto).

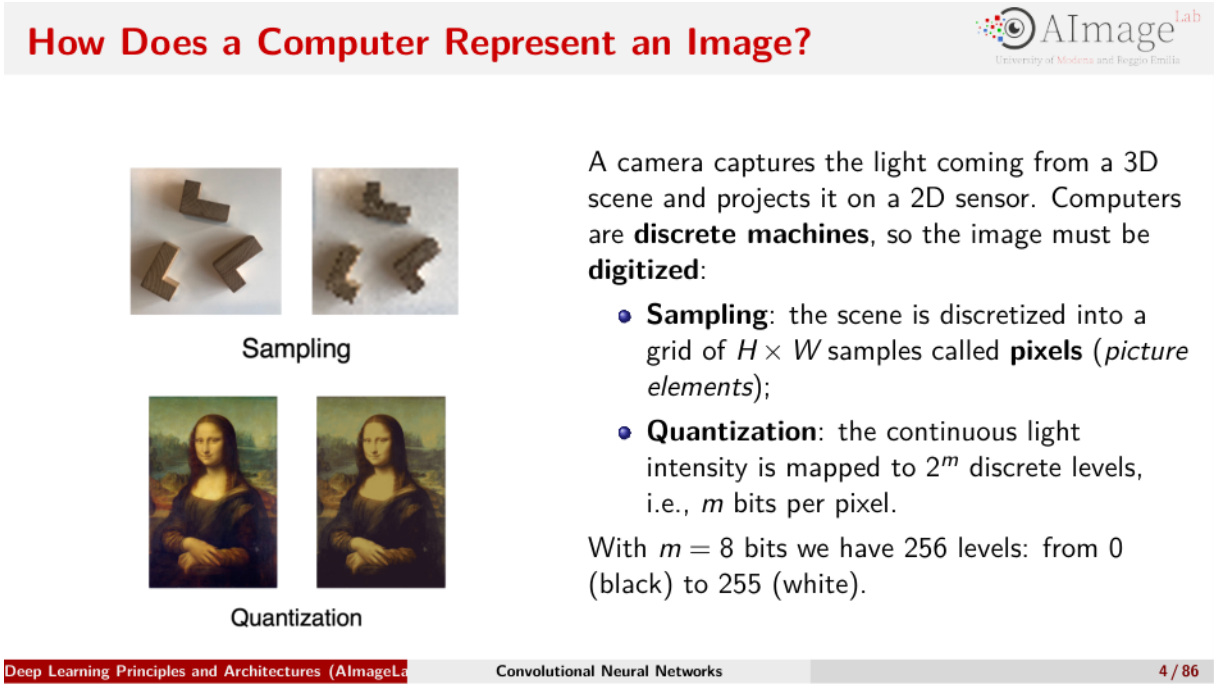


Figura 2.1: Slide 4 – How Does a Computer Represent an Image?

2.1.1 Immagini in Scala di Grigi (Grayscale)

Nel caso di un'immagine monocromatica (in scala di grigi), ciascun pixel memorizza unicamente l'intensità scalare della radianza luminosa. Un'immagine in scala di grigi di risoluzione $H \times W$ è a tutti gli effetti una matrice numerica:

$$\boxed{I \in \{0, 1, \ldots, 255\}^{H \times W}}$$

in cui l'elemento $I(i, j)$ rappresenta il livello di grigio del pixel posizionato alla riga i (con $1 \leq i \leq H$) e alla colonna j (con $1 \leq j \leq W$). La Slide 5 illustra questo concetto mostrando una porzione rettangolare (patch) di 6×8 pixel estratta da una fotografia di un gatto: ciò che l'occhio umano

percepisce come contorni morbidi e sfumature di pelo corrisponde, per la memoria del calcolatore, a una pura tabella bidimensionale di interi compresi tra 0 e 255.

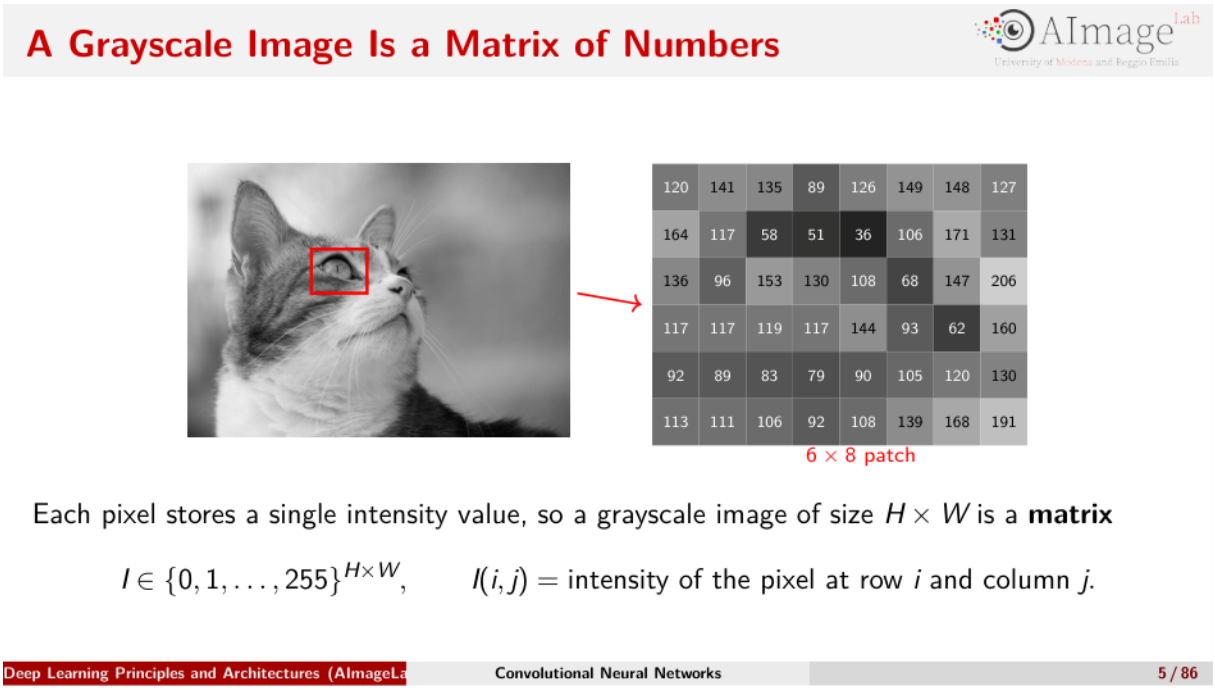


Figura 2.2: Slide 5 – A Grayscale Image Is a Matrix of Numbers

2.1.2 Immagini a Colori: Canali e Spazio RGB

La percezione cromatica umana è tricromatica: la retina dispone di tre tipologie distinte di coni fotosensibili, sintonizzati su differenti lunghezze d'onda dello spettro visibile. Nel calcolatore, la sintesi del colore viene implementata attraverso lo spazio **RGB (Red, Green, Blue)**, basato sul principio della sintesi additiva della luce.

A differenza della sintesi sottrattiva tipica delle miscele di pigmenti materiali (dove i colori primari tradizionali insegnati nella scuola dell'infanzia sono rosso, giallo e blu), i dispositivi digitali generano immagini emettendo radiazione luminosa direttamente da pixel composti da fosfori o LED colorati. La sovrapposizione delle tre lunghezze d'onda primarie alla massima intensità genera il bianco puro:

$$(R, G, B) = (255, 255, 255)$$

mentre l'azzeramento di tutte le sorgenti produce il nero:

$$(R, G, B) = (0, 0, 0)$$

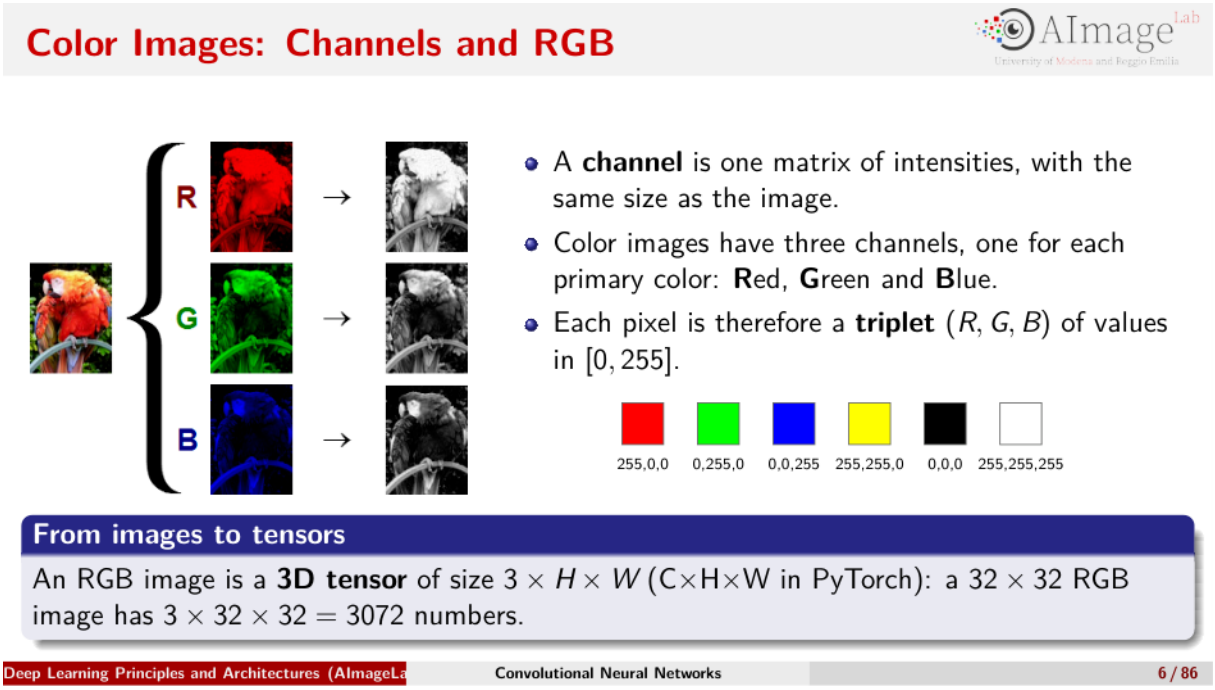


Figura 2.3: Slide 6 – Color Images: Channels and RGB

Un'immagine a colori non può essere descritta da una singola matrice bidimensionale: essa si articola in **tre matrici spaziali parallele sovrapposte**, denominate **canali** (*channels*), ciascuna di dimensioni $H \times W$, che registrano individualmente il contributo di intensità del rispettivo spettro cromatico nell'intervallo $[0, 255]$. Ciascun pixel è identificato da una terna ordinata (R, G, B).

Nel framework formale del Deep Learning e in librerie come PyTorch, un'immagine RGB viene rappresentata mediante un **tensore tridimensionale** di dimensioni:

$$\boxed{X \in \mathbb{R}^{C \times H \times W}}$$

dove $C = 3$ indica il numero di canali, H l'altezza e W la larghezza (notazione *channels-first*, standard per PyTorch, a differenza della convenzione *channels-last* $H \times W \times C$ utilizzata storicamente da altre librerie). Ad esempio, una minuscola immagine RGB di dimensioni 32×32 (come quelle del benchmark CIFAR-10) è formata da:

$$3 \times 32 \times 32 = 3072 \text{ valori scalari}$$

2.2 Paradigmi di Computer Vision ed Evoluzione Storica (slide 7–11)

La disciplina della **Computer Vision** (**CV**) si colloca all'intersezione tra l'analisi del segnale e l'intelligenza artificiale, ponendosi l'obiettivo formale di estrarre conoscenza e descrizioni semantiche esplicite a partire da flussi visivi ("elaborare immagini per ottenere dati comprensibili e decisioni operative").

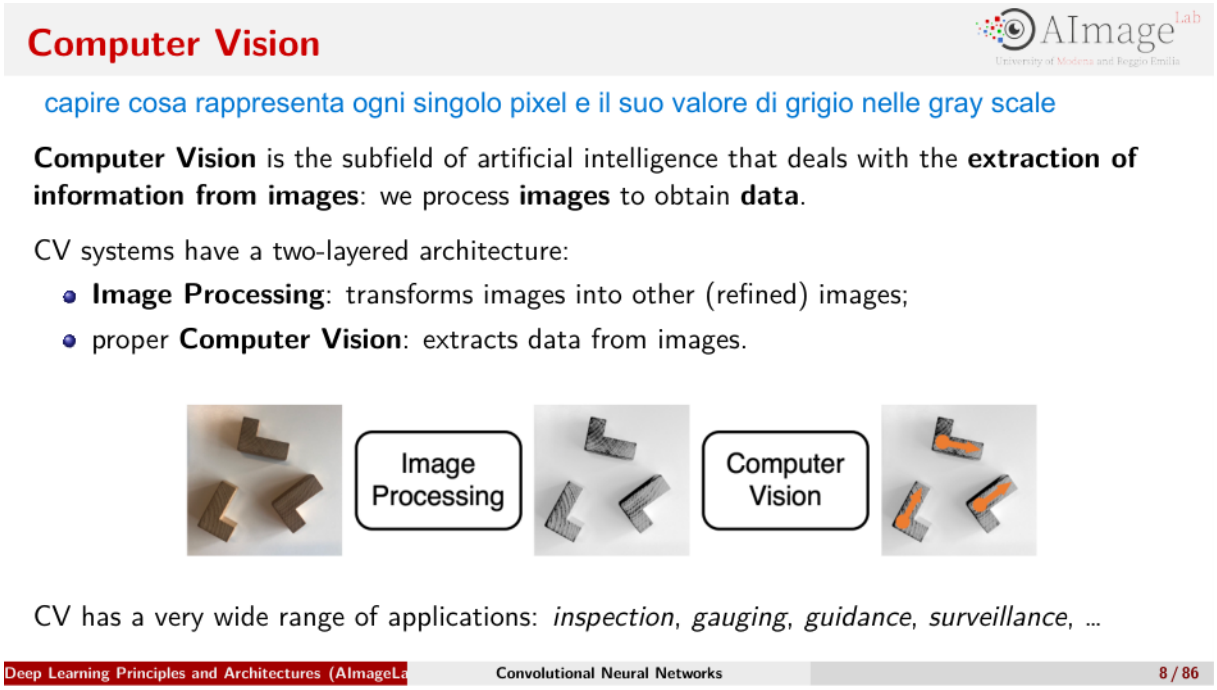


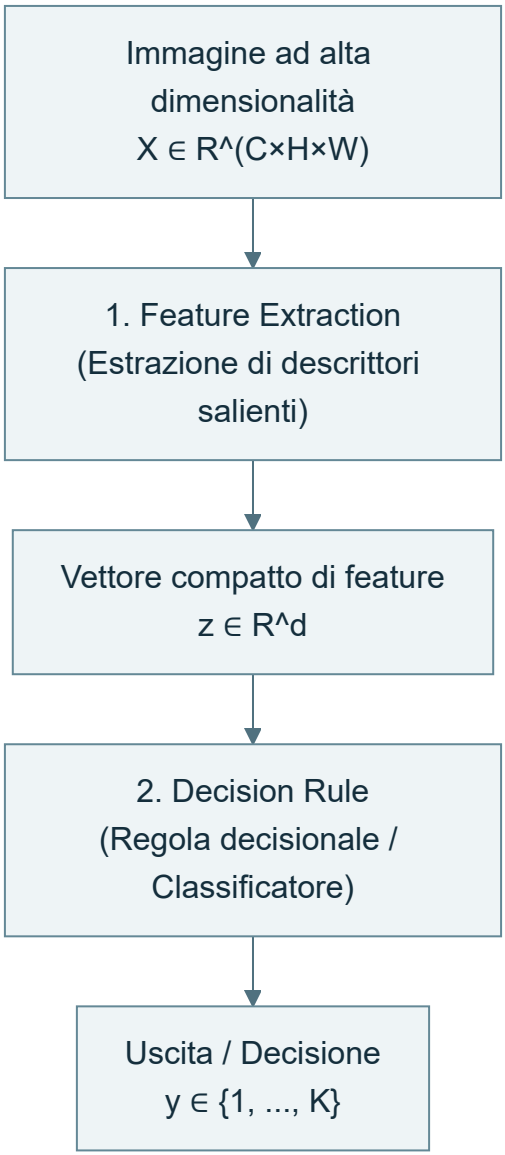
Figura 2.4: Slide 8 – Computer Vision

Nella tassonomia accademica dei sistemi visivi si distingue rigorosamente tra due livelli gerarchici di elaborazione:

- **Image Processing** (**Elaborazione delle immagini**): trasforma immagini di ingresso in altre immagini modificate o raffinate (ad esempio aumentando il contrasto, sopprimendo il rumore di acquisizione o correggendo aberrazioni cromatiche). Opera sui pixel trattandoli come entità numeriche astratte, senza comprendere in alcun modo il significato semantico degli oggetti rappresentati o la geometria tridimensionale della scena.
- **Computer Vision propriamente detta**: opera a un livello di astrazione semantica superiore. Il suo fine non è alterare i pixel, bensì interpretare cosa quei valori numerici rappresentino nel mondo reale (compiti di riconoscimento, misurazione volumetrica, guida di veicoli autonomi, videosorveglianza e controllo qualità industriale).

2.2.1 La Scomposizione Classica in Due Stadi (slide 9)

Poiché lo spazio delle configurazioni di pixel grezzi presenta una dimensionalità astronomica, la visione artificiale classica ha storicamente frammentato il problema in due stadi sequenziali disaccoppiati:



1. **Feature Extraction** (**Estrazione delle caratteristiche**): riduce la dimensionalità grezza dell'immagine calcolando una rappresentazione compatta ed espressiva (il vettore di *feature*), progettata per trattenere le informazioni salienti per lo specifico compito (es. profili dei contorni, orientamento dei gradienti, consistenza della trama) scartando le variazioni irrilevanti (come cambi di luminosità globale).
2. **Decision Rule** (**Regola decisionale**): prende in ingresso il vettore di feature compatto e formula il verdetto finale (assegnazione della classe, regressione delle coordinate o stima di parametri).

2.2.2 Le Tre Ere Paradigmatiche della Visione Artificiale (slide 10–11)

L'evoluzione della Computer Vision dagli anni '70 a oggi è scandita da tre paradigmi metodologici:

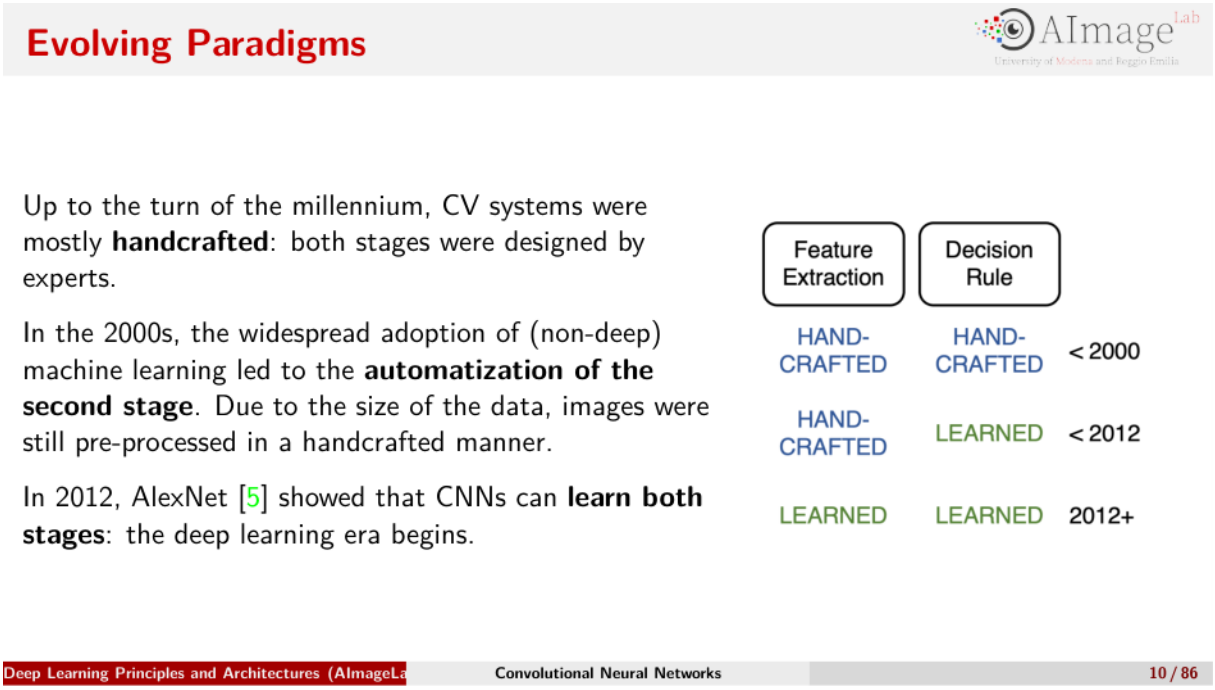


Figura 2.5: Slide 10 – Evolving Paradigms

Periodo Storico	Feature Extraction	Decision Rule	Caratteristiche del Modello
Fino al 2000	Progettata a mano (*Handcrafted*)	Progettata a mano (*Handcrafted*)	Regole euristiche basate su soglie fisse, allineamento di maschere rigide e modelli geometrici analitici formulati dall'esperto umano.
Anni 2000 – 2012	Progettata a mano (*Handcrafted*)	Appresa statisticamente (*Learned*)	Descrittori ingegnerizzati complessi (es. SIFT, HOG, SURF) combinati con classificatori statistici di Machine Learning (SVM, Random Forest, AdaBoost).
2012 – Oggi	**Appresa congiuntamente dai dati**	**Appresa congiuntamente dai dati**	**Deep Learning End-to-End**: reti neurali convoluzionali profonde (spartiacque storico: **AlexNet**, 2012). Estrazione delle feature e classificazione sono unificate e ottimizzate simultaneamente mediante differenziazione automatica.

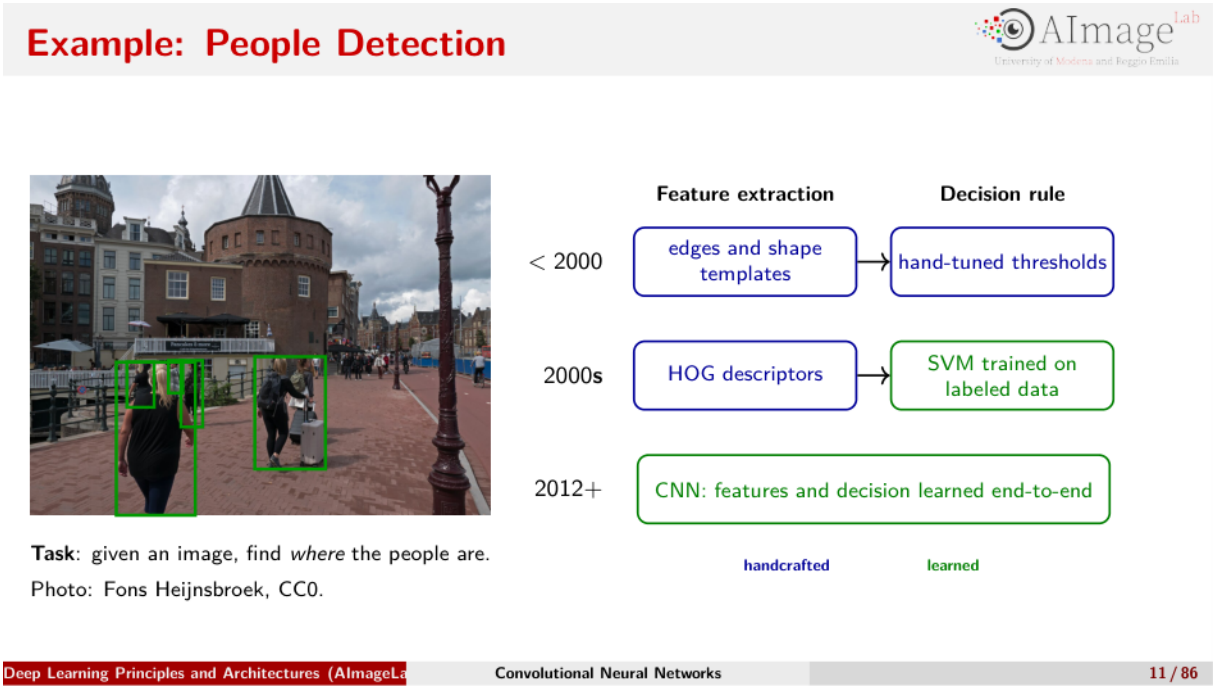


Figura 2.6: Slide 11 – Example: People Detection

L'esempio emblematico mostrato nella Slide 11 riguarda il compito di **rilevamento di pedoni** (**People Detection**): * **Prima del 2000**: si disegnavano a mano delle sagome geometriche standard (*template* stilizzati del corpo umano) e le si faceva scorrere sull'immagine, verificando mediante correlazione visiva se la sovrapposizione superava una certa soglia arbitraria. * **Negli anni 2000**: si calcolavano i descrittori **HOG** (**Histogram of Oriented Gradients**) per catturare le direzioni dominanti dei gradienti lungo la silhouette della persona, e si addestrava un classificatore Support Vector Machine (SVM) per discriminare tra la classe pedone e lo sfondo. * **Con l'avvento di AlexNet nel 2012**: si alimenta l'immagine direttamente a una rete convoluzionale profonda; il modello apprende dai dati grezzi sia i filtri per riconoscere la sagoma sia i pesi per la classificazione finale, superando nettamente ogni prestazione manuale precedente.

2.3 Limiti del Multilayer Perceptron per Dati Visivi (slide 12–18)

Le reti neurali convoluzionali non ripudiano la matematica del Multilayer Perceptron (Multilayer Perceptron), bensì ne condividono l'impalcatura teorica fondamentale (Slide 13):

- Sono costituite da strati di neuroni artificiali dotati di pesi e bias apprendibili.
- Ciascun neurone calcola una trasformazione lineare (prodotto scalare tra pesi e input) seguita da una funzione di attivazione non lineare.
- L'intera architettura definisce una **funzione continua e differenziabile a tratti**, addestrabile end-to-end tramite l'algoritmo di retropropagazione del gradiente (**Backpropagation**).

Tuttavia, l'applicazione diretta di un MLP alle immagini svela immediatamente limiti rilevanti, sia di natura computazionale sia di compatibilità geometrica.

What If the Inputs Are Images?

Let us imagine working with **small RGB images** of size 32×32, and trying to fit an **MLP** on them.

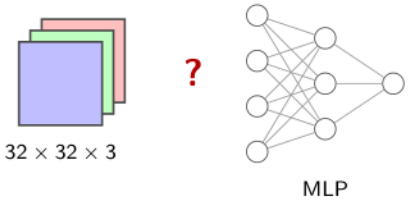
Question 1
How do we represent the image so that we can feed it to an MLP?

What If the Inputs Are Images?

Let us imagine working with **small RGB images** of size 32×32, and trying to fit an **MLP** on them.

Question 1
How do we represent the image so that we can feed it to an MLP?

Question 2
How many parameters does the MLP need?



2.3.1 Esplosione Parametrica e Inefficienza Computazionale (slide 14–16)

Nelle slide 14 e 15 vengono poste due domande fondamentali:

> **Domanda 1:** Come rappresentiamo un'immagine in modo da poterla fornire in ingresso a un MLP?
> **Domanda 2:** Di quanti parametri ha bisogno l'MLP per elaborarla?

La risposta alla Domanda 1 impone l'operazione di **appiattimento** (**Flattening**): poiché i neuroni di uno strato denso (Fully Connected) accettano unicamente ingressi monodimensionali, la matrice o il tensore dell'immagine deve essere "srotolato" in un unico vettore colonna $x \in \mathbb{R}^D$. Per una matrice RGB 32×32:

$$x \in \mathbb{R}^{3 \cdot 32 \cdot 32} = \mathbb{R}^{3072}$$

La risposta alla Domanda 2 svela la drammatica esplosione del numero di parametri. Consideriamo un MLP dotato di un singolo strato nascosto estremamente modesto, composto da soli $M = 512$ neuroni:

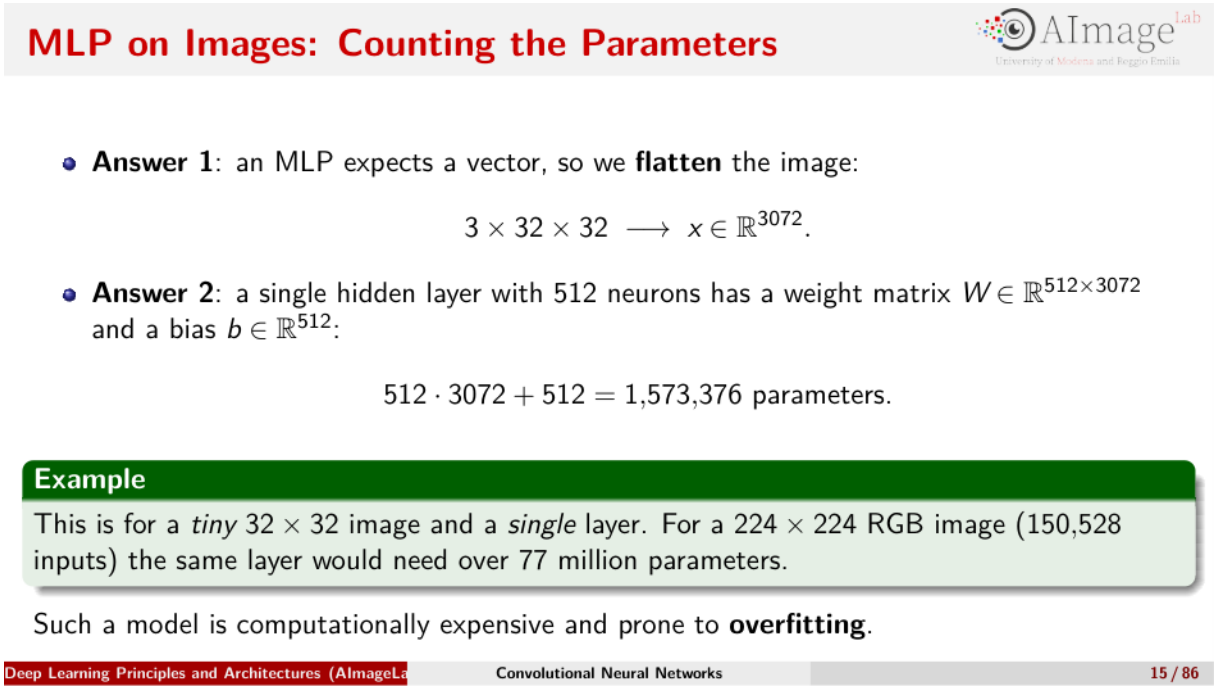


Figura 2.7: Slide 16 – MLP on Images: Counting the Parameters

La matrice dei pesi dello strato è $W \in \mathbb{R}^{512 \times 3072}$ e il vettore dei bias è $b \in \mathbb{R}^{512}$. Il conteggio esatto dei parametri liberi risulta:

$$\text{Parametri} = (512 \times 3072) + 512 = 1.572.864 + 512 = \mathbf{1.573.376}$$

Oltre un milione e mezzo di parametri per un'immagine giocattolo di appena 32×32 pixel! Se consideriamo una risoluzione comune per la visione artificiale moderna (come lo standard ImageNet da 224×224 pixel RGB), il vettore di ingresso contiene:

$$D = 3 \times 224 \times 224 = 150.528 \text{ componenti}$$

Collegando questo ingresso agli stessi 512 neuroni nascosti, il numero di parametri lievita a:

$$\text{Parametri} = (512 \times 150.528) + 512 = 77.070.336 + 512 = \mathbf{77.070.848}$$

Uno strato singolo richiede oltre **77 milioni di pesi**. Per un'immagine ad alta definizione (1000×1000 pixel) e uno strato nascosto proporzionato, si supererebbe ampiamente il miliardo di parametri per un solo livello. Questa mole di pesi ha un costo elevato di calcolo e memoria e può rendere più difficile generalizzare con pochi dati; il numero dei pesi da solo non dimostra overfitting.

2.3.2 Criticità Strutturali dell'MLP sulle Immagini (slide 17)

Al di là del costo computazionale, l'MLP denso non incorpora esplicitamente la struttura geometrica intrinseca dei dati visivi:

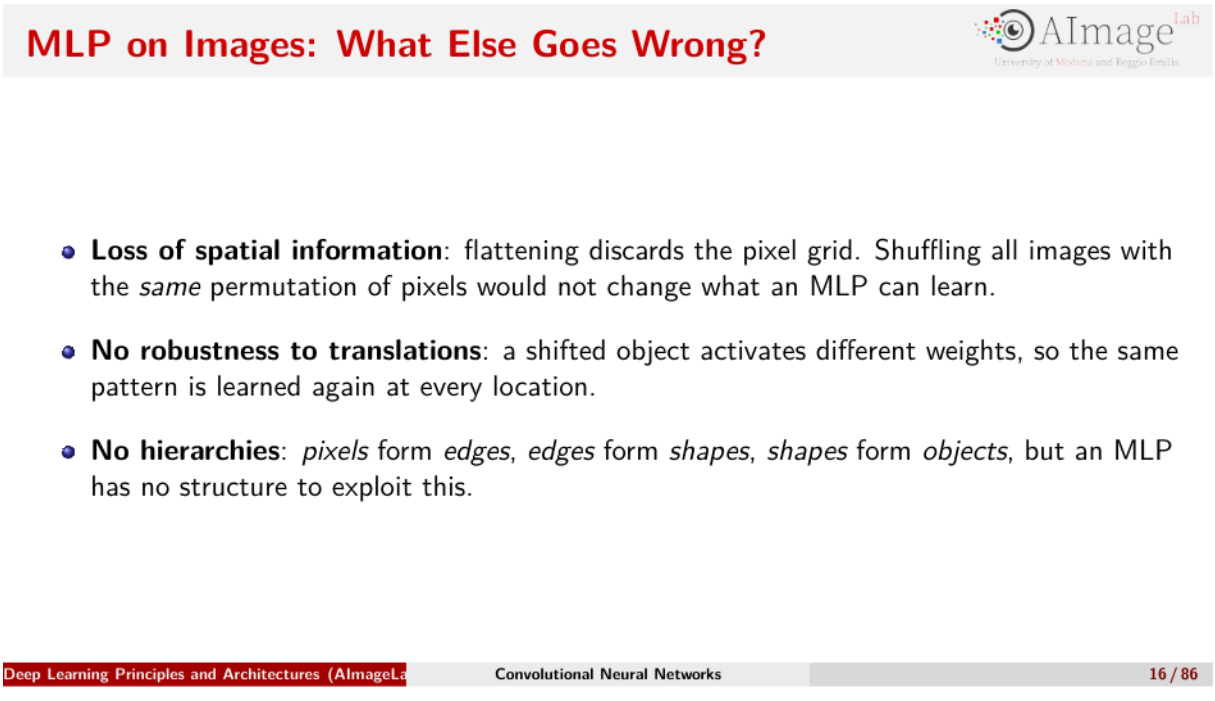


Figura 2.8: Slide 17 – MLP on Images: What Else Goes Wrong?

1. **Perdita della struttura topologica spaziale** (***Loss of spatial information***): Lo srotolamento è invertibile e non elimina i valori dei pixel; è il livello denso a non imporre esplicitamente la geometria della griglia. In un vettore piatto, la distanza tra il pixel $(0, 0)$ e il suo vicino adiacente $(0, 1)$ è trattata con la medesima neutralità della distanza tra il pixel $(0, 0)$ e l'angolo opposto $(31, 31)$. Se applicassimo a tutte le immagini di addestramento una permutazione casuale fissa di pixel, l'immagine risulterebbe incomprensibile per un essere umano, ma la capacità di apprendimento di un MLP rimarrebbe esattamente invariata.
2. **Mancanza di invarianza/equivarianza alla traslazione** (***No translation robustness***): in un MLP, a ogni indice del vettore di ingresso corrisponde un peso distinto. Se un pattern visivo (ad esempio il volto di un gatto) si sposta dall'angolo superiore sinistro a quello inferiore destro dell'immagine, esso attiverà componenti del vettore e pesi completamente diversi. L'MLP non impone la condivisione spaziale dei pesi: apprendere lo stesso pattern in posizioni diverse può richiedere più esempi.
3. **Assenza di una gerarchia locale imposta dall'architettura**: il mondo visivo naturale è strutturato secondo una gerarchia compositiva (i pixel formano contorni, i contorni si combinano in forme geometriche, le forme compongono parti di oggetti e le parti formano l'oggetto intero). La connettività densa e uniforme di un MLP non fornisce alcun vincolo architetturale che guidi la rete verso questa progressiva astrazione.

2.3.3 Le Proprietà Fisiche delle Immagini e l'Inductive Bias (slide 18)

I dati visivi naturali possiedono due regolarità utili in molti compiti:

1. **Proprietà del segnale: Località e Continuità Spaziale** (***Spatial Locality***): la luce riflessa dagli oggetti del mondo fisico varia nello spazio in modo continuo. Di conseguenza, i pixel

spazialmente vicini presentano una **fortissima correlazione statistica**: il valore di intensità di un pixel $I(x, y)$ fornisce un'enorme quantità di informazione sul valore del pixel contiguo $I(x + 1, y)$, mentre pixel a grande distanza hanno correlazione via via decrescente.

2. **Proprietà semantica: Indipendenza del contenuto dalla posizione** (***Translation Invariance***): l'identità semantica di un oggetto è invariante rispetto alla sua collocazione nel fotogramma. Un'automobile rimane un'automobile sia che si trovi al centro, a destra o a sinistra dell'inquadratura.

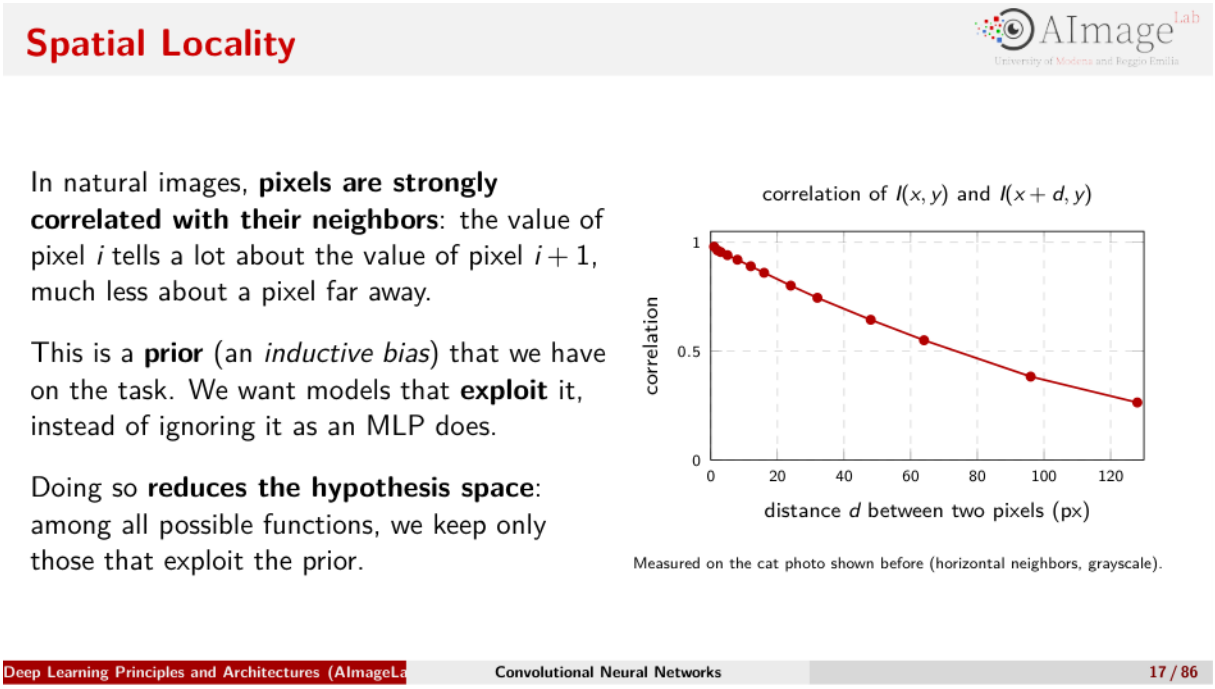


Figura 2.9: Slide 18 – Spatial Locality

Il grafico mostrato nella Slide 18 traccia la correlazione empirica tra $I(x, y)$ e $I(x + d, y)$ in funzione della distanza euclidea d in pixel: per distanze minime ($d \to 0$) la correlazione è quasi unitaria, per poi decadere rapidamente al crescere di d.

L'integrazione di queste due evidenze nella struttura di una rete neurale costituisce un potente **Inductive Bias** (bias induttivo): anziché costringere l'algoritmo a cercare una funzione predittiva nell'intero e sterminato spazio di tutte le funzioni astratte possibili (spazio delle ipotesi dell'MLP), restringiamo a priori l'architettura alle sole funzioni che rispettano la località spaziale e l'invarianza traslazionale. Ciò abbatte la complessità di calcolo e può favorire la generalizzazione quando queste ipotesi sono appropriate.

2.4 Principi Architetturali e Ispirazione Biologica (slide 19–23)

Le CNN riprendono un'intuizione della visione biologica: combinare risposte locali in rappresentazioni via via più complesse. Il campo recettivo descrive la porzione di ingresso che può influenzare un'unità. L'analogia fra elaborazione locale e riconoscimento globale motiva l'architettura, ma non significa che la rete riproduca esattamente il sistema visivo.

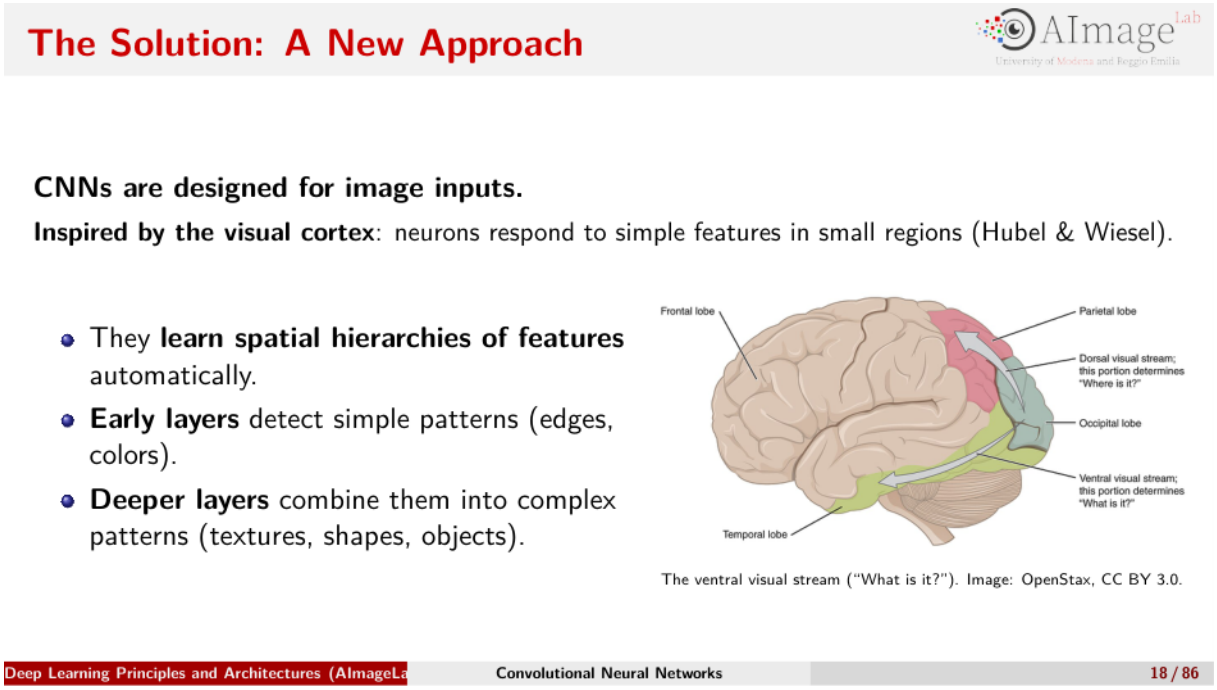


Figura 2.10: Slide 19 – Un approccio locale e gerarchico

2.4.1 I Tre Blocchi Costruttivi della CNN (slide 20–22)

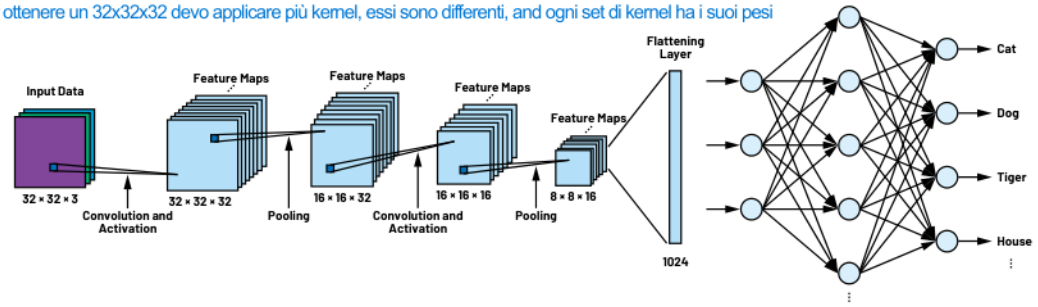
Per emulare artificialmente questo principio di calcolo, una rete convoluzionale standard combina tre blocchi architetturali (Slide 21):

1. **Livelli Convoluzionali** (**Convolutional Layers**): applicano banchi di filtri locali lineari che scorrono sull'immagine per estrarre mappe di feature, sfruttando la connettività sparsa e la condivisione dei parametri.
2. **Funzioni di Attivazione** (**Activation Functions**): introducono non linearità puntuali indispensabili per impedire il collasso algebrico degli strati e consentire l'apprendimento di funzioni complesse.
3. **Livelli di Pooling** (**Pooling Layers**): operano un sottocampionamento spaziale progressivo, riducendo la risoluzione geometrica delle mappe e allargando il campo recettivo per gli strati successivi.

Architectural Overview

A typical CNN architecture is composed of a sequence of layers stacked one on top of the other.

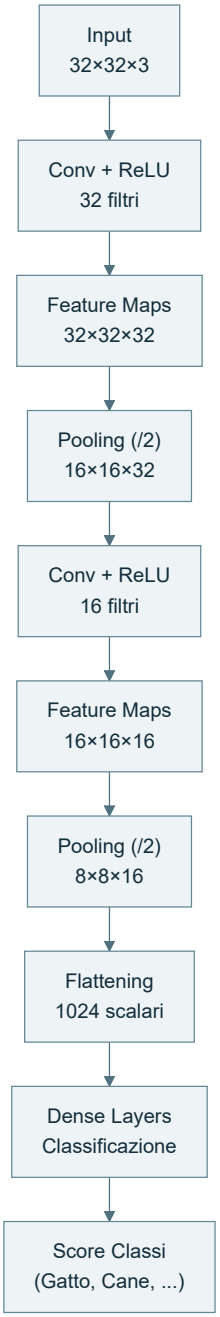
per ottenere un 32x32x32 devo applicare più kernel, essi sono differenti, and ogni set di kernel ha i suoi pesi



Each layer transforms an input 3D volume into an output 3D volume, using a differentiable function. Some layers have learnable parameters (e.g., convolutional layers), while others do not (e.g., pooling layers).

Figura 2.11: Slide 22 – Architectural Overview

Lo schema complessivo di una classica architettura convoluzionale (Slide 22) si struttura secondo la seguente sequenza:



Un tensore di ingresso (es. $32 \times 32 \times 3$) viene elaborato dal primo strato convoluzionale: le dimensioni spaziali rimangono controllate mentre il numero di canali viene espanso (es. a 32 canali). Il livello di pooling dimezza la risoluzione spaziale (portandola a 16×16), mantenendo inalterata la profondità dei canali. Una seconda convoluzione rielabora i canali (portandoli a 16), e un successivo pooling riduce la griglia a $8 \times 8 \times 16$.

Al termine dello stadio estrattivo, il volume compatto risultante viene appiattito in un vettore monodimensionale di feature ad alto livello ($8 \times 8 \times 16 = 1024$ elementi), che viene infine classificato da uno strato denso standard (MLP). **Il sistema è continuo e differenziabile a tratti**: il gradiente della funzione di perdita fluisce a ritroso lungo tutti i livelli, aggiornando simultaneamente sia i pesi del classificatore sia i parametri di ciascun filtro convoluzionale.

2.5 Operatori su Immagini e Sistemi Lineari Traslazione-Invarianti (LSI) (slide 24–34)

Per formalizzare matematicamente il funzionamento del livello convoluzionale occorre comprendere la natura degli operatori bidimensionali del processamento numerico dei segnali visivi (Slide 24):

- **Operatori puntuali (Point operators)**: il valore del pixel di uscita $o(x, y)$ dipende esclusivamente dal valore assunto dal pixel di ingresso nella medesima coordinata $i(x, y)$. Esempi tipici sono la correzione di contrasto/luminosità e la correzione gamma con parametri fissati.
- **Operatori locali o filtri spaziali (Local operators)**: il valore del pixel di uscita $o(x, y)$ viene determinato elaborando congiuntamente un insieme compatto di pixel appartenenti a un intorno geometrico (un vicinato locale) circostante la coordinata (x, y).

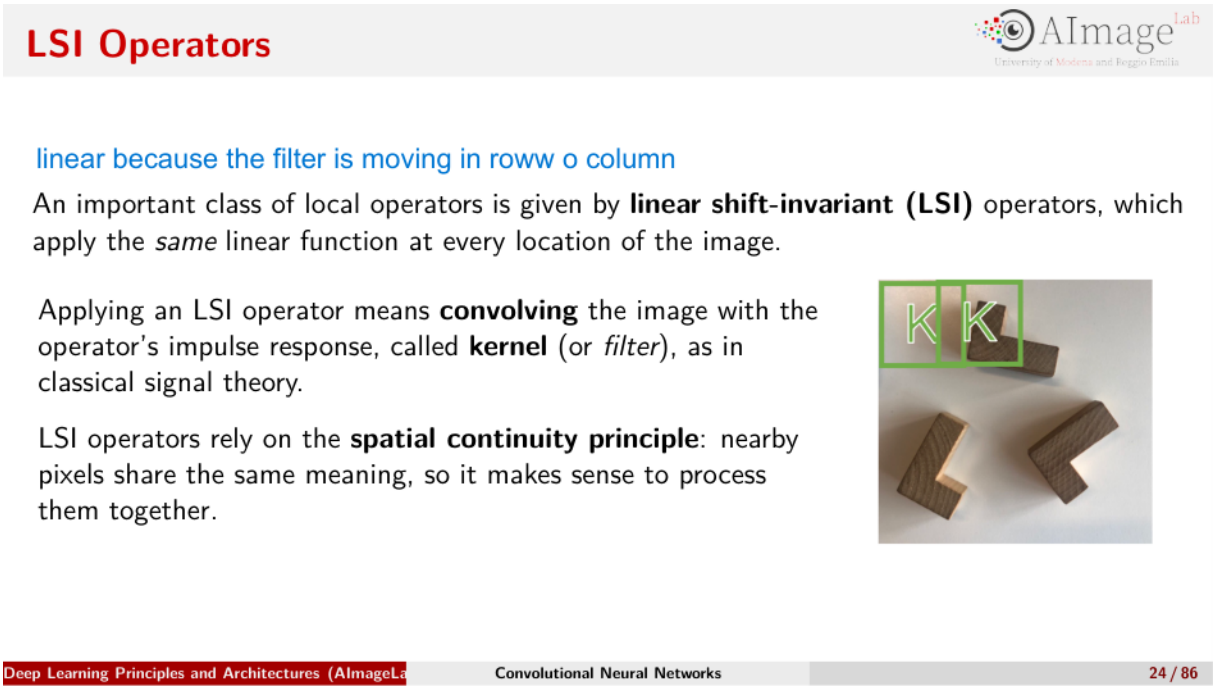


Figura 2.12: Slide 25 – LSI Operators

Una classe fondamentale di operatori locali è costituita dagli **operatori lineari e invarianti alla traslazione** (**LSI - Linear Shift-Invariant operators**, Slide 25). Un operatore locale $\mathcal{T}$ si dice LSI se soddisfa due condizioni: 1. **Linearità**: $\mathcal{T}(aX + bY) = a\mathcal{T}(X) + b\mathcal{T}(Y)$. L'annotazione della slide 25 che collega «linear» al movimento lungo righe e colonne non è corretta: la linearità riguarda la sovrapposizione dei segnali. Un filtro massimo scorre allo stesso modo ma non è lineare. Un bias non nullo rende la trasformazione affine. 2. **Invarianza alla traslazione** (**Shift-Invariance**): applica **la medesima identica funzione** a ogni posizione spaziale dell'immagine, indipendentemente dalla posizione della finestra.

Applicare un operatore LSI a un'immagine equivale algebricamente a eseguire la **convoluzione** tra l'immagine e la risposta impulsiva dell'operatore, denominata **kernel** (o filtro).

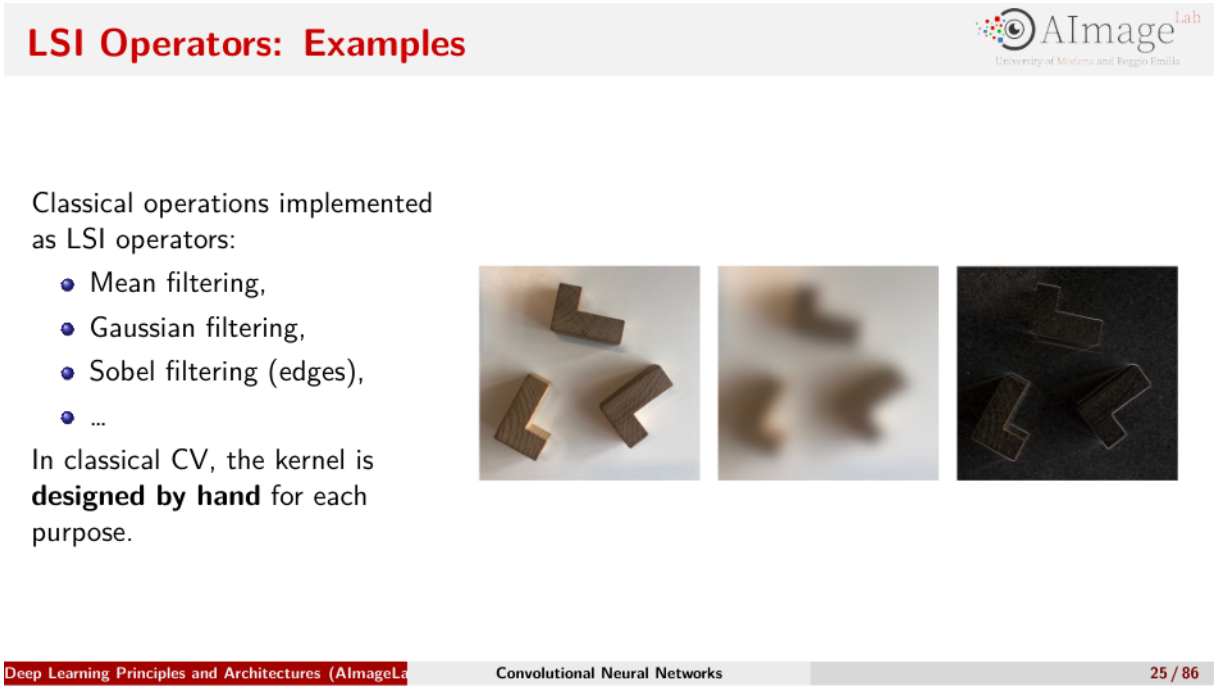


Figura 2.13: Slide 26 – LSI Operators: Examples

Tra gli operatori LSI classici ampiamente impiegati nella Computer Vision pre-deep learning figurano (Slide 26): * **Filtro di media (Mean filtering)**: calcola la media aritmetica non pesata dei pixel nel vicinato, producendo un effetto di sfocatura (*smoothing*). * **Filtro gaussiano (Gaussian filtering)**: assegna pesi decrescenti a campana gaussiana in base alla distanza dal centro, riducendo il rumore ad alta frequenza preservando meglio i contorni. * **Filtri derivativi (es. Sobel, Prewitt)**: calcolano le differenze discrete di intensità tra pixel adiacenti. Poiché la derivata prima spaziale evidenzia le discontinuità repentine di intensità, questi filtri estraggono i contorni (*edges*) degli oggetti.

2.5.1 Esempio Calcolato a Mano: Filtro di Media 3×3 (slide 27–31)

Nelle slide 27–31 viene illustrata l'applicazione di un filtro di media con kernel 3×3 su una porzione di immagine reale in scala di grigi.

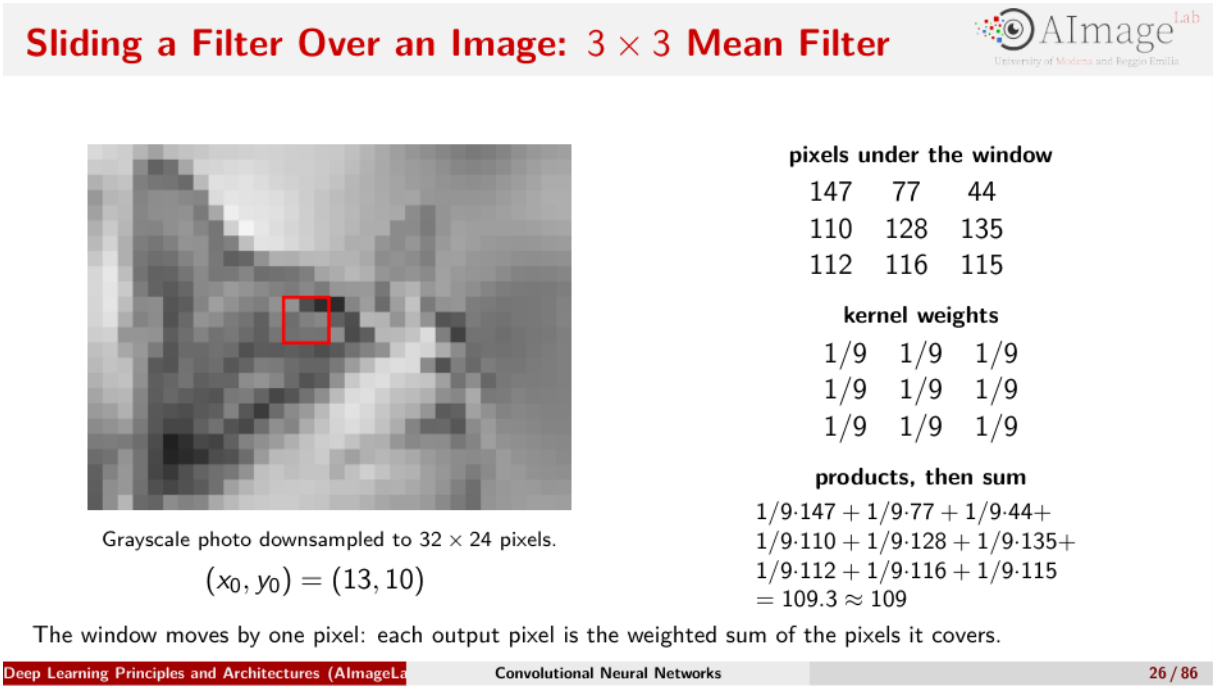


Figura 2.14: Slide 27 – Sliding a Filter Over an Image: 3×3 Mean Filter

Il kernel del filtro di media è una matrice quadrata 3×3 i cui coefficienti sommano a 1:

$$K = \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}$$

Alla coordinata spaziale iniziale $(x_0, y_0) = (13, 10)$, i pixel dell'immagine coperti dalla finestra del filtro sono:

$$P_{(13,10)} = \begin{bmatrix} 147 & 77 & 44 \\ 110 & 128 & 135 \\ 112 & 116 & 115 \end{bmatrix}$$

L'operazione locale consiste nel moltiplicare elemento per elemento ciascun pixel per il corrispondente coefficiente del kernel e sommare i 9 prodotti risultanti:

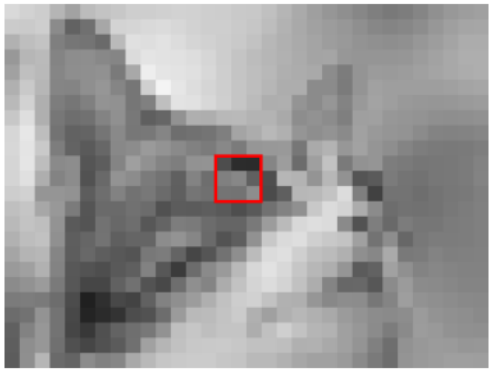
$$\begin{aligned} \text{Output} &= \frac{1}{9}(147) + \frac{1}{9}(77) + \frac{1}{9}(44) + \frac{1}{9}(110) + \frac{1}{9}(128) + \frac{1}{9}(135) + \frac{1}{9}(112) + \frac{1}{9}(116) + \frac{1}{9}(115) \\ &= \frac{147 + 77 + 44 + 110 + 128 + 135 + 112 + 116 + 115}{9} \\ &= \frac{984}{9} \approx 109.33 \longrightarrow 109 \end{aligned}$$

Il valore scalare risultante (109) viene trascritto nella mappa di uscita. Il filtro viene quindi traslato linearmente di un pixel verso destra lungo la riga:

- Alla coordinata $(14, 10)$ (Slide 28): la finestra copre un nuovo set di pixel con somma 895, producendo un'uscita di $\frac{895}{9} \approx 99.4 \rightarrow$ **99**.
- Alla coordinata $(15, 10)$ (Slide 29): la finestra copre il nuovo vicinato con somma 867, producendo $\frac{867}{9} \approx$ **96**.

- Alla coordinata $(16, 10)$ (Slide 30): la finestra copre il vicinato con somma 998, producendo $\frac{998}{9} \approx \mathbf{111}$.

Sliding a Filter Over an Image: 3×3 Mean Filter



Grayscale photo downsampled to 32×24 pixels.

pixels under the window

77	44	43
128	135	96
116	115	141

kernel weights

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

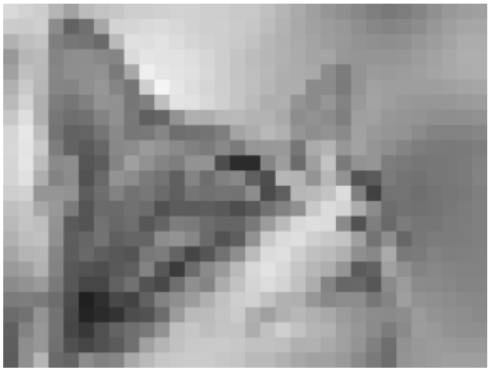
output pixel = weighted sum = 99

$(x_0, y_0) = (14, 10)$

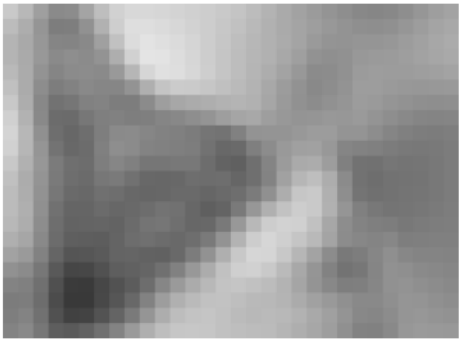
The window moves by one pixel: each output pixel is the weighted sum of the pixels it covers.

Deep Learning Principles and Architectures (AImageLa | Convolutional Neural Networks | 26 / 86

Sliding a Filter Over an Image: The Result



input (32×24)



output of the mean filter (30×22)

Repeating the same operation at every location produces a **smoothed** image. Without padding, the output is slightly smaller than the input.

Deep Learning Principles and Architectures (AImageLa | Convolutional Neural Networks | 27 / 86

Ripetendo l'operazione per tutte le coordinate si ottiene la mappa filtrata (Slide 31). In assenza di accorgimenti per i bordi, l'immagine risultante subisce una lieve riduzione di dimensioni (da 32×24 a 30×22).

2.5.2 Chiarimento Didattico: Dove Viene Collocato il Valore Calcolato?

Durante la spiegazione dell'operatore a finestra mobile emergono spesso dubbi sul significato dell'uscita e sul suo posizionamento:

1. **Un filtro locale modifica l'immagine sovrascrivendo i pixel coperti?**
   No. L'immagine di uscita è inizialmente una matrice vuota. L'applicazione del kernel su

un'intera finestra di 3×3 pixel non produce 9 valori, bensì **un singolo valore scalare**. Questo valore viene inserito in una coordinata specifica della nuova immagine di output, mentre i pixel originali rimangono intatti per i calcoli successivi.

2. **In quale cella dell'output viene inserito lo scalare calcolato?**
   Dipende dalla convenzione adottata per il sistema di coordinate del filtro: La scelta del centro o dell'angolo superiore sinistro è una convenzione di indicizzazione. Non distingue convoluzione e correlazione: la convoluzione ribalta il kernel lungo entrambi gli assi, mentre la correlazione lo applica senza ribaltarlo. Per un kernel simmetrico i risultati coincidono. Con patch $[1, 2]$ e pesi $[3, 4]$, la correlazione dà $1 \cdot 3 + 2 \cdot 4 = 11$, l'applicazione ribaltata dà $1 \cdot 4 + 2 \cdot 3 = 10$. Limiti interi inclusivi da -3 a 3 comprendono **7**, non 6, posizioni.

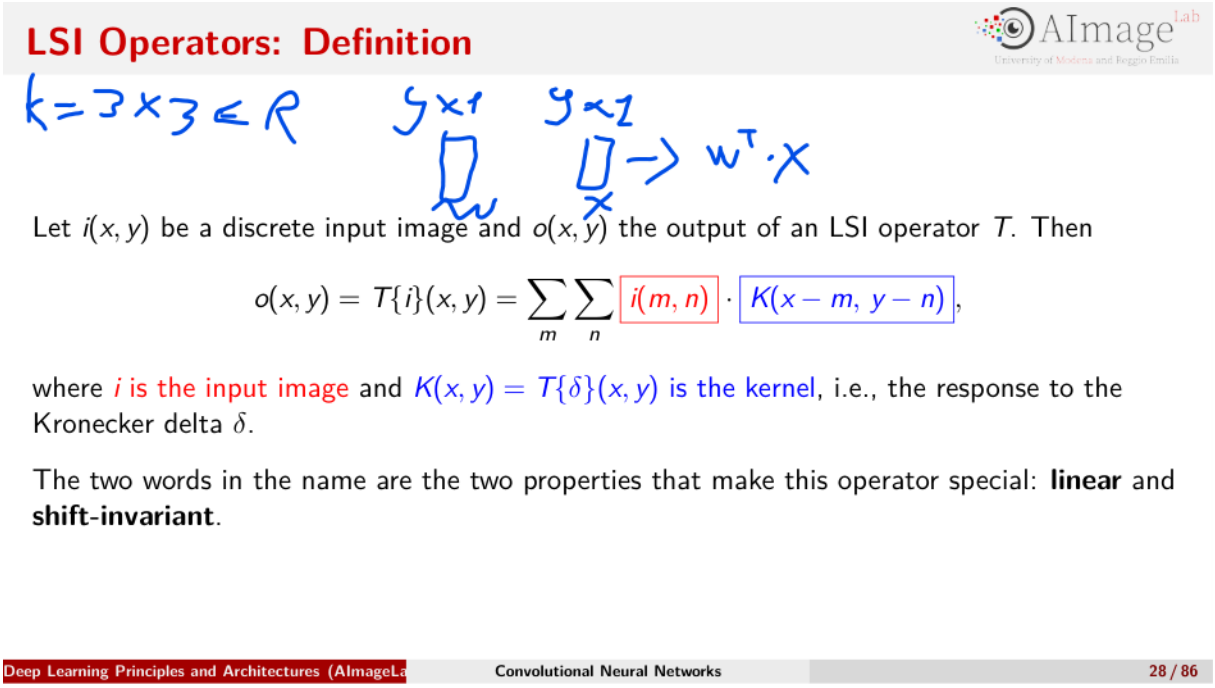
3. **Quante volte viene utilizzato ciascun pixel dell'input?**
   Un pixel interno viene condiviso e riutilizzato tante volte quante sono le posizioni del filtro che lo includono all'interno della propria finestra. Per un kernel 3×3 con passo unitario, ogni pixel interno entra nel calcolo di **9 finestre distinte**, garantendo un'altissima ridondanza e coerenza spaziale nel segnale estratto.

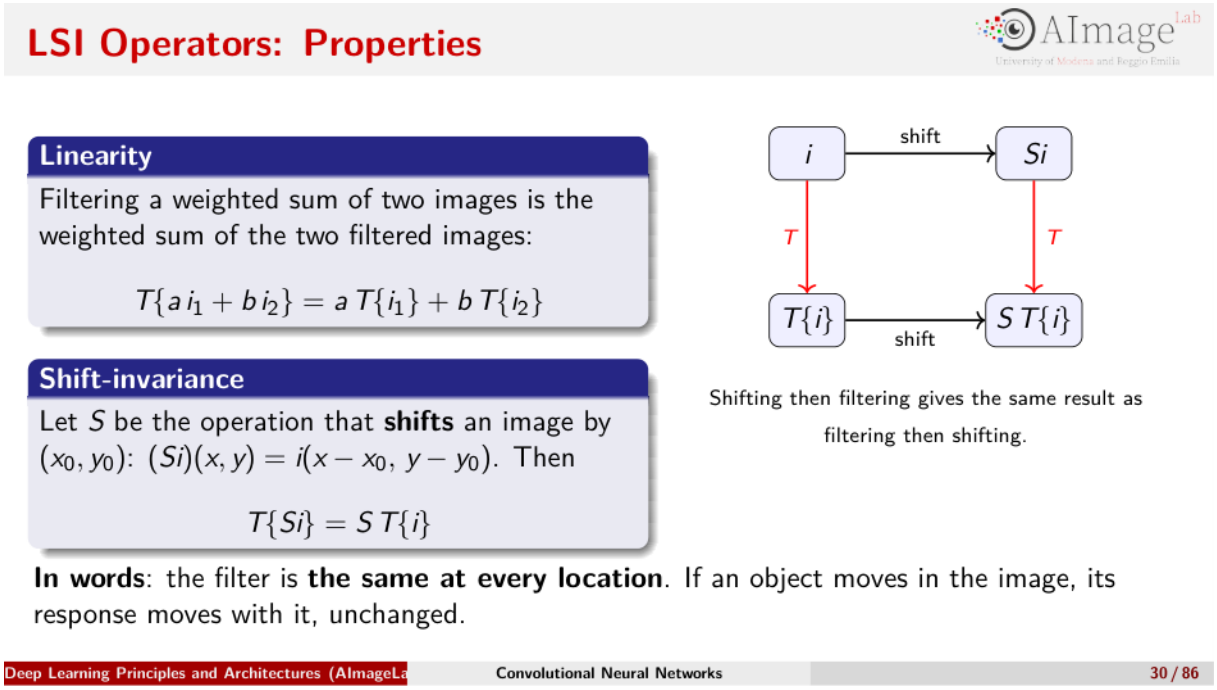
2.5.3 Formalizzazione Matematica dell'Operatore LSI (slide 32–34)

Data un'immagine discreta di ingresso $i(x, y)$, l'uscita $o(x, y)$ prodotta da un operatore LSI $\mathcal{T}$ con kernel discreto $K(x, y)$ è formalmente definita da una doppia sommatoria (Slide 32):

$$\boxed{o(x, y) = \mathcal{T}\{i\}(x, y) = \sum_m \sum_n i(m, n) \cdot K(x - m, y - n)}$$

dove $K(x, y) = \mathcal{T}\{\delta\}(x, y)$ rappresenta la **risposta all'impulso** dell'operatore applicato alla delta di Kronecker bidimensionale $\delta(x, y)$.





Le due proprietà algebriche fondamentali enunciate nella Slide 34 sono:

1. **Linearità**:
$$\mathcal{T}\{a \cdot i_1 + b \cdot i_2\} = a \cdot \mathcal{T}\{i_1\} + b \cdot \mathcal{T}\{i_2\}, \quad \forall a, b \in \mathbb{R}$$
2. **Invarianza alla traslazione (Shift-Invariance / Translation Equivariance)**:
Sia $S_{(x_0,y_0)}$ l'operatore di traslazione spaziale definito da $(Si)(x, y) = i(x - x_0, y - y_0)$. Allora:
$$\boxed{\mathcal{T}\{Si\} = S\mathcal{T}\{i\}}$$

In termini discorsivi: **traslare l'immagine e poi applicare il filtro produce lo stesso identico risultato che applicare il filtro e successivamente traslare la mappa di uscita**.

2.5.3.1 L'Intuizione Fisica: Risposta Massima e Sovrapposizione di Forme

Il significato profondo del prodotto di convoluzione tra un segnale e un kernel può essere compreso analizzando la risposta in 1D o 2D: il valore di uscita misura il grado di somiglianza locale (allineamento per prodotto scalare) tra il profilo del filtro e la porzione di segnale sottostante.

L'analogia delle forme a incastro aiuta a interpretare un filtro come rilevatore di pattern. Un buon allineamento può produrre una risposta alta, ma il prodotto scalare dipende anche dall'intensità e dai segni: non è una misura normalizzata di somiglianza e non genera in generale un unico impulso con zero altrove. La delta di Kronecker definisce invece l'impulso usato per identificare la risposta del sistema.

Se il pattern si sposta all'interno dell'immagine, la proprietà di shift-invariance garantisce che anche il picco della risposta si sposterà esattamente della medesima quantità, senza alterare la propria intensità o conformazione.

2.6 Il Nucleo della Convoluzione nel Deep Learning (slide 35–39)

Il passaggio rivoluzionario dalla Computer Vision classica al Deep Learning risiede in un cambio di paradigma fondamentale (Slide 35): **i coefficienti del kernel non vengono più progettati a mano da un esperto umano, ma vengono trattati come parametri liberi da apprendere direttamente dai dati mediante Backpropagation**.



Figura 2.15: Slide 35 – From LSI Operators to CNNs

2.6.1 La Convoluzione 2D come Percettrone Locale Scorrevole (slide 36)

Si consideri l'operazione di filtraggio locale su una porzione di matrice 3×3 con un filtro 2×2 (Slide 36):

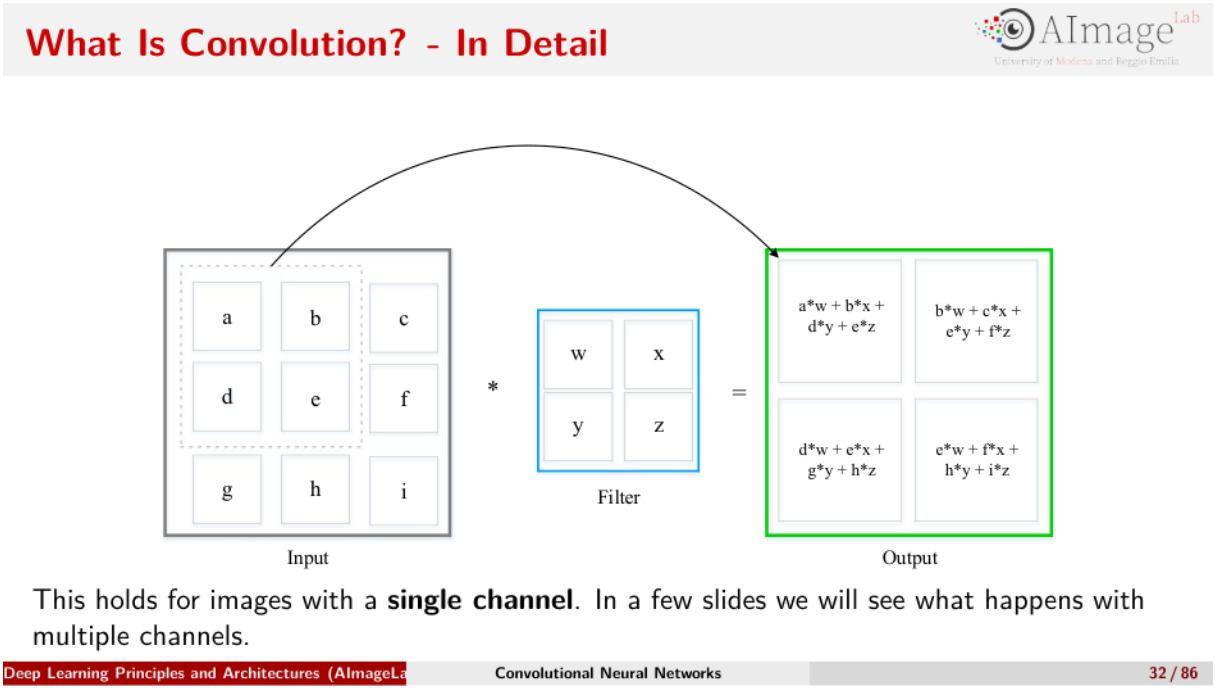


Figura 2.16: Slide 36 – What Is Convolution? - In Detail

$$\text{Input} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}, \qquad \text{Filtro} = \begin{bmatrix} w & x \\ y & z \end{bmatrix}$$

L'uscita nelle quattro posizioni spaziali è:

$$\text{Output} = \begin{bmatrix} a \cdot w + b \cdot x + d \cdot y + e \cdot z & b \cdot w + c \cdot x + e \cdot y + f \cdot z \\ d \cdot w + e \cdot x + g \cdot y + h \cdot z & e \cdot w + f \cdot x + h \cdot y + i \cdot z \end{bmatrix}$$

Se appiattiamo i 4 coefficienti del filtro in un vettore pesi $w = [w, x, y, z]^T \in \mathbb{R}^4$ e i 4 pixel della prima finestra locale in un vettore $x_{(1)} = [a, b, d, e]^T \in \mathbb{R}^4$, l'operazione calcolata coincide con:

$$\text{Out}_{(1,1)} = w^T x_{(1)} + b$$

Questa formulazione dimostra che **un'operazione convoluzionale non è altro che un piccolo perceptrone lineare (un micro-MLP) dotato di pochissimi pesi, che viene fatto scorrere linearmente su tutte le coordinate dell'immagine**.

2.6.2 I Due Principi Fondamentali: Connettività Sparsa e Condivisione dei Pesi (slide 37–39)

Le CNN modificano radicalmente l'approccio dell'MLP densamente connesso attraverso due regole di progettazione (Slide 37):

From LSI Operators to CNNs: Two Key Principles

CNNs build on this basic operator and modify the MLP through **two key principles**:

1. Sparse connectivity
Each neuron is connected only to a small **local neighborhood** of the input, as a kernel window, instead of to *all* the pixels.

2. Weight sharing
The **same weights** (the kernel) are reused at every spatial location, as in an LSI operator.

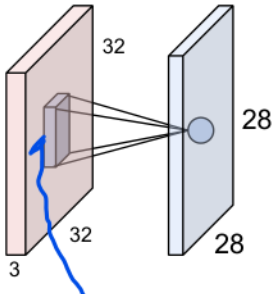
Figura 2.17: Slide 37 – From LSI Operators to CNNs: Two Key Principles

1. **Connettività Sparsa (Sparse Connectivity, Slide 38)**: ogni neurone dello strato convoluzionale non è connesso a tutti i pixel dell'immagine di ingresso, ma interagisce unicamente con una piccola regione spaziale locale e contigua. Questa regione delimitata costituisce il **Receptive Field** (campo recettivo) del neurone.
2. **Condivisione dei Pesi (Weight Sharing / Parameter Sharing, Slide 39)**: il medesimo insieme di coefficienti numerici (il kernel) e lo stesso termine di bias vengono riutilizzati per calcolare le risposte in **tutte** le posizioni spaziali dell'immagine. Tutti i neuroni che concorrono alla generazione di una specifica mappa di feature condividono identici pesi.

Convolutional Layers: Sparse Connectivity

Intuition: At its core, each neuron in a CNN still performs a weighted sum of its inputs plus a bias (as in MLPs):

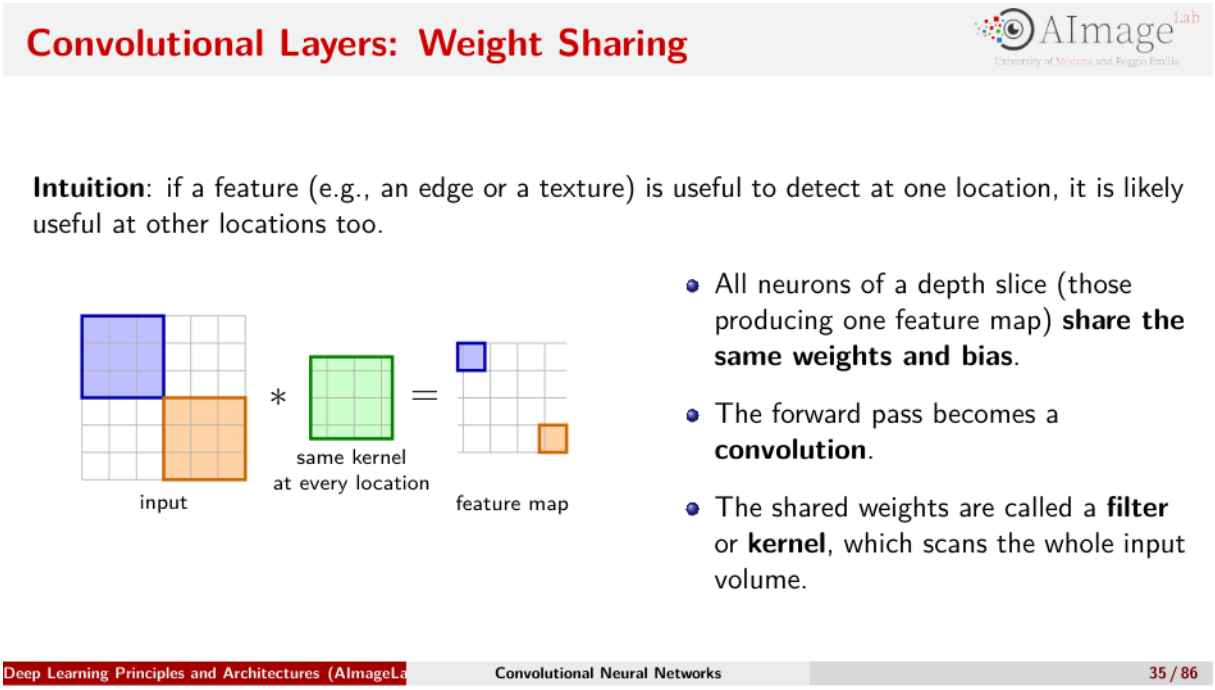
$$\sum_i w_i x_i + b$$



questo è importante più mi sposto in profondità maggiore ... (non ho sentito la risposta)

However, a key difference in convolutional layers is that neurons are only **locally connected** to a small, contiguous region of the input volume.

This small region that each neuron "sees" of the previous layer is called its **receptive field**.



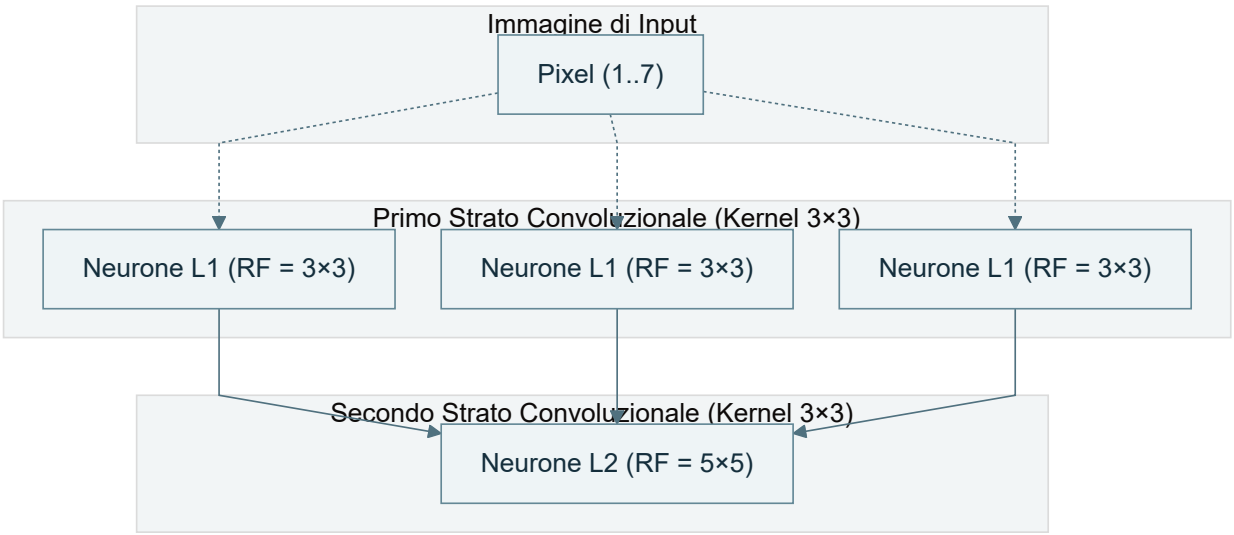
2.6.3 Risoluzione del Punto Aperto: Il Concetto e la Crescita del Receptive Field (slide 38)

Nella slide 38 compare una riflessione didattica lasciata aperta:

"Questo è importante: più mi sposto in profondità, maggiore è ... "

La risposta analitica e concettuale è: **il Receptive Field (campo recettivo teorico) cresce progressivamente all'aumentare della profondità della rete**.

Si analizzi la meccanica di composizione su più strati successivi: * Un neurone appartenente alla prima mappa di feature possiede un campo recettivo limitato, pari all'estensione spaziale del primo filtro (ad esempio 3×3 pixel dell'immagine di input). * Un neurone del secondo strato convoluzionale elabora una finestra 3×3 della mappa di feature prodotta dal primo strato. Ciascuno di questi 9 elementi, a sua volta, sintetizza l'informazione di una propria finestra 3×3 sull'immagine originaria. * Attraverso la sovrapposizione geometrica delle finestre, il singolo neurone del secondo strato "vede" indirettamente un'area considerevolmente più ampia dell'immagine originale (un patch di 5×5 pixel). * Procedendo verso strati via via più profondi (specialmente in presenza di stride o pooling), il campo recettivo teorico si espande a ventaglio fino a coprire decine o centinaia di pixel, arrivando a comprendere l'intero fotogramma.



Questa proprietà risolve un dilemma fondamentale: come può una rete neurale riconoscere oggetti di grandi dimensioni (come un'intera automobile o un volto umano) se i suoi filtri elementari sono minuscoli matrici 3×3? La risposta risiede proprio nella **crescita del campo recettivo**: non è necessario costruire filtri giganteschi (che comporterebbero troppi parametri), ma basta impilare strati con filtri piccoli per comporre gerarchicamente le informazioni locali in rappresentazioni globali.

2.6.4 Crescita del campo recettivo: risposta al dubbio della slide 38

Il campo recettivo teorico è l'insieme delle posizioni dell'immagine che possono influenzare un'uscita. Non coincide con l'importanza effettiva di ciascun pixel, che dipende dai pesi. Indicando con r_ℓ la sua ampiezza e con j_ℓ la distanza fra due uscite adiacenti misurata sull'immagine iniziale, per kernel di ampiezza k_ℓ e stride s_ℓ, senza dilatazione:

$$\boxed{r_\ell = r_{\ell-1} + (k_\ell - 1) j_{\ell-1}, \qquad j_\ell = s_\ell j_{\ell-1}.}$$

Si parte da $r_0 = j_0 = 1$. Una finestra unisce k_ℓ campi del livello precedente, i cui centri sono distanti $j_{\ell-1}$: dal primo all'ultimo si aggiungono $k_\ell - 1$ intervalli. Con due convoluzioni 3×3 a stride 1 si ottiene $r_1 = 3$, $r_2 = 5$.

L'esempio orale parte da una griglia 4×4. Un kernel 2×2 a stride 2 produce una griglia 2×2: ogni uscita riassume quattro pixel. Un altro kernel 2×2 unisce le quattro uscite in un solo valore: $r_2 = 2 + (2-1) \cdot 2 = 4$, quindi può dipendere dall'intera immagine. Si compongono pattern piccoli in pattern estesi senza apprendere fin dall'inizio un filtro grande quanto l'immagine. Non si può concludere che un filtro grande diretto sia più preciso: i modelli hanno parametrizzazioni differenti e, con attivazioni intermedie, non sono equivalenti.

2.7 Convoluzione Multicanale e Feature Maps (slide 40–42)

Nelle applicazioni reali le immagini di ingresso non sono scalari monodimensionali (possiedono $C_{in} = 3$ canali cromatici RGB), e gli strati intermedi operano su volumi composti da decine o centinaia di canali di feature.

2.7.1 Il Filtro Tridimensionale ($C_{in} > 1$, slide 40)

Quando il volume di ingresso ha forma $H \times W \times C_{in}$, un filtro convoluzionale **non è una matrice bidimensionale, bensì un volume tridimensionale** (**un parallelepipedo parametrico**) di dimensioni:

$$\boxed{K \times K \times C_{in}}$$

Il filtro si estende per l'intera profondità dei canali di ingresso. Esso è composto da una pila di C_{in} fette bidimensionali distinte di parametri, ciascuna di dimensione $K \times K$:

$$(K_1, K_2, \ldots, K_{C_{in}})$$

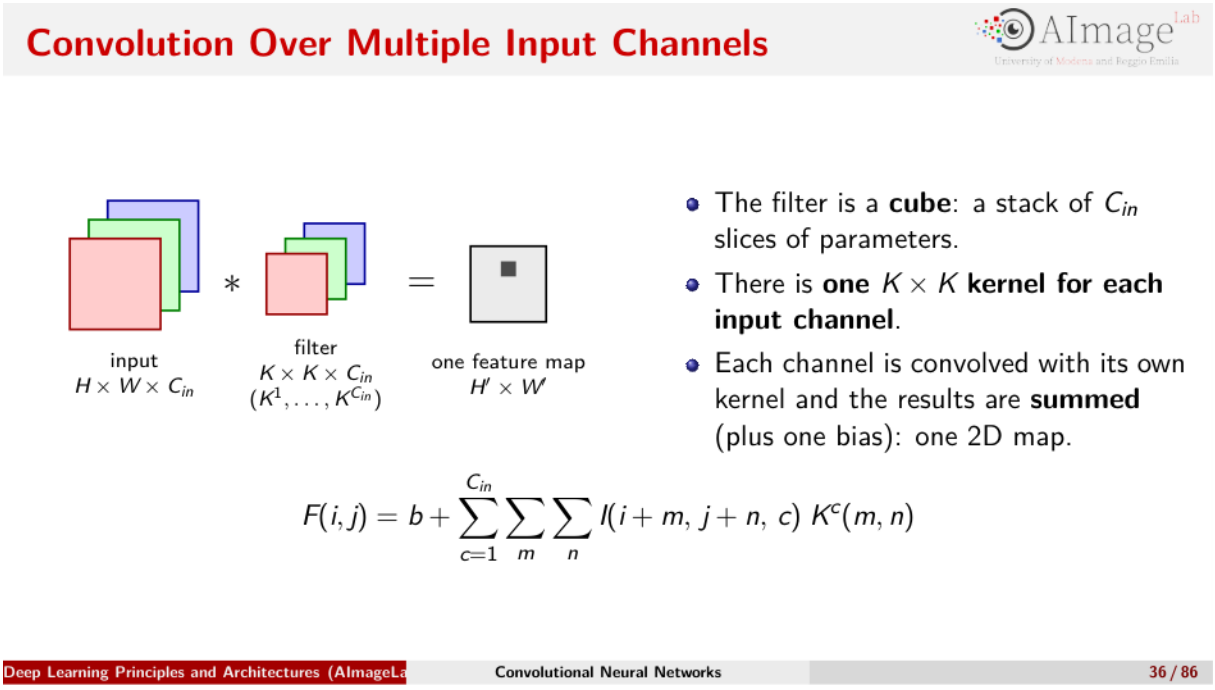


Figura 2.18: Slide 40 – Convolution Over Multiple Input Channels

La formula formale per il calcolo del valore nella mappa di feature $F(i, j)$ alla coordinata spaziale (i, j) è (Slide 40):

$$\boxed{F(i,j) = b + \sum_{c=1}^{C_{in}} \sum_{m} \sum_{n} I(i+m, j+n, c) \cdot K_c(m,n)}$$

dove $I(i+m, j+n, c)$ denota il pixel dell'input nel canale c, $K_c(m,n)$ rappresenta il peso corrispondente nella fetta c del filtro, e $b \in \mathbb{R}$ è l'unico termine scalare di bias associato al filtro.

Il calcolo procede dunque secondo tre passi: 1. Si calcola la convoluzione 2D tra il canale di ingresso c e la rispettiva fetta di kernel K_c. 2. Si sommano i contributi scalari ottenuti attraverso **tutti** i C_{in} canali di ingresso. 3. Si aggiunge il bias scalare b.

Il risultato dell'applicazione di questo filtro tridimensionale $K \times K \times C_{in}$ è **una e una sola mappa di feature bidimensionale** di dimensioni $H' \times W'$.

2.7.2 Generazione di Più Canali di Uscita ($C_{out} > 1$, slide 41)

Se un singolo filtro tridimensionale produce sempre un'unica mappa 2D, come è possibile espandere la profondità del tensore (ad esempio trasformare un ingresso $32\times32\times3$ in un volume $32\times32\times32$, come visto nella Slide 22)?

A questo proposito è utile richiamare una metafora didattica efficace: se un pescatore prende una trota ogni volta che getta la canna da pesca, per ottenere 32 trote non può pretendere che una sola canna catturi magicamente 32 pesci insieme: deve compiere l'azione di pesca 32 volte distinte.

Allo stesso modo, **per ottenere C_{out} mappe di feature distinte dobbiamo applicare C_{out} filtri tridimensionali indipendenti**:

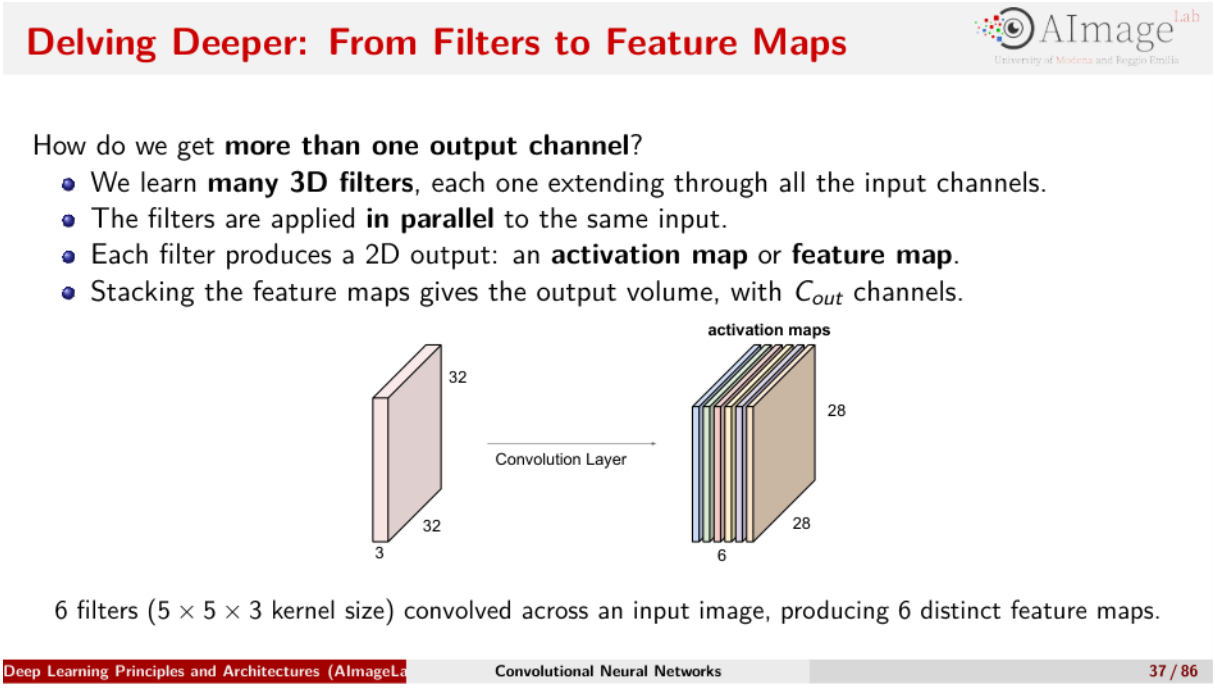


Figura 2.19: Slide 41 – Delving Deeper: From Filters to Feature Maps

Ciascuno dei C_{out} filtri possiede un proprio set esclusivo di pesi di dimensione $K \times K \times C_{in}$ e un proprio bias b_k. Ciascun filtro scorre sull'intero volume di ingresso e genera la propria mappa bidimensionale $H' \times W'$. Impilando (*stacking*) le C_{out} mappe di attivazione lungo l'asse della profondità si ottiene il volume di uscita:

$$\boxed{Y \in \mathbb{R}^{H' \times W' \times C_{out}}}$$

Ad esempio, nell'illustrazione della Slide 41: * Ingresso: $32 \times 32 \times 3$ ($C_{in} = 3$). * Strato convoluzionale: $C_{out} = 6$ filtri indipendenti di dimensione $5 \times 5 \times 3$. * Uscita: volume di 6 mappe di feature di dimensione 28×28, ovvero forma complessiva $28 \times 28 \times 6$.

2.8 Iperparametri del Livello Convoluzionale: Kernel, Stride e Padding (slide 43–46)

La configurazione operativa di uno strato convoluzionale è governata da quattro iperparametri cardinali (Slide 43–44):

1. **Numero di filtri** (C_{out}): determina la profondità del volume di uscita, ovvero quante feature maps distinte vengono estratte in parallelo.
2. **Dimensione spaziale del kernel** ($K \times K$): l'ampiezza geometrica del campo di vista locale dei filtri (generalmente quadrati e dispari: 3×3, 5×5, 7×7). La terza dimensione coincide per definizione con il numero di canali di ingresso C_{in}.
3. **Passo di scorrimento** (**Stride** S): il numero di pixel di cui la finestra del filtro si sposta a ogni passo lungo l'asse orizzontale e verticale.
4. **Padding spaziale** (P): il numero di pixel fittizi aggiunti attorno al perimetro dell'immagine di ingresso.

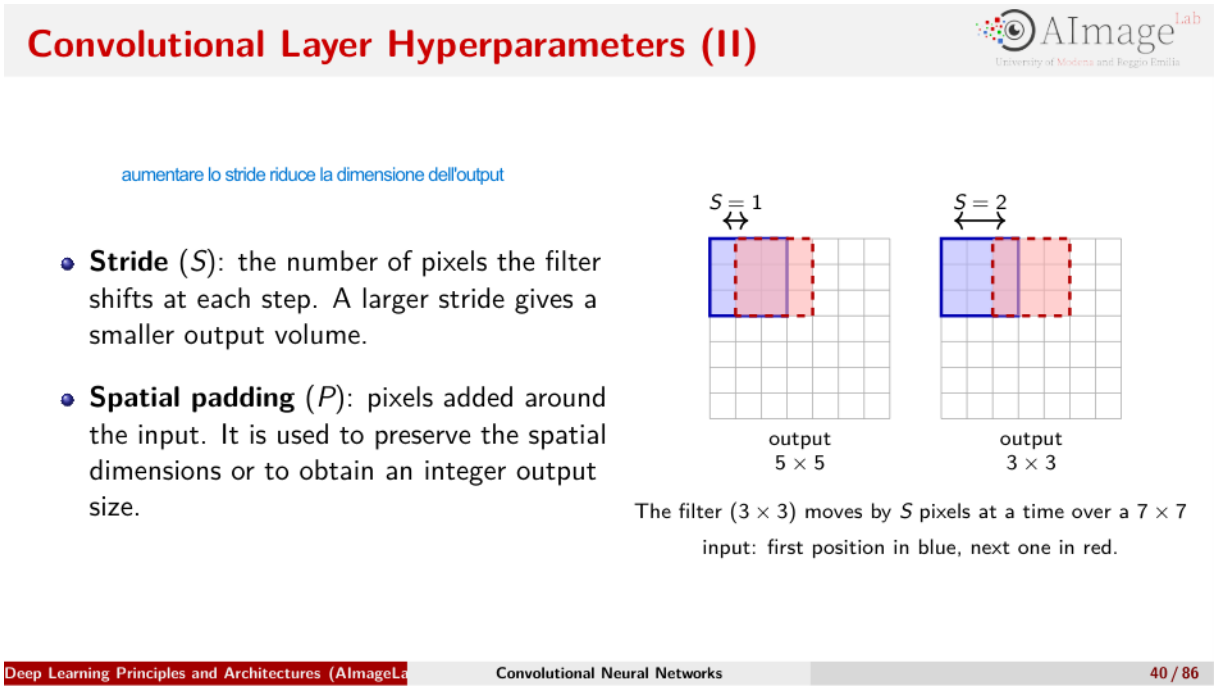


Figura 2.20: Slide 44 – Convolutional Layer Hyperparameters (II)

2.8.1 L'Effetto dello Stride (S)

- Con $S = 1$, il filtro trasla di una singola unità di pixel adiacente alla volta, scandendo capillarmente ogni coordinata.
- Con $S > 1$ (tipicamente $S = 2$), il filtro salta pixel intermedi. L'effetto immediato è una decisa riduzione spaziale delle dimensioni della mappa prodotta (un sottocampionamento diretto): con $S = 2$ le dimensioni spaziali dell'uscita risultano approssimativamente dimezzate rispetto all'ingresso.

La Slide 44 mostra un input 7×7 elaborato con filtro 3×3: * Con $S = 1$ e nessun padding: il filtro trasla di 1 pixel producendo una griglia 5×5. * Con $S = 2$ e nessun padding: il filtro compie salti di 2 pixel (posizioni alle coordinate 1, 3, 5), generando una mappa ridotta 3×3.

2.8.2 Il Fenomeno dell'Erosione dei Bordi e il Ruolo del Padding (P)

Quando un kernel di ampiezza $K \times K$ scorre su un'immagine senza uscire dai bordi, le coordinate periferiche non dispongono di un numero sufficiente di vicini per centrare il filtro. Questo comporta due gravi inconvenienti: 1. **Erosione spaziale** (***Border eating phenomenon***): a ogni strato

convoluzionale privo di padding le dimensioni dell'immagine si restringono forzatamente. In reti profonde con decine di livelli, le dimensioni spaziali collasserebbero a 1×1 dopo pochissimi strati. 2. **Sotto-rappresentazione dei bordi**: i pixel situati sugli angoli e lungo le cornici dell'immagine vengono inclusi in un numero di finestre molto inferiore rispetto ai pixel centrali, subendo una perdita ingiustificata di informazione.

Per ovviare a questo problema si introduce il **Padding** (P), ovvero una cornice di pixel sintetici aggiunta attorno all'immagine prima del filtraggio (Slide 45–46).

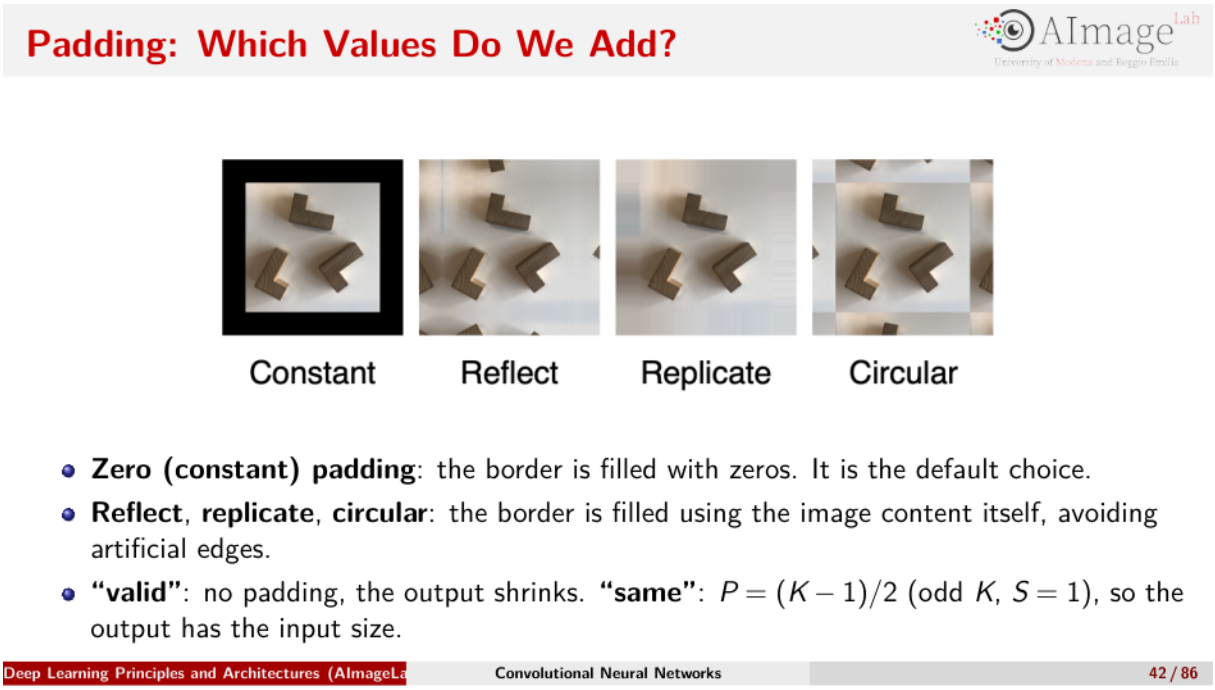


Figura 2.21: Slide 46 – Padding: Which Values Do We Add?

Le modalità principali di padding sono: * **Zero Padding** (**Constant Padding**): i pixel di bordo vengono impostati a zero. Rappresenta la scelta di gran lunga prevalente e predefinita nel Deep Learning per via della sua semplicità computazionale. * **Reflect Padding**: i valori di bordo vengono riempiti specchiando specularmente i pixel interni verso l'esterno, attenuando la formazione di discontinuità artificiali ad alto contrasto sui bordi. * **Replicate Padding**: copia e ripete l'ultimo valore valido di riga o colonna fino al limite del bordo. * **Circular Padding**: assume una topologia toroidale periodica, ricopiando il margine opposto dell'immagine.

2.8.2.1 Le Convenzioni "Valid" e "Same"

- **Padding "valid"**: corrisponde a $P = 0$. Il filtro elabora unicamente posizioni in cui è interamente contenuto nell'immagine originaria; la mappa di uscita risulta più piccola dell'ingresso.
- **Padding "same"**: dimensiona P in modo tale che, impostando uno stride unitario ($S = 1$), **le dimensioni spaziali della mappa di uscita siano esattamente identiche a quelle dell'ingresso** ($H' = H, W' = W$). Per un kernel quadrato di lato dispari K, la quantità di padding necessaria per ciascun lato è determinata analiticamente da:

$$\boxed{P = \frac{K-1}{2}}$$

- Per kernel $3 \times 3 \implies P = \frac{3-1}{2} = 1$ pixel di zero-padding su tutti e quattro i lati.

- Per kernel $5 \times 5 \implies P = \frac{5-1}{2} = 2$ pixel di padding per lato.
- Per kernel $7 \times 7 \implies P = \frac{7-1}{2} = 3$ pixel di padding per lato.

2.8.3 Formula Universale per le Dimensioni dell'Uscita

Dato un volume di ingresso di dimensioni spaziali $H \times W$, elaborato con kernel $K \times K$, padding simmetrico P e stride S, le dimensioni della mappa di feature di uscita (H', W') sono rigorosamente date da:

$$\boxed{H' = \left\lfloor \frac{H - K + 2P}{S} \right\rfloor + 1, \qquad W' = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1}$$

dove $\lfloor \cdot \rfloor$ denota la funzione parte intera inferiore (*floor*).

Verifiche analitiche: 1. *Esempio Slide 41:* Input 32×32, kernel 5×5, $P = 0$, $S = 1$:

$$H' = \frac{32 - 5 + 0}{1} + 1 = 27 + 1 = 28 \implies \mathbf{28 \times 28}$$

2. *Esempio Slide 44 (S=1):* Input 7×7, kernel 3×3, $P = 0$, $S = 1$:

$$H' = \frac{7 - 3 + 0}{1} + 1 = 4 + 1 = 5 \implies \mathbf{5 \times 5}$$

3. *Esempio Slide 44 (S=2):* Input 7×7, kernel 3×3, $P = 0$, $S = 2$:

$$H' = \left\lfloor \frac{7 - 3 + 0}{2} \right\rfloor + 1 = \frac{4}{2} + 1 = 3 \implies \mathbf{3 \times 3}$$

Per ricavare la dimensione dell'uscita, si considera una sola direzione: l'input con padding ha $H + 2P$ posizioni. Una finestra di ampiezza K che parte in qS entra interamente se $qS + K \leq H + 2P$. Il massimo indice è $q_{\max} = \lfloor (H+2P-K)/S \rfloor$; contando anche l'indice zero si ottiene $H' = q_{\max}+1$. La stessa derivazione vale per la larghezza. Occorre che la finestra sia compatibile con l'input padded; si assume dilatazione 1.

2.9 Calcolo dei Parametri Apprendibili (slide 47)

L'estrema efficienza delle reti convoluzionali rispetto agli strati densamente connessi emerge con evidenza dal calcolo formale dei parametri apprendibili di uno strato convoluzionale (Slide 47).

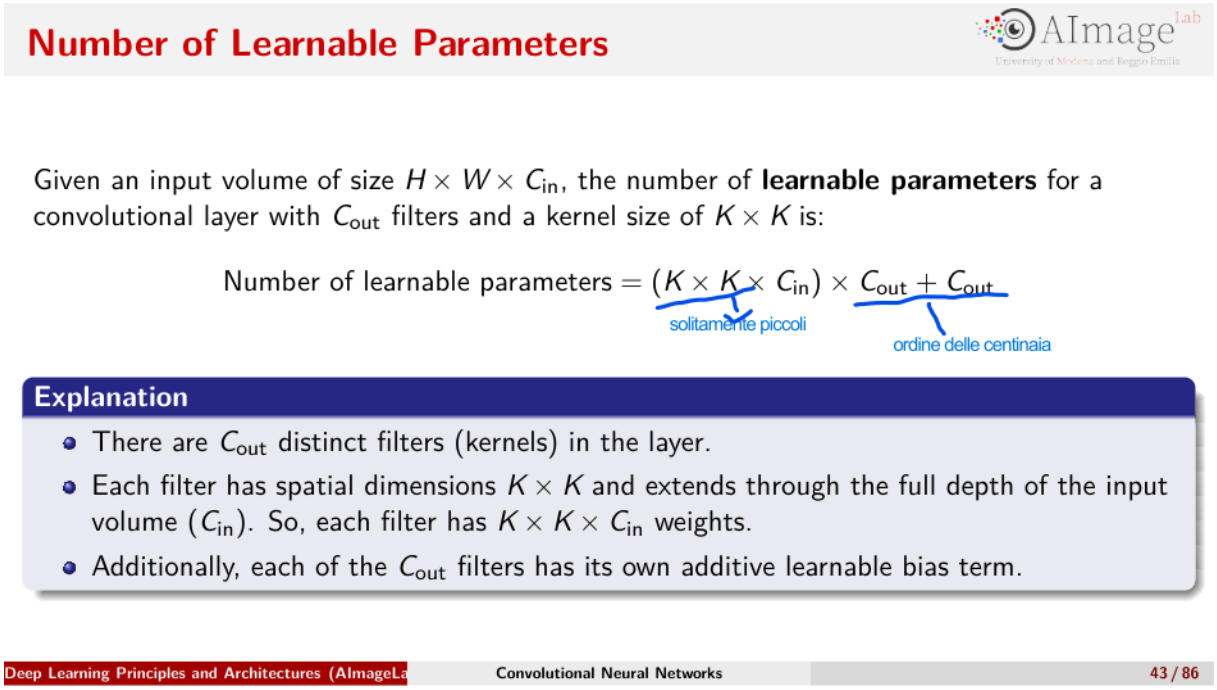


Figura 2.22: Slide 47 – Number of Learnable Parameters

Dato un volume di ingresso di forma $H \times W \times C_{in}$ e uno strato convoluzionale definito da C_{out} filtri di dimensione $K \times K$, il numero totale di parametri apprendibili è:

$$\boxed{\text{Parametri} = (K \times K \times C_{in}) \times C_{out} + C_{out} = C_{out} \cdot (K^2 \cdot C_{in} + 1)}$$

2.9.1 Dimostrazione e Scomposizione

1. Ciascun filtro tridimensionale possiede dimensione spaziale $K \times K$ ed estensione in profondità pari a C_{in}. Il numero di coefficienti moltiplicativi (pesi) per ogni singolo filtro è quindi:
$$\text{Pesi per filtro} = K \times K \times C_{in}$$
2. Essendo presenti C_{out} filtri distinti, il numero complessivo di pesi moltiplicativi ammonta a:
$$(K \times K \times C_{in}) \times C_{out}$$
3. Ciascuno dei C_{out} filtri dispone di un proprio termine di **bias additivo scalare** $b_k \in \mathbb{R}$, condiviso su tutte le coordinate della mappa $H' \times W'$. Si hanno pertanto esattamente C_{out} bias apprendibili.
4. Sommando pesi e bias si ottiene la formula finale.

2.9.2 Il Risultato Chiave: Indipendenza dalla Risoluzione dell'Immagine

La proprietà teorica e computazionale più straordinaria di questa formula è che **il numero di parametri apprendibili di uno strato convoluzionale è totalmente indipendente dalle dimensioni spaziali dell'immagine** (H **e** W).

Si consideri un livello convoluzionale con filtri standard 3×3, $C_{in} = 32$ canali in ingresso e $C_{out} = 64$ filtri in uscita:

$$\text{Parametri} = (3 \times 3 \times 32) \times 64 + 64 = 288 \times 64 + 64 = 18.432 + 64 = \mathbf{18.496}$$

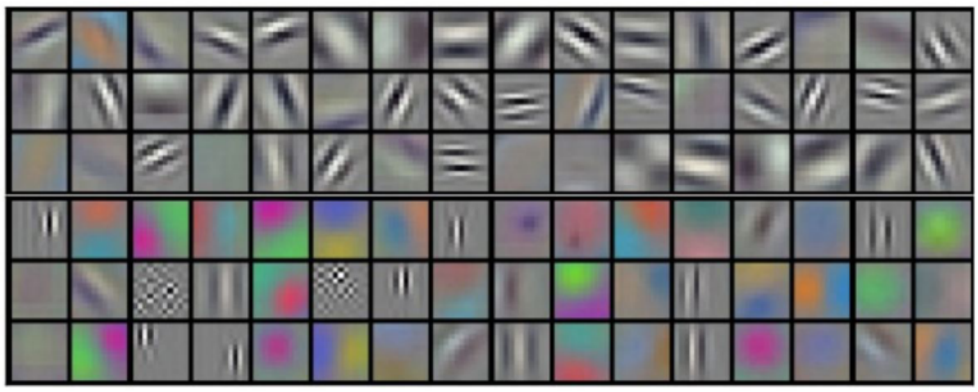
Che l'immagine di ingresso misuri 32×32 pixel, oppure 1000×1000 pixel, il numero di parametri liberi dello strato rimane rigorosamente **18.496** (nell'ordine delle decine di migliaia). Al contrario, come dimostrato nella Sezione 2.3.1, un MLP per immagini da 1000×1000 pixel richiederebbe centinaia di milioni o miliardi di parametri. L'aumento della risoluzione dell'immagine incrementa il numero di operazioni aritmetiche nel forward pass, ma **non modifica il numero di pesi di quel livello**; non permette però di concludere che il rischio di overfitting dell'intero modello resti invariato.

2.10 Visualizzazione dei Filtri e Gerarchia delle Rappresentazioni (slide 48–51)

Uno dei contributi scientifici più rilevanti dell'architettura **AlexNet** (Krizhevsky et al., 2012) è consistito nella possibilità di ispezionare visivamente i parametri appresi dal primo strato convoluzionale.

Visualizing Learned Convolutional Filters

the brain obtain this result from the sight (or similar)



Examples of learned weights from the first convolutional layer of AlexNet [5].

What Do the Learned Filters Look Like?

- Each of the 96 filters is of size $11 \times 11 \times 3$, and is shown as a small RGB image.
- They are **learned**, not designed: many resemble oriented edge detectors and color blobs.
- Each filter is **shared** by all the spatial positions (the 55×55 neurons of a depth slice).
- Detecting a horizontal edge is useful everywhere in the image: the parameter sharing assumption is reasonable, and gives **translation equivariance** (shifting the input shifts the feature map).

Il primo strato di AlexNet riceveva immagini RGB ($C_{in} = 3$) ed era composto da $C_{out} = 96$ filtri con dimensione 11×11. Poiché ciascun filtro ha dimensione $11 \times 11 \times 3$, esso può essere direttamente convertito e renderizzato come una minuscola immagine a colori di 11×11 pixel (Slide 49).

L'ispezione dei 96 filtri svela pattern regolari e significativi: * Molti filtri apprendono risposte a **bordi orientati** (**oriented edge detectors**) ad angolazioni specifiche (orizzontali, verticali, diagonali), con bande alternate di luce e buio. * Altri filtri mostrano selettività cromatica, manifestando gradienti netti o chiazze di colore opposte (*color blobs*, ad esempio transizioni dal rosso al ciano o dal giallo al blu). * Altri ancora assomigliano a filtri di Gabor su diverse frequenze spaziali.

I filtri visualizzati emergono dall'addestramento: alcuni rilevano bordi orientati, contrasti di colore o texture. L'analogia con la visione biologica è qualitativa e non dimostra identità dei meccanismi.

2.10.0.1 Forma vs Texture (*Shape over Texture*)

Bordi, forme e texture possono contribuire al riconoscimento. Le visualizzazioni mostrano feature locali utili, ma non stabiliscono una priorità universale della forma sulla texture: il risultato dipende dal compito e dall'addestramento.

2.10.1 Le Mappe di Attivazione (Activation Maps, slide 50)

Mentre i filtri rappresentano i parametri statici appresi dalla rete, le **mappe di attivazione** (**Activation Maps**) rappresentano le risposte dinamiche dei neuroni a una specifica immagine di ingresso.

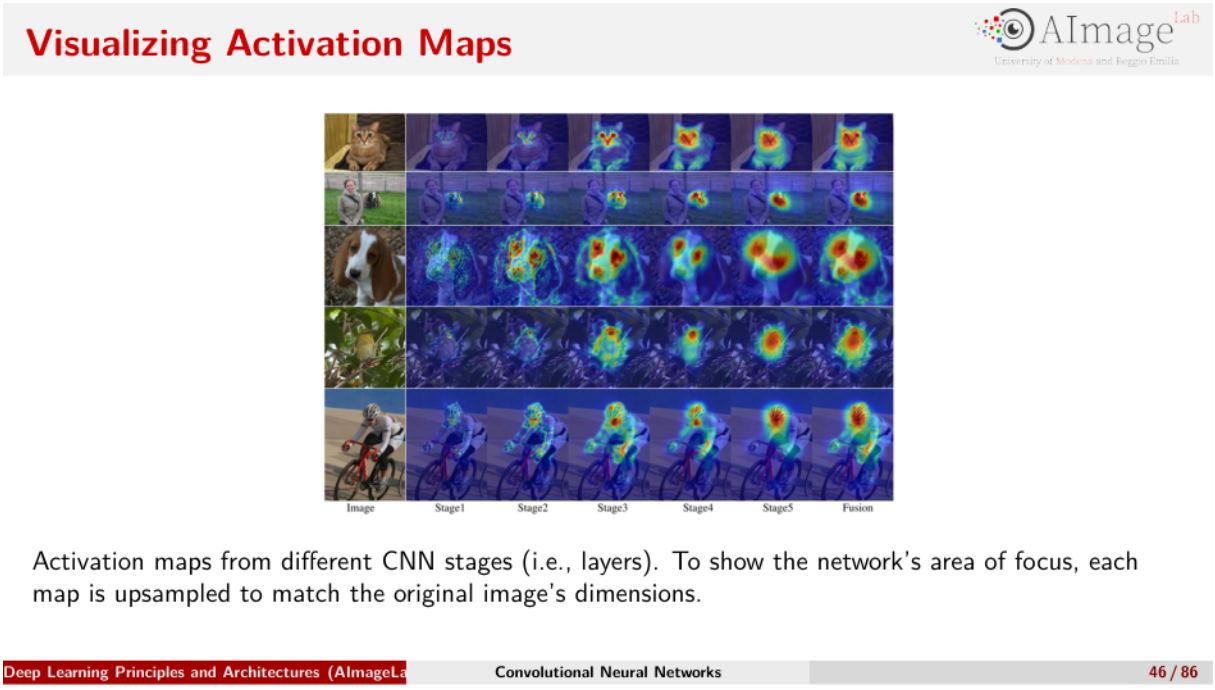


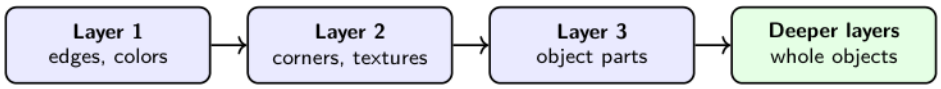
Figura 2.23: Slide 50 – Visualizing Activation Maps

Come mostrato nella Slide 50 (in cui le mappe vengono ricampionate alle dimensioni dell'immagine originale per renderle interpretabili): * Nei primi strati, le mappe si accendono in corrispondenza di singoli pixel o linee sottili che presentano l'orientamento o il colore ricercato dal filtro. * Negli strati intermedi e profondi, le mappe di attivazione non rispondono più a singoli pixel, ma si concentrano su intere regioni coerenti della scena (il muso di un animale, le ruote di un veicolo, una figura umana), evidenziando le aree semantiche su cui la rete focalizza la propria attenzione decisionale.

2.10.2 La Gerarchia dei Concetti e il Transfer Learning (slide 51)

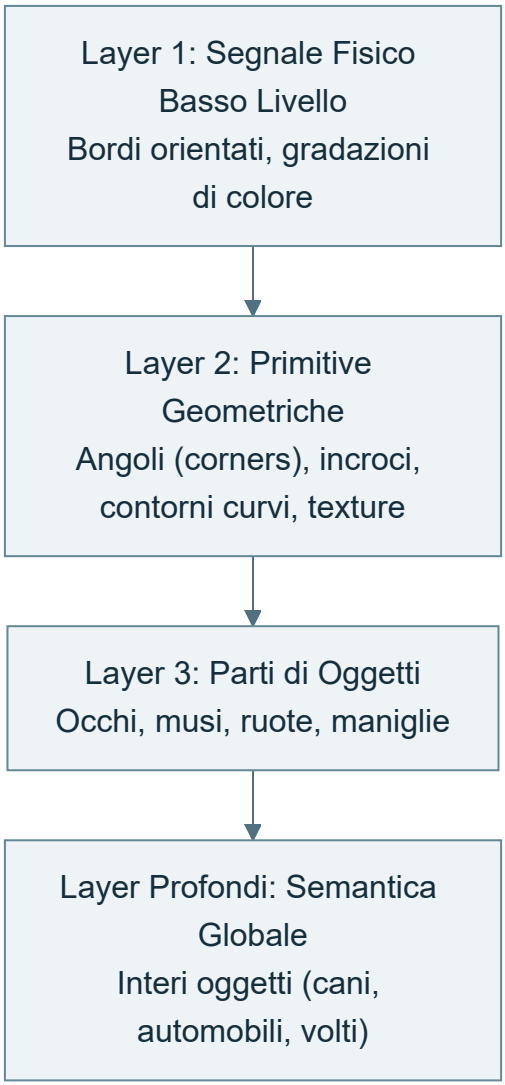
L'impilamento sequenziale di strati convoluzionali realizza una transizione graduale dal segnale grezzo all'astrazione semantica (Slide 51):

Stacking Layers: A Hierarchy of Concepts



- Each layer combines the patterns of the previous one: *edges* → *shapes* → *parts* → *objects*.
- The more **gradual** the hierarchy (the **deeper** the network), the more intermediate concepts it can extract.
- Intermediate concepts are **shared among different classes** (e.g., an *eye* for cats and dogs): features are reused, not relearned for each class.

Figura 2.24: Slide 51 – Stacking Layers: A Hierarchy of Concepts



1. **Strato 1**: rileva bordi, linee e variazioni elementari di contrasto.
2. **Strato 2**: combina linearmente i bordi intercettando angoli (*corners*), incroci a T, archi e trame tessili.
3. **Strato 3**: combina le forme geometriche per assemblare parti anatomiche o meccaniche riconoscibili (occhi, nasi, ruote di automobile).
4. **Strati profondi**: assemblano le parti per identificare l'oggetto intero nella sua configurazione globale.

2.10.2.1 Implicazioni per il Transfer Learning e il Fine-Tuning

- Le feature iniziali, come bordi e contrasti, possono essere riutilizzabili fra compiti e domini diversi. La trasferibilità non è universale: dipende dalla compatibilità fra le immagini di partenza e quelle del nuovo problema.

2.11 Funzioni di Attivazione e Superamento della ReLU Standard (slide 52–54)

2.11.1 La Necessità Fondamentale della Non-Linearità (slide 53)

Uno strato convoluzionale puro calcola unicamente una combinazione lineare dei pixel coperti dal kernel. Perché è tassativo interporre una funzione di attivazione non lineare tra livelli convoluzionali successivi?

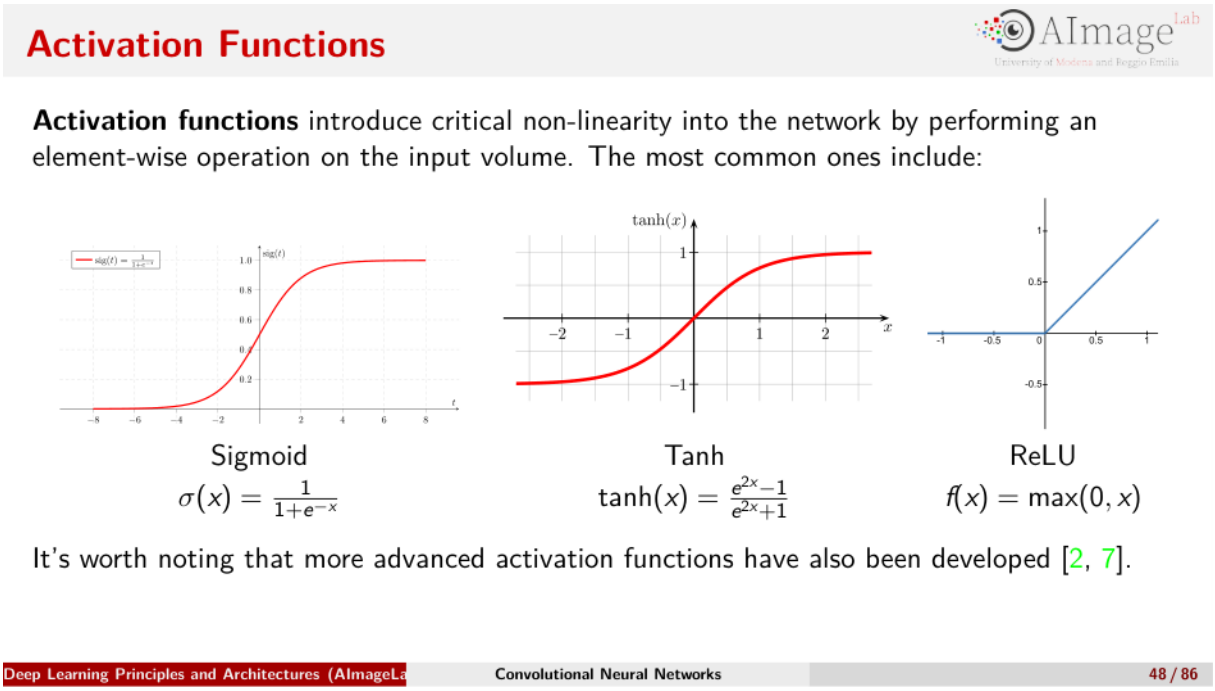
Si considerino due trasformazioni convoluzionali lineari applicate in cascata:

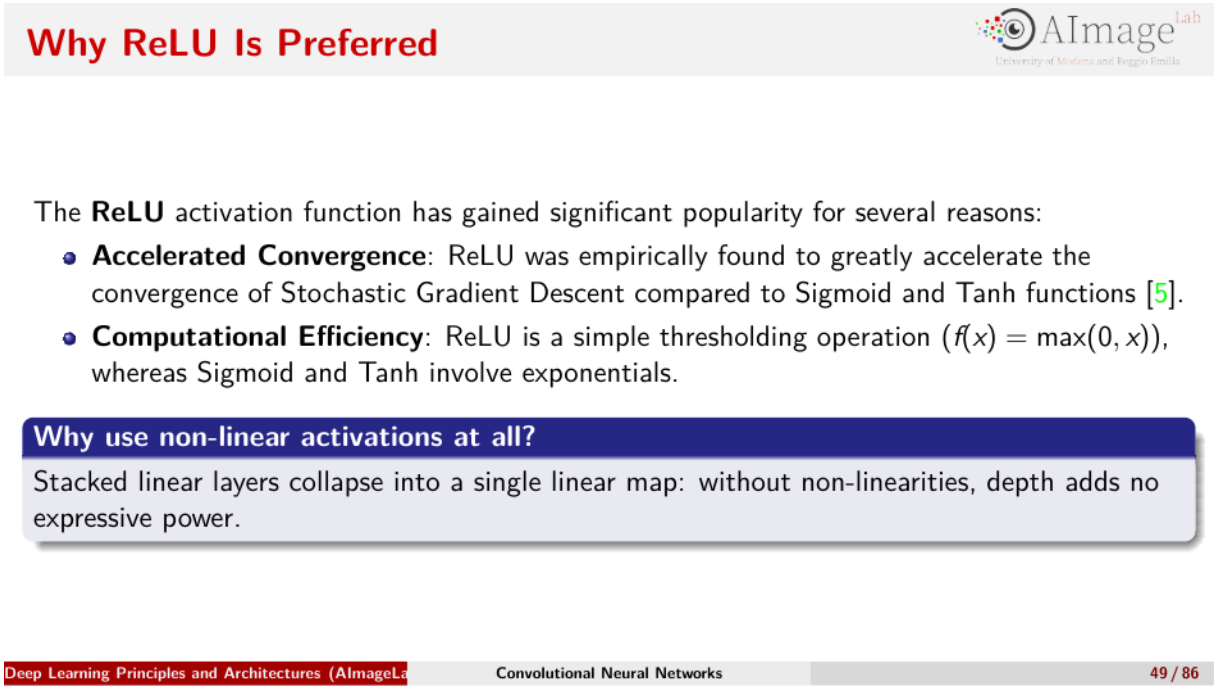
$$y = \mathcal{T}_2(\mathcal{T}_1(x)) = W_2 * (W_1 * x)$$

Per la proprietà associativa dell'operatore di convoluzione, la composizione di due convoluzioni lineari equivale a un'unica convoluzione con un kernel equivalente W_{eq}:

$$W_2 * (W_1 * x) = (W_2 * W_1) * x = W_{eq} * x$$

In assenza di funzioni di attivazione non lineari, una rete profonda composta da cento strati convoluzionali collasserebbe algebricamente in un unico, banale strato convoluzionale lineare. La profondità non apporterebbe alcun incremento di capacità espressiva. L'inserimento di una non-linearità puntuale rompe questa equivalenza algebrica e conferisce alla rete la capacità di approssimare varietà decisionali complesse.





2.11.2 Dai Limiti di Sigmoide e Tanh al Successo della ReLU (slide 52–53)

Storicamente, le prime reti neurali impiegavano come attivazioni la **Sigmoide** o la **Tangente Iperbolica (Tanh)** (Slide 52):

$$\sigma(x) = \frac{1}{1+e^{-x}}, \qquad \tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}} = \frac{e^{2x}-1}{e^{2x}+1}$$

Entrambe queste funzioni presentano due gravi svantaggi nell'addestramento di reti profonde: 1. **Saturazione e scomparsa del gradiente** (*Vanishing Gradient*): per valori di ingresso elevati (positivi o negativi), la curva si appiattisce asintoticamente. La derivata prima tende esponenzialmente a zero ($\sigma'(x) \approx 0$). Durante la retropropagazione, moltiplicando gradienti vicini allo zero lungo molti strati, il segnale di errore si annulla prima di raggiungere gli strati iniziali, bloccando l'apprendimento dei filtri. 2. **Costo computazionale elevato**: il calcolo degli esponenziali (e^{-x}) è operativamente dispendioso rispetto a operazioni aritmetiche elementari.

La **ReLU (Rectified Linear Unit)**, introdotta sistematicamente su larga scala proprio con AlexNet (Slide 53), definisce una semplice sogliatura a zero:

$$\boxed{f(x) = \max(0, x)}$$

La ReLU garantisce vantaggi decisivi: * **Accelerazione drammatica della convergenza**: per tutti i valori positivi ($x > 0$), la derivata è costante e pari a 1 ($\frac{df}{dx} = 1$). Il gradiente non è attenuato dal solo fattore derivativo della ReLU sul semiasse positivo; rimangono gli effetti dei pesi e degli altri strati, consentendo alla discesa stocastica del gradiente (SGD) di convergere molto più rapidamente. * **Efficienza di calcolo estrema**: richiede una banale operazione di confronto logico a livello di registri hardware, senza esponenziali né divisioni.

2.11.3 La Patologia del Dying ReLU e le Attivazioni Avanzate: Leaky ReLU ed ELU (slide 54)

Nonostante i suoi benefici, la ReLU standard manifesta una debolezza intrinseca, evidenziata dall'annotazione presente sulla Slide 54:

"Nella ReLU normale qualsiasi valore negativo viene tagliato a 0 e questo ferma l'apprendimento del modello."

- Un'attivazione negativa annulla il gradiente **attraverso quella posizione**, non l'apprendimento dell'intero modello. Un filtro condiviso può ricevere contributi da altre posizioni o esempi dove è attivo. Il problema del *dying ReLU* riguarda unità che rimangono inattive su tutti o quasi tutti gli input rilevanti; non segue da un solo valore negativo e non è necessariamente irreversibile.

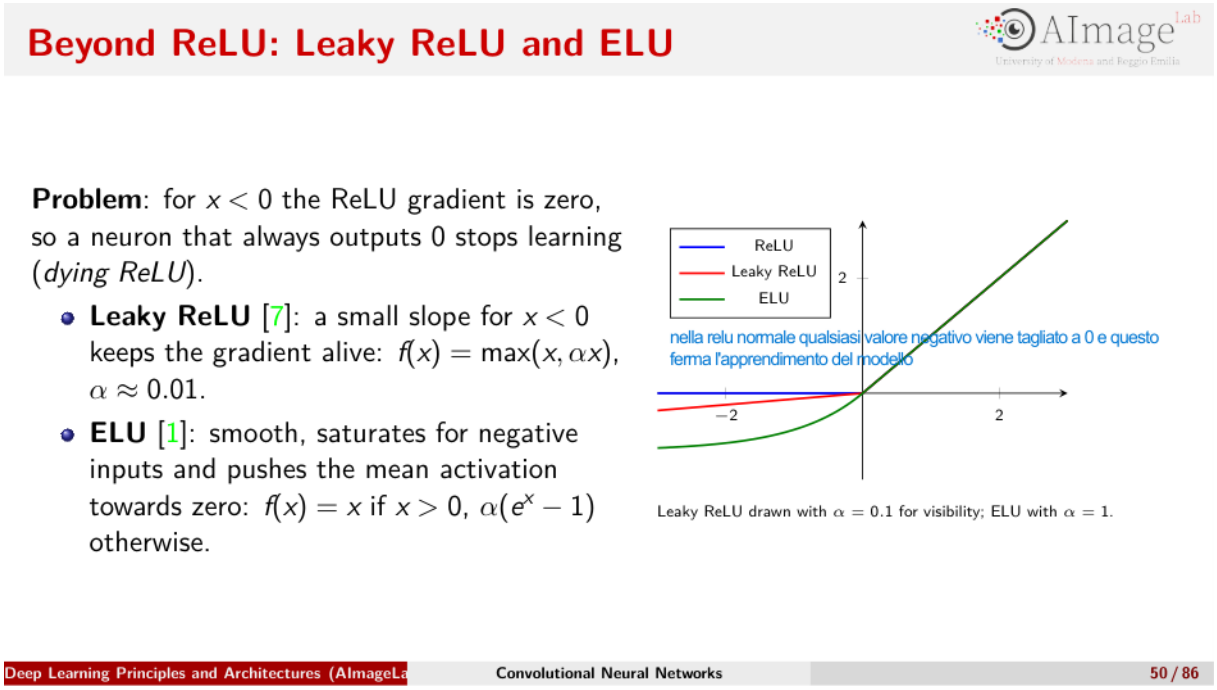


Figura 2.25: Slide 54 – Beyond ReLU: Leaky ReLU and ELU

Per sanare questa criticità sono state sviluppate due estensioni fondamentali (Slide 54):

2.11.3.1 Leaky ReLU

La **Leaky ReLU** introduce una piccola pendenza lineare per la regione dei valori negativi, impedendo al gradiente di azzerarsi:

$$\boxed{f(x) = \max(x, \alpha x) = \begin{cases} x & \text{se } x \geq 0 \\ \alpha x & \text{se } x < 0 \end{cases}}$$

dove α è un iperparametro positivo molto piccolo (tipicamente fissato a $\alpha = 0.01$; nella figura della Slide 54 è tracciato con $\alpha = 0.1$ a scopo di visibilità grafica).

La derivata della Leaky ReLU vale:

$$f'(x) = \begin{cases} 1 & \text{se } x > 0 \\ \alpha & \text{se } x < 0 \end{cases}$$

Poiché per $x < 0$ la derivata vale $\alpha \neq 0$, **il gradiente rimane vivo anche per attivazioni negative**, consentendo ai pesi del filtro di continuare a ricevere correzioni e di riattivarsi nelle iterazioni successive.

2.11.3.2 ELU (Exponential Linear Unit)

L'**ELU** (**Exponential Linear Unit**) sostituisce la retta spezzata con un andamento esponenziale asintotico liscio nella semiretta negativa:

$$\boxed{f(x) = \begin{cases} x & \text{se } x > 0 \\ \alpha(e^x - 1) & \text{se } x \leq 0 \end{cases}}$$

(tipicamente impostata con $\alpha = 1$).

L'ELU è continua per ogni $\alpha > 0$. Le derivate laterali in zero sono α e 1, quindi è differenziabile in zero **soltanto se** $\alpha = 1$. Per $x < 0$ la derivata è αe^x: è positiva ma tende a zero per ingressi molto negativi. L'uscita tende a $-\alpha$ e i valori negativi possono avvicinare la media delle attivazioni a zero, senza garantirne l'annullamento.

Con $x = -2$, ReLU dà 0 e derivata 0; Leaky ReLU con $\alpha = 0.01$ dà -0.02 e derivata 0.01; ELU con $\alpha = 1$ dà $e^{-2} - 1 \approx -0.8647$ e derivata $e^{-2} \approx 0.1353$. Questi sono calcoli scalari applicati indipendentemente a ciascuna componente del tensore, senza cambiarne le dimensioni.

2.12 Livelli di pooling: aggregazione locale e riduzione spaziale (slide 55–58)

Una convoluzione produce mappe di attivazione che descrivono pattern in posizioni diverse. Conservare tutte le posizioni a ogni profondità ha un costo di memoria e mantiene dettagli talvolta superflui per classificare. Il **pooling** riassume una finestra in un solo valore, riducendo la risoluzione quando lo stride è maggiore di uno.

L'operazione si applica **separatamente a ogni canale**: se entrano 16 mappe, ne escono 16. Il «kernel» indica qui la finestra di aggregazione, non pesi apprendibili. Dimensione della finestra, stride e funzione di aggregazione sono iperparametri; max e average pooling ordinari hanno **zero parametri apprendibili**, come annotato nella slide 55.

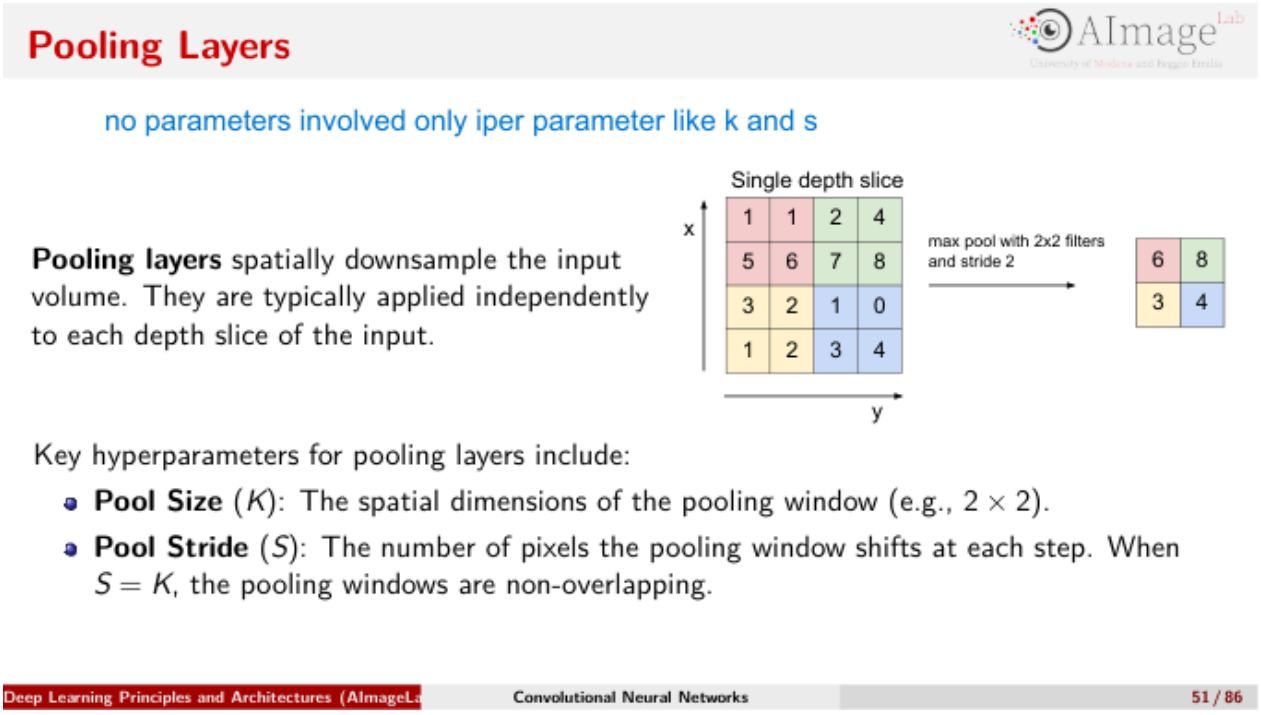


Figura 2.26: Slide 55 – Finestra di max pooling e annotazione sui parametri

2.12.1 Formule e dimensioni (slide 55–56)

Si usa $X \in \mathbb{R}^{N\times C\times H\times W}$: batch N, canali C, altezza H e larghezza W. Con finestra quadrata $K \times K$, stride S, nessun padding né dilatazione e indici spaziali da zero:

$$\boxed{\begin{aligned} Y^{\max}_{n,c,i,j} &= \max_{0\le u,v<K} X_{n,c,iS+u,jS+v}, \\ Y^{\text{avg}}_{n,c,i,j} &= \frac{1}{K^2}\sum_{u=0}^{K-1}\sum_{v=0}^{K-1} X_{n,c,iS+u,jS+v}. \end{aligned}}$$

Ogni uscita è uno scalare: il massimo seleziona il valore più alto fra i K^2 elementi, la media ne somma i contributi e divide per il loro numero. Il massimo privilegia una risposta forte anche isolata; la media misura l'attivazione complessiva della regione. Un'attivazione non è necessariamente una probabilità: un valore 100 può essere semplicemente uno score.

La finestra di indice i parte dalla riga iS e deve soddisfare $iS + K \le H$. Quindi

$$\begin{aligned} i &\le (H-K)/S, \\ i_{\max} &= \lfloor (H-K)/S \rfloor, \\ \boxed{H'} &= \boxed{\lfloor (H-K)/S \rfloor + 1}, \qquad \boxed{W'} = \boxed{\lfloor (W-K)/S \rfloor + 1}. \end{aligned}$$

L'uscita è $Y \in \mathbb{R}^{N\times C\times H'\times W'}$. Con $K = S = 2$ e dimensioni pari, entrambi gli assi si dimezzano e le attivazioni diventano un quarto. Batch e canali restano invariati. Con $S = 1$, invece, una griglia 4×4 diventa 3×3. Le finestre sono non sovrapposte per $S = K$.

Stride e riduzione

Il riferimento orale isolato a una configurazione usuale $K = 2, S = 1$ contrasta con l'esempio e con la slide 57. La configurazione che dimezza entrambi gli assi è $K = 2, S = 2$.

2.12.2 Esempio completo della slide 55 (slide 55–56)

Per un esempio e un canale:

$$X = \begin{bmatrix} 1 & 1 & 2 & 4 \\ 5 & 6 & 7 & 8 \\ 3 & 2 & 1 & 0 \\ 1 & 2 & 3 & 4 \end{bmatrix} \in \mathbb{R}^{4\times 4}.$$

Con $K = S = 2$ si ottengono quattro finestre:

Uscita	Valori	Massimo	Media
$(0,0)$	$1,1,5,6$	6	$13/4 = 3.25$
$(0,1)$	$2,4,7,8$	8	$21/4 = 5.25$
$(1,0)$	$3,2,1,2$	3	$8/4 = 2$
$(1,1)$	$1,0,3,4$	4	$8/4 = 2$

$$\boxed{Y^{\max} = \begin{bmatrix} 6 & 8 \\ 3 & 4 \end{bmatrix}}, \qquad \boxed{Y^{\text{avg}} = \begin{bmatrix} 3.25 & 5.25 \\ 2 & 2 \end{bmatrix}}.$$

Le forme tornano: $H' = W' = \lfloor (4-2)/2 \rfloor + 1 = 2$. La riduzione degli scalari è $(16-4)/16 = 75\%$. Un input $N \times 16 \times 8 \times 8$ diventa $N \times 16 \times 4 \times 4$: la medesima operazione viene ripetuta sui canali senza apprendere 16 nuovi filtri.

Zero parametri non significa assenza di backpropagation. Con massimo unico, la derivata rispetto agli elementi della finestra vale 1 nella posizione selezionata e 0 nelle altre; per la media vale $1/K^2$ in ogni posizione. Sono le derivate delle due funzioni scalari. Se il gradiente in arrivo sulla prima uscita è 2, il massimo lo invia al valore 6; la media invia $2/4 = 0.5$ a ciascun valore. Con massimi uguali occorre una convenzione per il sottogradiente; con finestre sovrapposte i contributi alla stessa posizione si sommano.

2.12.3 Robustezza locale, gerarchia e campo recettivo (slide 57)

Il massimo di $(100, 2, 4, 3)$ rimane 100 qualunque sia la posizione del 100 **entro la finestra**. Se il canale rileva un cerchio, interessa sapere che una forte risposta esiste nella regione, anche perdendone la coordinata precisa. La media è invece $109/4 = 27.25$: una risposta isolata viene attenuata dai valori piccoli circostanti. Questa è la motivazione intuitiva per usare spesso il massimo negli stadi locali e la media per una descrizione complessiva; non dimostra che il max pooling sia sempre superiore.

La robustezza è limitata: se il 100 attraversa il confine fra due finestre cambia l'uscita. Pooling e stride non garantiscono invarianza a qualunque traslazione dell'immagine.

L'annotazione «gerarchia di concetti» si riferisce alla composizione progressiva delle rappresentazioni: i primi filtri descrivono bordi e contrasti, i successivi combinano risposte su regioni più estese. Eliminare dettagli di posizione può favorire l'astrazione, ma il pooling non produce da solo significati semantici: questi dipendono dai filtri appresi e dal compito.

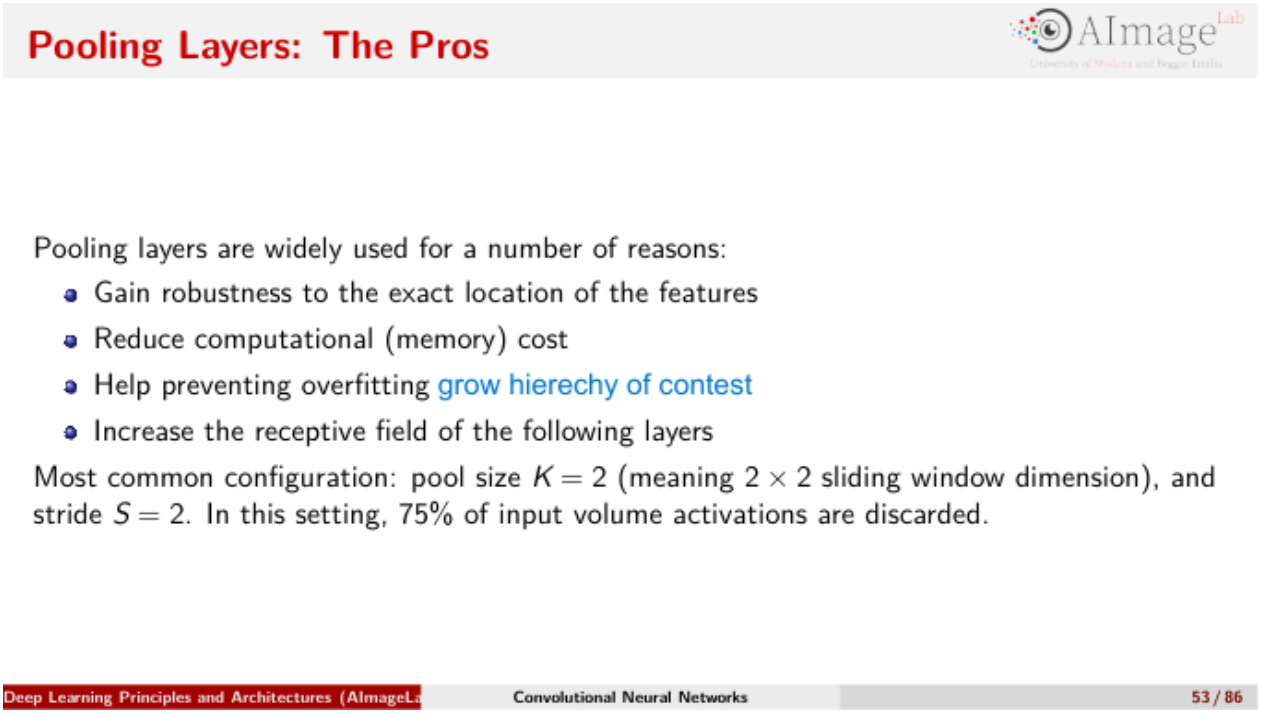


Figura 2.27: Slide 57 – Vantaggi del pooling e gerarchia

Con campo recettivo r e distanza j fra uscite adiacenti, misurata sull'immagine iniziale:

$$\boxed{r_{\text{out}} = r_{\text{in}} + (K-1)j_{\text{in}}, \qquad j_{\text{out}} = S j_{\text{in}}.}$$

La prima formula unisce K campi i cui centri distano j_{in}; la seconda esprime il salto di S posizioni. Dopo una convoluzione 3×3 a stride 1 si ha $r = 3, j = 1$. Un pooling 2×2 a stride 2 dà $r = 4, j = 2$; un'altra convoluzione 3×3 dà $r = 4 + 2 \cdot 2 = 8$. L'uscita può dipendere da una regione 8×8 dell'input originale. Si tratta del campo recettivo potenziale, distinto dalle posizioni effettivamente selezionate dal massimo.

Il pooling riduce la memoria delle attivazioni e il lavoro successivo. La perdita di dettagli può limitare dipendenze da peculiarità del training set, ma non garantisce l'assenza di overfitting.

2.12.4 Il limite: perdita di localizzazione (slide 58)

Nella classificazione può bastare sapere che un oggetto è presente; nella segmentazione semantica occorre assegnare una classe a ogni pixel. Una riduzione aggressiva della risoluzione può cancellare confini e oggetti piccoli.

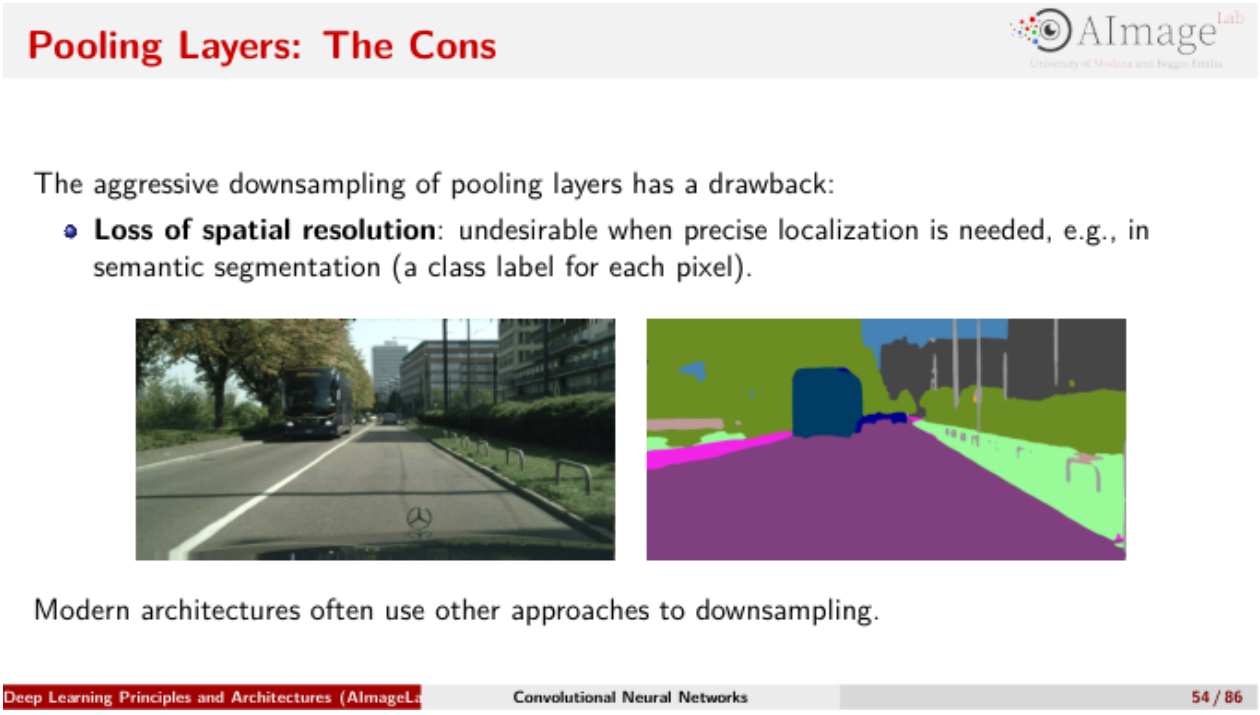


Figura 2.28: Slide 58 – Risoluzione e segmentazione

Il pooling va scelto in funzione del compito. Questo non implica che ogni modello di segmentazione debba escluderlo, ma che la gestione della risoluzione richieda attenzione. Le alternative architetturali non vengono sviluppate qui.

2.13 Dalle feature al classificatore: pooling adattivo (slide 59)

L'estrattore produce $X \in \mathbb{R}^{N\times C\times h\times w}$. Un livello denso richiede un numero fisso di ingressi, perché la sua matrice dei pesi ha un numero fisso di colonne. Occorre quindi rendere compatibile una griglia dipendente dalla risoluzione con un classificatore di forma fissata.

2.13.1 Flattening e risoluzione dell'ingresso (slide 59)

Il flattening riordina gli elementi senza aggregarli:

$$\boxed{N \times C \times h \times w \;\longrightarrow\; N \times D, \qquad D = Chw.}$$

Da $16\times8\times8$ si ottengono 1024 elementi. Se un'immagine più grande produce $16\times12\times12$, diventano 2304: una matrice $W \in \mathbb{R}^{q\times1024}$ non può moltiplicare un vettore di lunghezza 2304. La flessibilità spaziale della convoluzione non rende automaticamente flessibile la testa densa.

Se la risoluzione è fissa, il flattening diretto è valido e conserva tutte le attivazioni. Non è necessariamente migliore: richiede più pesi e impone quella forma. Ridimensionare preventivamente le immagini è possibile, ma introduce interpolazione e, se si altera il rapporto degli assi, deformazione geometrica.

2.13.2 Global average pooling e posizione esatta nella rete (slide 59)

La media globale sostituisce ciascun canale con un solo scalare:

$$\boxed{z_{n,c} = \frac{1}{hw} \sum_{i=0}^{h-1} \sum_{j=0}^{w-1} X_{n,c,i,j}.}$$

È l'average pooling con una finestra estesa a tutta la mappa: si sommano hw elementi e si divide per il loro numero. Il volume diventa $N \times C \times 1 \times 1$ e un successivo flattening produce $N \times C$, indipendentemente da h, w.

Per due canali

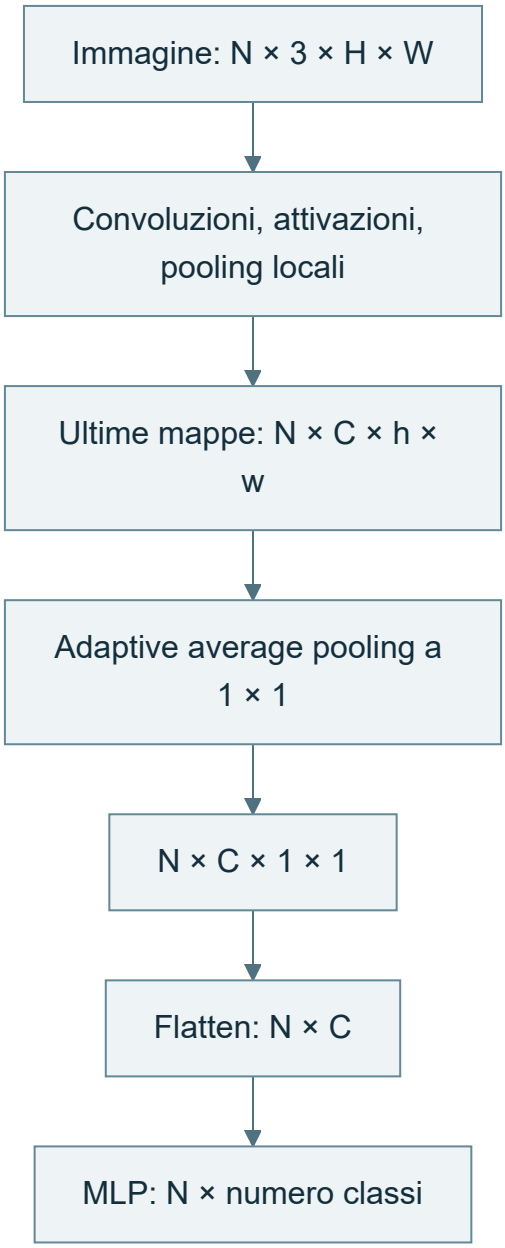
$$X_1 = \begin{bmatrix} 1 & 3 \\ 5 & 7 \end{bmatrix}, \qquad X_2 = \begin{bmatrix} 2 & 2 \\ 6 & 10 \end{bmatrix},$$

si ottiene $z = [16/4, 20/4]^T = [4, 5]^T \in \mathbb{R}^2$. Un classificatore con q uscite impiega $W \in \mathbb{R}^{q \times C}$, $b \in \mathbb{R}^q$:

$$\boxed{s = Wz + b \in \mathbb{R}^q, \qquad P = qC + q = q(C + 1).}$$

Con $q = 3$, $W = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & -1 \end{bmatrix}$ e $b = 0$, gli score sono $[4, 5, -1]^T$. I sei pesi e tre bias danno nove parametri: sono del classificatore, non del pooling. Una softmax finale può convertirli in probabilità del modello.

Il dubbio «solo nell'ultimo layer: dove?» si risolve localizzando l'interfaccia: **dopo le ultime mappe dell'estrattore convoluzionale, prima del flattening e dell'MLP**, non dopo gli score finali.


Immagine: N × 3 × H × W
Convoluzioni, attivazioni, pooling locali
Ultime mappe: N × C × h × w
Adaptive average pooling a 1 × 1
N × C × 1 × 1
Flatten: N × C
MLP: N × numero classi

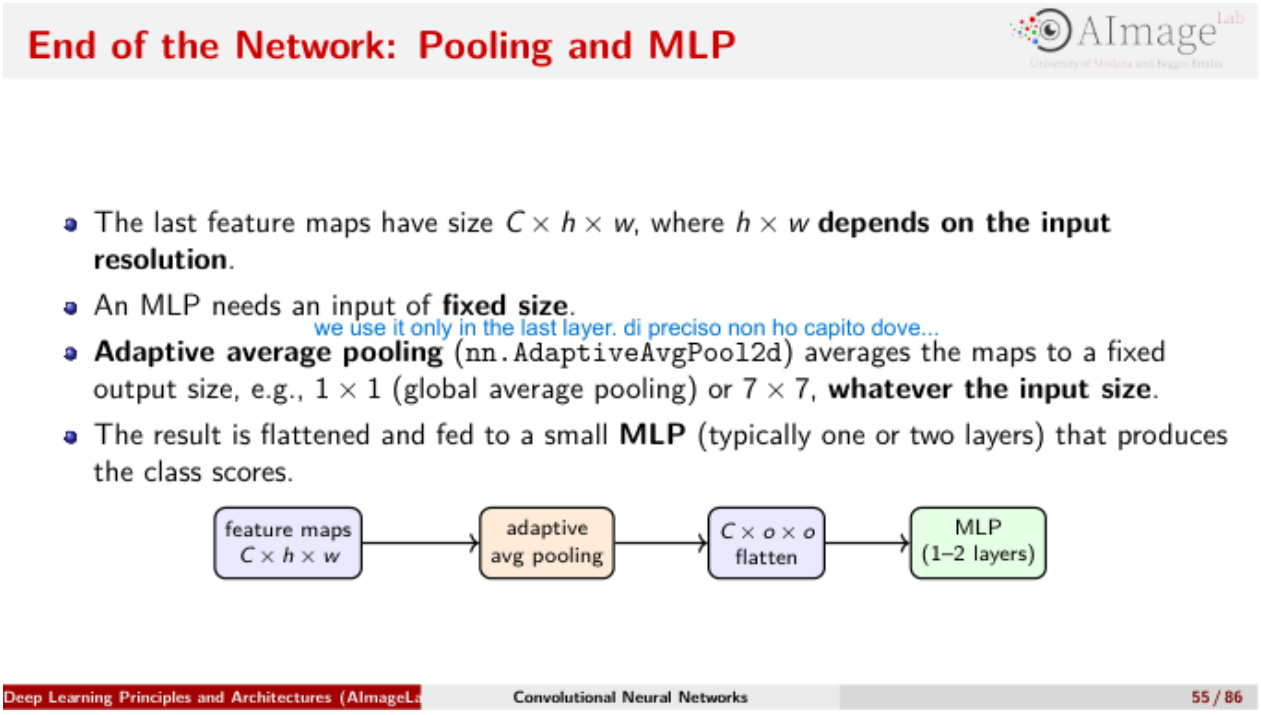


Figura 2.29: Slide 59 – Collegamento fra estrattore e MLP

Il pooling **adattivo** fissa la dimensione di uscita adattando le regioni di aggregazione all'ingresso. L'uscita 1×1 coincide con la media globale; una griglia fissa $o \times o$, come 7×7 nella slide, produce Co^2 ingressi per l'MLP. Non si elimina il flattening: lo si applica dopo aver fissato la forma. Questa collocazione è il disegno discusso, non un divieto matematico di usare pooling adattivo altrove.

Con $C = 16, q = 10$, il flattening diretto di 8×8 richiede $10(1024 + 1) = 10250$ parametri; la media globale richiede $10(16 + 1) = 170$. Si risparmiano pesi perdendo disposizione spaziale. Più canali possono arricchire la descrizione, ma non ricostruiscono automaticamente le posizioni eliminate.

2.14 LeNet e AlexNet: evoluzione delle CNN (slide 60–62)

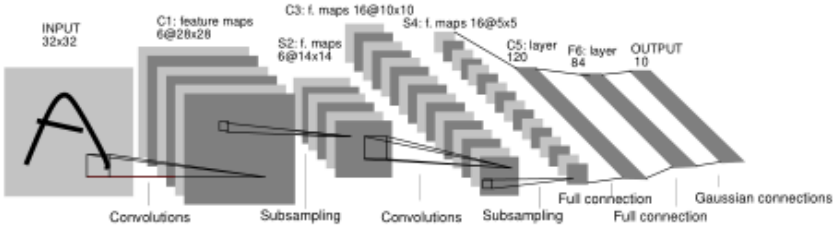
Le architetture condividono convoluzioni, attivazioni, riduzione spaziale e classificazione, ma differiscono per profondità, kernel e canali. Queste scelte determinano campo recettivo e costo.

2.14.1 LeNet: riconoscimento di cifre (slide 61)

LeNet mostra una pipeline completa per caratteri manoscritti: un'immagine a un canale 32×32 attraversa filtri 5×5 e sottocampionamenti. Il modello storico usa non linearità sigmoidi e sottocampionamento basato su medie; non va reinterpretato come una rete con ReLU e max pooling in ogni blocco.

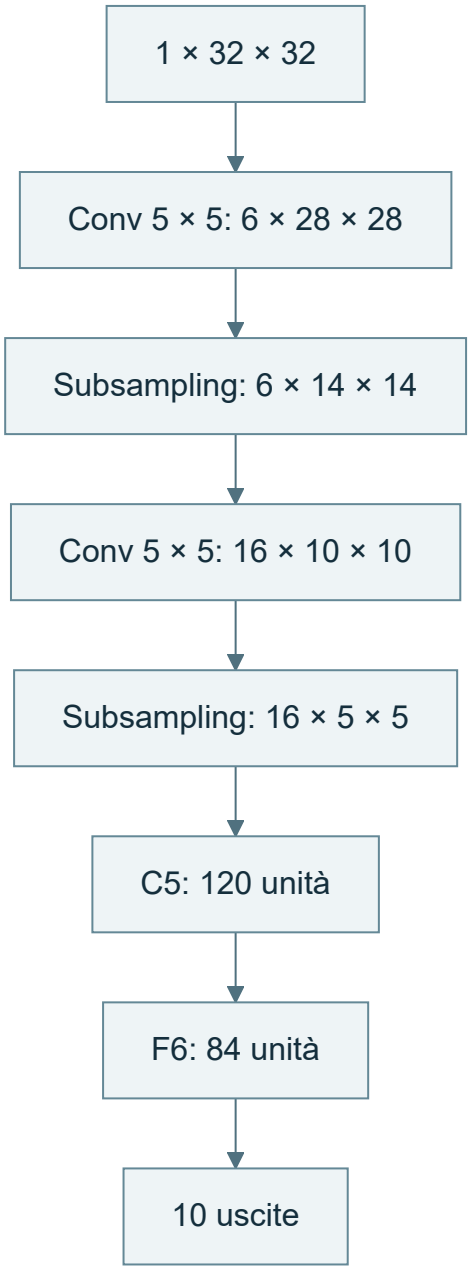
From the Origins... - LeNet

LeNet was introduced in [6]. It laid much of the groundwork for modern CNNs, demonstrating their efficacy for tasks like handwritten digit recognition.



- **Pioneering CNN:** Demonstrated a complete, **end-to-end** trainable CNN architecture.
- **Convolutional Layers:** Employed learnable filters (5x5 kernels) for feature extraction.
- **Subsampling Layers:** Utilized **average pooling** (2x2 windows with stride 2) to reduce spatial resolution.
- **Activation Functions:** Primarily used **Sigmoid** non-linearities.

Figura 2.30: Slide 61 – LeNet



Senza padding, $32 - 5 + 1 = 28$; il sottocampionamento dimezza a 14. La convoluzione successiva dà $14 - 5 + 1 = 10$ e il pooling porta a 5. Una finestra 5×5 sull'ultima mappa copre tutta la sua estensione e dà un valore per filtro. La figura riporta connessioni gaussiane all'uscita, non una moderna softmax. Il conteggio zero dei parametri del pooling puro non va esteso automaticamente a ogni variante storica di sottocampionamento.

2.14.2 AlexNet: scala computazionale e ReLU (slide 62)

AlexNet è associata al risultato ImageNet del 2012. Contiene **otto livelli con parametri**, cinque convoluzionali e tre fully connected; il conteggio non include separatamente ReLU e pooling. Utilizza un kernel iniziale 11×11, ReLU e addestramento su due GPU nella realizzazione illustrata.

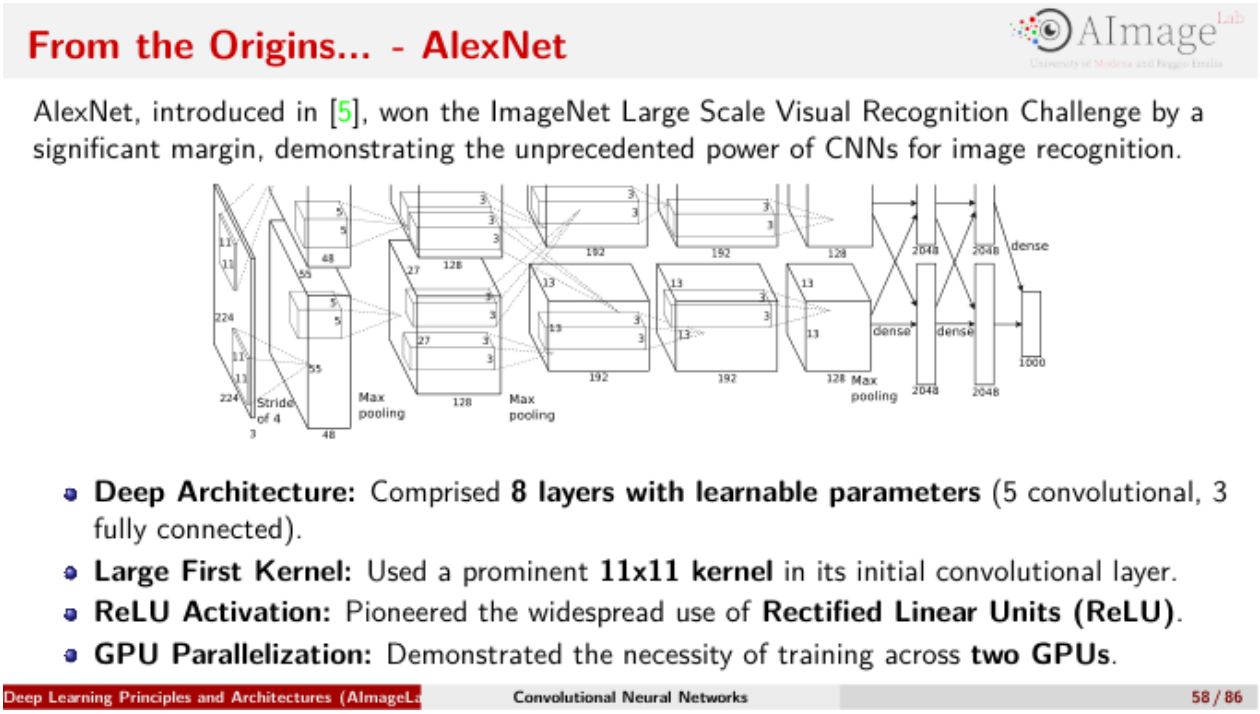


Figura 2.31: Slide 62 – AlexNet

Un kernel ampio aumenta rapidamente il campo recettivo quando ci sono pochi stadi convoluzionali, pagando più pesi. A canali fissati, passare da 3×3 a 11×11 moltiplica i pesi per $121/9 \approx 13.44$. Il kernel grande non riduce da solo la risoluzione di un fattore fissato: contano anche stride e padding.

La GPU ha reso praticabile l'addestramento su larga scala; la ReLU evita la saturazione sul semiasse positivo. Non ne segue che AlexNet sia la prima rete profonda in assoluto o che due GPU siano necessarie per ogni CNN. Le indicazioni storiche esitanti nella trascrizione non vengono trasformate in fatti aggiuntivi.

2.15 VGG: distribuire la capacità lungo la profondità (slide 63–68)

VGG, dal nome **Visual Geometry Group**, usa sequenze regolari di piccole convoluzioni 3×3. L'elaborazione è distribuita lungo la profondità, costruendo gradualmente rappresentazioni più ampie. I pesi preaddestrati hanno inoltre favorito il trasferimento delle rappresentazioni fra compiti.

2.15.1 Piccoli kernel e campo recettivo (slide 63–64)

A stride 1 e senza dilatazione, ogni convoluzione 3×3 aggiunge due posizioni per asse:

$$\boxed{r_L = 1 + \sum_{\ell=1}^{L} (3-1) = 1 + 2L.}$$

Due livelli hanno campo recettivo 5×5, tre 7×7. Non sono però equivalenti a un solo filtro grande: le attivazioni intermedie introducono ulteriori non linearità.

Supponendo C **canali in ingresso, in uscita e in tutti gli strati intermedi**, ogni coppia di canali possiede K^2 pesi. Pertanto:

$$\boxed{\begin{aligned} P_{7\times7} &= 49C^2, \quad P_{3\times(3\times3)} = 27C^2, \\ P_{5\times5} &= 25C^2, \quad P_{2\times(3\times3)} = 18C^2. \end{aligned}}$$

Nel primo confronto il risparmio è $22/49 \approx 44.90\%$. Con $C = 64$, si passa da 200704 a 110592 pesi. Includendo i bias, sono 200768 contro 110784, perché tre strati hanno tre vettori di bias. La sequenza offre tre ReLU invece di una. Con canali intermedi diversi il conteggio va rifatto: il risparmio non vale per qualunque architettura.

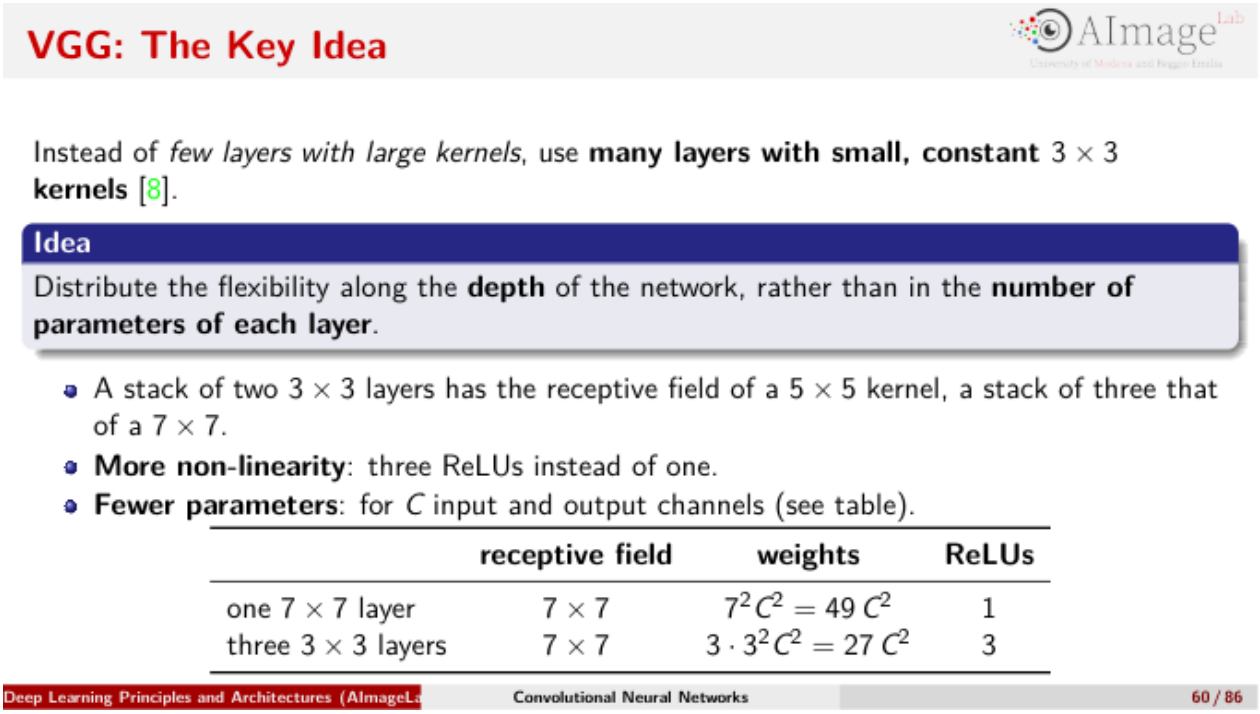


Figura 2.32: Slide 64 – Kernel grande e sequenza di kernel piccoli

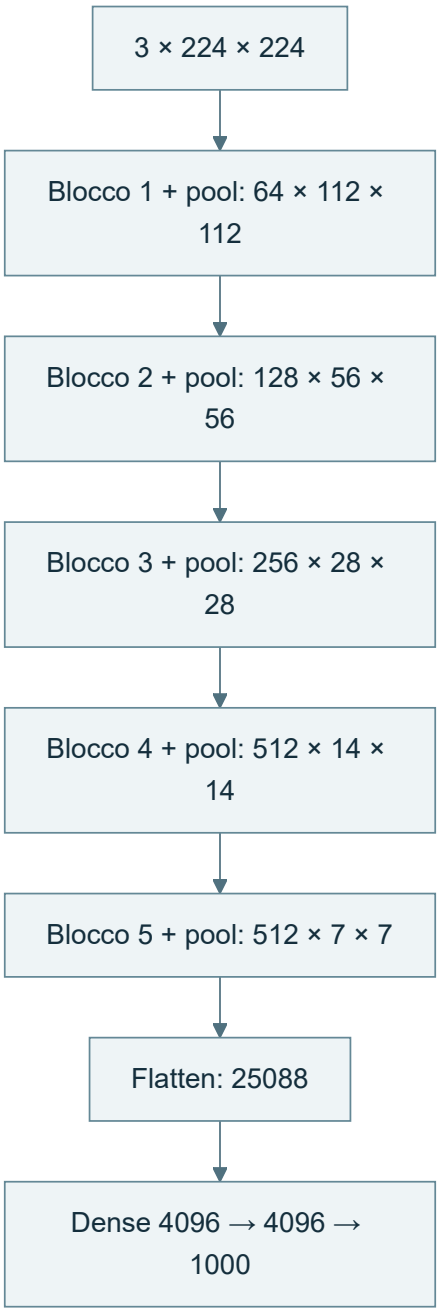
Una costruzione graduale lascia spazio alla condivisione di feature fra classi: bordi e texture possono contribuire a categorie diverse prima di essere combinati in parti specifiche. È una motivazione progettuale, non una garanzia che aggiungere livelli migliori sempre l'accuratezza.

2.15.2 VGG16: percorso delle dimensioni (slide 65–66)

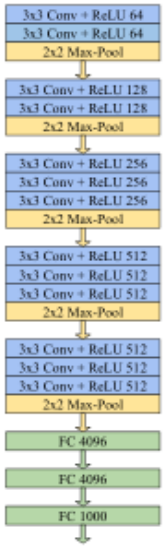
VGG16 contiene 13 convoluzioni e 3 livelli densi: **16 livelli con pesi**. Le convoluzioni 3×3 hanno stride 1 e padding 1, mentre cinque max pooling 2×2 a stride 2 dimezzano progressivamente gli assi dell'ingresso RGB 224×224.

Blocco	Successione dei canali	Forma dopo le convoluzioni, senza batch	Forma dopo il pooling
1	$3 \to 64 \to 64$	$64 \times 224 \times 224$	$64 \times 112 \times 112$
2	$64 \to 128 \to 128$	$128 \times 112 \times 112$	$128 \times 56 \times 56$
3	$128 \to 256 \to 256 \to 256$	$256 \times 56 \times 56$	$256 \times 28 \times 28$
4	$256 \to 512 \to 512 \to 512$	$512 \times 28 \times 28$	$512 \times 14 \times 14$
5	$512 \to 512 \to 512 \to 512$	$512 \times 14 \times 14$	$512 \times 7 \times 7$

Le forme derivano da $(H - 3 + 2) + 1 = H$ per le convoluzioni e $H/2$ per i pooling su dimensioni pari. La catena spaziale è $224 \to 112 \to 56 \to 28 \to 14 \to 7$. Il flattening produce $512 \cdot 7 \cdot 7 = 25088$ ingressi per i livelli $4096 \to 4096 \to 1000$.

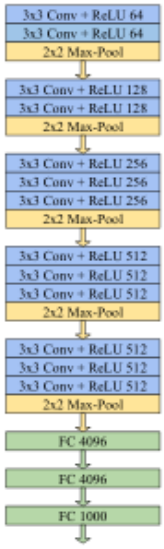


VGG16 Architecture Details (I)



- **Input:** Fixed 224x224 RGB images, mean-subtracted during pre-processing.
- **Convolutional Filters:** Uses consistent 3x3 receptive fields with stride 1. Stacking these small filters effectively captures complex patterns and increases non-linearity.
- **ReLU Activation:** Applied after every convolutional and fully connected layer.

VGG16 Architecture Details (II)



- **Spatial Pooling:** Five max-pooling layers, each performed over a 2x2 pixel window with stride 2, for downsampling feature maps.
- **Fully Connected Layers:** Three layers. The first two have 4096 neurons; the final layer has 1000 neurons (for ImageNet classes) followed by a Softmax activation for classification.

La sottrazione della media all'ingresso è pre-processamento, distinto dalla Batch Normalization. Le ReLU seguono convoluzioni e livelli densi nascosti; gli score finali alimentano la softmax. L'espressione «dopo ogni fully connected» della slide 65 va letta insieme alla slide 66, che distingue l'uscita di classificazione.

2.15.3 Dove sono i 138 milioni di parametri? (slide 67–68)

Un livello denso ha un peso per ogni coppia ingresso–uscita e un bias per uscita:

$$\boxed{P_{\text{dense}} = d_{\text{out}} d_{\text{in}} + d_{\text{out}}.}$$

La domanda della slide 67 riguarda il **primo fully connected**, che riceve tutti i 25088 elementi:

$$
\begin{aligned}
P_{\text{FC1}} &= 25088 \cdot 4096 + 4096 = 102764544, \\
P_{\text{FC2}} &= 4096 \cdot 4096 + 4096 = 16781312, \\
P_{\text{FC3}} &= 4096 \cdot 1000 + 1000 = 4097000, \\
P_{\text{FC}} &= 123642856.
\end{aligned}
$$

Per le convoluzioni si applica $P = C_{\text{out}}(9C_{\text{in}} + 1)$:

Blocco	Calcolo, bias inclusi	Totale
1	$64(9 \cdot 3 + 1) + 64(9 \cdot 64 + 1)$	38720
2	$128(9 \cdot 64 + 1) + 128(9 \cdot 128 + 1)$	221440
3	$256(9 \cdot 128 + 1) + 2 \cdot 256(9 \cdot 256 + 1)$	1475328
4	$512(9 \cdot 256 + 1) + 2 \cdot 512(9 \cdot 512 + 1)$	5899776
5	$3 \cdot 512(9 \cdot 512 + 1)$	7079424
Totale convoluzioni	somma dei cinque blocchi	**14714688**

Il totale è $14714688 + 123642856 = 138357544$, circa 138 milioni. La parte densa contiene l'89.36% dei parametri. Le cifre della slide 68 sono arrotondate in milioni e rappresentano soprattutto i pesi, perciò differiscono leggermente dai conti con bias. La percentuale dei pesi non coincide con quella del tempo di calcolo: una convoluzione riutilizza ciascun peso in molte posizioni.

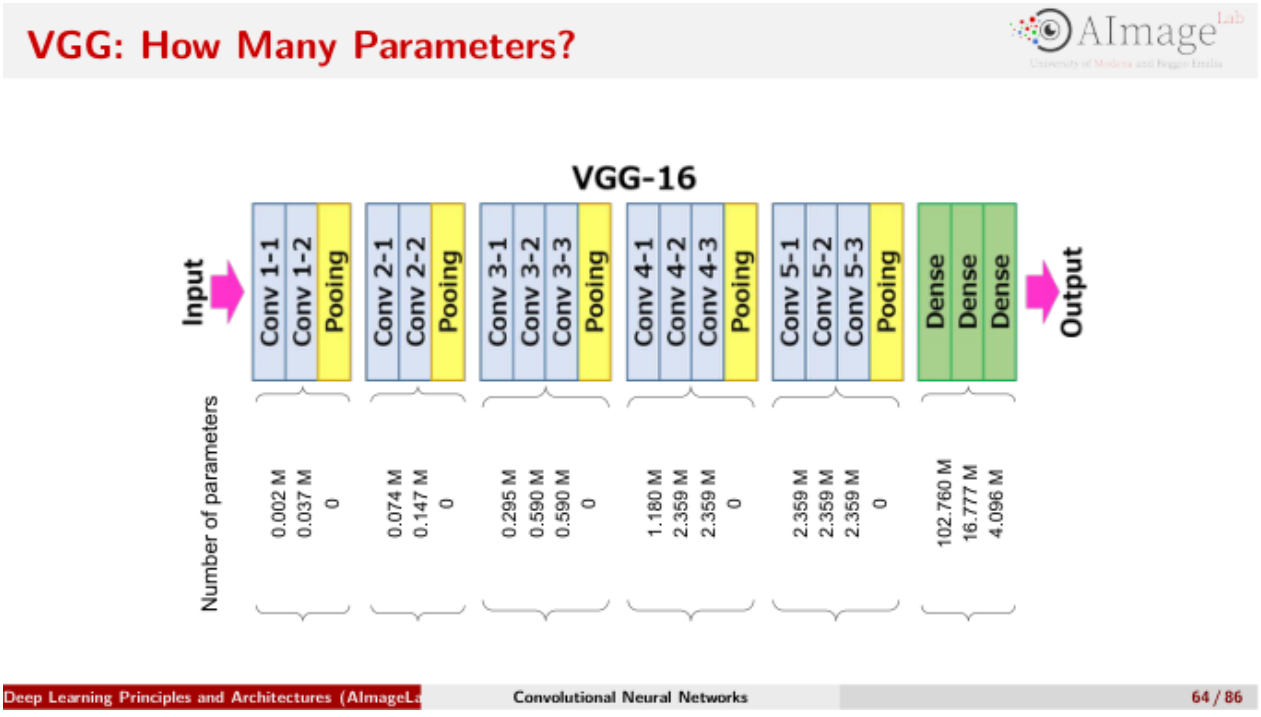


Figura 2.33: Slide 68 – Distribuzione dei parametri VGG16

2.16 Profondità e residual learning: ResNet (slide 69–75)

2.16.1 Il problema della degradazione (slide 69–72)

Aggiungere livelli a una rete tradizionale può peggiorare **sia training sia test**. Questo caso non è spiegato semplicemente dall'overfitting: una rete che memorizza meglio il training avrebbe tipicamente un errore di training inferiore; qui peggiora già l'ottimizzazione sui dati noti.

In linea di principio la rete più profonda potrebbe riprodurre quella corta facendo calcolare l'identità ai livelli aggiuntivi. La difficoltà è trovare quella configurazione durante l'addestramento: maggiore capacità non implica automaticamente maggiore facilità di ottimizzazione.

VGG Limits: Findings

Beyond a certain point, simply stacking more layers in traditional CNNs leads to performance degradation, not improvement:

- This is not due to overfitting: deeper models show higher training and test error.

ResNet: The Degradation Problem

Observation ([3], Introduction): when the depth of a plain network increases, accuracy gets saturated and then **degrades**.

- It is **not overfitting**: the deeper network has higher *training* error too (e.g., 56 vs. 20 layers on CIFAR-10).
- Yet a deeper model could be **at least as good**: take a shallower model and add layers that compute the **identity**.
- In practice, the solvers are not able to find solutions that are as good or better.

Idea

Give the network the tools to **skip** some of the layers: if the identity is optimal, it is easier to just *do nothing*.

Per la catena scalare $h_\ell = f_\ell(h_{\ell-1})$, applicando ripetutamente la regola della catena:

$$\boxed{\frac{\partial \mathcal{L}}{\partial h_0} = \frac{\partial \mathcal{L}}{\partial h_L} \prod_{\ell=1}^{L} f'_\ell(h_{\ell-1}).}$$

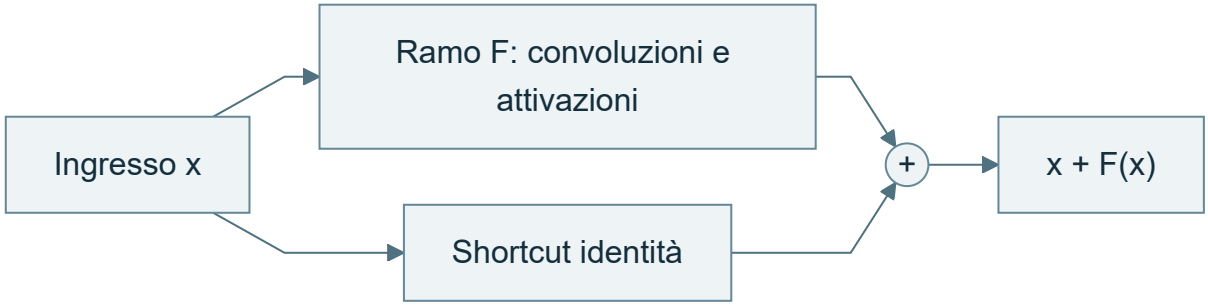
Ogni passo introduce un fattore: $\partial \mathcal{L}/\partial h_{\ell-1} = (\partial \mathcal{L}/\partial h_\ell) f'_\ell$. Venti fattori 0.5 danno $9.54 \cdot 10^{-7}$; venti fattori 2 danno 1048576. Sono esempi di gradiente che svanisce o esplode. Nel caso vettoriale compaiono prodotti di Jacobiane. L'instabilità è una difficoltà importante, ma non dimostra di esaurire ogni causa della degradazione.

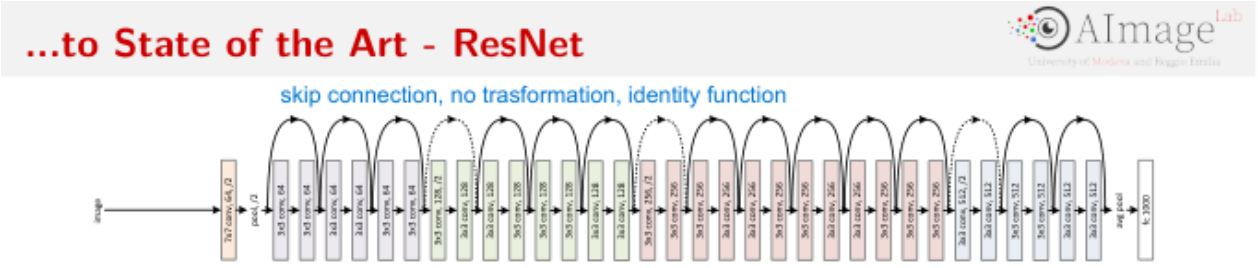
2.16.2 Blocco residuo e significato della shortcut (slide 71–74)

Invece di apprendere direttamente $H(x)$, si apprende la correzione rispetto all'ingresso:

$$\boxed{F(x;\theta) = H(x) - x, \qquad H(x) = x + F(x;\theta).}$$

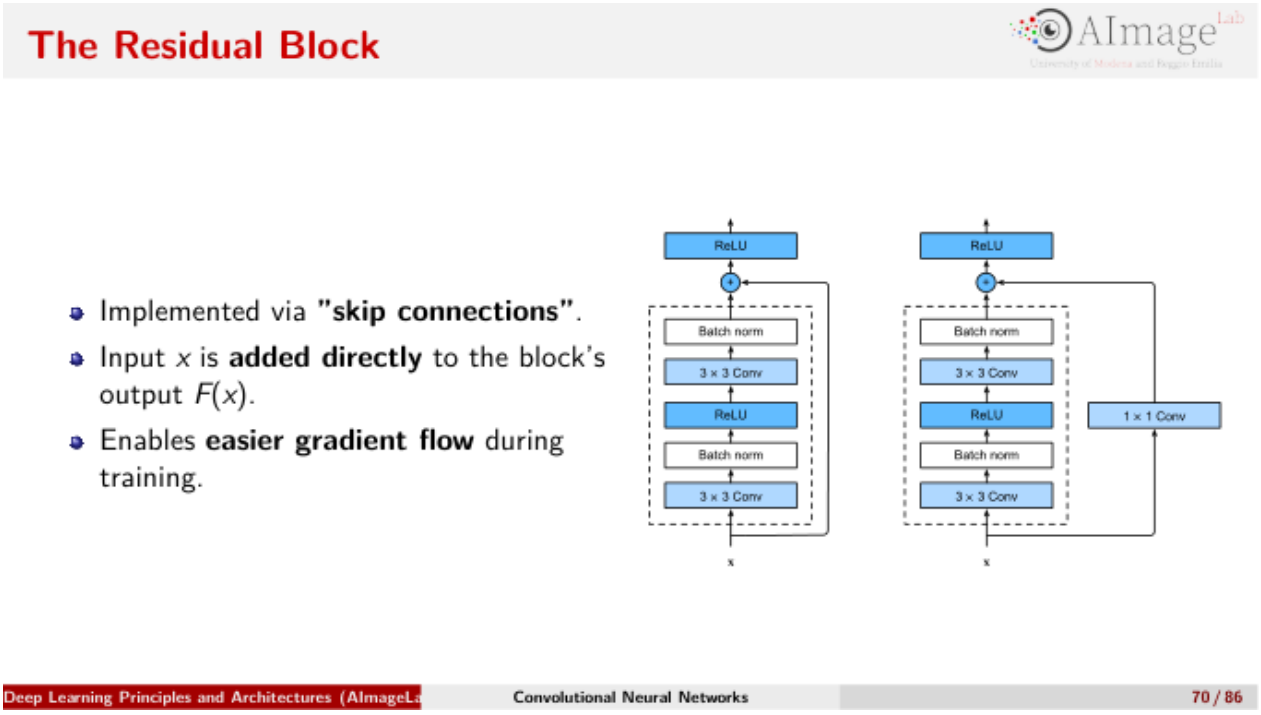
La relazione si ricava isolando la differenza fra uscita e ingresso. Per $x \in \mathbb{R}^{N\times C\times H\times W}$, il ramo F deve avere la stessa forma: la somma è elemento per elemento. Per rappresentare l'identità basta porre $F(x) = 0$.





Context: Introduced in [3]. Winner of ImageNet Large Scale Visual Recognition Challenge (ILSVRC 2015) and COCO 2015 (Detection, Segmentation, Localization).

- **Input**: 224 × 224 RGB images.
- **Structure**: Introduced ultra-deep architectures, with common versions like ResNet-50, ResNet-101, and ResNet-152 (up to 152 layers, far deeper than VGG).
- **Output**: 1000 classes (ImageNet).



Entrambi i rami sono calcolati e poi sommati. Non si evita fisicamente l'esecuzione di un ramo, non si confrontano i risultati per scegliere il migliore e non si scarta il ramo meno utile. «Ignorare il blocco» significa che il suo contributo appreso può essere vicino a zero, pur rimanendo il costo del calcolo.

Con $x = [2, -1]^T$ e $F(x) = 0.25x$, il ramo produce $[0.5, -0.25]^T$ e la somma è $[2.5, -1.25]^T$. Se il coefficiente diventa zero, l'uscita è x. La stessa funzione può produrre correzioni diverse per input diversi senza un selettore esterno di percorsi.

L'annotazione «nessuna trasformazione, identità» vale quando le forme coincidono. La figura 74 mostra anche una convoluzione 1×1 sulla shortcut, necessaria quando cambiano canali o risoluzione. In tal caso

$$\boxed{H(x) = P(x) + F(x)}$$

con $P(x)$ e $F(x)$ della stessa forma, ma P non più identità. Per $64 \times 8 \times 8 \to 128 \times 4 \times 4$, una convoluzione 1×1 a stride 2 può adattare la shortcut. Ha $128 \cdot 64 = 8192$ pesi, oppure $128(64+1) = 8320$ parametri includendo il bias. È il vincolo di compatibilità illustrato dalla figura, non un nuovo tipo di architettura.

2.16.3 Backpropagation: i gradienti si sommano (slide 73–74)

Rappresentando il tensore come vettore $x \in \mathbb{R}^d$, siano $J_F \in \mathbb{R}^{d \times d}$ la Jacobiana e $g = \nabla_H \mathcal{L} \in \mathbb{R}^d$ il gradiente in arrivo. Si deriva la somma componente per componente:

$$\frac{\partial H_i}{\partial x_j} = \delta_{ij} + \frac{\partial F_i}{\partial x_j},$$
$$\frac{\partial \mathcal{L}}{\partial x_j} = \sum_i \frac{\partial \mathcal{L}}{\partial H_i} \left(\delta_{ij} + \frac{\partial F_i}{\partial x_j} \right),$$
$$\boxed{\nabla_x \mathcal{L}} = \boxed{g + J_F^T g.}$$

Il termine g attraversa l'identità senza moltiplicazioni per pesi o attivazioni interne a F; $J_F^T g$ attraversa il ramo trasformato. **Non si sceglie il gradiente più grande: si sommano tutti i contributi.** Questo risponde al dubbio sui percorsi multipli nel backward.

Con $F(x) = 0.25x$ e $g = [3, -2]^T$, i contributi sono $[3, -2]^T$ e $[0.75, -0.5]^T$: il risultato è $[3.75, -2.5]^T$. Con $F = 0$ e Jacobiana nulla, ritorna g.

L'identità facilita la propagazione, ma non conserva necessariamente la norma del gradiente **totale**: i contributi possono amplificarsi o cancellarsi. Con $J_F = -I$ la somma sarebbe zero. La figura contiene inoltre una ReLU dopo la somma: se $y = \text{ReLU}(H)$, prima si calcola $g = \nabla_y \mathcal{L} \odot \mathbf{1}[H > 0]$, poi si applica la formula. Con shortcut proiettata, il termine diretto è invece $J_P^T g$.

2.16.4 Conseguenze architetturali (slide 75)

Le connessioni residue rendono più praticabile l'ottimizzazione di reti profonde, fra cui ResNet-50, ResNet-101 e ResNet-152. La novità è il grafo delle connessioni: l'informazione può raggiungere un livello oltrepassando più trasformazioni. Il principio è riutilizzato in altre famiglie, senza svilupparle qui.

La possibilità di una correzione nulla non garantisce un miglioramento delle prestazioni a ogni livello aggiunto. I riferimenti orali a ipotetiche «VGG34» e «VGG100» illustrano l'aumento di profondità, non definiscono nuove varianti canoniche da aggiungere al programma.

2.17 Batch Normalization: statistiche e trasformazione appresa (slide 76–86)

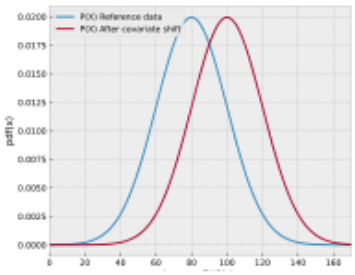
2.17.1 Perché normalizzare fra due livelli (slide 76–80)

Aggiornare un livello modifica le sue attivazioni, cioè gli ingressi dei livelli successivi. Un livello che riceveva valori soprattutto fra 0.5 e 2 può trovarsi davanti a una scala diversa dopo gli aggiornamenti a monte. La variazione delle distribuzioni interne durante il training è descritta nelle slide come **internal covariate shift**.

Si introduce quindi una trasformazione che controlli posizione e scala delle attivazioni. Non esistono due aggiornamenti distinti, uno «per la loss» e uno «per inseguire il livello precedente»: rimane un unico gradiente della funzione obiettivo. L'analogia descrive l'accoppiamento fra trasformazioni apprese.

What Is Internal Covariate Shift?

- **Internal Covariate Shift**: the distribution of a layer's inputs changes during training, because the parameters of the preceding layers change.
- Small changes in earlier layers get amplified as they propagate through the network.
- Training slows down: each layer chases a moving target.



Question

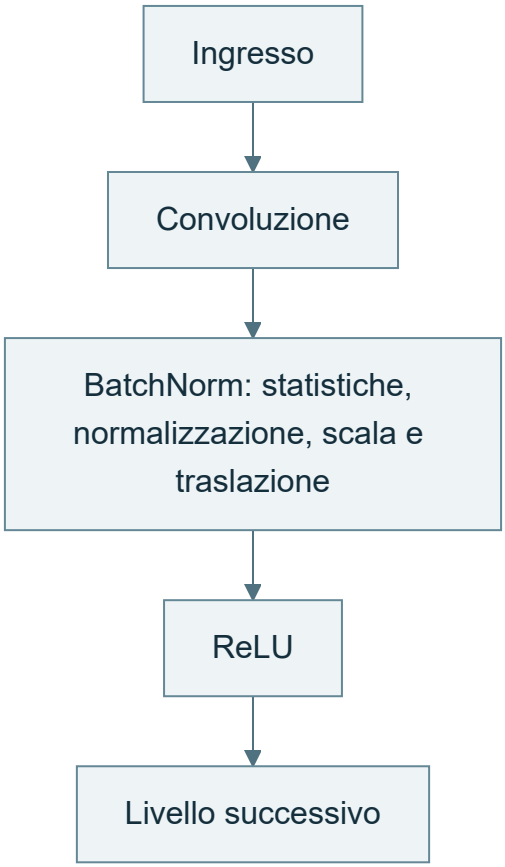
Can we **decouple** the activity of each layer from the distributions produced by the previous ones?

Batch Normalization: The Idea

- **Fix the distribution** of each layer's inputs: zero mean and unit variance, computed on the current mini-batch.
- Each layer then sees **stable inputs**, whatever the previous layers do.
- Then a learnable **scale and shift** restore the expressiveness of the layer.
- Normalization is a **differentiable layer**, trained with backpropagation, usually placed between the convolution and the activation.



La **Batch Normalization** standardizza con statistiche del mini-batch, poi applica scala e traslazione apprendibili. Nel blocco illustrato l'ordine è convoluzione → BatchNorm → ReLU. La stabilità delle distribuzioni è una motivazione: la trasformazione non rende immutabile l'intera distribuzione, indipendenti i livelli o gaussiani i valori.



2.17.2 Formule e assi di aggregazione (slide 81–84)

Sia $X \in \mathbb{R}^{N \times C \times H \times W}$. Nelle CNN le statistiche sono calcolate **per canale**, aggregando batch e posizioni spaziali. Per ogni canale ci sono $M = NHW$ valori. Media e varianza sono vettori in $\mathbb{R}^C$, non una sola statistica per l'intero volume.

$$
\begin{aligned}
\mu_{B,c} &= \frac{1}{M} \sum_{n=1}^{N} \sum_{h=1}^{H} \sum_{w=1}^{W} X_{n,c,h,w}, \\
v_{B,c} &= \frac{1}{M} \sum_{n=1}^{N} \sum_{h=1}^{H} \sum_{w=1}^{W} (X_{n,c,h,w} - \mu_{B,c})^2, \\
\widehat{X}_{n,c,h,w} &= \frac{X_{n,c,h,w} - \mu_{B,c}}{\sqrt{v_{B,c} + \epsilon}}, \\
Y_{n,c,h,w} &= \gamma_c \widehat{X}_{n,c,h,w} + \beta_c.
\end{aligned}
$$

$\epsilon > 0$ è una costante di stabilità; $\gamma, \beta \in \mathbb{R}^C$ sono apprendibili. $X, \widehat{X}, Y$ hanno la stessa forma. Le statistiche e i parametri vengono applicati mediante broadcasting a tutte le posizioni del rispettivo canale. Per un input denso $N \times d$ si considera separatamente ogni feature, aggregando i N esempi.

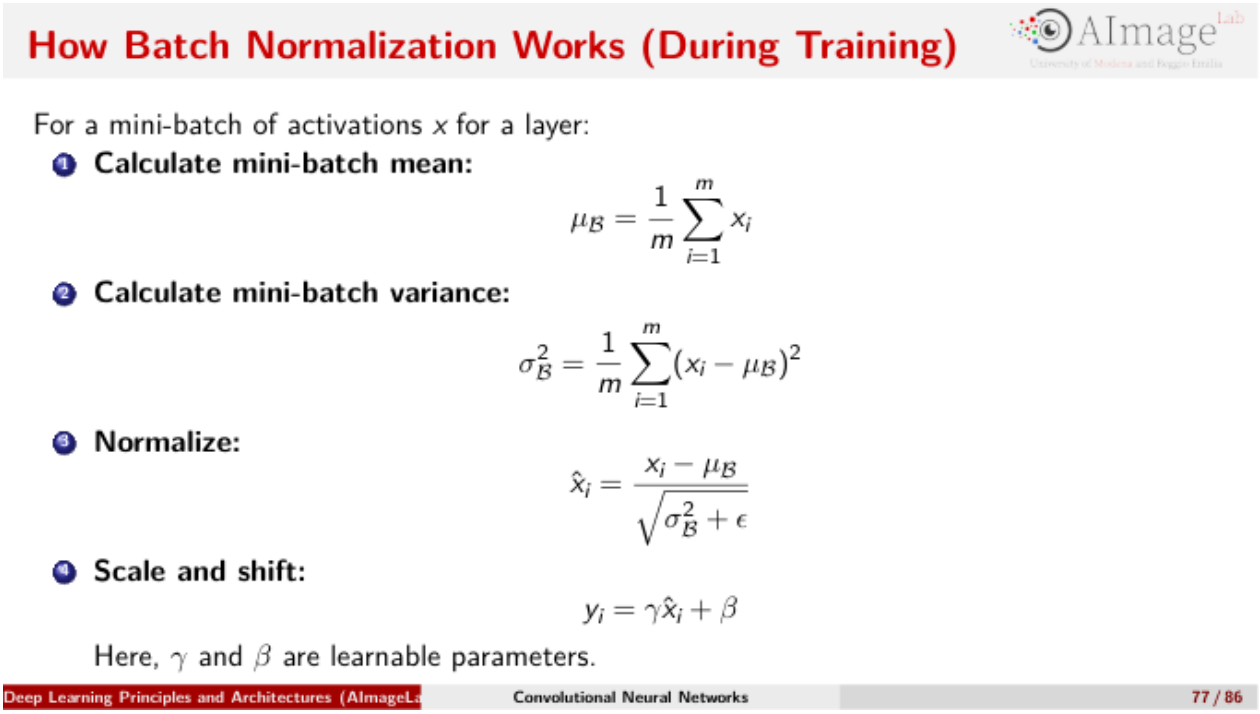


Figura 2.34: Slide 84 – Quattro passaggi della Batch Normalization

La media è il centro aritmetico; sottrarla produce scarti con somma zero. La varianza è la media dei quadrati degli scarti e ha unità al quadrato. Per questo si divide per **la radice della varianza**, cioè la deviazione standard stabilizzata, non per la varianza.

Indicizzando i valori di un canale con a:

$$\frac{1}{M}\sum_a (X_a - \mu_B) = \frac{1}{M}\sum_a X_a - \mu_B = 0,$$
$$\frac{1}{M}\sum_a \widehat{X}_a = \frac{0}{\sqrt{v_B + \epsilon}} = 0,$$
$$\frac{1}{M}\sum_a \widehat{X}_a^2 = \frac{\frac{1}{M}\sum_a (X_a - \mu_B)^2}{v_B + \epsilon} = \frac{v_B}{v_B + \epsilon}.$$

La media normalizzata è zero; la varianza è vicina a 1 per $v_B \gg \epsilon$, non esattamente 1. Se tutti i valori coincidono, $v_B = 0$ e le uscite normalizzate sono zero. Controllare questi due momenti non azzera le covarianze fra canali e non rende gaussiana la distribuzione.

2.17.3 Esempio numerico completo (slide 81–85)

Si prendano $N = 2, C = 1, H = 1, W = 2$, quindi $M = 4$: le due mappe sono $[1, 3]$ e $[5, 7]$.

$$\mu_B = (1 + 3 + 5 + 7)/4 = 4,$$
$$v_B = \frac{(1-4)^2 + (3-4)^2 + (5-4)^2 + (7-4)^2}{4}$$
$$= (9 + 1 + 1 + 9)/4 = 5.$$

Con $\epsilon = 10^{-5}$, il denominatore è $\sqrt{5.00001} \approx 2.23607$ e

$$\widehat{X} \approx (-1.34164, -0.44721, 0.44721, 1.34164).$$

La varianza vale $5/5.00001 \approx 0.999998$. Con $\gamma = 2, \beta = 1$:

$$\boxed{Y \approx (-1.68328, 0.10557, 1.89443, 3.68328).}$$

La forma rimane $2 \times 1 \times 1 \times 2$. La media finale è 1, la varianza circa 4: la trasformazione affine non mantiene obbligatoriamente media zero e varianza unitaria.

2.17.4 Perché apprendere gamma e beta? (slide 85)

Una scala imposta permanentemente può limitare la rappresentazione. La normalizzazione fornisce una base controllata; γ_c e β_c scelgono scala e offset utili al compito.

$$\boxed{\text{media}_B(Y_c) = \beta_c, \qquad \text{var}_B(Y_c) = \gamma_c^2 \frac{v_{B,c}}{v_{B,c} + \epsilon}.}$$

La prima relazione segue dalla linearità della media. La seconda deriva dal fatto che la traslazione non modifica gli scarti, mentre la moltiplicazione per γ ne moltiplica i quadrati per γ^2. L'esempio con $\gamma = 2, \beta = 1$ verifica entrambe.

A statistiche fissate, scegliere $\gamma = \sqrt{v_B + \epsilon}$ e $\beta = \mu_B$ ricostruisce l'ingresso:

$$\gamma \frac{x - \mu_B}{\sqrt{v_B + \epsilon}} + \beta = (x - \mu_B) + \mu_B = x.$$

Questo mostra la capacità di ripristinare la scala originale; due parametri fissi non invertono però esattamente ogni mini-batch se le sue statistiche cambiano.

Il numero dei parametri è

$$\boxed{P_{\text{BN}} = C + C = 2C.}$$

Con 64 canali si apprendono 128 scalari. Le statistiche mobili sono **stato del livello**, non parametri ottimizzati tramite gradient descent. Ponendo $g_a = \partial \mathcal{L} / \partial Y_a$, la catena applicata a $Y_a = \gamma \widehat{X}_a + \beta$ dà

$$\boxed{\frac{\partial \mathcal{L}}{\partial \beta} = \sum_a g_a, \qquad \frac{\partial \mathcal{L}}{\partial \gamma} = \sum_a g_a \widehat{X}_a.}$$

Si sommano i contributi perché gli stessi parametri sono condivisi dalle posizioni del canale. Con $g = (1, 0, 0, 0)$ nell'esempio, i gradienti sono 1 e circa -1.34164.

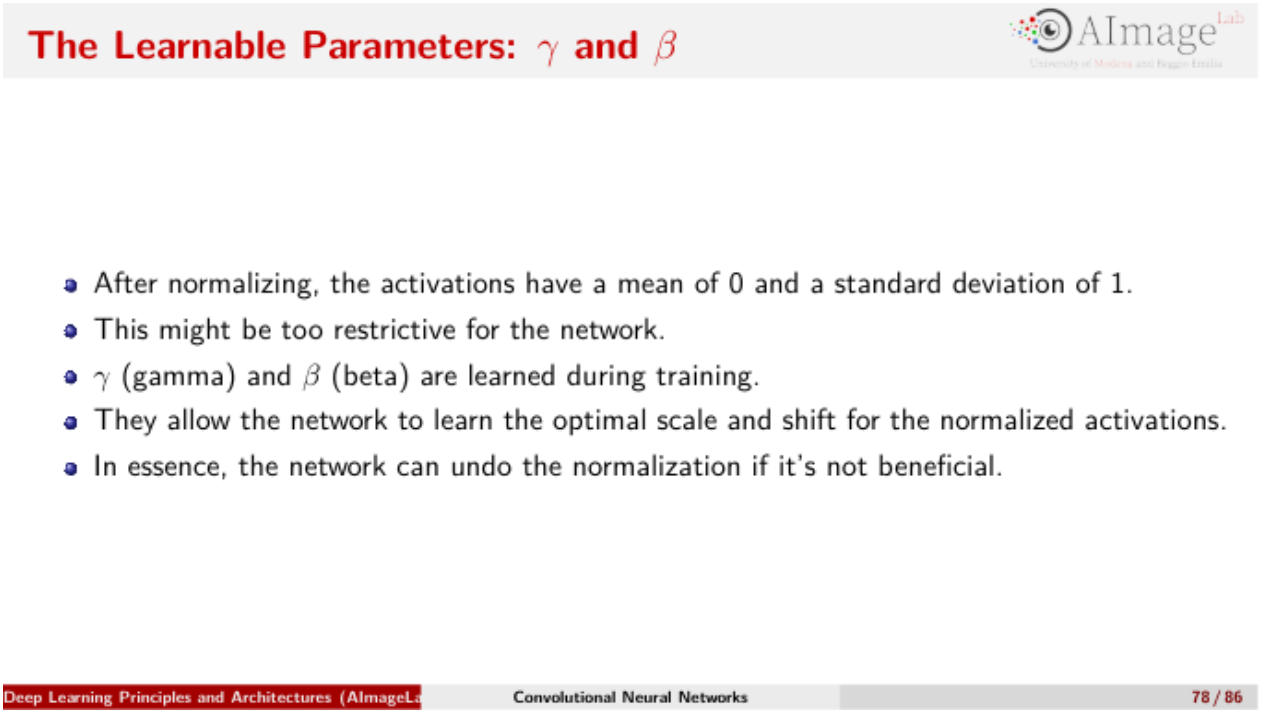


Figura 2.35: Slide 85 – Parametri apprendibili gamma e beta

2.17.5 Inferenza con statistiche memorizzate (slide 86)

In training è disponibile un mini-batch; in inferenza può arrivare una sola immagine o un elemento di un flusso. Ricalcolare le statistiche renderebbe inoltre la predizione dipendente dagli altri esempi presenti nel batch di valutazione. Si conservano quindi statistiche durante il training e si riutilizzano dopo.

Con medie e varianze memorizzate $\bar{\mu}, \bar{v} \in \mathbb{R}^C$:

$$\boxed{Y_{n,c,h,w} = \gamma_c \frac{X_{n,c,h,w} - \bar{\mu}_c}{\sqrt{\bar{v}_c + \epsilon}} + \beta_c.}$$

Si sostituiscono le statistiche correnti con quelle fissate dopo il training. A modello e ingresso fissati, questo livello non dipende dalla composizione del batch di inferenza. Con $x = 7$, $\bar{\mu} = 4$, $\bar{v} = 5$, $\gamma = 2$, $\beta = 1$ ed $\epsilon = 10^{-5}$, l'uscita è circa 3.68328.

Una media mobile esponenziale può aggiornare lo stato secondo

$$\boxed{\begin{aligned} \bar{\mu}_{t+1} &= (1-\rho)\bar{\mu}_t + \rho\mu_{B,t}, \\ \bar{v}_{t+1} &= (1-\rho)\bar{v}_t + \rho\widetilde{v}_{B,t}. \end{aligned}}$$

$0 < \rho \leq 1$ pesa la nuova osservazione e $\widetilde{v}_B$ è la stima scelta per l'accumulatore. È una media pesata con coefficienti che sommano a 1. Con $\rho = 0.1$, media vecchia 2 e nuova osservazione 4 si ottiene $0.9 \cdot 2 + 0.1 \cdot 4 = 2.2$. Con varianza vecchia 3 e nuova stima 5 si ottiene 3.2. La convenzione esatta dipende dall'implementazione: una media di varianze non coincide con il quadrato della media delle deviazioni standard.

Un solo esempio non significa sempre un solo valore. Per un unico scalare la varianza empirica con divisore 1 è definita e vale zero; quella corretta con divisore $M-1$ non è definita per $M = 1$. Manca

comunque informazione sulla dispersione della popolazione. Nella BatchNorm convoluzionale si aggregano $M = NHW$ valori: anche con $N = 1$ possono esserci molti elementi spaziali. Usare statistiche fisse resta la procedura di inferenza discussa, ma non si giustifica affermando che ogni immagine singola abbia necessariamente varianza nulla.

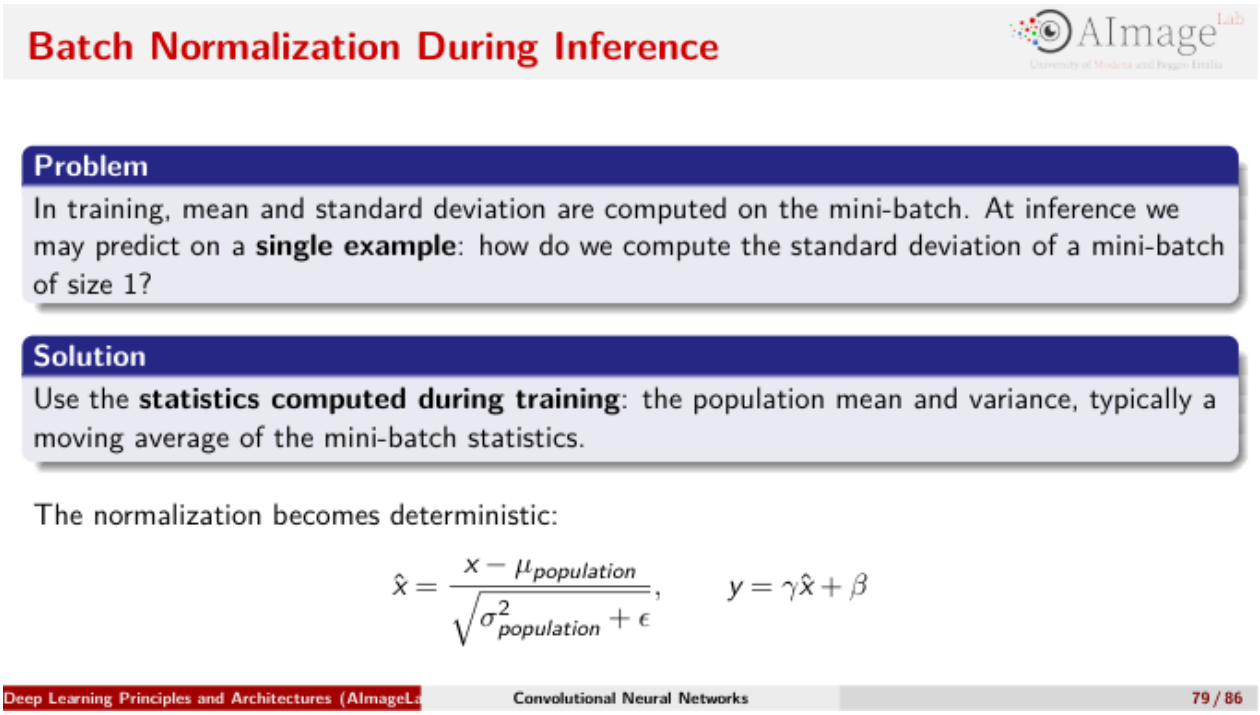


Figura 2.36: Slide 86 – Batch Normalization in inferenza

2.17.6 Modalità training ed evaluation (slide 86 e chiusura della trascrizione)

Livello	Training	Evaluation/inferenza
BatchNorm con statistiche mobili	Usa il batch corrente e aggiorna lo stato	Usa le statistiche memorizzate
Dropout ordinario	Campiona una maschera che azzera alcune attivazioni	Non elimina casualmente le attivazioni

In PyTorch si usano `net.train()` e `net.eval()`. `net.training` è lo stato booleano, non la chiamata per impostarlo. Queste modalità governano i livelli; `eval()` non disabilita da solo il calcolo dei gradienti. Impostare correttamente la modalità è quindi necessario per valutare il comportamento previsto del modello.

Stop-line verificata — 08/10/2026

La trattazione termina alla **pagina PDF 86 di `03_conv.pdf`**, "Batch Normalization During Inference", numerata **79/86** nel piè di pagina. La trascrizione si chiude con le statistiche di inferenza e il confronto BatchNorm/dropout nelle modalità training/eval. La successiva pagina PDF 87, "Benefits of Batch Normalization", non risulta spiegata nella registrazione e non viene sviluppata. Titoli e nomi delle immagini usano le pagine fisiche del PDF.

2.18 Riepilogo Formule Operative

Concetto	Formula Matematica	Significato e Vincoli
Tensore Immagine RGB	$X \in \mathbb{R}^{C \times H \times W}$	Standard PyTorch: $C = 3$ canali, H altezza, W larghezza
Operatore LSI Discreto	$o(x, y) = \sum_m \sum_n i(m, n)K(x - m, y - n)$	Convoluzione 2D con risposta all'impulso K
Mappa di Feature Multicanale	$F(i, j) = b + \sum_{c=1}^{C_{in}} \sum_m \sum_n I(i + m, j + n, c)K_c(m, n)$	Somma attraverso tutti i C_{in} canali di ingresso più bias scalare b
Dimensioni Spaziali Uscita	$H' = \lfloor \frac{H-K+2P}{S} \rfloor + 1,\ W' = \lfloor \frac{W-K+2P}{S} \rfloor + 1$	K: kernel, P: padding, S: stride, $\lfloor \cdot \rfloor$: floor
Padding per Dimensione Costante ("same")	$P = \frac{K-1}{2}$	Valido per kernel dispari K con stride unitario $S = 1$
Parametri Apprendibili Strato Conv	Parametri $= C_{out} \cdot (K^2 \cdot C_{in} + 1)$	Completamente indipendente dalla risoluzione spaziale $H \times W$
Attivazione ReLU	$f(x) = \max(0, x)$	Gradiente 1 per $x > 0$; gradiente 0 per $x < 0$ (rischio dying ReLU)
Attivazione Leaky ReLU	$f(x) = \max(x, \alpha x)$	Pendenza $\alpha \approx 0.01$ per $x < 0$ (mantiene vivo il gradiente)
Attivazione ELU	$f(x) = x$ se $x > 0,\ \alpha(e^x - 1)$ se $x \leq 0$	Asintoto a $-\alpha$ per $x \to -\infty$; media attivazioni vicina a 0
Pooling: forma, senza padding	$H' = \lfloor (H - K)/S \rfloor + 1,$ $W' = \lfloor (W - K)/S \rfloor + 1$	N, C invariati; dilatazione 1, finestre compatibili
Max pooling	$Y_{n,c,i,j} = \max_{u,v} X_{n,c,iS+u,jS+v}$	$0 \leq u, v < K$; zero parametri apprendibili
Average pooling	$Y_{n,c,i,j} = K^{-2} \sum_{u,v} X_{n,c,iS+u,jS+v}$	Media sulla finestra; zero parametri apprendibili
Global average pooling	$z_{n,c} = (hw)^{-1} \sum_{i,j} X_{n,c,i,j}$	$N \times C \times h \times w \to N \times C \times 1 \times 1$
Pool adattivo a griglia fissa	$N \times C \times h \times w \to N \times C \times o \times o$	Dopo flatten: Co^2 ingressi per esempio
Campo recettivo	$r_\ell = r_{\ell-1} + (k_\ell - 1)j_{\ell-1},$ $j_\ell = s_\ell j_{\ell-1}$	$r_0 = j_0 = 1$, senza dilatazione
Parametri dense	$d_{\text{out}}(d_{\text{in}} + 1)$	Bias incluso
Blocco residuo	$H = x + F(x)$	Stesse forme; entrambi i rami calcolati
Gradiente residuo	$\nabla_x \mathcal{L} = g + J_F^T g$	Somma, non selezione, dei contributi
BN: media e varianza	$\mu_{B,c} = M^{-1} \sum_a X_{a,c},$ $v_{B,c} = M^{-1} \sum_a (X_{a,c} - \mu_{B,c})^2$	$M = NHW$, una coppia per canale
BN: training	$Y_c = \gamma_c(X_c - \mu_{B,c})/\sqrt{v_{B,c} + \epsilon} + \beta_c$	Forma invariata; $2C$ parametri apprendibili
BN: inferenza	$Y_c = \gamma_c(X_c - \bar{\mu}_c)/\sqrt{\bar{v}_c + \epsilon} + \beta_c$	Statistiche memorizzate; nessun nuovo calcolo sul batch

2.19 Errata e precisazioni principali

Riferimento	Precisazione
Slide 25	Linearità significa sovrapposizione dei segnali, non movimento rettilineo della finestra.
Slide 32 e spiegazione orale	Convoluzione e correlazione differiscono per il ribaltamento del kernel, non per la scelta del centro dell'uscita.
Slide 38	Il campo recettivo teorico cresce componendo regioni; due kernel 3×3 a stride 1 danno 5×5.

Riferimento	Precisazione
Slide 54	Una ReLU negativa non blocca l'intero modello; ELU è differenziabile in zero solo per $\alpha = 1$.
Slide 55–57	Il dimezzamento dei due assi richiede $K = S = 2$ nell'esempio; pooling non cambia i canali.
Slide 59	Il pooling adattivo precede flattening e MLP; non li sostituisce entrambi.
Slide 65–66	Le ReLU dei livelli nascosti sono distinte dall'uscita finale di classificazione.
Slide 71–74	Shortcut identità solo a forme coincidenti; i rami e i gradienti vengono sommati.
Slide 80–85	BN non impone gaussianità né decorrelazione; con $\epsilon > 0$ la varianza normalizzata non è esattamente 1.
Slide 86	Nelle CNN $N = 1$ non implica $NHW = 1$; `train()`/`eval()` cambiano la modalità dei livelli.

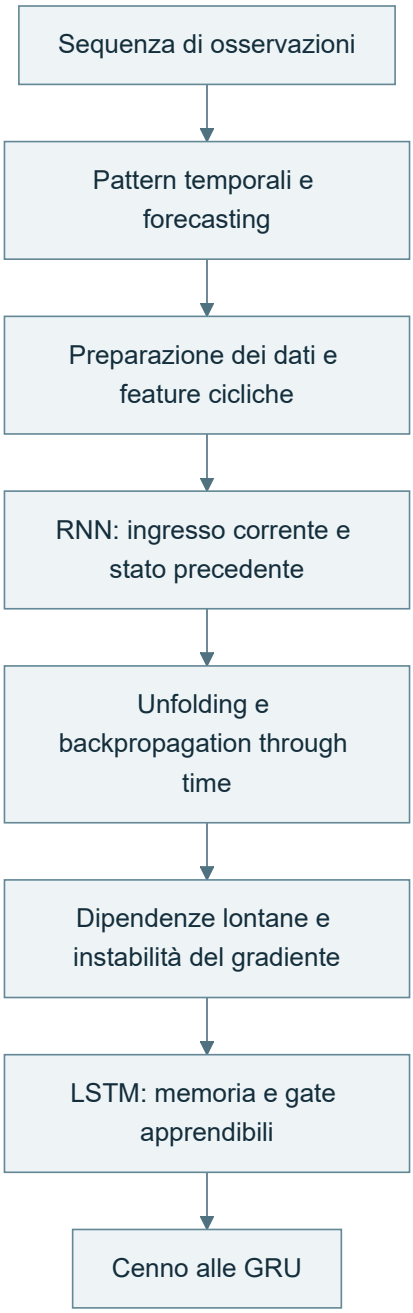
Argomento 3

RNN - Reti neurali ricorrenti

Contenuti dell'argomento

- Serie temporali e sequenze di lunghezza variabile (slide 3–4)
- Pattern temporali: trend, stagionalità e residuo (slide 5–8)
- Forecasting: continuare una sequenza (slide 9–12)
- Autoregressione e richiamo ai modelli statistici (slide 13–14)
- Preparazione dei dati e codifica ciclica (slide 15–18)
- Reti ricorrenti e forme dei problemi sequenziali (slide 19–21)
- Vanilla RNN: equazioni, dimensioni e significato (slide 22)
- Hidden state: un riassunto utile, non il modello appreso (slide 23)
- Unfolding: esplicitare il calcolo nel tempo (slide 24–26)
- BPTT: sommare i contributi ai parametri condivisi (slide 27)
- Dipendenze a lungo termine e instabilità del gradiente (slide 28–29)
- Memoria selettiva e architetture con gate (slide 30)
- LSTM: stato interno e quadro delle equazioni (slide 31)
- Gate: interruttori continui apprendibili (slide 32)
- Stato candidato: proporre un nuovo contenuto (slide 33)
- Aggiornare la memoria ed esporre lo stato (slide 34)
- Perché separare cell state e hidden state (slide 35)
- Riepilogo operativo (slide 4–35)
- Errata e precisazioni da conservare nel ripasso (slide 4–35)

Le reti convoluzionali sfruttano la struttura spaziale delle immagini. Le **reti neurali ricorrenti**, o **RNN**, sfruttano invece l'ordine di una sequenza: a ogni passo combinano una nuova osservazione con una rappresentazione di quanto è stato elaborato prima. Il problema centrale è costruire e apprendere una memoria utile al compito, anche quando le sequenze hanno lunghezze diverse.



3.1 Serie temporali e sequenze di lunghezza variabile (slide 3–4)

Una **serie temporale** è una successione ordinata di osservazioni raccolte nel tempo. Esempi sono un segnale audio o una canzone, la temperatura misurata giorno per giorno, il prezzo di un titolo finanziario e le misure di un sensore. L'ordine è parte dell'informazione: scambiare due osservazioni può modificare il significato della sequenza.

Si distinguono un tempo continuo, in cui il segnale è definito su un intervallo temporale, e un tempo discreto, in cui si considerano istanti separati. Qui si lavora con sequenze discrete osservate per un numero finito di passi:

$$\boxed{X = (x_1, x_2, \ldots, x_T), \qquad x_t \in \mathbb{R}^d.}$$

T è la **lunghezza della sequenza**; d è il **numero di feature di ogni osservazione**. Con vettori colonna, ogni x_t ha forma $d \times 1$. Disporre una osservazione per riga produce una rappresentazione tabellare $X \in \mathbb{R}^{T \times d}$.

- Una serie **univariata** ha $d = 1$: per esempio il solo numero di passeggeri.
- Una serie **multivariata** ha $d > 1$: per esempio più misure meteorologiche per ciascun giorno. Aggiungere una codifica della data alle misure aumenta il numero di feature.

La differenza fra una e più componenti è analoga a quella fra un valore grayscale e i tre canali RGB di reti convoluzionali; un'immagine RGB isolata, però, non è per questo una serie temporale. Una successione di immagini, come un video, possiede anche l'asse temporale. Analogamente, una nuvola di punti LiDAR isolata è un insieme spaziale, mentre misure LiDAR successive possono formare una sequenza.

Se il campionamento è regolare, gli istanti soddisfano

$$\boxed{t_k = t_0 + k\Delta t, \qquad f_c = \frac{1}{\Delta t}.}$$

Il primo istante è t_0 e ogni passo aggiunge lo stesso intervallo Δt. In un secondo entrano $1/\Delta t$ intervalli, da cui la frequenza di campionamento f_c. Per esempio, una misura ogni 0.5 secondi corrisponde a $f_c = 2$ Hz. Per dati mensili il passo è invece un mese: si usa l'unità temporale appropriata.

Errata slide 4 — Discreto non significa finito

Il tempo discreto è definito dall'insieme separato degli indici, non dalla finitezza della serie. Una serie discreta può continuare indefinitamente; è il campione osservato nel dataset a essere finito. Il campionamento regolare richiede inoltre un intervallo costante, non soltanto osservazioni discrete.

3.1.1 Sequenze di parole e limite dell'ingresso fisso (slide 4)

Una frase è una sequenza in cui l'indice t indica la posizione, senza dover rappresentare un tempo fisico. Ogni elemento può essere un token, identificato da un indice in un vocabolario e fornito alla rete mediante una rappresentazione numerica. Due frasi possono contenere numeri diversi di token.

Per classificare il sentiment di una frase, un MLP applicato direttamente alla concatenazione degli ingressi richiede una dimensione iniziale fissata. Con T osservazioni di d feature, la concatenazione ha Td componenti: se T cambia, cambia anche il numero di ingressi richiesto al primo livello denso. È la stessa rigidità della testa densa discussa in reti convoluzionali.

Non significa che un MLP non possa mai essere usato con dati temporali: finestre e rappresentazioni di lunghezza fissata sono possibili. La RNN affronta direttamente il problema riutilizzando **lo stesso aggiornamento per ogni elemento**, per quante osservazioni siano presenti. Può variare T, mentre la dimensione d dell'osservazione e quella dello stato rimangono fissate dal modello.

3.2 Pattern temporali: trend, stagionalità e residuo (slide 5–8)

Il grafico dei passeggeri aerei internazionali contiene misure mensili espresse in migliaia. Mostra contemporaneamente due regolarità: picchi che tendono a ripetersi e un livello complessivo che cresce. La prima suggerisce una componente stagionale; la seconda un trend crescente. Le annotazioni della slide evidenziano proprio la coesistenza delle due componenti.

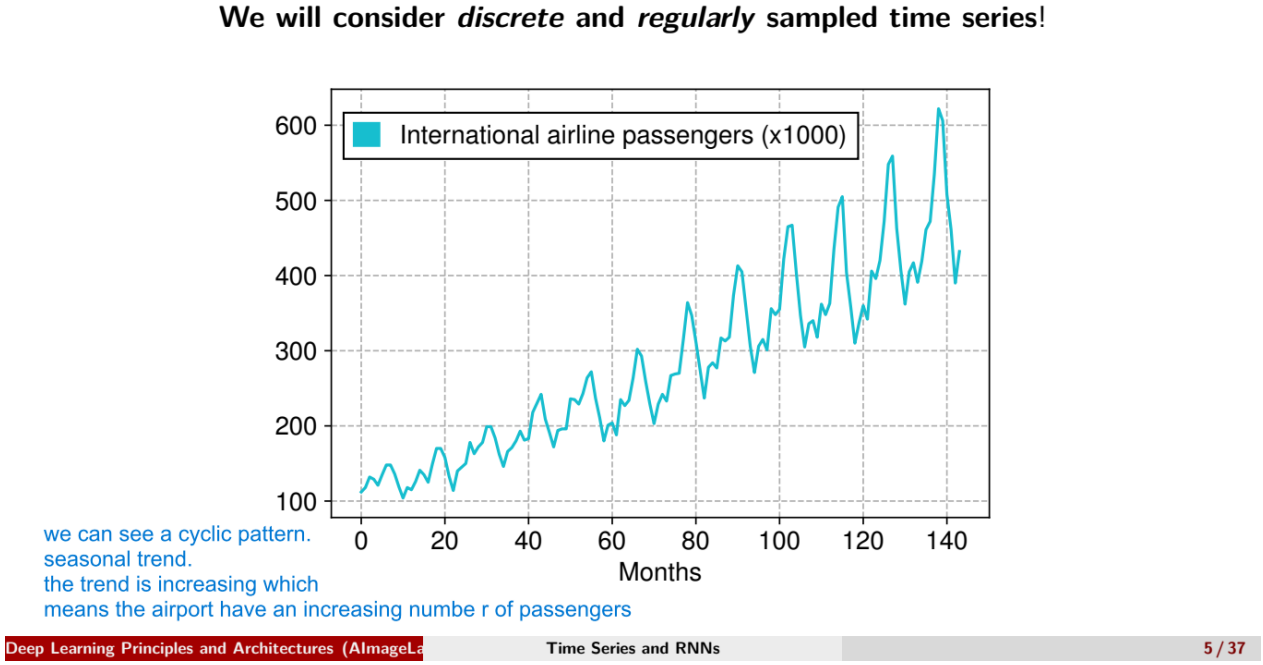


Figura 3.1: Slide 5 — Serie dei passeggeri con annotazioni su periodicità e trend

Componente	Significato	Lettura dell'esempio
Trend	Tendenza di fondo verso valori più alti o più bassi	Il numero di passeggeri cresce negli anni
Stagionalità	Fluttuazione che si ripete secondo un periodo di riferimento	Mesi corrispondenti tendono a mostrare comportamenti ricorrenti
Ciclo	Oscillazione attorno al trend	Una variazione ricorrente, non necessariamente legata a un calendario rigidamente fissato
Outlier	Osservazione insolita rispetto al comportamento atteso	Un valore eccezionalmente alto o basso, da interpretare nel suo contesto
Residuo	Parte non spiegata dalle componenti considerate	Rumore, irregolarità e pattern rimasti fuori dalla decomposizione

Le misure possono contenere rumore di acquisizione; inoltre, l'evoluzione reale può dipendere da fattori non osservati. Nei dati finanziari, per esempio, molte cause influenzano il prezzo. Una componente stocastica non implica soltanto un sensore impreciso: può descrivere anche incertezza del processo o fattori non modellati.

Una scomposizione additiva permette di leggere il grafico come

$$\boxed{x_t = a_t + s_t + r_t,}$$

dove a_t è il trend, s_t la stagionalità e r_t il residuo, tutti scalari con le stesse unità di x_t. Si ottiene il residuo sottraendo le parti spiegate:

$$\begin{aligned} x_t &= a_t + s_t + r_t && \text{somma delle componenti,} \\ x_t - a_t - s_t &= r_t && \text{sottrazione di trend e stagionalità.} \end{aligned}$$

Se in un mese si osservano 135 migliaia di passeggeri, con trend stimato 120 e componente stagionale 10, il residuo è $135 - 120 - 10 = 5$. La ricostruzione trend più stagionalità vale 130; la differenza di 5 è ciò che quella ricostruzione non descrive. Non è automaticamente rumore bianco.

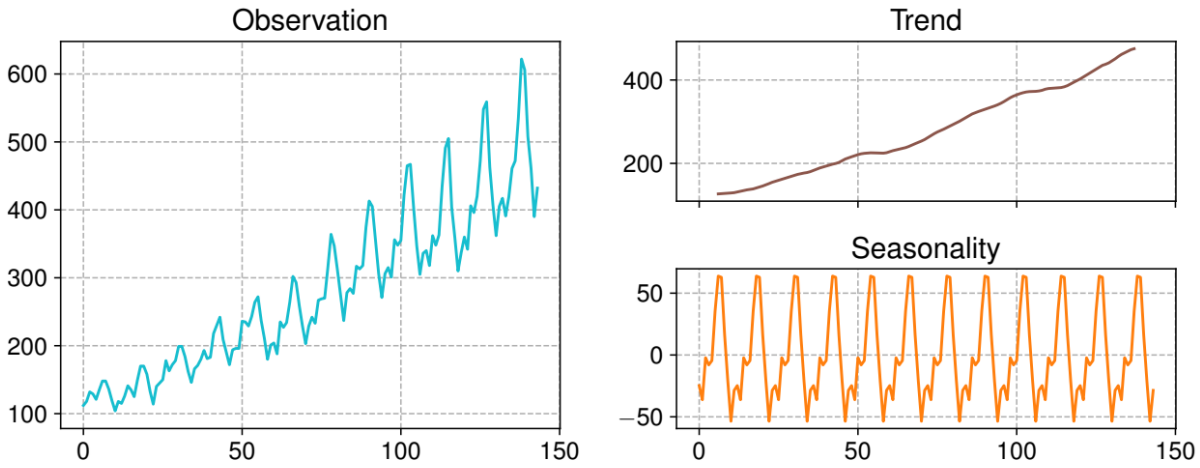


Figura 3.2: Slide 7 — Osservazione, trend e componente stagionale

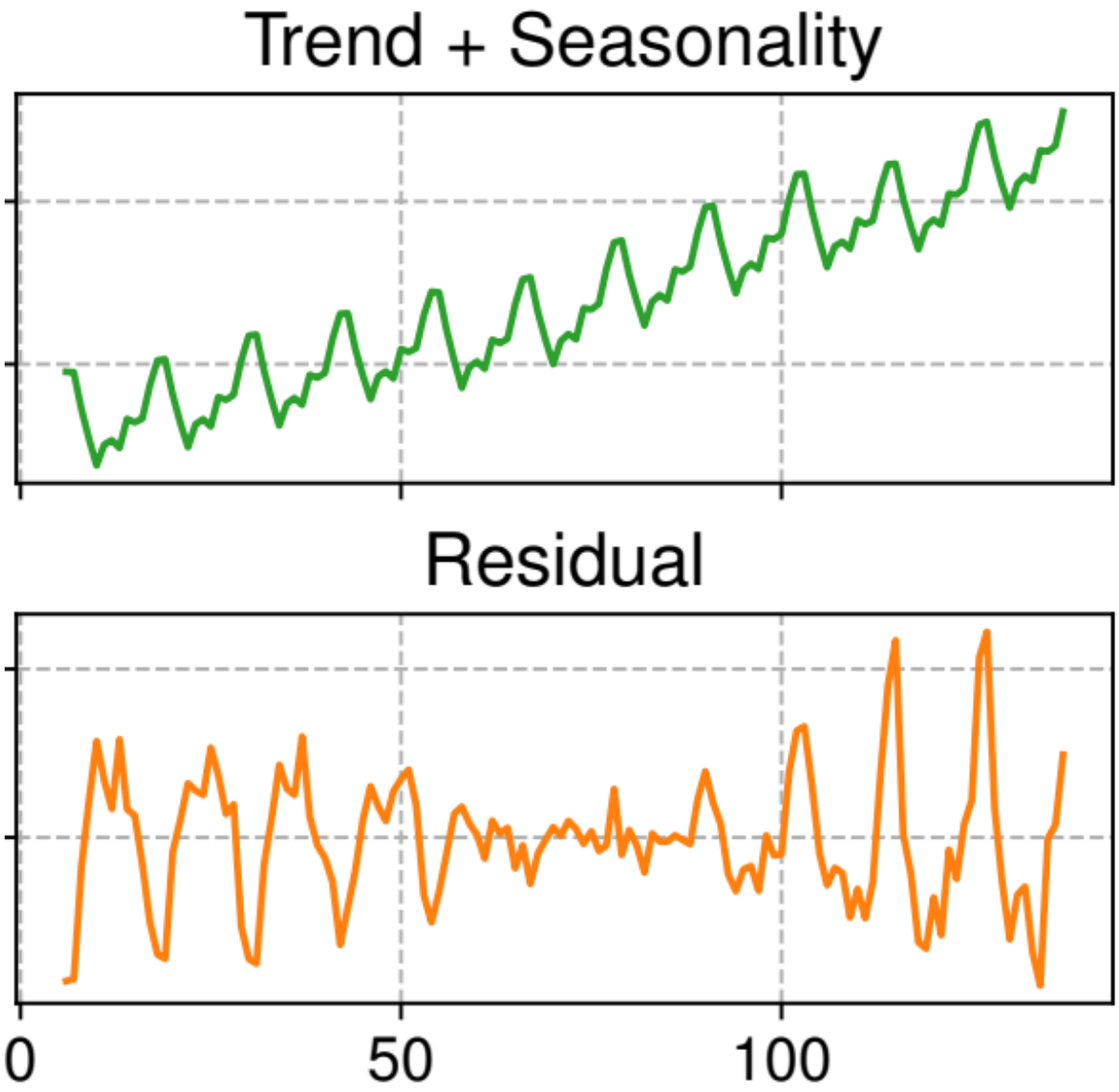


Figura 3.3: Slide 8 — Somma delle componenti e residuo

La **stazionarietà** descrive proprietà statistiche che non cambiano nel tempo. Una media che aumenta con il trend è già sufficiente a escludere la stazionarietà. Media e varianza costanti, da sole, non esauriscono tutte le condizioni: per la stazionarietà debole anche la covarianza deve dipendere dalla distanza temporale, non dall'istante assoluto. Qui il concetto serve a leggere i dati, senza introdurre test o procedure di stazionarizzazione.

Slide 6–8 — Formalizzazione presente nelle slide, non approfondita

Le slide nominano la decomposizione **STL**, basata su regressione LOESS per separare trend e stagionalità, e la definizione di stazionarietà. La lettura qualitativa dei pattern è centrale; l'algoritmo LOESS, i test statistici e i dettagli della procedura STL non sono sviluppati e non vengono aggiunti.

3.3 Forecasting: continuare una sequenza (slide 9–12)

Il **forecasting** consiste nel prevedere valori futuri usando osservazioni disponibili fino a un certo istante. Indichiamo con T l'ultimo passo osservato e con $h > 0$ l'**orizzonte di previsione**:

$$\boxed{\widehat{x}_{T+h} = F_\theta(x_1, \ldots, x_T; h).}$$

F_θ è il modello con parametri θ; $\widehat{x}_{T+h}$ è la stima della futura osservazione. Nel caso multivariato appartiene a $\mathbb{R}^d$, come x_t. Per ottenere più passi si può considerare il vettore delle previsioni da $T+1$ a $T+H$.

La costruzione del problema avviene in tre passaggi: si sceglie dove terminano i dati osservati; si decide quanto avanti guardare; si confronta la previsione con il valore futuro effettivamente osservato. Per esempio, osservando i valori mensili $(100, 110, 120)$, una prosecuzione illustrativa della crescita di 10 al mese darebbe 130 a orizzonte 1 e 140 a orizzonte 2. È una possibile previsione, non una conseguenza certa dei tre numeri.

Tra le applicazioni ci sono la previsione della traiettoria di un veicolo, della domanda elettrica, della domanda di prodotti e dei valori finanziari. Prevedere la domanda permette di preparare l'offerta; prevedere una traiettoria aiuta ad anticipare dove si troverà un oggetto.

3.3.1 Previsione puntuale e incertezza (slide 10–11)

La linea tratteggiata nella figura rappresenta una possibile previsione puntuale. La fascia attorno alla linea esprime invece un intervallo di valori plausibili. Nell'esempio la fascia si allarga all'aumentare dell'orizzonte: più ci si allontana dalle osservazioni, maggiore è l'incertezza rappresentata.

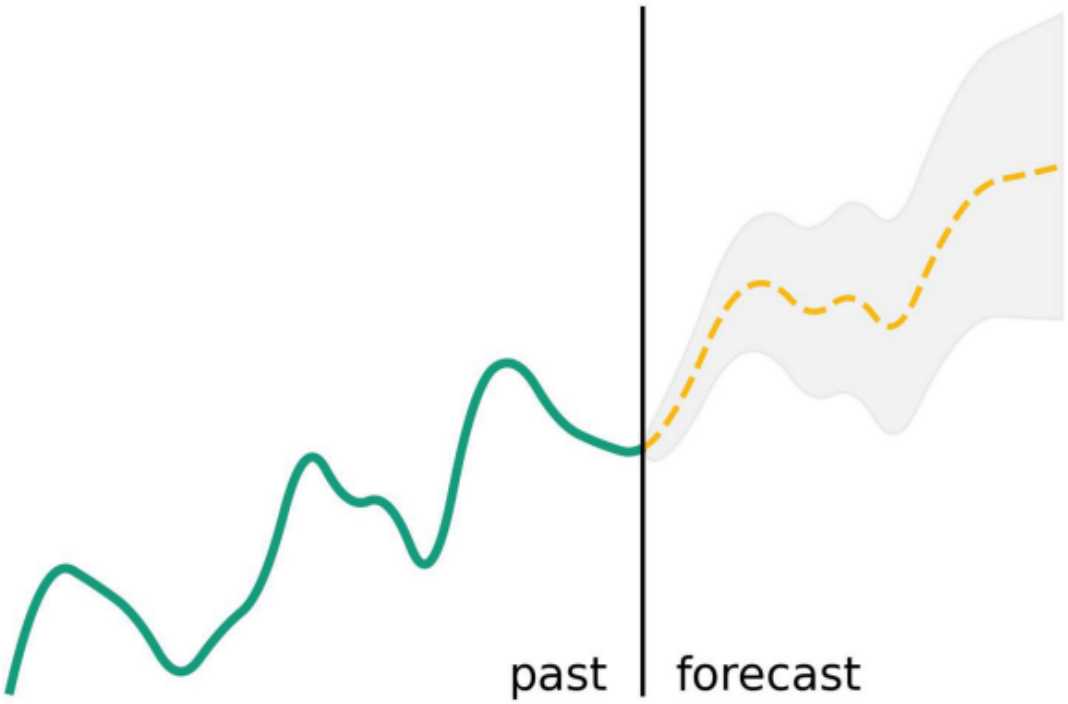


Figura 3.4: Slide 11 — Passato osservato, previsione e fascia di incertezza

Un intervallo dichiarato al 90% mira a contenere il valore futuro con quella copertura, sotto le ipotesi e la calibrazione del metodo. Per esempio, una previsione 130 accompagnata dall'intervallo $[120, 140]$ comunica sia un valore centrale sia la sua incertezza. La sola presenza di una fascia nel disegno non dimostra la copertura del 90%, né impone una distribuzione gaussiana. La costruzione di tali intervalli non viene sviluppata qui.

Errata slide 10 — Conoscere il processo non elimina il caso

Conoscere il processo generatore dei dati permette di descrivere correttamente la distribuzione futura; non garantisce di prevedere perfettamente ogni realizzazione se rimangono innovazioni casuali. Una previsione esatta a ogni orizzonte richiederebbe anche assenza di imprevedibilità residua e disponibilità delle informazioni necessarie.

3.3.2 Finestre di osservazione e componente casuale (slide 12)

Spesso la previsione usa una finestra limitata di osservazioni precedenti. Per prevedere il passo t a partire da q ritardi, l'ingresso è $(x_{t-1}, \ldots, x_{t-q})$: sono q osservazioni. La funzione appresa cerca di sfruttare le relazioni temporali contenute in quella finestra.

Slide 12 — Schema statistico generale non sviluppato

La slide scrive $x_t = f(x_{t-1}, \ldots, x_{t-q}; \phi) + g(\xi_{t-1}, \ldots, \xi_{t-q}; \theta)$, separando una funzione dei valori passati da una dei disturbi. ϕ, θ sono parametri e ξ indica disturbi casuali. È una rappresentazione introduttiva, non l'equazione della RNN né una definizione universale dei modelli di forecasting. Non si sviluppano qui stima dei disturbi o modelli combinati.

Errata slide 12 — Indici della finestra

L'insieme $\{x_t, \ldots, x_{t-q}\}$ riportato nel testo contiene $q+1$ osservazioni e include il valore da prevedere nella formula sottostante. Per prevedere x_t con q soli valori passati si usa $\{x_{t-1}, \ldots, x_{t-q}\}$. Inoltre lo schema dei disturbi non esplicita un'innovazione corrente ξ_t: non va letto come un modello stocastico completo senza ulteriori ipotesi.

3.4 Autoregressione e richiamo ai modelli statistici (slide 13–14)

Un modello è **autoregressivo** quando descrive un valore attraverso i valori precedenti della stessa sequenza. Nel modello lineare AR di ordine p:

$$\boxed{x_t = \sum_{i=1}^{p} \phi_i x_{t-i} + \xi_t.}$$

Qui x_t, x_{t-i}, ξ_t sono scalari; $\phi = (\phi_1, \ldots, \phi_p) \in \mathbb{R}^p$ contiene i coefficienti da stimare. Il termine ξ_t rappresenta un'innovazione a media zero. Il rumore bianco ha assenza di correlazione fra istanti distinti; la gaussianità è una possibile ipotesi aggiuntiva, non la sua definizione.

L'equazione nasce scegliendo una combinazione lineare di p ritardi e aggiungendo ciò che non è previsto da quella combinazione. Con $p = 2$:

$$\begin{aligned} x_t &= \phi_1 x_{t-1} + \phi_2 x_{t-2} + \xi_t && \text{sviluppo della somma,} \\ \widehat{x}_t &= \phi_1 x_{t-1} + \phi_2 x_{t-2} && \text{previsione con innovazione a media condizionale zero.} \end{aligned}$$

Con coefficienti 0.6 e 0.2 e osservazioni precedenti 120 e 100:

$$\widehat{x}_t = 0.6 \cdot 120 + 0.2 \cdot 100 = 72 + 20 = 92.$$

Se l'innovazione realizzata è 3, il valore osservato è 95. La differenza rende concreta la distinzione fra previsione e osservazione. L'equazione mostrata omette l'intercetta; una trasformazione affine può includere anche un termine costante.

Fundamental approaches

AImage Lab
University of Modena and Reggio Emilia

An autoregressive model of order p, denoted as AR(p), is defined as:

xt is a linear transformation of the previous generate value

$$x_t = \sum_{i=1}^{p} \phi_i x_{t-i} + \xi_t$$

x_t is generated as a function of its *past values*, plus a random variation, also called *white noise*.

- ϕ are the learnable parameters of the function.
- ξ, the white noise, is usually modelled as a normal distribution with mean 0.

Deep Learning Principles and Architectures (AImageLa | Time Series and RNNs | 13 / 37

Figura 3.5: Slide 13 — Modello AR e annotazione sulla dipendenza dai valori precedenti

Il principio autoregressivo ricompare nella generazione di testo: il nuovo token è condizionato al contesto già disponibile, compresi i token generati prima. Questo non rende un generatore linguistico un modello AR lineare: il principio di condizionamento è condiviso, la funzione che lo realizza può essere non lineare.

Slide 14 — Moving Average, presente nelle slide ma non sviluppato

Il modello MA di ordine q è riportato come $x_t = \mu + \xi_t + \sum_{i=1}^{q} \theta_i \xi_{t-i}$: combina innovazione corrente e innovazioni precedenti mediante coefficienti θ_i. Qui μ è la media teorica del processo quando i disturbi hanno media zero; la media empirica può stimarla, ma non coincide per definizione con essa. Non è una media mobile dei valori osservati. Stima e applicazioni del modello MA non sono state sviluppate.

Errata slide 14 — Media del processo e media campionaria

Identificare direttamente il parametro μ con la media empirica confonde una quantità del modello con un suo possibile stimatore. Sono concetti distinti anche se la seconda viene usata per stimare la prima.

3.5 Preparazione dei dati e codifica ciclica (slide 15–18)

Preparare una serie richiede di distinguere il **valore osservato** dall'**informazione temporale** associata. Una stringa come `1949-01` deve essere interpretata come una data, non come una generica etichetta: anno e mese possono essere estratti per organizzare e rappresentare le osservazioni. Le prime righe mostrate nelle slide associano, per esempio, gennaio 1949 a 112 migliaia di passeggeri e febbraio a 118.

Una seconda scelta riguarda la rappresentazione del segnale. Nel **dominio del tempo** si mantengono le osservazioni ordinate; nel **dominio delle frequenze** si descrivono componenti oscillatorie,

per esempio mediante Fourier. Per vibrazioni di motori, le componenti in frequenza possono essere informative. Le reti ricorrenti qui descritte lavorano sugli ingressi nel dominio del tempo, senza richiedere una trasformata di Fourier preliminare.

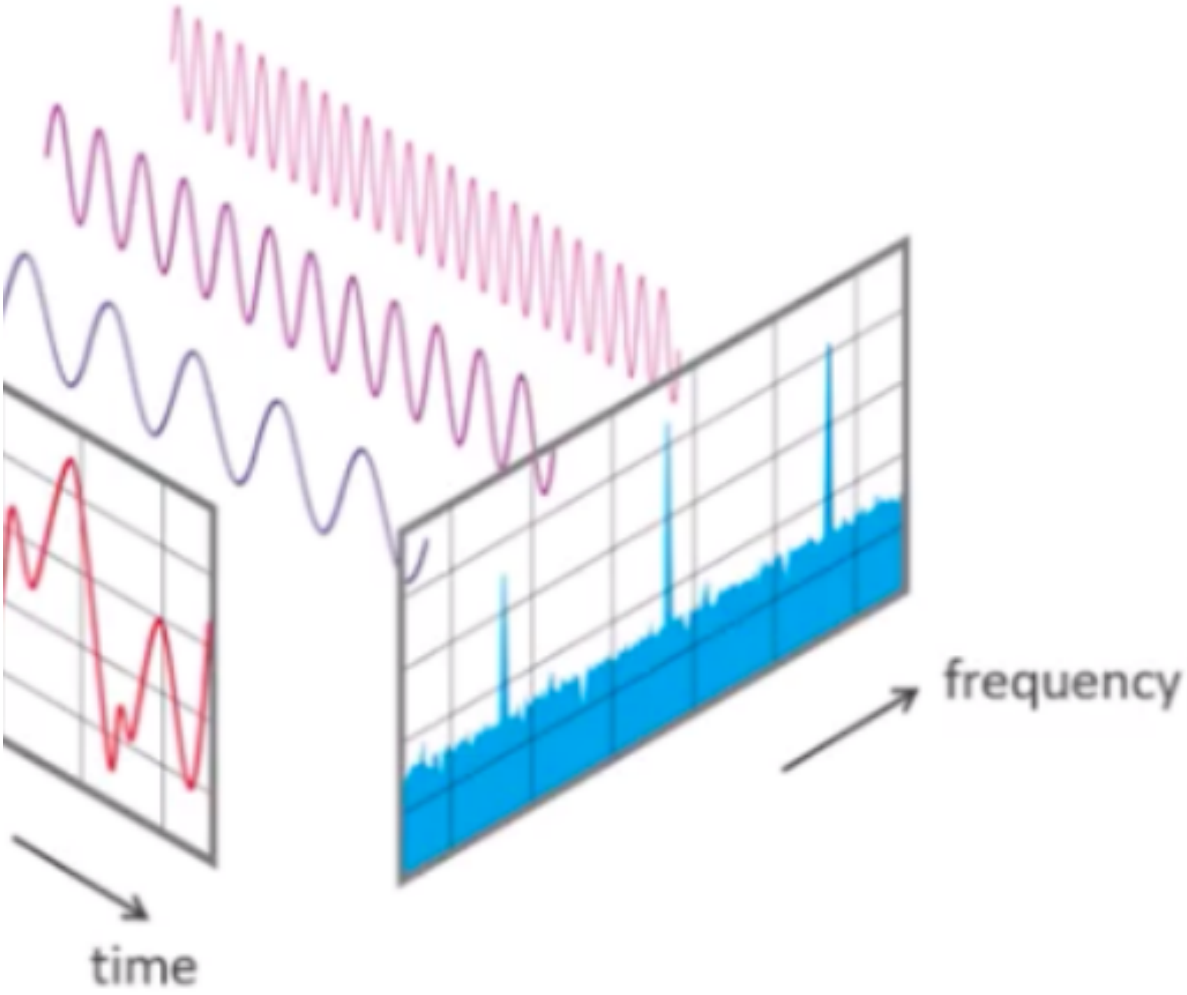


Figura 3.6: Slide 16 — Rappresentazione nel tempo e nelle frequenze

Slide 16 — Trasformate soltanto elencate

La slide cita anche wavelet e Hilbert–Huang. Non ne vengono sviluppate definizioni, formule o algoritmi; rimangono indicazioni di possibili rappresentazioni alternative.

3.5.1 Perché i mesi non sono un semplice asse numerico (slide 18)

Numerare gennaio con 0, febbraio con 1, fino a dicembre con 11 mantiene una differenza di 1 fra molti mesi consecutivi, ma separa dicembre e gennaio di 11 unità. Nel calendario, invece, sono adiacenti.

Per una variabile con periodo P, una codifica su una circonferenza conserva questa vicinanza:

$$\boxed{\theta_k = \frac{2\pi k}{P}, \qquad e(k) = \begin{bmatrix}\cos\theta_k \\ \sin\theta_k\end{bmatrix} \in \mathbb{R}^2.}$$

Si divide un giro completo, 2π radianti, in P intervalli; l'indice k determina l'angolo. Coseno e seno forniscono le coordinate del punto sulla circonferenza unitaria. L'ordine delle due coordinate può essere invertito, purché si mantenga la convenzione.

Per le ore, $P = 24$; per i mesi con indice $0, \ldots, 11$, $P = 12$. La periodicità deriva direttamente da seno e coseno:

$$\begin{aligned}\theta_{k+P} &= \frac{2\pi(k+P)}{P} = \theta_k + 2\pi, \\ \cos(\theta_k + 2\pi) &= \cos\theta_k, \\ \sin(\theta_k + 2\pi) &= \sin\theta_k, \\ e(k+P) &= e(k).\end{aligned}$$

Mese	k	Angolo	$\cos\theta_k$	$\sin\theta_k$
Gennaio	0	0	1	0
Febbraio	1	$\pi/6$	$\sqrt{3}/2 \approx 0.8660$	$1/2$
Dicembre	11	$11\pi/6$	$\sqrt{3}/2 \approx 0.8660$	$-1/2$

La distanza gennaio–dicembre è

$$\begin{aligned}\|e(0) - e(11)\|_2 &= \sqrt{(1-\sqrt{3}/2)^2 + (0+1/2)^2} && \text{distanza euclidea,} \\ &= \sqrt{2-\sqrt{3}} \approx 0.5176 && \text{sviluppo dei quadrati.}\end{aligned}$$

È la stessa distanza di gennaio–febbraio. Usando mesi da 1 a 12 si può porre $k = m - 1$; usare direttamente m produce soltanto una rotazione comune della codifica.

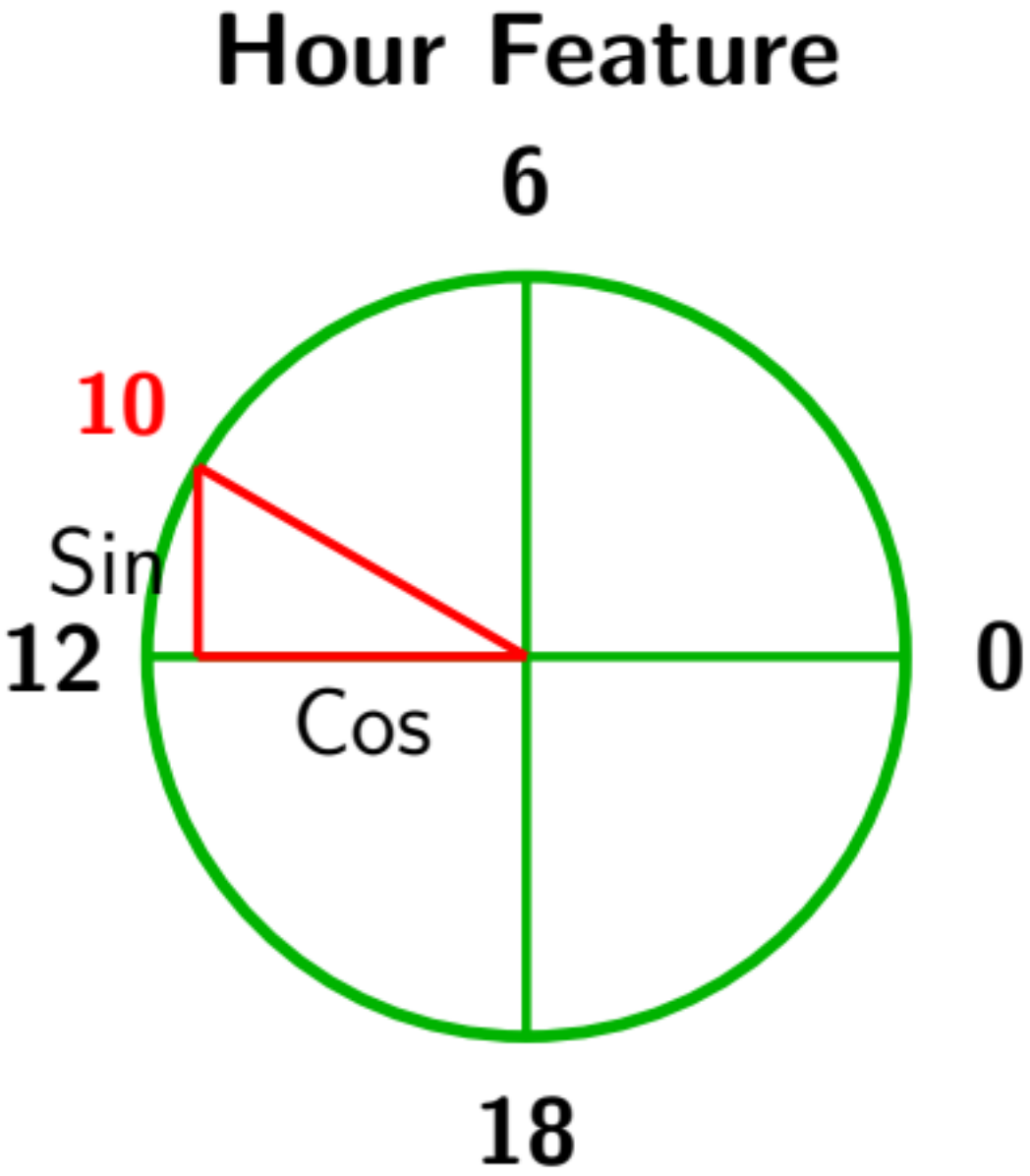


Figura 3.7: Slide 18 — Ora del giorno rappresentata sulla circonferenza

Servono entrambe le coordinate per distinguere posizioni che una sola funzione confonderebbe: il seno, per esempio, vale zero sia a mezzanotte sia a mezzogiorno. Per l'ora 10, l'angolo è $5\pi/6$ e la coppia $(\cos, \sin)$ è $(-\sqrt{3}/2, 1/2)$, coerente con il punto nel secondo quadrante della figura.

La codifica periodica è appropriata per grandezze **effettivamente cicliche**. L'anno assoluto non riparte automaticamente da zero: non gli si impone un ciclo senza una ragione legata al problema. Inoltre coordinate limitate fra -1 e 1 non garantiscono media empirica zero o una particolare varianza se i dati non coprono uniformemente il ciclo.

Il preprocessing costruisce le feature fornite al modello; non cambia la regola di ricorrenza. I richiami alla scala degli ingressi si collegano a Multilayer Perceptron e, per il diverso caso della normalizzazione delle attivazioni interne, a reti convoluzionali.

3.6 Reti ricorrenti e forme dei problemi sequenziali (slide 19–21)

In una rete feedforward il calcolo procede dall'ingresso attraverso livelli intermedi fino all'uscita. Una RNN introduce una connessione di ritorno: **lo stato calcolato a un passo viene utilizzato nel passo successivo**. Il feedback riguarda qui la rappresentazione nascosta, non necessariamente la predizione finale.

La sequenza può comparire all'ingresso, all'uscita o in entrambi. I termini «one» e «many» contano gli elementi della sequenza, non il numero di componenti del singolo vettore.

Schema	Ingresso e uscita	Interpretazione
One-to-one	Un elemento → un risultato	Situazione ordinaria non sequenziale
One-to-many	Un elemento → una sequenza	Un ingresso avvia più passi di uscita
Many-to-one	Una sequenza → un risultato	Sentiment o categoria di una frase
Many-to-many con uscita successiva	Sequenza osservata → sequenza prodotta	Traduzione o prosecuzione futura di una serie
Many-to-many allineato	Una uscita associata a ogni ingresso	Ingressi e uscite sono indicizzati lungo la stessa sequenza

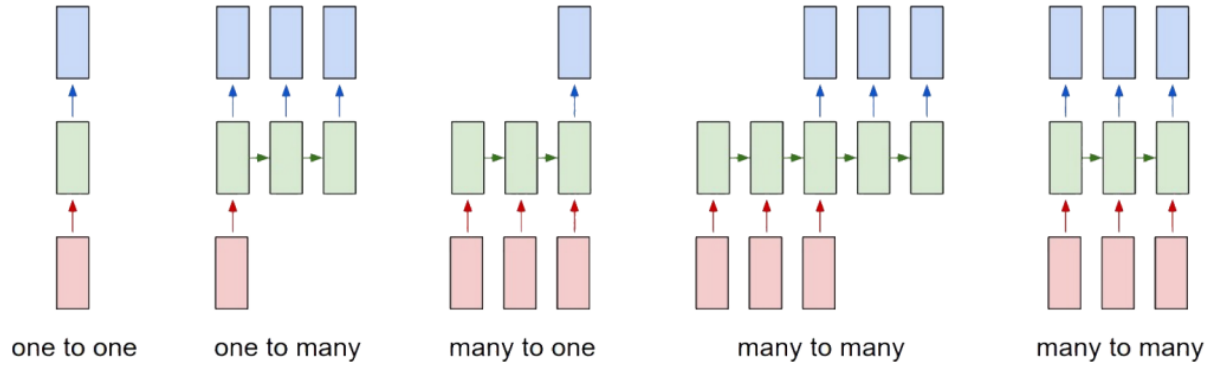


Figura 3.8: Slide 21 — Schemi di ingresso e uscita delle reti sequenziali

Una frase può contenere molti token ma avere un solo target di classificazione; quel target può essere rappresentato da un vettore di score. Nel forecasting multipasso si osservano più valori e si prevedono più valori futuri. Nella traduzione, lunghezza dell'ingresso e lunghezza dell'uscita possono essere differenti. Anche domanda e risposta testuale illustrano il rapporto fra una sequenza in ingresso e una sequenza in uscita, senza che questo determini da solo l'architettura usata.

3.7 Vanilla RNN: equazioni, dimensioni e significato (slide 22)

La forma più semplice, detta **vanilla RNN**, aggiorna lo stato mediante due contributi: l'osservazione corrente e lo stato precedente. Adottiamo vettori colonna e la notazione della slide 22:

$$\boxed{\begin{aligned} a_t &= W h_{t-1} + U x_t, \\ h_t &= \phi(a_t), \\ o_t &= V h_t. \end{aligned}}$$

Simbolo	Forma	Ruolo
x_t	$d \times 1$	Osservazione al passo t
h_{t-1}, h_t	$m \times 1$	Stato nascosto precedente e corrente
a_t	$m \times 1$	Pre-attivazione, prima della non linearità
U	$m \times d$	Proiezione ingresso $\rightarrow$ stato
W	$m \times m$	Trasformazione dello stato precedente
ϕ	Applicata componente per componente	Attivazione, per esempio tanh
V	$r \times m$	Proiezione stato $\rightarrow$ uscita
o_t	$r \times 1$	Uscita o score del passo corrente

La dimensione m dello stato è una scelta architetturale, come la larghezza di un livello nascosto di un MLP. Anche una serie univariata può essere proiettata in uno stato di 256 componenti: $d = 1$ non impone $m = 1$.

3.7.1 Come si costruisce un aggiornamento (slide 22)

La formula si legge verificando prima le dimensioni:

$$
\begin{aligned}
u_t &= Ux_t \in \mathbb{R}^m && (m \times d)(d \times 1) = m \times 1, \\
v_t &= Wh_{t-1} \in \mathbb{R}^m && (m \times m)(m \times 1) = m \times 1, \\
a_t &= u_t + v_t \in \mathbb{R}^m && \text{somma di vettori della stessa forma}, \\
h_t &= \phi(a_t) \in \mathbb{R}^m && \text{attivazione componente per componente}, \\
o_t &= Vh_t \in \mathbb{R}^r && (r \times m)(m \times 1) = r \times 1.
\end{aligned}
$$

Le trasformazioni portano ingresso e memoria nello stesso spazio; la somma li fonde; l'attivazione produce il nuovo stato. È lo schema del neurone di Multilayer Perceptron, con un secondo ingresso che arriva dal passato.

Le slide omettono i bias per semplicità. Se presenti, si aggiungono un vettore $b_h \in \mathbb{R}^m$ prima di ϕ e un vettore $b_o \in \mathbb{R}^r$ nell'uscita. L'omissione non implica che un'implementazione debba esserne priva.

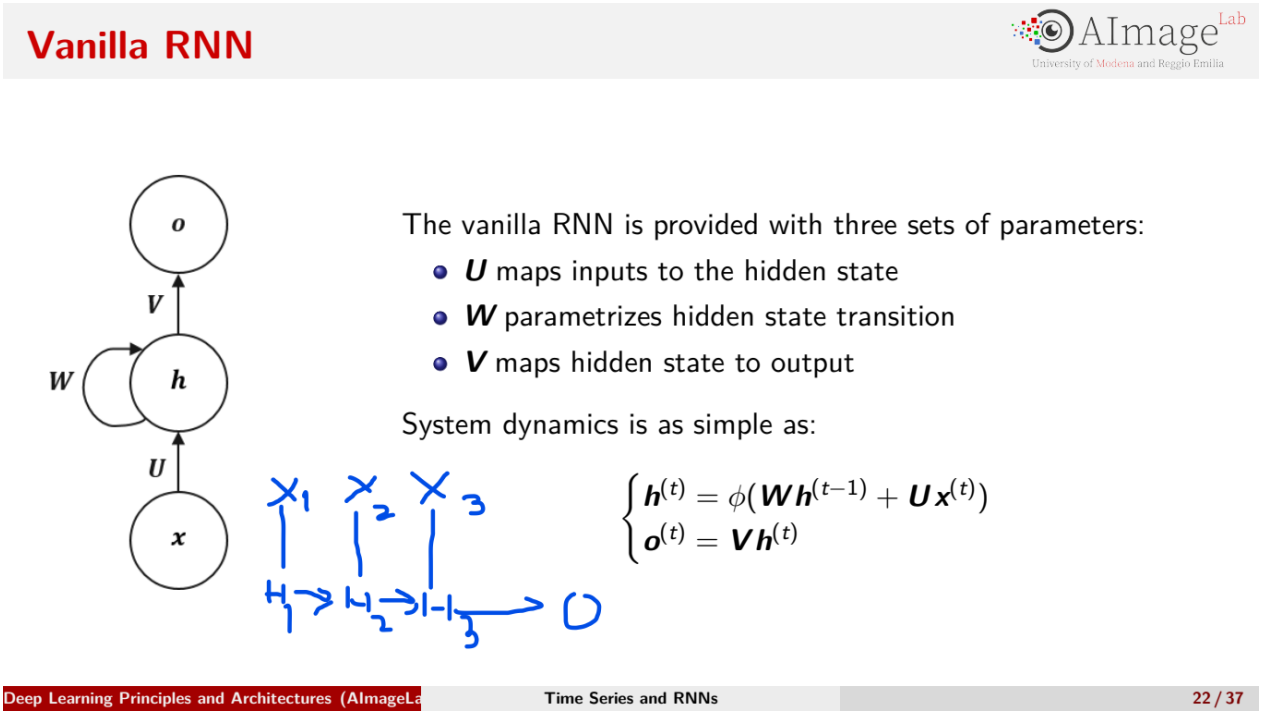


Figura 3.9: Slide 22 — RNN con annotazione a mano dello srotolamento in tre passi

3.7.2 Esempio numerico di un passo (slide 22)

Consideriamo due feature di ingresso, due unità nascoste e una uscita: $d = m = 2$, $r = 1$. Siano

$$x_t = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad h_{t-1} = \begin{bmatrix} 0.5 \\ -0.5 \end{bmatrix}, \quad U = \begin{bmatrix} 1 & 0 \\ 0 & 0.5 \end{bmatrix}, \quad W = \begin{bmatrix} 0.2 & 0 \\ 0 & 0.2 \end{bmatrix}, \quad V = \begin{bmatrix} 1 & -1 \end{bmatrix}.$$

Applicando i passaggi precedenti:

$$\begin{aligned} Ux_t &= \begin{bmatrix} 1 \cdot 1 + 0 \cdot 2 \\ 0 \cdot 1 + 0.5 \cdot 2 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \\ Wh_{t-1} &= \begin{bmatrix} 0.2 \cdot 0.5 \\ 0.2 \cdot (-0.5) \end{bmatrix} = \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix}, \\ a_t &= \begin{bmatrix} 1.1 \\ 0.9 \end{bmatrix}, \\ h_t &= \tanh(a_t) \approx \begin{bmatrix} 0.800499 \\ 0.716298 \end{bmatrix}, \\ o_t &= 1 \cdot 0.800499 - 1 \cdot 0.716298 \approx 0.084201. \end{aligned}$$

L'uscita è qui uno scalare generico. In classificazione può essere uno score su cui costruire la loss appropriata; in regressione può rappresentare la quantità da stimare. Una trasformazione lineare Vh_t non produce automaticamente probabilità.

3.7.3 Lunghezza variabile e pesi condivisi (slide 22)

Per una sequenza di tre elementi il modello calcola:

$$h_1 = \phi(Wh_0 + Ux_1),$$
$$h_2 = \phi(Wh_1 + Ux_2),$$
$$h_3 = \phi(Wh_2 + Ux_3).$$

Se arriva un quarto elemento si esegue un altro aggiornamento con **le stesse matrici**. Non si crea un nuovo insieme di pesi per ogni posizione. La ricorrenza cambia il numero delle iterazioni richieste, non la forma delle trasformazioni.

Senza bias e senza apprendere h_0, il conteggio deriva dal numero di elementi delle tre matrici:

$$P_U = md, \qquad P_W = m^2, \qquad P_V = rm,$$
$$\boxed{P_{\text{RNN}}} = \boxed{md + m^2 + rm.}$$

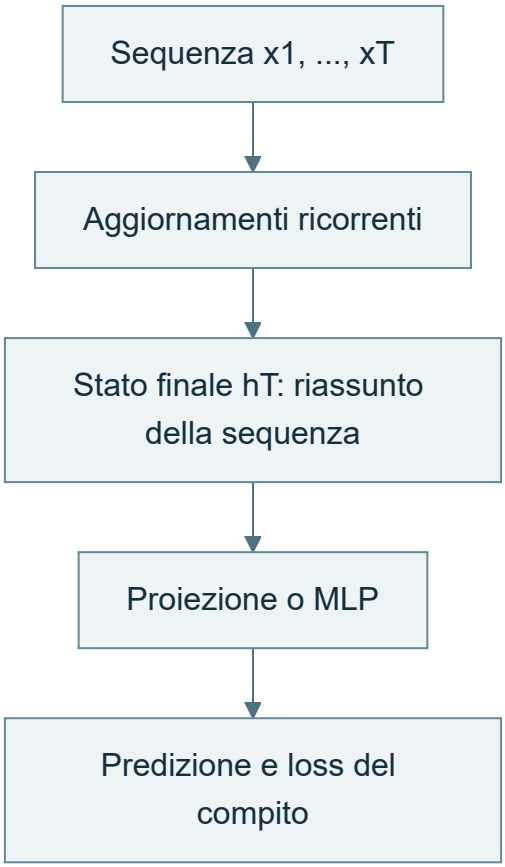
Nell'esempio $d = m = 2, r = 1$ sono $4+4+2 = 10$ pesi, sia per una sequenza di 3 passi sia per una di 100. Due bias aggiungerebbero $m+r = 3$ parametri; uno stato iniziale apprendibile aggiungerebbe altri $m = 2$ valori. È il conteggio della parametrizzazione descritta, non una formula universale per qualsiasi implementazione multilivello.

La condivisione è temporale: nelle CNN lo stesso filtro viene riutilizzato in posizioni spaziali differenti; qui la stessa regola viene riutilizzata in istanti differenti. Più passi significano comunque più calcolo.

3.8 Hidden state: un riassunto utile, non il modello appreso (slide 23)

Lo stato h_t può essere influenzato da tutti gli ingressi elaborati fino a t, perché h_t dipende da h_{t-1}, che dipende da h_{t-2}, e così via. È un **riassunto della storia disponibile**, orientato al compito e generalmente con perdita di informazione. Una dimensione fissa m non equivale a memorizzare esplicitamente una lista che cresce con la sequenza.

Per una classificazione many-to-one si può usare lo stato finale h_T per ottenere la decisione. La RNN agisce da estrattore di feature temporali, come la parte convoluzionale estrae feature spaziali; sopra lo stato o la sua proiezione si può collegare un MLP.



Il riassunto non è necessariamente completo né sempre efficace: proprio l'apprendimento di informazioni lontane sarà il problema successivo. Il legame strutturale con il passato non garantisce che tutti gli elementi conservino la stessa influenza.

3.8.1 Che cosa rimane dopo il training? (slide 23)

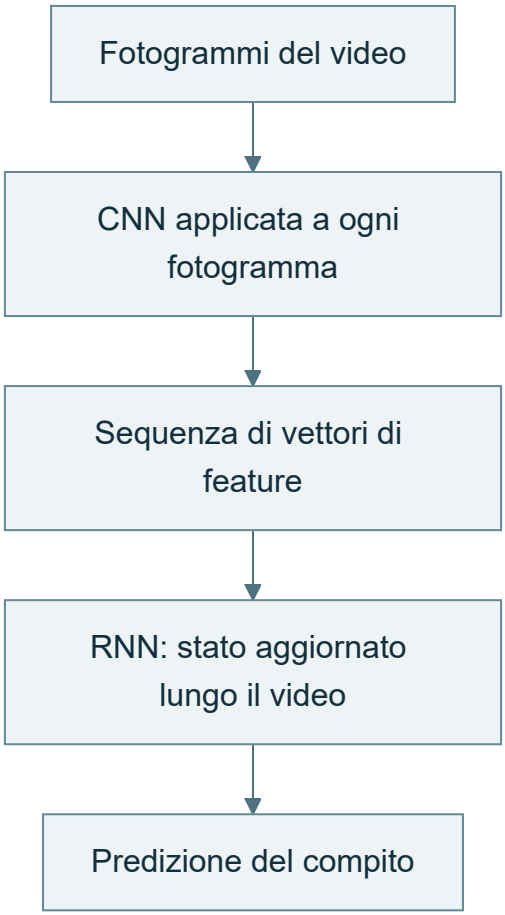
Le matrici U, W, V sono **parametri**, condivisi fra esempi. Lo stato h_t è una **attivazione**, calcolata per una specifica sequenza. Il risultato dell'apprendimento sono le matrici e gli eventuali altri parametri, non l'ultimo stato prodotto dall'ultimo esempio di training.

Su una nuova sequenza si inizializza lo stato e si ripete il forward. h_0 può essere fissato, spesso a zero, oppure essere un parametro apprendibile. Per proseguire una stessa sequenza in inferenza è sufficiente mantenere lo stato corrente e aggiornarlo con la nuova osservazione; per esempi indipendenti non si trasferisce automaticamente la storia dell'esempio precedente.

In training, per calcolare i gradienti lungo la sequenza, servono invece gli intermedi del grafo o un modo di ricostruirli: dimensione fissa dello stato non significa memoria di addestramento indipendente da T.

3.8.2 Comporre CNN e RNN per un video (slide 23)

Un video è una sequenza di immagini. Una CNN può elaborare ciascun fotogramma e produrre un vettore di feature; la RNN può elaborare la successione di quei vettori. I due blocchi trattano rispettivamente struttura spaziale e ordine temporale.



È un esempio di composizione dei blocchi già studiati, senza introdurre una nuova regola di apprendimento.

3.9 Unfolding: esplicitare il calcolo nel tempo (slide 24–26)

Il disegno con un arco che torna sullo stato è una rappresentazione compatta. Lo **srotolamento**, o **unfolding/unrolling**, esplicita un nodo per ogni aggiornamento. Ogni copia usa gli stessi parametri: non rappresenta una cella con pesi nuovi.

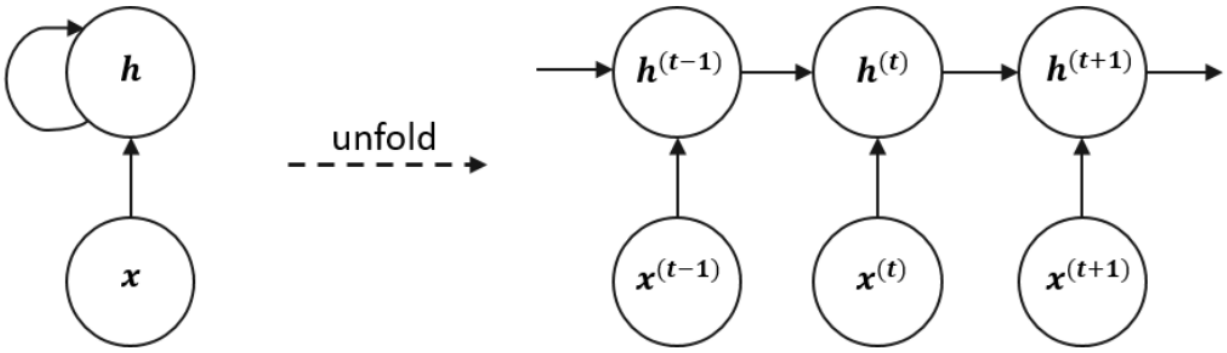


Figura 3.10: Slide 24 — Grafo ricorrente e srotolamento

Per tre osservazioni e una sola uscita finale:

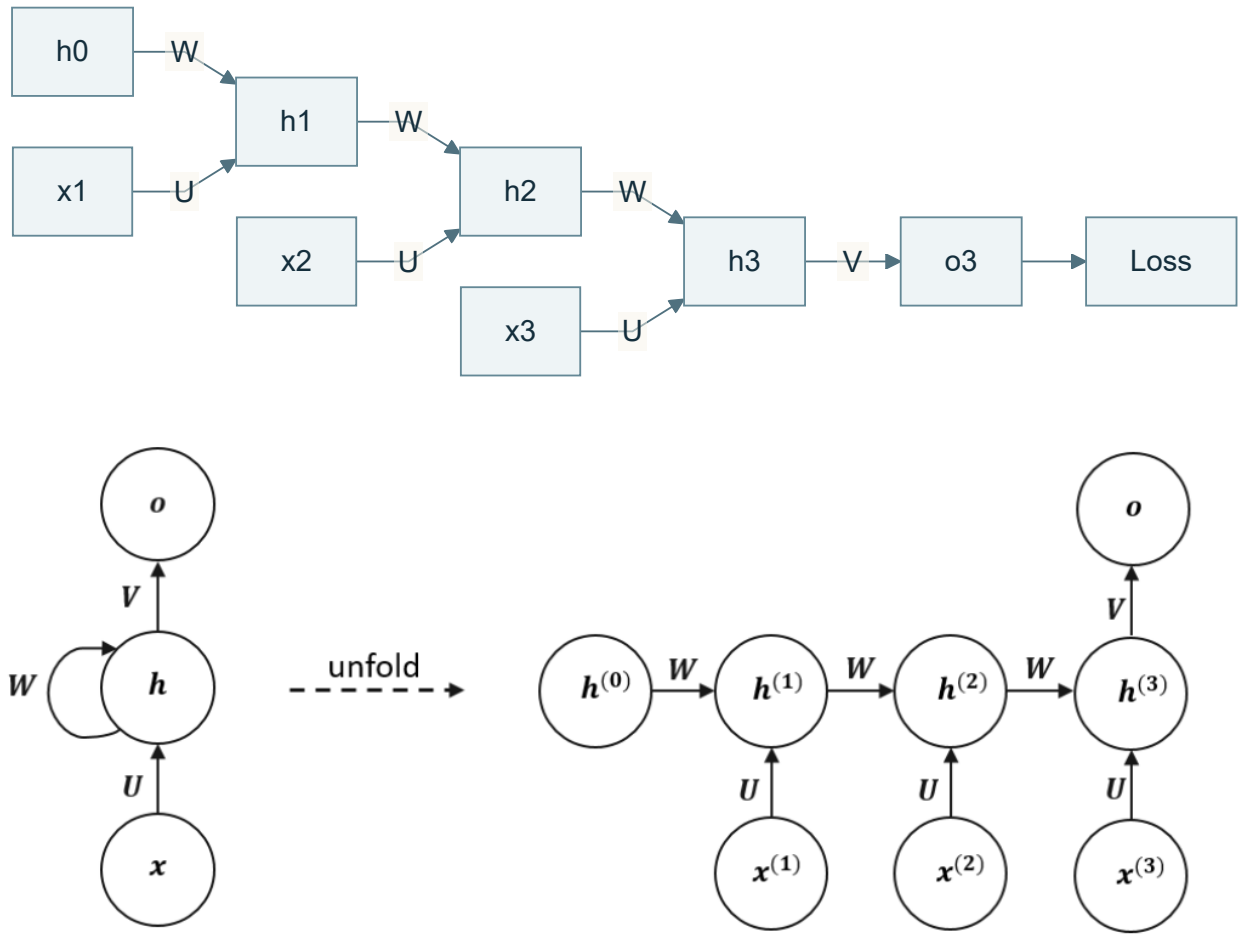


Figura 3.11: Slide 26 — Esempio many-to-one con tre passi e stato iniziale

Una volta fissata una sequenza finita, il grafo srotolato è un **DAG**, un grafo diretto senza cicli: h_1 dipende da h_0, h_2 da h_1, e nessun nodo torna indietro modificando il valore di un nodo precedente nello stesso forward. L'arco di ritorno del disegno compatto indica il riutilizzo al passo successivo.

Si può quindi applicare la backpropagation ordinaria. Poiché il backward attraversa i passi temporali in ordine inverso, si parla di **Backpropagation Through Time**, abbreviata **BPTT**. La regola della catena è quella di Multilayer Perceptron; qui le stesse matrici compaiono in più punti del grafo. Un sistema di differenziazione automatica può registrare questi passaggi e accumulare i gradienti corrispondenti.

3.10 BPTT: sommare i contributi ai parametri condivisi (slide 27)

Consideriamo il caso mostrato: $T = 3$, un'uscita finale $o_3 = Vh_3$ e una loss differenziabile $\mathcal{L}(y, o_3)$. Manteniamo vettori colonna, h_0 inizialmente indipendente da U, W e bias omessi.

Definiamo:

$$g = \nabla_{o_3}\mathcal{L} \in \mathbb{R}^r, \qquad D_t = \operatorname{diag}\big(\phi'(a_t)\big) \in \mathbb{R}^{m\times m}.$$

La matrice diagonale D_t contiene una derivata di attivazione per ogni unità. Come già visto per l'MLP, la derivata della loss rispetto alla pre-attivazione combina il gradiente in arrivo con la derivata locale.

3.10.1 Gradiente della proiezione finale V (slide 27)

Per ogni componente:

$$
\begin{aligned}
o_{3,j} &= \sum_{i=1}^{m} V_{ji} h_{3,i}, \\
\frac{\partial o_{3,j}}{\partial V_{ji}} &= h_{3,i} && \text{derivata locale della combinazione lineare,} \\
\frac{\partial \mathcal{L}}{\partial V_{ji}} &= g_j h_{3,i} && \text{regola della catena,} \\
\boxed{\nabla_V \mathcal{L}} &= \boxed{g h_3^T \in \mathbb{R}^{r \times m}.}
\end{aligned}
$$

È un **prodotto esterno**: un gradiente per ciascun peso. Nel grafo considerato V compare soltanto nella proiezione finale, quindi c'è un solo contributo locale. Il suo valore dipende comunque da h_3, che riassume la sequenza: non significa che il passato sia irrilevante per V.

3.10.2 Gradiente attraverso gli stati (slide 27)

Poniamo $\delta_t = \nabla_{a_t} \mathcal{L} \in \mathbb{R}^m$. Dall'uscita finale si torna prima allo stato e poi alla pre-attivazione:

$$
\begin{aligned}
\nabla_{h_3} \mathcal{L} &= V^T g && \text{backward della trasformazione lineare,} \\
\delta_3 &= D_3 V^T g && \text{backward dell'attivazione,} \\
\nabla_{h_2} \mathcal{L} &= W^T \delta_3 && \text{dipendenza di } a_3 \text{ da } h_2, \\
\delta_2 &= D_2 W^T \delta_3 && \text{backward dell'attivazione al passo 2,} \\
\delta_1 &= D_1 W^T \delta_2 && \text{stessa regola al passo 1.}
\end{aligned}
$$

In generale, quando la loss è soltanto sull'uscita finale:

$$
\boxed{\delta_T = D_T V^T g, \qquad \delta_t = D_t W^T \delta_{t+1} \quad (t < T).}
$$

Tutti i prodotti restituiscono un vettore di m componenti. Se ci sono loss anche su uscite intermedie si aggiungono i contributi locali di quelle uscite: il caso della slide rimane quello con una sola loss finale.

3.10.3 Perché compare una somma per W e U (slide 27)

La componente i della pre-attivazione è

$$
a_{t,i} = \sum_j W_{ij} h_{t-1,j} + \sum_\ell U_{i\ell} x_{t,\ell}.
$$

Trattando gli ingressi al singolo nodo come fissati, le derivate **locali** sono:

$$
\begin{aligned}
\left.\frac{\partial a_{t,i}}{\partial W_{ij}}\right|_{\text{locale}} &= h_{t-1,j}, \\
\left.\frac{\partial a_{t,i}}{\partial U_{i\ell}}\right|_{\text{locale}} &= x_{t,\ell}.
\end{aligned}
$$

Moltiplicando per $\delta_{t,i}$ si ottiene il contributo di quella specifica occorrenza del parametro. Poiché il parametro è lo stesso in ogni passo, si sommano tutte le occorrenze:

$$\boxed{\begin{aligned}\nabla_W \mathcal{L} &= \sum_{t=1}^{T} \delta_t h_{t-1}^T \in \mathbb{R}^{m\times m},\\ \nabla_U \mathcal{L} &= \sum_{t=1}^{T} \delta_t x_t^T \in \mathbb{R}^{m\times d}.\end{aligned}}$$

Per $T = 3$, per esempio:

$$\nabla_W \mathcal{L} = \delta_1 h_0^T + \delta_2 h_1^T + \delta_3 h_2^T.$$

La **sovrapposizione degli effetti** consiste nel tenere conto di tutti i modi in cui un parametro condiviso influenza l'uscita. I prodotti dentro δ_t trasportano l'effetto lungo un percorso; la somma combina le diverse occorrenze del peso. Somma fra contributi e prodotto lungo una catena hanno ruoli differenti.

Errata slide 27 — Derivate locali e stato iniziale

Le somme con $\partial h^{(k)}/\partial W$ e $\partial h^{(k)}/\partial U$ richiedono di intendere queste derivate come contributi **locali** dell'occorrenza del parametro al passo k. Se fossero derivate totali, si conterebbero più volte percorsi già inclusi. Con aggiornamenti indicizzati da 1 a T e h_0 indipendente dai pesi, le occorrenze sono $k = 1, \dots, T$; il termine $k = 0$ della slide è nullo. Un h_0 appreso ha il proprio gradiente, non un'ulteriore occorrenza di W o U.

Se h_0 è apprendibile, il suo gradiente in questo esempio è $W^T \delta_1$. Se c'è un bias nascosto condiviso, il contributo locale è δ_t e il gradiente del bias è la loro somma. Sono le stesse regole di accumulo.

3.10.4 Esempio numerico completo su tre passi (slide 26–27)

Per rendere trasparenti i conti, usiamo **solo in questo esempio** una ricorrenza scalare con attivazione identità: $h_t = W h_{t-1} + U x_t$. Non è il caso non lineare con tanh dell'esempio precedente; serve a isolare condivisione e regola della catena.

Siano $h_0 = 0$, $U = 1$, $W = 0.5$, $V = 1$, ingressi $(x_1, x_2, x_3) = (1, 2, 0)$ e target $y = 1$. La loss è $\mathcal{L} = \frac{1}{2}(o_3 - y)^2$, collegata alla MSE di Multilayer Perceptron.

Passo	Calcolo dello stato	Risultato
1	$0.5 \cdot 0 + 1 \cdot 1$	$h_1 = 1$
2	$0.5 \cdot 1 + 1 \cdot 2$	$h_2 = 2.5$
3	$0.5 \cdot 2.5 + 1 \cdot 0$	$h_3 = 1.25$

Seguono $o_3 = 1.25$ e $\mathcal{L} = \frac{1}{2}(0.25)^2 = 0.03125$. La derivata della loss è $g = o_3 - y = 0.25$; con attivazione identità tutti i fattori D_t valgono 1:

$$\begin{aligned}\delta_3 &= 1 \cdot 1 \cdot 0.25 = 0.25,\\ \delta_2 &= 1 \cdot 0.5 \cdot 0.25 = 0.125,\\ \delta_1 &= 1 \cdot 0.5 \cdot 0.125 = 0.0625.\end{aligned}$$

Passo	Contributo a $\partial\mathcal{L}/\partial W$	Contributo a $\partial\mathcal{L}/\partial U$
1	$\delta_1 h_0 = 0$	$\delta_1 x_1 = 0.0625$
2	$\delta_2 h_1 = 0.125$	$\delta_2 x_2 = 0.25$
3	$\delta_3 h_2 = 0.625$	$\delta_3 x_3 = 0$
Somma	**0.75**	**0.3125**

Per la proiezione finale, $\partial\mathcal{L}/\partial V = g h_3 = 0.25 \cdot 1.25 = 0.3125$. Si può verificare anche sviluppando la ricorrenza:

$$\begin{aligned} h_1 &= Ux_1, \\ h_2 &= WUx_1 + Ux_2, \\ h_3 &= W^2Ux_1 + WUx_2 + Ux_3, \\ \frac{\partial h_3}{\partial W} &= 2WUx_1 + Ux_2 = 1 + 2 = 3, \\ \frac{\partial \mathcal{L}}{\partial W} &= gV\frac{\partial h_3}{\partial W} = 0.25 \cdot 1 \cdot 3 = 0.75. \end{aligned}$$

Il risultato coincide con la somma dei tre contributi. Il primo contributo locale a W è nullo perché $h_0 = 0$, non perché il primo ingresso sia ignorato: x_1 influenza anche gli stati successivi e il gradiente di U.

Con un passo di gradient descent di ampiezza $\eta = 0.1$, tutti i gradienti vengono prima calcolati con gli stessi valori correnti, poi i parametri diventano $W' = 0.425$, $U' = 0.96875$ e $V' = 0.96875$. Non si modifica W a metà del backward per usarlo nella derivazione degli altri contributi.

3.11 Dipendenze a lungo termine e instabilità del gradiente (slide 28–29)

Una dipendenza a lungo termine si presenta quando un'informazione ricevuta molto prima è utile alla predizione corrente. La RNN possiede un percorso che collega i due istanti, ma il gradiente deve attraversare tutti gli aggiornamenti intermedi per apprendere come conservare quell'informazione.

Definiamo la Jacobiana di una transizione:

$$\boxed{J_t = \frac{\partial h_t}{\partial h_{t-1}} = D_t W \in \mathbb{R}^{m \times m}.}$$

La formula si ricava da $h_t = \phi(a_t)$ e $a_t = Wh_{t-1} + Ux_t$: la derivata dell'attivazione è D_t; la derivata della pre-attivazione rispetto allo stato precedente è W. Per la regola della catena i due fattori si moltiplicano in quest'ordine.

3.11.1 Il prodotto lungo il tempo (slide 28)

Con tre passi:

$$\begin{aligned} \frac{\partial h_3}{\partial h_1} &= \frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial h_1} && \text{regola della catena,} \\ &= J_3 J_2 && \text{definizione delle Jacobiane,} \\ &= D_3 W D_2 W && \text{sostituzione delle derivate locali.} \end{aligned}$$

In generale, per $t > k$:

$$\boxed{\frac{\partial h_t}{\partial h_k} = J_t J_{t-1} \cdots J_{k+1}.}$$

Il gradiente rispetto allo stato iniziale del tratto si ottiene moltiplicando il gradiente in arrivo per la trasposta di questo prodotto. Nel caso scalare sono semplicemente prodotti di derivate.

Se ogni fattore scalare vale 0.5, dieci transizioni danno $0.5^{10} = 0.0009765625$; venti danno circa $9.54 \cdot 10^{-7}$. Un contributo partito con ampiezza 1 può quindi arrivare molto attenuato. Nell'esempio lineare della sezione precedente, $J_t = W = 0.5$ e $\partial h_3 / \partial h_1 = 0.25$.

Il **vanishing gradient** rende debole il segnale che dovrebbe insegnare alla rete a usare gli ingressi lontani. Non implica che ogni gradiente della rete sia nullo o che non rimanga alcuna informazione del passato: indica una difficoltà di apprendimento crescente lungo percorsi che contraggono ripetutamente il segnale.

3.11.2 Derivata dell'attivazione e matrice ricorrente (slide 28)

Per tanh e sigmoide, richiamando Multilayer Perceptron:

$$\boxed{\tanh'(a) = 1 - \tanh^2(a) \le 1, \qquad \sigma'(a) = \sigma(a)(1 - \sigma(a)) \le \tfrac{1}{4}.}$$

Il limite della sigmoide si verifica completando il quadrato: posto $s = \sigma(a) \in (0, 1)$,

$$\begin{aligned} s(1-s) &= s - s^2, \\ &= \frac{1}{4} - \left(s - \frac{1}{2}\right)^2, \\ &\le \frac{1}{4}. \end{aligned}$$

Per tanh, il quadrato è non negativo, quindi $1 - \tanh^2(a) \le 1$; in saturazione il quadrato è vicino a 1 e la derivata è piccola. Per esempio, $\tanh'(0) = 1$; $\tanh'(2) \approx 0.07065$. Per la sigmoide, $\sigma'(0) = 0.25$.

Questi sono i limiti delle **derivate delle attivazioni**, non della transizione completa. Per la norma spettrale, la proprietà $\|AB\|_2 \le \|A\|_2 \|B\|_2$ dà:

$$\begin{aligned} \|J_t\|_2 &= \|D_t W\|_2 \le \|D_t\|_2 \|W\|_2, \\ \left\| \frac{\partial h_t}{\partial h_k} \right\|_2 &\le \prod_{j=k+1}^{t} \|J_j\|_2. \end{aligned}$$

Se ogni fattore è al più $c < 1$, il prodotto è al più c^{t-k}, che diminuisce geometricamente. Con $c = 0.5$ si ritrovano i numeri precedenti.

Errata slide 28 — Il limite non vale senza W

I limiti 1 per tanh e $1/4$ per sigmoide riguardano D_t. La Jacobiana temporale è $J_t = D_t W$ e dipende anche dai pesi. Per esempio, nel caso scalare con $W = 2$ e pre-attivazione zero, $\tanh'(0) = 1$ ma $J_t = 2$. Non si può dedurre dalla sola attivazione che ogni transizione abbia norma al più 1 o $1/4$.

3.11.3 Exploding gradient e mitigazioni (slide 29)

L'effetto opposto si verifica quando i prodotti amplificano il gradiente. Dieci fattori scalari uguali a 2 danno 1024; venti danno 1 048 576. Aggiornamenti molto grandi possono rendere instabile l'ottimizzazione e produrre problemi numerici. Si parla di **exploding gradient**.

Un limite superiore grande non dimostra che ogni direzione venga amplificata: nei vettori contano anche i pesi, le attivazioni e la direzione del gradiente. Gli esempi scalari isolano il meccanismo.

Inizializzazione e scelta delle attivazioni influenzano questi prodotti. Il **gradient clipping** limita gradienti eccessivamente grandi; per chiarirne il significato, un clipping sulla norma ha la forma:

$$\boxed{g_{\text{clip}} = g \min\left(1, \frac{c}{\|g\|_2}\right), \qquad c > 0,}$$

con $g_{\text{clip}} = 0$ se $g = 0$. Se $\|g\|_2 \leq c$, il minimo vale 1 e il gradiente non cambia. Se la norma supera c, lo si moltiplica per $c/\|g\|_2$, preservandone la direzione e portandone la norma a c. Per $g = (6, 8)$ la norma è 10; con $c = 5$ il risultato è $(3, 4)$, la cui norma è 5.

Errata slide 29 — Il clipping non recupera gradienti svaniti

Il clipping controlla soprattutto l'esplosione del gradiente. Non ricostruisce un contributo già diventato trascurabile. Inizializzazione, attivazioni e architettura intervengono sul flusso dei gradienti in modi differenti; non sono rimedi intercambiabili ai due problemi.

Una RNN srotolata su molti passi crea una grande profondità temporale anche se contiene un solo livello ricorrente. Problemi analoghi possono comparire nelle reti feedforward profonde, come già discusso in reti convoluzionali per le connessioni residue.

La condivisione dei pesi è una proprietà dell'architettura; la BPTT ne calcola correttamente le conseguenze, non la introduce. La difficoltà deriva dai lunghi prodotti di Jacobiane, non dalla sola presenza di parametri condivisi. Non basta inserire un logaritmo nel backward: il gradiente deve rimanere la derivata della funzione effettivamente calcolata. Per cambiarne i percorsi si interviene su modello e calcolo, non sostituendo arbitrariamente la regola della catena.

3.12 Memoria selettiva e architetture con gate (slide 30)

La difficoltà delle dipendenze lontane motiva celle che possano imparare **quali informazioni conservare, quali aggiornare e quali esporre**. Le **Long Short-Term Memory**, o **LSTM**, introducono una memoria interna e gate apprendibili. Le **Gated Recurrent Unit**, o **GRU**, sono un'altra variante ricorrente, qui soltanto richiamata.

L'intuizione è accettare una memoria di dimensione fissata e imparare a gestirla. Non si può trattare lo stato come un archivio esplicito che contiene tutto il passato: conviene selezionare ciò che aiuta a ridurre la loss. Anche la vanilla RNN apprende una trasformazione selettiva degli ingressi; la differenza è che nella LSTM la selezione viene resa esplicita mediante gate e un percorso additivo della memoria.

Un **gate** si può immaginare come un interruttore graduabile. Un interruttore rigido lascia passare tutto o niente; un gate continuo può attenuare una componente in misura intermedia. I valori vicini a 1 lasciano passare quasi tutto, quelli vicini a 0 quasi nulla. Questa regolazione è appresa attraverso i pesi e la loss del compito, non impostata manualmente per ciascun ingresso.

Per le GRU basta qui ricordare il cenno conclusivo: impiegano **due gate**, comunemente detti aggiornamento e reset, e una struttura più compatta rispetto alla LSTM standard a parità di dimensioni. Non vengono sviluppate le equazioni GRU, né si deduce che due gate siano universalmente sufficienti per qualsiasi problema.

3.13 LSTM: stato interno e quadro delle equazioni (slide 31)

La LSTM mantiene due vettori di dimensione m:

- il **cell state** C_t, la memoria interna;
- lo **hidden state** h_t, la rappresentazione esposta dalla cella.

Entrambi vengono aggiornati lungo la sequenza. Il simbolo o_t ora indica l'**output gate**: non è l'uscita $o_t = Vh_t$ usata per la vanilla RNN. Per evitare ambiguità, una eventuale proiezione finale della LSTM sarà chiamata s_t.

Le slide LSTM usano prodotti con vettori riga, mentre la slide 22 usava vettori colonna. Manteniamo qui la convenzione a colonne: la trasformazione $x_t U_i$ della slide diventa $U_i x_t$, con la matrice trasposta rispetto alla convenzione a righe. Non cambia l'operazione rappresentata.

Come nelle slide, i bias sono omessi. Il passo completo è:

$$
\begin{aligned}
i_t &= \sigma(U_i x_t + W_i h_{t-1}),\\
f_t &= \sigma(U_f x_t + W_f h_{t-1}),\\
o_t &= \sigma(U_o x_t + W_o h_{t-1}),\\
\widetilde{C}_t &= \tanh(U_C x_t + W_C h_{t-1}),\\
C_t &= f_t \odot C_{t-1} + i_t \odot \widetilde{C}_t,\\
h_t &= o_t \odot \tanh(C_t).
\end{aligned}
$$

Quantità	Forma	Significato
x_t	$d \times 1$	Osservazione corrente
h_{t-1}, h_t	$m \times 1$	Stato nascosto precedente e corrente
C_{t-1}, C_t	$m \times 1$	Memoria precedente e corrente
i_t, f_t, o_t	$m \times 1$	Gate di ingresso, mantenimento e uscita
$\widetilde{C}_t$	$m \times 1$	Contenuto candidato da scrivere
U_i, U_f, U_o, U_C	$m \times d$	Pesi che elaborano l'ingresso
W_i, W_f, W_o, W_C	$m \times m$	Pesi che elaborano lo stato precedente
$\odot$	Operazione fra vettori della stessa forma	Prodotto elemento per elemento

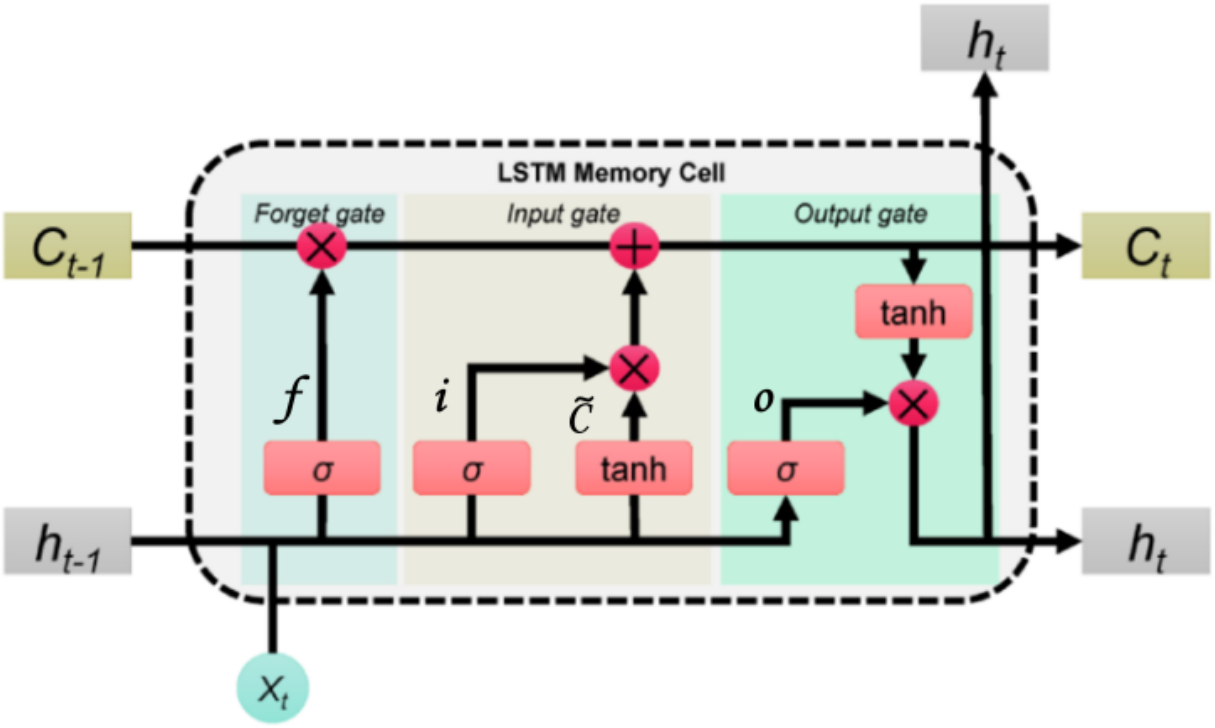


Figura 3.12: Slide 31 — Schema della cella LSTM

Le equazioni non sono un risultato da dedurre dalla vanilla RNN: **definiscono la nuova architettura**. Per comprenderle si segue il flusso del calcolo: si costruiscono i gate e il candidato da x_t, h_{t-1}; si aggiorna la memoria; si ricava lo stato nascosto. I passaggi seguenti esplicitano ogni operazione.

3.14 Gate: interruttori continui apprendibili (slide 32)

Tutti i gate ricevono ingresso corrente e stato nascosto precedente, ma hanno **matrici proprie**. Per un gate generico g_t:

$$z_{g,t} = U_g x_t + W_g h_{t-1} \in \mathbb{R}^m \qquad \text{combinazione delle due sorgenti,}$$
$$g_{t,j} = \sigma(z_{g,t,j}) = \frac{1}{1+e^{-z_{g,t,j}}} \in (0,1) \qquad \text{sigmoide per ciascuna componente.}$$

La sigmoide dà una quantità interpretabile come frazione da lasciare passare. Per esempio:

Pre-attivazione	Gate risultante	Interpretazione
0	0.5	Passaggio a metà
$\ln 3$	$1/(1+1/3) = 0.75$	Passaggio al 75%
$-\ln 3$	$1/(1+3) = 0.25$	Passaggio al 25%

Con pre-attivazioni finite, una sigmoide matematica è strettamente fra 0 e 1. I casi esatti 0 e 1 sono idealizzazioni utili per capire i meccanismi. La sigmoide è la scelta della cella descritta: il requisito concettuale è una selezione continua compatibile con l'apprendimento, non un teorema che impedisce ogni altro possibile gate.

3.14.1 Prodotto di Hadamard (slide 31–32)

Il simbolo $\odot$ è il **prodotto di Hadamard**:

$$\boxed{(a \odot b)_j = a_j b_j.}$$

Due vettori di m componenti producono un vettore di m componenti. Ogni posizione viene moltiplicata soltanto per la posizione corrispondente, senza sommare fra coordinate.

L'esempio dell'interruttore selettivo è:

$$\begin{bmatrix}1\\2\\3\end{bmatrix} \odot \begin{bmatrix}0\\1\\0\end{bmatrix} = \begin{bmatrix}1\cdot 0\\2\cdot 1\\3\cdot 0\end{bmatrix} = \begin{bmatrix}0\\2\\0\end{bmatrix}.$$

Un gate può quindi conservare una componente e sopprimerne altre. È diverso sia da un prodotto scalare, che restituirebbe un solo numero, sia dalla trasformazione tramite una matrice che mescola componenti.

3.14.2 I tre ruoli distinti (slide 32)

Gate	Dove interviene	Cosa controlla
Forget f_t	$f_t \odot C_{t-1}$	Quanto della memoria precedente conservare
Input i_t	$i_t \odot \widetilde{C}_t$	Quanto del candidato scrivere nella memoria
Output o_t	$o_t \odot \tanh(C_t)$	Quanto dello stato trasformato esporre come h_t

Il nome *forget gate* può trarre in inganno: un valore grande **conserva** il contenuto precedente; un valore piccolo lo attenua. La quantità scartata, lungo quel percorso, cresce quando f_t diminuisce. Il gate di ingresso filtra il candidato, che dipende anche da h_{t-1}, non semplicemente i valori grezzi di x_t.

Errata slide 32 — Interpretazione del forget gate

La descrizione in termini di memoria da «sovrascrivere» va letta attraverso l'equazione: il valore del forget gate è la frazione di memoria **conservata** sul percorso diretto. Un valore vicino a 1 non significa dimenticare tutto.

I gate non sono tre RNN indipendenti con tre memorie nascoste separate: sono tre trasformazioni apprendibili che leggono lo stesso x_t e lo stesso h_{t-1} e operano in punti diversi della cella.

3.15 Stato candidato: proporre un nuovo contenuto (slide 33)

Il candidato usa una combinazione di ingresso e stato precedente simile alla vanilla RNN:

$$\boxed{\widetilde{C}_t = \tanh(U_C x_t + W_C h_{t-1}) \in \mathbb{R}^m.}$$

Prima si proiettano le due sorgenti nello stesso spazio, poi si sommano e si applica tanh. Le matrici sono proprie del candidato, distinte da quelle dei gate. Ogni componente del candidato è compresa fra -1 e 1: può proporre un contributo positivo o negativo.

Per esempio, se una componente della pre-attivazione vale $\ln 3$, allora

$$\begin{aligned}\widetilde{C}_{t,j} = \tanh(\ln 3) &= \frac{e^{\ln 3} - e^{-\ln 3}}{e^{\ln 3} + e^{-\ln 3}} \\ &= \frac{3 - 1/3}{3 + 1/3} = \frac{8/3}{10/3} = 0.8.\end{aligned}$$

È un contenuto **candidato**, non ancora la nuova memoria. Se il gate di ingresso corrispondente vale 0.5, la componente effettivamente scritta è $0.5 \cdot 0.8 = 0.4$. Il modello separa così la proposta del contenuto dalla decisione su quanto scriverne.

3.16 Aggiornare la memoria ed esporre lo stato (slide 34)

La memoria nuova combina due percorsi:

$$\boxed{C_t = f_t \odot C_{t-1} + i_t \odot \widetilde{C}_t.}$$

Per ciascuna componente:

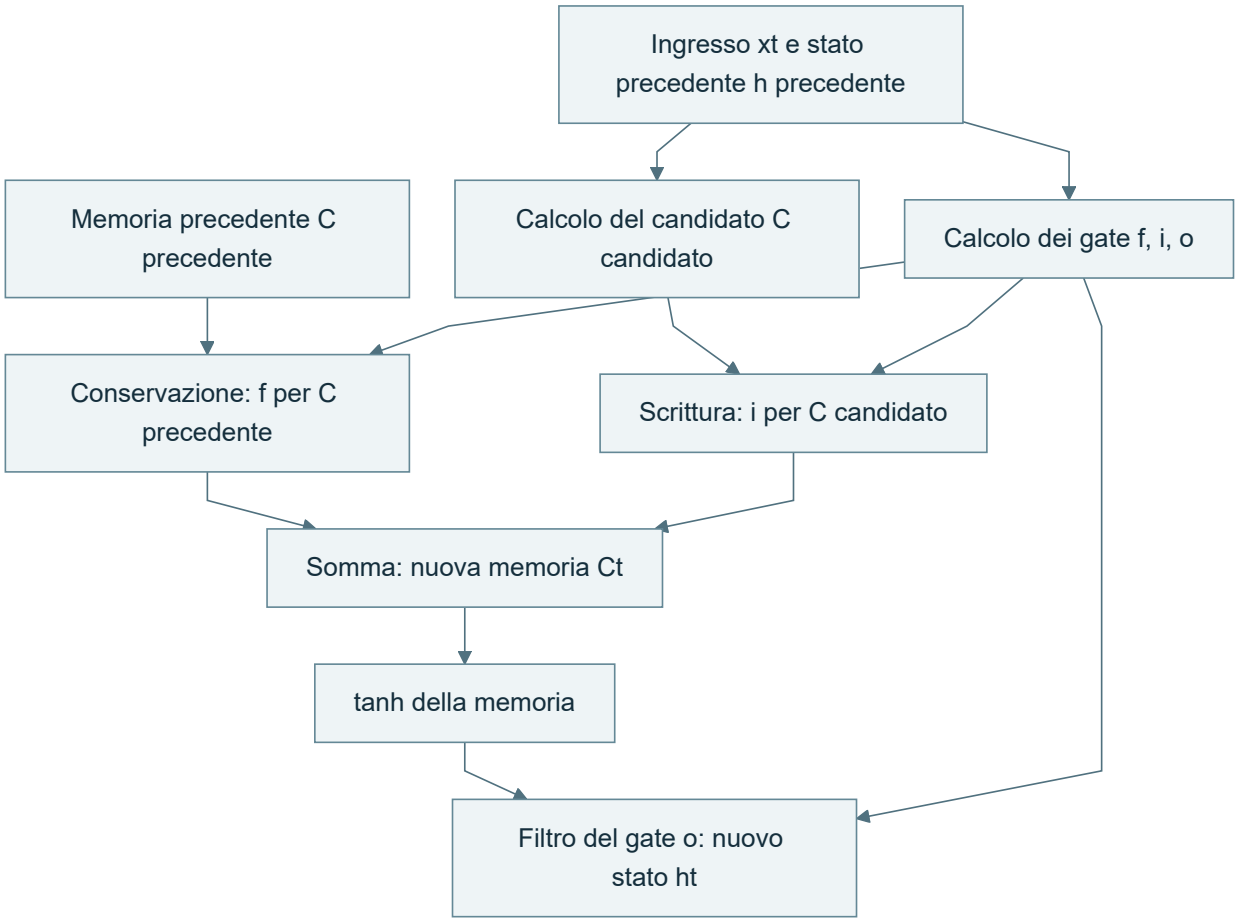
$$\begin{aligned} p_{t,j} &= f_{t,j} C_{t-1,j} && \text{parte conservata del passato,} \\ n_{t,j} &= i_{t,j} \widetilde{C}_{t,j} && \text{parte accettata del candidato,} \\ C_{t,j} &= p_{t,j} + n_{t,j} && \text{somma dei contributi.} \end{aligned}$$

Non è obbligatoriamente una media pesata: $f_{t,j} + i_{t,j}$ non deve essere 1. Le due selezioni sono apprese separatamente. Lo stato C_t non è perciò vincolato all'intervallo $[-1, 1]$ del candidato.

Il secondo passaggio costruisce lo stato nascosto:

$$\boxed{h_t = o_t \odot \tanh(C_t).}$$

Prima si trasforma ogni componente della memoria con tanh, poi si applica il gate di uscita. La cella può conservare un contenuto in C_t e decidere di non esporlo pienamente in h_t. Il risultato rimane un vettore di m componenti.



3.16.1 Che cosa accade quando un gate si chiude? (slide 34)

I casi limite rendono esplicito il significato delle formule:

Caso ideale, componente per componente	Nuova memoria	Significato
$f_t = 1, i_t = 0$	$C_t = C_{t-1}$	Conservare il passato senza nuova scrittura
$f_t = 0, i_t = 1$	$C_t = \widetilde{C}_t$	Sostituire la memoria con il candidato
$f_t = 0, i_t = 0$	$C_t = 0$	Eliminare entrambi i contributi
$i_t = 0, f_t = 0.5$	$C_t = 0.5C_{t-1}$	Nessuna nuova scrittura, ma attenuazione del passato

Chiudere soltanto l'input gate non garantisce di conservare lo stato identico: rimane il fattore f_t. Anche l'espressione «ignorare l'ingresso corrente» va intesa sul percorso di scrittura, perché x_t può continuare a influenzare gli altri gate.

Se l'output gate vale zero, $h_t = 0$ su quella componente, ma C_t può rimanere diverso da zero. Chiudere l'esposizione non equivale a cancellare la memoria.

Errata slide 34 — Indice della memoria precedente

Il testo descrittivo indica la memoria del passo precedente come $C^{(t)}$: il simbolo corretto è $C^{(t-1)}$. L'equazione dell'aggiornamento nella slide usa già l'indice corretto. Il forget gate pesa il contenuto precedente prima della somma con il candidato.

3.16.2 Esempio scalare completo della cella (slide 31–34)

Prendiamo $d = m = 1$, $x_t = 1$, $h_{t-1} = 0$ e $C_{t-1} = 2$. Poniamo tutti i pesi ricorrenti W_i, W_f, W_o, W_C a zero e scegliamo $U_i = U_o = 0$, $U_f = U_C = \ln 3$. I bias rimangono omessi.

Quantità	Calcolo	Valore
Input gate	$i_t = \sigma(0 \cdot 1 + 0 \cdot 0)$	0.5
Forget gate	$f_t = \sigma((\ln 3) \cdot 1 + 0 \cdot 0)$	0.75
Output gate	$o_t = \sigma(0 \cdot 1 + 0 \cdot 0)$	0.5
Candidato	$\widetilde{C}_t = \tanh((\ln 3) \cdot 1 + 0 \cdot 0)$	0.8
Memoria conservata	$f_t C_{t-1} = 0.75 \cdot 2$	1.5
Nuova scrittura	$i_t \widetilde{C}_t = 0.5 \cdot 0.8$	0.4
Nuova memoria	$C_t = 1.5 + 0.4$	1.9
Stato nascosto	$h_t = 0.5 \tanh(1.9)$	≈ 0.478119

L'esempio verifica due punti: la memoria può superare 1 e la sua esposizione può essere attenuata dal gate di uscita. Il valore iniziale $C_{t-1} = 2$ è ammesso; con output gate precedente nullo si può anche avere $h_{t-1} = 0$ pur conservando memoria non nulla.

Con un'eventuale proiezione scalare $s_t = 2h_t$ si avrebbe $s_t \approx 0.956237$. Questa proiezione è esterna al meccanismo dei gate: il gate $o_t = 0.5$ non è la predizione finale.

3.17 Perché separare cell state e hidden state (slide 35)

Il **cell state** offre un percorso di conservazione e aggiornamento additivo della memoria. Lo **hidden state** espone una versione trasformata e filtrata di tale memoria al resto della rete e al calcolo dei passi successivi.

L'interpretazione «memoria a lungo termine» per C_t e «a breve termine» per h_t descrive i loro ruoli, non due durate fissate. Non c'è un numero prestabilito di passi oltre il quale un contenuto debba scomparire: la conservazione dipende dai gate e dall'apprendimento.

3.17.1 Percorso diretto della memoria e gradiente (slide 30–35)

Per chiarire il vantaggio rispetto alla vanilla RNN, consideriamo il percorso diretto da C_{t-1} a C_t, mantenendo fissi gate e candidato. Per la componente j:

$$
\begin{aligned}
C_{t,j} &= f_{t,j} C_{t-1,j} + i_{t,j} \widetilde{C}_{t,j}, \\
\left.\frac{\partial C_{t,j}}{\partial C_{t-1,j}}\right|_{\text{diretto}} &= f_{t,j} && \text{derivata del termine lineare nella memoria,} \\
\left.\frac{\partial C_t}{\partial C_{t-1}}\right|_{\text{diretto}} &= \operatorname{diag}(f_t) && \text{aggiornamento separato per componente.}
\end{aligned}
$$

Se un gate di mantenimento è vicino a 1 per più passi, questo percorso può trasportare la memoria e il relativo gradiente senza imporre ogni volta una nuova moltiplicazione per W e una nuova derivata di tanh come nella catena della vanilla RNN. Per esempio, dieci fattori 0.99 danno $0.99^{10} \approx 0.9044$, contro $0.5^{10} \approx 0.0009766$.

Si tratta del **contributo diretto**, non della derivata totale della cella: gate e candidato dipendono anche da h_{t-1} e quindi introducono altri percorsi. Le LSTM mitigano la difficoltà delle dipendenze lontane; non garantiscono memoria infinita né assenza di vanishing o exploding gradient in ogni condizione.

La conservazione rimane orientata alla loss: i gate non hanno un criterio esterno che stabilisca cosa sia interessante. La BPTT aggiorna le loro matrici affinché la selezione aiuti la predizione del target. L'intuizione di un interruttore è utile, ma la selezione è continua e differenziabile, non una decisione simbolica separata.

3.17.2 Confine della trattazione e collegamenti (slide 35; riferimenti 36–37)

La trattazione delle slide termina con i due stati della LSTM. Il cenno alle GRU resta limitato alla variante con due gate e alla maggiore compattezza; non sono state sviluppate le relative equazioni. La bibliografia della slide 36 richiama i lavori su LSTM, GRU e difficoltà dell'addestramento ricorrente; la slide 37 contiene ringraziamenti.

I Transformer sono soltanto anticipati come successivo argomento: possono evitare la specifica catena di transizioni ricorrenti lungo il tempo, ma non è corretto trasformare questo richiamo nella garanzia che eliminino ogni difficoltà dei gradienti. La loro architettura non viene aggiunta a questa nota. Il laboratorio RNN è annunciato, senza sviluppare qui codice PyTorch non presente nelle fonti della lezione.

I collegamenti utili per il ripasso sono Multilayer Perceptron per neurone, attivazioni, loss e regola della catena; reti convoluzionali per condivisione dei pesi, composizione dei blocchi e connessioni residue; manuale introduttivo di Machine Learning (documento separato) per regressione e ottimizzazione.

3.18 Riepilogo operativo (slide 4–35)

Concetto	Formula o forma	Interpretazione e condizioni
Sequenza	$X \in \mathbb{R}^{T \times d}$	T passi, d feature per passo; T può variare
Codifica periodica	$(\cos(2\pi k/P), \sin(2\pi k/P))$	Preserva la vicinanza a cavallo del ciclo
AR(p)	$x_t = \sum_{i=1}^{p} \phi_i x_{t-i} + \xi_t$	Dipendenza lineare dai ritardi più innovazione
Stato RNN	$h_t = \phi(W h_{t-1} + U x_t)$	Ingresso corrente e storia compressa
Uscita RNN	$o_t = V h_t$	Score o quantità di uscita, non automaticamente probabilità
Dimensioni dei pesi	$U : m \times d$, $W : m \times m$, $V : r \times m$	Convenzione a vettori colonna
Pesi RNN	$md + m^2 + rm$	Senza bias o stato iniziale appreso; indipendenti da T
Jacobiana temporale	$J_t = D_t W$	Include derivata dell'attivazione e pesi ricorrenti
Dipendenza lontana	$\partial h_t / \partial h_k = J_t \cdots J_{k+1}$	Prodotto delle transizioni intermedie
BPTT finale	$\delta_T = D_T V^T g$, $\delta_t = D_t W^T \delta_{t+1}$	Una sola loss finale; $g = \nabla_{o_T} \mathcal{L}$
Gradiente di V	$g h_T^T$	Prodotto esterno alla proiezione finale
Gradienti condivisi	$\nabla_W \mathcal{L} = \sum_t \delta_t h_{t-1}^T$, $\nabla_U \mathcal{L} = \sum_t \delta_t x_t^T$	Si sommano le occorrenze dello stesso parametro
Gate LSTM	$g_t = \sigma(U_g x_t + W_g h_{t-1})$	Un valore fra 0 e 1 per componente; $g = i, f, o$
Candidato LSTM	$\widetilde{C}_t = \tanh(U_C x_t + W_C h_{t-1})$	Proposta di contenuto da filtrare con i_t
Memoria LSTM	$C_t = f_t \odot C_{t-1} + i_t \odot \widetilde{C}_t$	Conservazione più scrittura; non necessariamente media

Concetto	Formula o forma	Interpretazione e condizioni
Hidden state LSTM	$h_t = o_t \odot \tanh(C_t)$	Esposizione filtrata della memoria

Da saper spiegare

Una RNN riusa le stesse matrici per aggiornare uno stato di dimensione fissata, quindi può elaborare sequenze di lunghezza variabile. La BPTT applica la regola della catena al grafo srotolato e somma i contributi ai pesi condivisi. I lunghi prodotti di Jacobiane possono attenuare o amplificare il gradiente. La LSTM introduce una memoria con aggiornamento additivo e tre gate apprendibili: conservare il passato, scrivere il candidato ed esporre lo stato sono operazioni distinte.

3.19 Errata e precisazioni da conservare nel ripasso (slide 4–35)

Riferimento	Correzione o precisazione
Slide 4	Discreto non significa finito; il campione osservato è finito, il processo può proseguire.
Slide 6–8	Stagionalità e ciclo non richiedono la stessa periodicità rigida; residuo non equivale automaticamente a rumore bianco. Media e varianza non esauriscono la stazionarietà.
Slide 10	Un processo stocastico noto può mantenere innovazioni imprevedibili: conoscere il DGP non dà sempre il valore futuro esatto.
Slide 11	Una fascia nel grafico non prova copertura al 90% o gaussianità; l'incertezza è rappresentata, non derivata qui.
Slide 12	Per prevedere x_t con q ritardi usare $x_{t-1}, \ldots, x_{t-q}$; lo schema dei disturbi non esplicita l'innovazione corrente.
Slide 13–14	Rumore bianco non implica gaussianità; nel MA μ è la media del processo, stimabile mediante la media empirica.
Slide 18	Usare codifiche cicliche per periodi effettivi; l'anno assoluto non è automaticamente ciclico. Media empirica zero non è garantita dalla sola coppia seno-coseno.
Slide 22–23	U, W, V sono parametri; h_t è un'attivazione della sequenza. Lunghezza variabile non significa numero di feature variabile.
Slide 24–27	Lo srotolamento riusa i pesi; la BPTT non crea la condivisione. La profondità temporale cresce con i passi.
Slide 27	Le derivate nella somma devono essere locali; per h_0 indipendente dai pesi il termine $k = 0$ è nullo. Il gradiente di V usa lo stato finale, che dipende dalla storia.
Slide 28	La Jacobiana della transizione è $D_t W$, non la sola derivata dell'attivazione.
Slide 29	Il clipping limita gradienti grandi, senza ricostruire contributi già svaniti.

Riferimento	Correzione o precisazione
Slide 30–32	I gate sono continui, non bit rigidi; il forget gate alto conserva, basso attenua. La LSTM non è l'unico modello con memoria di dimensione fissata.
Slide 31	Notazione LSTM a righe convertita coerentemente a colonne; l'output gate o_t è distinto dallo score della RNN.
Slide 34	La memoria precedente è C_{t-1}. Con $i_t = 0$ rimane $f_t \odot C_{t-1}$: identità solo se anche $f_t = 1$ nelle componenti considerate.
Slide 35	LSTM e GRU mitigano il problema delle dipendenze lontane; non garantiscono conservazione illimitata o stabilità assoluta.

Le forme trascritte come «Adamart/Adamard», «EFO» e «bad propagation» sono ricondotte rispettivamente a **Hadamard**, ai gate i, f, o e alla **backpropagation**, coerentemente con formule e contesto. I frammenti non intelligibili della domanda sul confronto con altri regressori non consentono di ricostruire un metodo preciso e non sono trasformati in contenuti aggiuntivi.

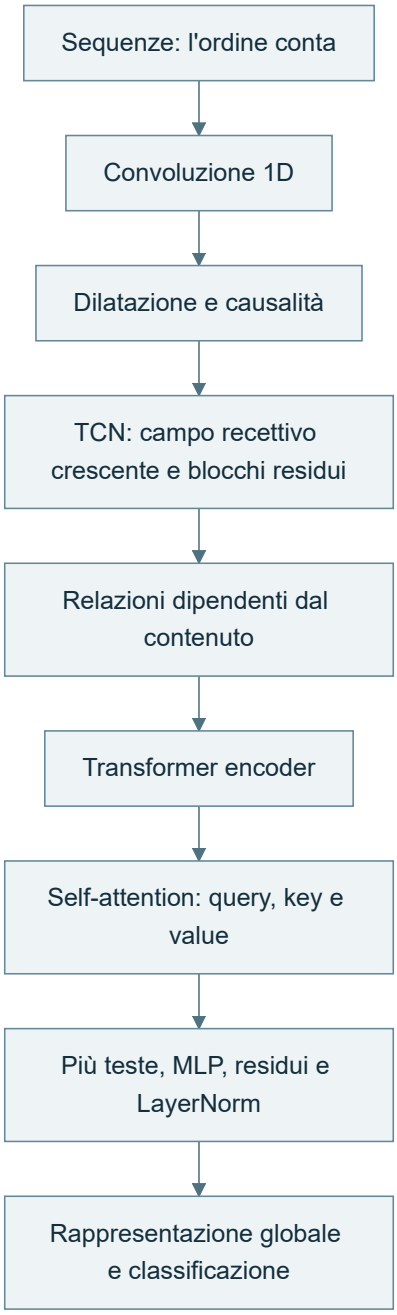
Argomento 4

TCN - Temporal Convolutional Networks

Contenuti dell'argomento

- Modelli per sequenze: l'ordine è informazione (slide 3–4)
- Dalla convoluzione 2D alla convoluzione 1D (slide 5–8)
- Dilatazione: ampliare la regione osservata senza nuovi pesi (slide 9)
- Causalità: non leggere un futuro indisponibile (slide 10)
- Temporal Convolutional Network: gerarchia e blocchi residui (slide 11)
- Caso di studio finanziario: dati, baseline e risultati (slide 12–13)
- TCN e RNN: memoria, parallelismo e spiegazioni (slide 14)

Le sequenze possono essere elaborate senza una ricorrenza sullo stato nascosto. Una prima possibilità consiste nello spostare un filtro convoluzionale lungo il tempo; una seconda nel costruire relazioni fra elementi mediante **attention**, con coefficienti che dipendono dall'ingresso corrente. Le **Temporal Convolutional Networks** (TCN) e i **Transformer** realizzano queste due idee.



4.1 Modelli per sequenze: l'ordine è informazione (slide 3–4)

Un modello sequenziale riceve una successione di elementi e produce un risultato oppure un'altra successione. Gli elementi possono essere misure numeriche, parole, lettere, segnali audio o rappresentazioni di immagini. Una traduzione può trasformare «Je suis étudiant» in «I am a student»: il numero di elementi in ingresso e in uscita può essere diverso.

L'ordine modifica il significato. Nel testo, cambiare la posizione di soggetto e oggetto può cambiare chi compie un'azione; nelle serie temporali, scambiare due misure cambia l'evoluzione osservata. Traduzione, forecasting, descrizione di video, generazione audio e riassunto di testi sono esempi di compiti sequenziali.

Una sequenza di T osservazioni con C feature per osservazione si può disporre in una matrice $X \in \mathbb{R}^{T \times C}$, con un'osservazione per riga. Qui l'indice temporale può anche indicare semplicemente

una posizione nel testo. La distinzione fra lunghezza e numero di feature è quella introdotta in reti ricorrenti.

Una RNN elabora gli ingressi successivamente e li riassume nello stato. Le alternative qui considerate mantengono invece una sequenza di rappresentazioni intermedie: le convoluzioni la trasformano con finestre locali; l'attention mette in relazione elementi in base al loro contenuto.

4.2 Dalla convoluzione 2D alla convoluzione 1D (slide 5–8)

Una convoluzione sulle immagini sposta il kernel lungo altezza e larghezza. Per una sequenza basta un solo asse di scorrimento: il tempo o la posizione. Le feature di ogni osservazione svolgono il ruolo dei canali dell'immagine.

Una convoluzione 1D ha un solo asse di scorrimento, non necessariamente un solo peso per posizione. Un filtro su una finestra di k elementi, ciascuno con C_{in} feature, contiene kC_{in} pesi, più un eventuale bias. Il kernel copre tutti i canali e scorre lungo la sequenza, come descritto per i canali in reti convoluzionali.

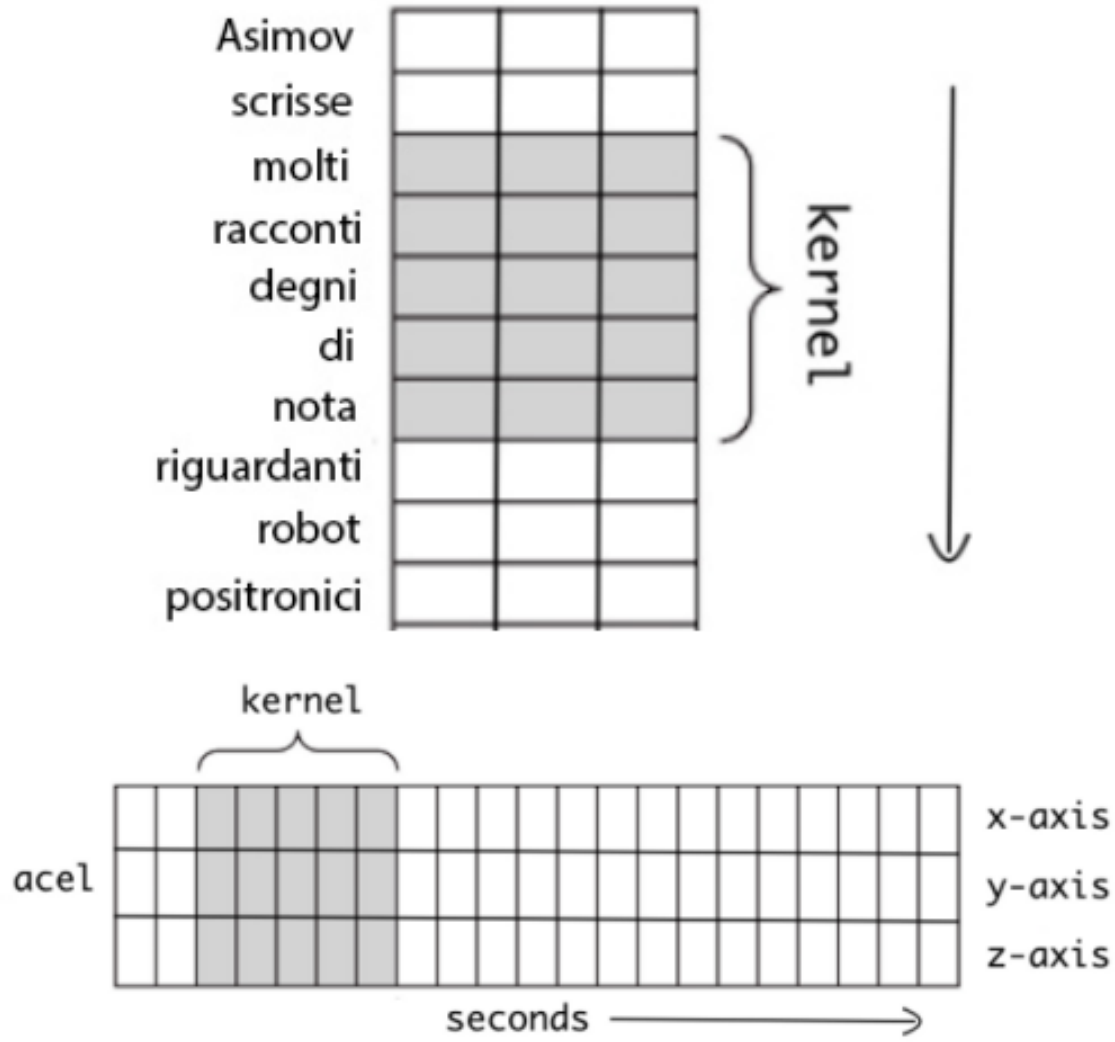


Figura 4.1: Slide 7 — Finestre su testo e segnale multivariato

4.2.1 Operazione scalare e multicanale (slide 7)

La convoluzione discreta scalare riportata nelle slide è

$$\boxed{y_t = (x * \omega)(t) = \sum_{i=0}^{k-1} x * (t-i)\omega_i.}$$

x_t, y_t, ω_i sono scalari; k è il numero di coefficienti del filtro. Si prende una finestra di valori, si moltiplica ogni valore per il peso associato e si sommano i risultati. Per $k = 3$:

$$y_t = x_t\omega_0 + x_{t-1}\omega_1 + x_{t-2}\omega_2 \quad \text{sviluppo della sommatoria.}$$

Con valori correnti e passati $(3, 2, 1)$ e pesi $(1, 0, -1)$ si ottiene $3 \cdot 1 + 2 \cdot 0 + 1 \cdot (-1) = 2$. La formula usa pesi ordinati per ritardo; all'inizio della serie occorre precisare come trattare gli indici mancanti, per esempio mediante padding.

Per l'esempio a finestre delle slide è più comoda la notazione di **correlazione incrociata** senza ribaltamento del kernel, come già chiarito in reti convoluzionali. Per una finestra che parte in t, con stride 1:

$$\boxed{Y_{t,o} = b_o + \sum_{i=0}^{k-1} \sum_{c=1}^{C_{\text{in}}} X_{t+i,c} K_{i,c,o}.}$$

X è la sequenza di feature; $K_{i,c,o}$ è il peso all'offset temporale i, canale di ingresso c, filtro di uscita o; b_o è il bias di quel filtro. Si somma prima sui canali e poi sulle posizioni della finestra. Ogni filtro produce una feature per posizione, quindi C_{out} filtri producono C_{out} canali di uscita.

Non cambia il principio di apprendimento: la differenza fra le due convenzioni riguarda orientamento dei pesi e indici, non l'idea di applicare un prodotto scalare locale condiviso. La finestra $t, t+1, t+2$ descrive qui l'inizio della patch; non va confusa con un'uscita causalmente allineata al suo primo istante.

4.2.2 Esempio completo: «Asimov scrisse molti racconti sui robot positronici» (slide 8)

Ogni parola è rappresentata da quattro feature:

Parola	Feature 1	Feature 2	Feature 3	Feature 4
Asimov	0.2	0.1	-0.3	0.4
scrisse	0.5	0.2	-0.3	-0.1
molti	-0.1	-0.3	-0.2	0.4
racconti	0.3	-0.3	0.1	0.1
sui	0.2	-0.3	0.4	0.2
robot	0.1	0.2	-0.1	-0.1
positronici	-0.4	-0.4	0.2	0.3

La finestra temporale è lunga 3, perciò un filtro ha forma 3×4:

$$K = \begin{bmatrix} 3 & 1 & 2 & -3 \\ -1 & 2 & 1 & -3 \\ 1 & 1 & -1 & 1 \end{bmatrix}.$$

Le tre righe corrispondono alle tre posizioni della finestra; le quattro colonne alle feature. Non è una convoluzione che scorre anche sull'asse delle feature: quell'asse è coperto interamente, mentre lo scorrimento avviene lungo le parole.

Sulla prima finestra, con bias omesso, i tre prodotti scalari sono:

$$\begin{aligned}
u_1 &= 0.2 \cdot 3 + 0.1 \cdot 1 + (-0.3) \cdot 2 + 0.4 \cdot (-3) \\
&= 0.6 + 0.1 - 0.6 - 1.2 = -1.1, \\
u_2 &= 0.5 \cdot (-1) + 0.2 \cdot 2 + (-0.3) \cdot 1 + (-0.1) \cdot (-3) \\
&= -0.5 + 0.4 - 0.3 + 0.3 = -0.1, \\
u_3 &= (-0.1) \cdot 1 + (-0.3) \cdot 1 + (-0.2) \cdot (-1) + 0.4 \cdot 1 \\
&= -0.1 - 0.3 + 0.2 + 0.4 = 0.2, \\
y_1 &= u_1 + u_2 + u_3 = -1.1 - 0.1 + 0.2 = \boxed{-1}.
\end{aligned}$$

Spostando la finestra di una parola alla volta:

Finestra	Risultato verificato
Asimov – scrisse – molti	-1.0
scrisse – molti – racconti	-0.5
molti – racconti – sui	-3.6
racconti – sui – robot	-0.2
sui – robot – positronici	0.3

Asimov	0.2	0.1	-0.3	0.4
scrisse	0.5	0.2	-0.3	-0.1
molti	-0.1	-0.3	-0.2	0.4
racconti	0.3	-0.3	0.1	0.1
sui	0.2	-0.3	0.4	0.2
robot	0.1	0.2	-0.1	-0.1
positronici	-0.4	-0.4	0.2	0.3

Apply a **filter** (or **kernel**) of size 3

3	1	2	-3
-1	2	1	-3
1	1	-1	1

Asimov scrisse molti	-1
scrisse molti racconti	-0.5
molti racconti sui	-3.6
racconti sui robot	-0.2
sui robot positronici	0.3

Question: Why is the kernel a matrix 3×4?

Figura 4.2: Slide 8 — Feature delle parole, filtro e risultati

Da 7 posizioni si ottengono $7 - 3 + 1 = 5$ uscite con stride 1 e senza padding. Un filtro produce un canale; per generare più feature si usano più filtri distinti.

Il conteggio per uno strato standard, con bias, è

$$\boxed{P_{\text{Conv1D}} = C_{\text{out}}(kC_{\text{in}} + 1).}$$

Si contano kC_{in} pesi e un bias per filtro, poi si moltiplica per il numero di filtri. Con $k = 3$, $C_{\text{in}} = 4$ e 8 filtri sono $8(12 + 1) = 104$ parametri; nell'esempio con un solo filtro senza bias sono 12 pesi. La lunghezza della sequenza non compare nel conteggio.

Precisazione slide 5–6 — Richiamo storico e significato di 1D

LeNet-5 è un riferimento storico importante, ma non segna la prima comparsa in assoluto delle idee convoluzionali. La distinzione 1D/2D riguarda il numero di assi lungo cui scorre il filtro; con più canali, un filtro 1D non è semplicemente un vettore di soli k scalari.

4.3 Dilatazione: ampliare la regione osservata senza nuovi pesi (slide 9)

Una finestra piccola coglie relazioni locali. Per osservare elementi più lontani si può introdurre una **dilatazione**: i pesi vengono applicati a posizioni distanziate, lasciando intervalli fra un campione e il successivo.

$$\boxed{y_t = \sum_{i=0}^{k-1} x_{t-di}\omega_i, \qquad d \geq 1.}$$

d è il fattore di dilatazione, intero positivo. Nell'indice del campione si sostituisce il ritardo i con di. Per $k = 3$:

$$y_t = x_t\omega_0 + x_{t-d}\omega_1 + x_{t-2d}\omega_2.$$

La distanza fra primo e ultimo campione è $(k-1)d$; contando entrambi gli estremi, l'ampiezza temporale coperta è

$$\boxed{k_{\text{eff}} = 1 + (k-1)d.}$$

Con $k = 3, d = 2$ si usano gli offset $0, 2, 4$, quindi si copre un intervallo di 5 posizioni con **tre** pesi per canale. Con $d = 4$, gli offset sono $0, 4, 8$ e l'intervallo è lungo 9. La dilatazione non trasforma il kernel in un filtro denso di 5 o 9 pesi.



Figura 4.3: Slide 9 — Campioni distanziati nella finestra dilatata

La differenza dallo **stride** è essenziale:

- la dilatazione modifica la distanza fra i punti letti all'interno della finestra;
- lo stride modifica la distanza fra le posizioni di uscita a cui si applica la finestra.

Si può usare $d > 1$ mantenendo stride 1, senza sottocampionare la griglia delle uscite. È utile quando un evento molto breve, quasi impulsivo, non deve scomparire fra due posizioni campionate. Anche nelle immagini un sottocampionamento può perdere dettagli: la differenza è una scelta legata al compito, non una garanzia assoluta dei dati visivi.

Un livello dilatato legge solo alcuni punti della sua griglia di ingresso; se i livelli precedenti hanno già combinato osservazioni locali, quei punti rappresentano però regioni più estese. Saltare una posizione intermedia a un livello profondo non equivale necessariamente a scartare il corrispondente dato grezzo dall'intera rete.

Precisazione slide 9–11 — Indici, stride e crescita del campo recettivo

Con kernel di tre elementi e dilatazione 2 si leggono, in avanti, le posizioni 1, 3 e 5, non 1, 3 e 6. La dilatazione non richiede stride 2. Inoltre anche convoluzioni non dilatate a stride 1 fanno crescere il campo recettivo quando si impilano livelli: la dilatazione ne accelera la crescita.

4.4 Causalità: non leggere un futuro indisponibile (slide 10)

Per produrre un risultato al tempo t usando soltanto ciò che è già disponibile, una convoluzione deve dipendere da ingressi con indice **minore o uguale a** t. Questo è il vincolo di **causalità temporale**. Una finestra centrata su t che leggesse anche $t+1$ o $t+2$ violerebbe il vincolo.

La formula dilatata precedente è già scritta in forma causale, perché usa $t, t-d, \ldots, t-(k-1)d$. All'inizio mancano alcuni valori passati: un padding soltanto a sinistra permette di trattarli senza leggere valori futuri. Per stride 1 e uscita lunga quanto l'ingresso:

$$\boxed{P_{\text{sinistra}} = (k-1)d, \qquad P_{\text{destra}} = 0.}$$

Infatti il kernel effettivo è lungo $k_{\text{eff}} = 1 + (k-1)d$. Partendo da una sequenza di lunghezza T:

$$\begin{aligned} T_{\text{out}} &= T + P_{\text{sinistra}} - k_{\text{eff}} + 1 && \text{conteggio delle finestre a stride 1,} \\ &= T + (k-1)d - [1 + (k-1)d] + 1 = T && \text{sostituzione e semplificazione.} \end{aligned}$$

Con $k = 3, d = 2$ servono quattro valori di padding a sinistra. Se i pesi sono tutti 1 e i primi dati sono $(1, 2, 3, 4, 5)$, l'uscita al terzo istante usa $x_3 + x_1 + x_{-1} = 3 + 1 + 0 = 4$; al quinto usa $x_5 + x_3 + x_1 = 5 + 3 + 1 = 9$. Nessuno dei due risultati richiede valori futuri.

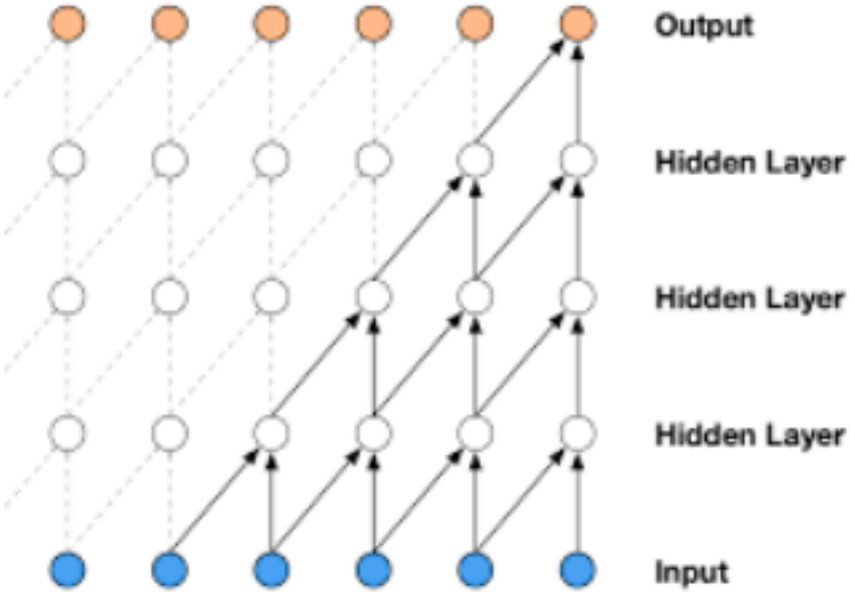


Figura 4.4: Slide 10 — Collegamenti causali fra i livelli

La causalità indica un vincolo di disponibilità dell'informazione, non una prova che un ingresso causi fisicamente un evento. Inoltre y_t può dipendere da x_t: se il compito consiste nel prevedere il valore ancora sconosciuto x_t, occorre allineare i target rispetto agli ingressi, per esempio prevedendo x_{t+1} dai dati fino a t. Causalità del filtro e definizione della previsione devono essere coerenti.

4.5 Temporal Convolutional Network: gerarchia e blocchi residui (slide 11)

Una **TCN** combina convoluzioni causali e dilatate in una gerarchia. La figura mostra kernel di ampiezza 3 e dilatazioni $1, 2, 4$: salendo di livello, ogni elemento riassume una porzione più lunga della sequenza originale.

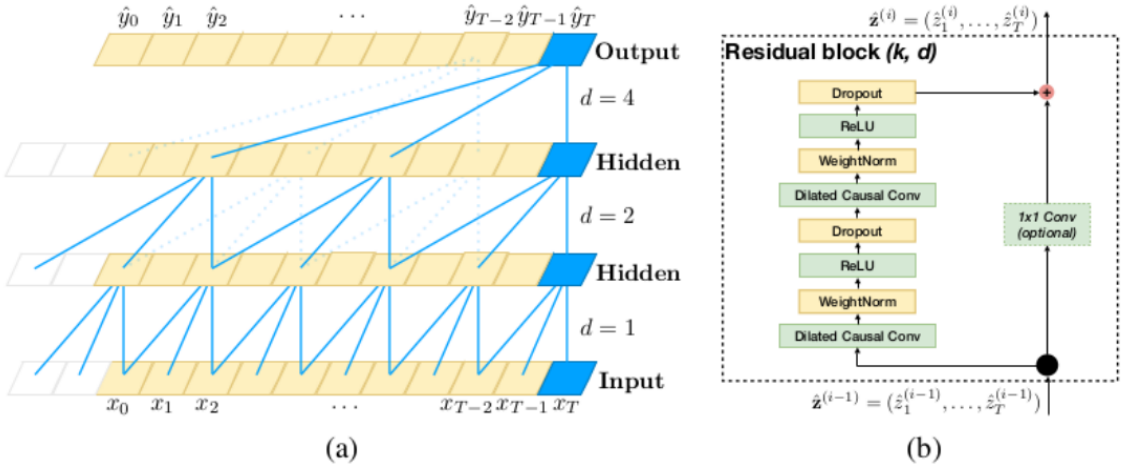


Figura 4.5: Slide 11 — Crescita del campo recettivo e blocco residuo TCN

4.5.1 Campo recettivo lungo la sequenza (slide 11)

Riprendiamo da reti convoluzionali il campo recettivo r_ℓ e la distanza j_ℓ fra centri di uscite adiacenti, misurati sull'ingresso originale. Con kernel k_ℓ, dilatazione d_ℓ e stride s_ℓ:

$$\boxed{\begin{aligned} r_\ell &= r_{\ell-1} + (k_\ell - 1) d_\ell j_{\ell-1}, \\ j_\ell &= s_\ell j_{\ell-1}, \qquad r_0 = j_0 = 1. \end{aligned}}$$

Fra primo e ultimo punto della finestra ci sono $k_\ell - 1$ intervalli; ognuno vale d_ℓ posizioni della griglia precedente, distanziate di $j_{\ell-1}$ sull'ingresso. Da qui il termine aggiunto. Lo stride determina invece la nuova distanza fra uscite.

Con stride sempre 1 si mantiene $j_\ell = 1$. Per i tre livelli della figura:

Livello	k	d	Calcolo	Campo recettivo
Ingresso	—	—	Valore iniziale	1
1	3	1	$1 + 2 \cdot 1$	3
2	3	2	$3 + 2 \cdot 2$	7
3	3	4	$7 + 2 \cdot 4$	15

Un'uscita finale può quindi dipendere da 15 istanti, pur usando kernel di soli tre campioni. I salti di un livello si combinano con le regioni già elaborate dal precedente.

4.5.2 Che cosa contiene il blocco reale (slide 11)

Il blocco a destra della figura comprende due sequenze convoluzione causale dilatata $\rightarrow$ normalizzazione dei pesi $\rightarrow$ ReLU $\rightarrow$ dropout, oltre a una connessione residua. La **WeightNorm** riguarda i pesi: non è la BatchNorm sulle attivazioni di reti convoluzionali e non viene qui sviluppata matematicamente.

La shortcut somma ingresso e ramo trasformato. Se i canali non coincidono, una convoluzione 1×1 può adattarli. È la stessa condizione di compatibilità delle forme già incontrata con ResNet: i due rami vengono calcolati e sommati, senza scegliere quale eseguire.

Il disegno a sinistra conta **una convoluzione per livello**. Se invece si impilano tre blocchi con due convoluzioni ciascuno alle dilatazioni $1, 2, 4$, il campo recettivo massimo del ramo profondo diventa $1 + 2 \cdot 2(1 + 2 + 4) = 29$. Il conteggio va fatto sulle operazioni effettive, non confondendo un blocco con una singola convoluzione.

Le convoluzioni possono elaborare sequenze di lunghezza diversa mantenendo gli stessi pesi. Una eventuale testa densa su una concatenazione completa può tuttavia reintrodurre il vincolo di lunghezza fissa; la flessibilità dell'estrattore non rende automaticamente flessibile qualsiasi classificatore finale.

4.6 Caso di studio finanziario: dati, baseline e risultati (slide 12–13)

Il caso illustrato combina tre fonti: valori giornalieri dell'indice DJIA, titoli di notizie e conoscenza strutturata da grafi. Il compito è classificare il movimento successivo come rialzo o ribasso. Le feature provenienti da più fonti possono essere allineate e affiancate lungo l'asse dei canali, mantenendo la struttura temporale.

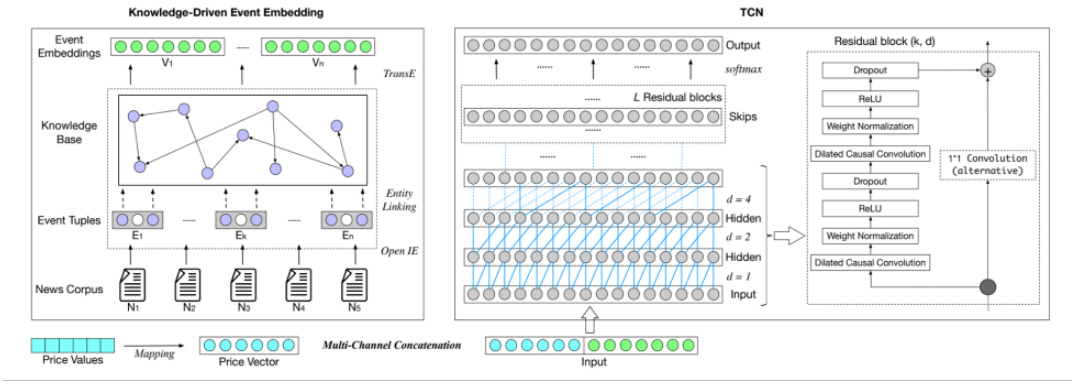


Figura 4.6: Slide 12 — Fusione delle fonti e classificazione del movimento

Aggiungere canali non cambia il principio convoluzionale, ma **aumenta i pesi del filtro**, perché ciascun filtro deve coprire anche le nuove feature. Non significa inserire dati eterogenei senza prima renderli rappresentabili e temporalmente coerenti. La costruzione dei grafi e degli embedding del caso di studio non è sviluppata.

Average results

Model	Accuracy	F1 Score
ARIMA	52.40% ± 0.25%	0.3844 ± 0.0052
LSTM	56.17% ± 0.26%	**0.5348 ± 0.0039**
CNN	55.41% ± 0.25%	0.5280 ± 0.0021
TCN	**60.96% ± 0.26%**	0.5317 ± 0.0036

Model	Input	
	Raw Data	Processed Training Data
ARIMA [2]	$\mathcal{X}$	$\mathcal{X}$
LSTM	$\mathcal{X}$	$\mathcal{X}$
CNN	$\mathcal{X}$	$\mathcal{X}$
TCN	$\mathcal{X}$	$\mathcal{X}$
WB-TCN	$\mathcal{N}$	*word embedding*
EB-TCN	$\mathcal{N}$	*event embedding*$^{(a)}$
PVWB-TCN	$\mathcal{X} + \mathcal{N}$	*price vector + word embedding*
PVEB-TCN	$\mathcal{X} + \mathcal{N}$	*price vector + event embedding*$^{(a)}$
KDEB-TCN	$\mathcal{N} + \mathcal{G}$	*event embedding*$^{(b)}$
KDTCN	$\mathcal{X} + \mathcal{N} + \mathcal{G}$	*price vector + event embedding*$^{(b)}$

Results on varying covariates infos

Model	Accuracy	F1 Score
TCN	60.96% ± 0.26%	0.5317 ± 0.0036
WB-TCN	62.96% ± 0.25%	0.5934 ± 0.0038
EB-TCN	64.74% ± 0.26%	0.6698 ± 0.0031
PVWB-TCN	66.49% ± 0.24%	0.5845 ± 0.0043
PVEB-TCN	67.51% ± 0.25%	0.6979 ± 0.0031
KDEB-TCN	68.51% ± 0.25%	0.6959 ± 0.0032
KDTCN	**69.77% ± 0.25%**	**0.7073 ± 0.0030**

Model	Accuracy	F1 Score
TCN	60.96% ± 0.26%	0.5317 ± 0.0036
WB-TCN	62.96% ± 0.25%	0.5934 ± 0.0038
EB-TCN	64.74% ± 0.26%	0.6698 ± 0.0031
PVWB-TCN	66.49% ± 0.24%	0.5845 ± 0.0043
PVEB-TCN	67.51% ± 0.25%	0.6979 ± 0.0031
KDEB-TCN	68.51% ± 0.25%	0.6959 ± 0.0032
KDTCN	**69.77% ± 0.25%**	**0.7073 ± 0.0030**

Results on abrupt changes

Figura 4.7: Slide 13 — Tabelle dei risultati e confronto delle feature

Modello, confronto iniziale	Accuracy media	F1
ARIMA	$52.40\% \pm 0.25\%$	0.3844 ± 0.0052
LSTM	$56.17\% \pm 0.26\%$	0.5348 ± 0.0039
CNN	$55.41\% \pm 0.25\%$	0.5280 ± 0.0021
TCN	$60.96\% \pm 0.26\%$	0.5317 ± 0.0036

La TCN migliora l'accuracy della LSTM di $60.96 - 56.17 = 4.79$ **punti percentuali**. La F1 è però leggermente più bassa: $0.5317 < 0.5348$. Dire che un modello è migliore richiede quindi di specificare la metrica, oltre al dataset e al protocollo.

Nella tabella con diverse covariate, il modello KDTCN che combina prezzi, notizie e conoscenza strutturata raggiunge $69.77\% \pm 0.25\%$ di accuracy e 0.7073 ± 0.0030 di F1. La tabella sui cambiamenti

bruschi riporta lo stesso valore centrale di accuracy, con dispersione 0.24%: le due tabelle non vanno fuse attribuendo loro un'identica condizione sperimentale.

Un'accuracy attorno al 60% è un risultato del caso studiato, non una garanzia di previsione finanziaria o redditività. Una baseline casuale al 50% di accuracy presuppone una scelta binaria equiprobabile; per le metriche e il dataset reale servono confronti appropriati. In particolare una F1 pari a 0.5 non è una baseline universale: dipende da prevalenza della classe positiva e regola di predizione. Il significato delle metriche si collega a manuale introduttivo di Machine Learning (documento separato).

4.7 TCN e RNN: memoria, parallelismo e spiegazioni (slide 14)

Una RNN conserva uno stato di dimensione fissata. Riassumere 20 osservazioni o 4000 osservazioni in 128 numeri pone un diverso carico alla stessa rappresentazione: è l'idea del **riassunto con perdita** discussa in reti ricorrenti. Una TCN conserva invece mappe di feature indicizzate lungo la sequenza, fino a un'eventuale aggregazione finale.

La maggiore lunghezza delle mappe ha un costo di memoria. A parità di canali, raddoppiare la lunghezza raddoppia le attivazioni di un livello che la conserva. Inoltre una TCN di profondità fissata vede un campo recettivo finito: accettare ingressi più lunghi non amplia automaticamente la storia osservabile da ogni uscita.

Il vantaggio computazionale è l'assenza di una ricorrenza che imponga di completare h_t prima di calcolare h_{t+1}. Le posizioni di una convoluzione nello stesso livello possono essere calcolate in parallelo, oltre ai filtri; rimane la dipendenza fra livelli successivi.

Per l'apprendimento si applica la backpropagation al grafo convoluzionale, senza srotolare una cella ricorrente per ogni istante. **La backpropagation non scompare.** Le connessioni residue e la struttura della rete possono facilitarla; non eliminano ogni possibile instabilità.

> **Errata slide 14 — Nessuna garanzia assoluta sui gradienti**
>
> L'affermazione «no vanishing/exploding gradients» è troppo forte. La TCN evita la specifica catena ricorrente attraverso tutti gli istanti, ma una rete convoluzionale profonda può comunque avere gradienti che svaniscono o esplodono. I risultati favorevoli rispetto a LSTM e GRU descrivono confronti sperimentali, non una superiorità universale.

La struttura convoluzionale consente anche di adattare strumenti di localizzazione come Grad-CAM per evidenziare porzioni della sequenza associate a una decisione. Non segue che le RNN siano impossibili da analizzare: anche un grafo ricorrente srotolato può essere studiato con metodi di attribuzione. Il richiamo alla spiegabilità indica una possibilità pratica delle TCN, senza sviluppare qui l'algoritmo Grad-CAM né trasformare una mappa di attribuzione in una prova causale.

Argomento 5

Transformer

Contenuti dell'argomento

- Dai collegamenti regolari alle relazioni dipendenti dal contenuto (slide 15–18)
- Encoder e decoder: ruoli e perimetro (slide 19–20)
- Dall'ingresso ai token e alla posizione (slide 21)
- Blocco encoder: attention, MLP, residui e normalizzazione (slide 22–25)
- Layer Normalization: normalizzare le feature del singolo token (slide 26–28)
- Attention: ricostruire ogni token dal contesto (slide 29–32)
- Query, key e value: tre proiezioni dello stesso ingresso (slide 33)
- Single-head attention: dal confronto alla somma pesata (slide 34)
- Multi-Head Attention: più confronti in parallelo (slide 35–36)
- Dalla sequenza di rappresentazioni alla classificazione (slide 37–38)
- TCN e Transformer: confronto, modalità e costi (slide 39–41)
- Confine della spiegazione e materiale rinviato (slide 42–60)
- Riepilogo delle forme e delle formule (slide 5–41)
- Errata e precisazioni principali (slide 5–41)

5.1 Dai collegamenti regolari alle relazioni dipendenti dal contenuto (slide 15–18)

Le convoluzioni impongono un'organizzazione regolare dei collegamenti. La dilatazione può ampliare la distanza fra campioni, ma la regola dei salti è fissata dall'architettura: una volta scelti kernel e dilatazioni, le posizioni potenzialmente collegate sono determinate.

Molte dipendenze non seguono una distanza costante. In una frase, un nome può essere collegato al suo articolo e poi richiamato molto più avanti; fra i due possono comparire quantità diverse di parole. Nel codice, la chiusura di un ciclo `for` è collegata alla sua apertura, ma la distanza cambia con la lunghezza del blocco. Non basta quindi scegliere un unico salto periodico adatto a tutti gli esempi.

La domanda è: **si possono calcolare, per ogni ingresso, coefficienti che descrivano quali elementi mettere in relazione?** L'attention costruisce proprio questi coefficienti. Non cambia i parametri del modello durante il forward: usa parametri già appresi per produrre relazioni che dipendono dai dati correnti.

Il Transformer presentato in *Attention Is All You Need* del 2017 combina meccanismi di attention con trasformazioni lineari, MLP, connessioni residue e normalizzazione. Non è una rete priva di struttura o di bias induttivi: cambia il modo di far interagire le posizioni, rendendolo dipendente dal contenuto.

5.2 Encoder e decoder: ruoli e perimetro (slide 19–20)

L'architettura illustrata ha due rami:

- l'**encoder** trasforma una sequenza in una sequenza di rappresentazioni;

- il **decoder** costruisce una sequenza di uscita, eventualmente condizionandosi anche alle rappresentazioni dell'encoder.

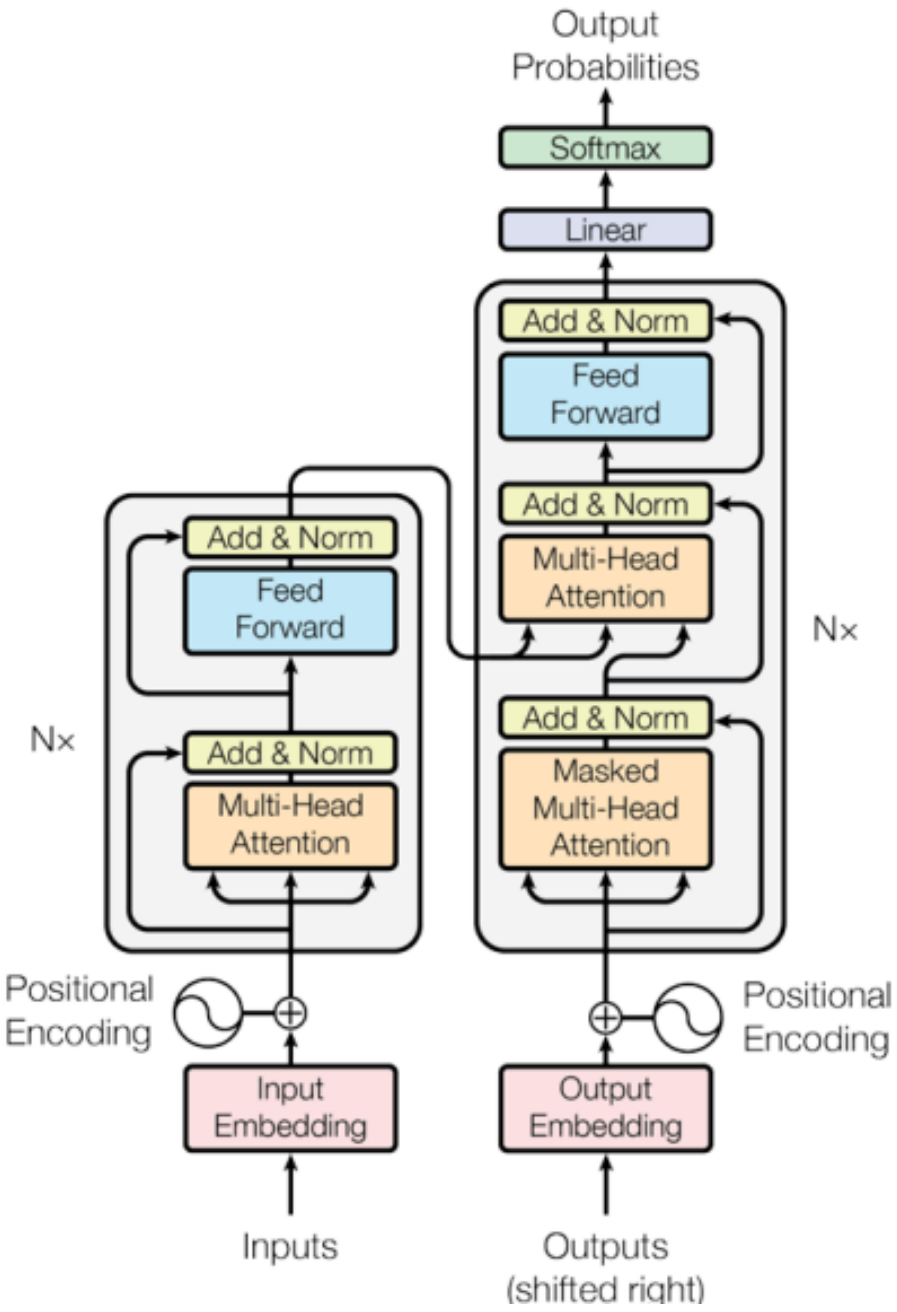


Figura 5.1: Slide 19 — Architettura encoder–decoder di riferimento

Nella traduzione, l'encoder rappresenta la frase nella lingua di partenza e il decoder produce quella nella lingua di arrivo. Per classificare una frase come positiva o negativa può bastare un encoder con una testa di classificazione. Una rete di generazione può invece essere basata sul solo decoder, producendo progressivamente nuovi elementi condizionati al contesto.

La coppia encoder–decoder non serve soltanto a cambiare modalità: una traduzione fra lingue rimane interamente testuale. Analogamente, usare token di modalità differenti richiede rappresentazioni e addestramento coerenti, non necessariamente proprio lo schema completo della figura.

Il prompt «Elon Musk che mangia un Kinder Bueno» illustra il principio di **condizionare** una generazione di immagini con una descrizione testuale: il testo deve influire sul contenuto prodotto. L'esempio non identifica l'architettura esatta di ogni sistema di generazione d'immagini, né implica che tutti seguano il decoder testuale disegnato nella figura.

Il valore $N = 6$ riportato nelle slide è il numero di livelli della configurazione di riferimento, non una costante obbligatoria per ogni Transformer.

Slide 20 — Componenti non sviluppate in dettaglio

La trattazione si concentra sull'encoder. Del positional encoding viene data l'intuizione qualitativa; non vengono sviluppate le sue formule sinusoidali. Decoder, masking e cross-attention restano richiami di contesto: non si aggiungono le equazioni o la descrizione completa di questi moduli.

5.3 Dall'ingresso ai token e alla posizione (slide 21)

Per entrare nel Transformer, ogni elemento deve essere rappresentato mediante un vettore di dimensione fissata $D = d_{\text{model}}$. Con una riga per token:

$$\boxed{X \in \mathbb{R}^{T \times D}.}$$

T è il numero di token; D è il numero di feature di ogni token. Per esempio, 100 token con 10 feature danno una matrice 100×10. Il blocco encoder descritto mantiene questa forma: non esegue pooling temporale fra un livello e il successivo.

5.3.1 Tokenizzazione ed embedding sono passaggi distinti (slide 21)

Nel caso testuale, la **tokenizzazione** suddivide il testo in unità e le associa a identificativi; l'**embedding** associa agli identificativi dei vettori numerici. Nel linguaggio del calcolo, «token» può indicare anche il vettore che rappresenta quell'unità, ma è utile distinguere i due passaggi.

Una parola non corrisponde necessariamente a un solo token: un tokenizer può produrre sottoparole o altre unità. La corrispondenza «un ingresso, un vettore» descrive la sequenza dopo che quelle unità sono state definite. Il numero di feature per vettore rimane fissato, mentre il numero di vettori può variare.

Word2Vec è un esempio richiamato per l'idea di rappresentare parole con vettori. Non occorre che i vettori siano progettati manualmente; possono essere appresi e possono provenire da un modello già disponibile. Il modulo di embedding di un Transformer può anche essere addestrato insieme al resto della rete. Non si deve estendere al suo embedding l'idea che il tokenizer discreto venga normalmente scelto prima del training del modello.

Precisazione slide 21 — Token, parola e rappresentazione appresa

«Un token per parola» è una semplificazione dell'esempio, non una proprietà generale. Tokenizzazione ed embedding non sono la stessa operazione: la prima definisce le unità e gli identificativi, il secondo costruisce i vettori e può ricevere gradienti durante l'addestramento end-to-end.

5.3.2 Perché aggiungere la posizione (slide 21)

Confrontare il contenuto dei token, da solo, non assegna un significato alla loro posizione nella frase. Per rendere accessibile l'ordine si introduce una rappresentazione della posizione, qui descritta come un vettore aggiunto alle feature:

$$\boxed{x_t = e_t + p_t, \qquad e_t, p_t, x_t \in \mathbb{R}^D.}$$

e_t è l'embedding del contenuto e p_t codifica la posizione. La somma è possibile perché le forme coincidono. Per esempio, nell'illustrazione più semplice si potrebbe aggiungere l'indice a ogni feature: con $e_1 = (0.2, 0.4)$ e $p_1 = (1, 1)$ si ottiene $x_1 = (1.2, 1.4)$; alla seconda posizione si potrebbe usare $p_2 = (2, 2)$. È un esempio intuitivo, non la codifica sinusoidale dell'architettura di riferimento né una raccomandazione progettuale.

Nel caso descritto, le funzioni di posizione sono fissate; altri dettagli di progettazione non sono necessari per seguire il blocco encoder. Il collegamento con le feature cicliche di reti ricorrenti è l'uso di funzioni numeriche per rappresentare una struttura dell'indice, senza identificare i due meccanismi.

5.4 Blocco encoder: attention, MLP, residui e normalizzazione (slide 22–25)

Il blocco contiene due trasformazioni principali: **Multi-Head Attention** e **Feed-Forward Network**. Dopo ciascuna si somma l'ingresso del sottoblocco e si applica LayerNorm. L'ordine mostrato è quindi quello con normalizzazione successiva alla somma.

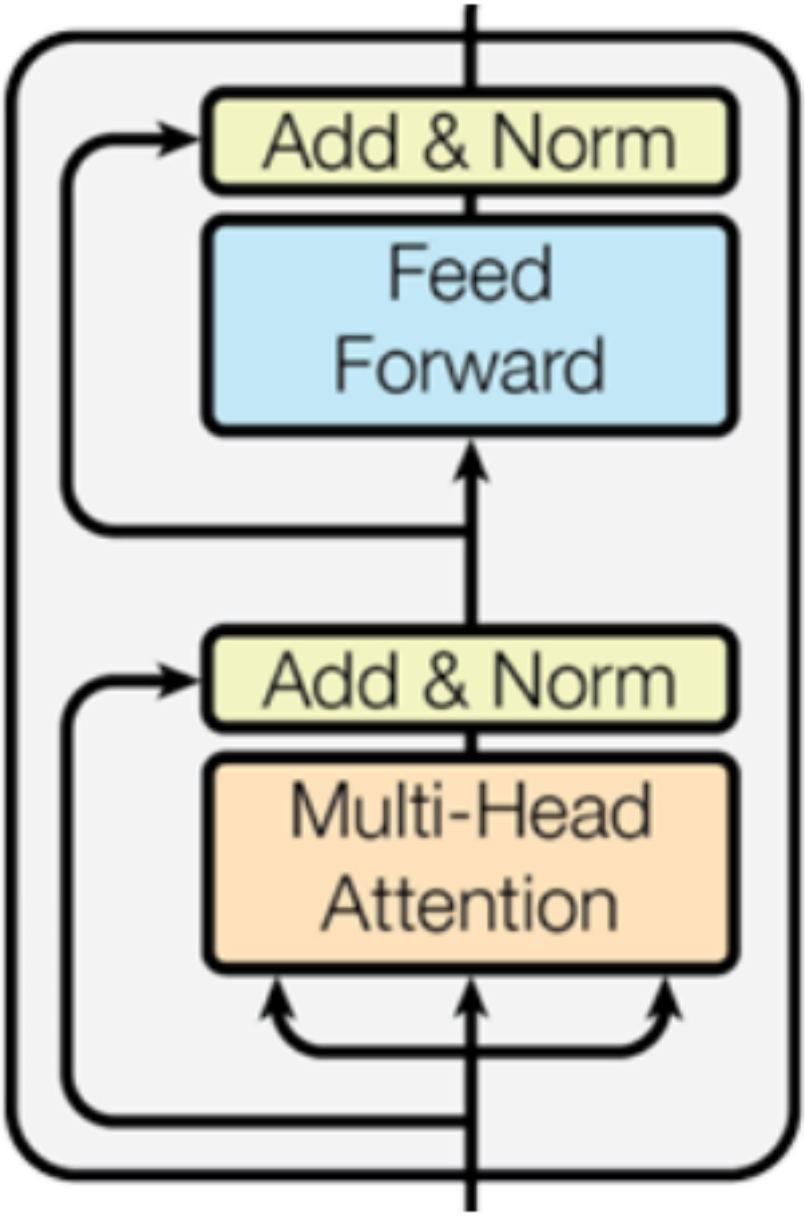
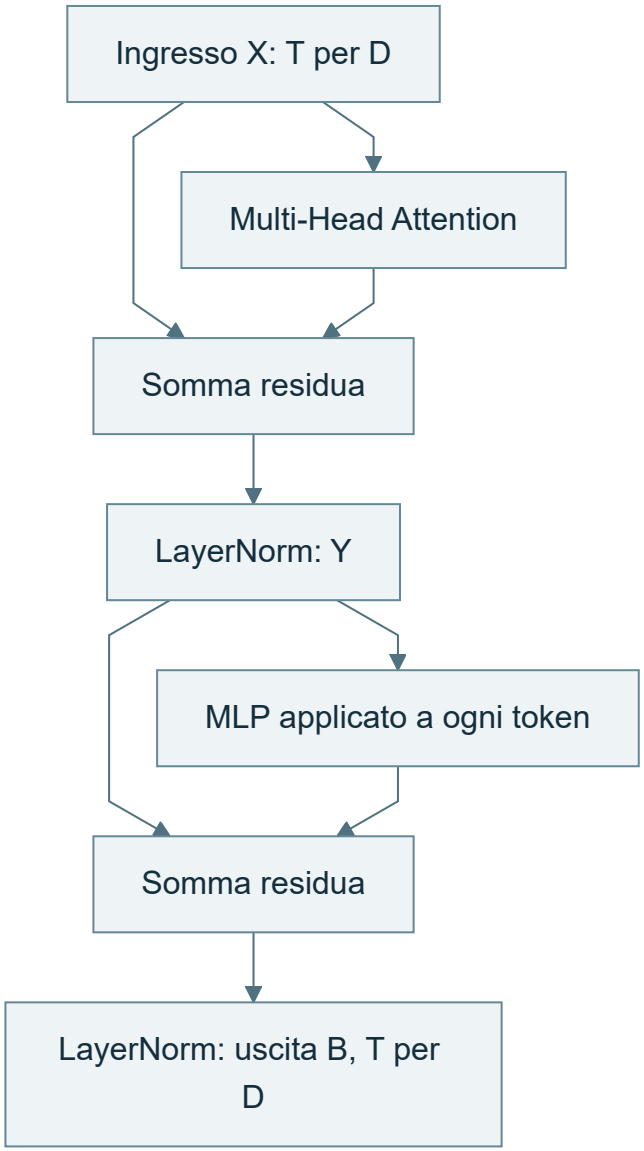


Figura 5.2: Slide 25 — Blocco encoder con due connessioni residue

Scrivendo MHA per la multi-head attention e FFN per il piccolo MLP:

$$\boxed{\begin{aligned} Y &= \mathrm{LN}_1\big(X + \mathrm{MHA}(X)\big), \\ B &= \mathrm{LN}_2\big(Y + \mathrm{FFN}(Y)\big). \end{aligned}}$$

X, Y, B hanno forma $T \times D$. Perché le somme siano definite, anche MHA e FFN devono restituire $T \times D$. Le due LayerNorm hanno propri parametri. La forma resta uguale, ma il contenuto dei vettori cambia: ogni livello può affinare le rappresentazioni.



5.4.1 MLP per token e condivisione dei pesi (slide 23–25)

Il feed-forward comprende due trasformazioni lineari con ReLU in mezzo. Per un token riga $y_t \in \mathbb{R}^{1\times D}$:

$$\boxed{\mathrm{FFN}(y_t) = \mathrm{ReLU}(y_t W_1 + b_1)W_2 + b_2.}$$

La dimensione interna F può essere diversa da D:

Quantità	Forma
W_1	$D \times F$
b_1	$1 \times F$
$y_t W_1 + b_1$	$1 \times F$
W_2	$F \times D$
b_2	$1 \times D$
$\mathrm{FFN}(y_t)$	$1 \times D$

Il calcolo prima proietta da D a F, applica la non linearità e poi torna da F a D. **Lo stesso MLP, con gli stessi pesi, viene applicato a ogni posizione.** «Indipendentemente» significa che questa

operazione non mescola token differenti, non che ogni token abbia un MLP distinto. Le posizioni possono essere elaborate in parallelo.

Per $D = 2, F = 3$, con bias nulli:

$$y_t = \begin{bmatrix} 1 & -1 \end{bmatrix}, \quad W_1 = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 1 \end{bmatrix}, \quad W_2 = \begin{bmatrix} 2 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}.$$

Si ottiene:

$$\begin{aligned} y_t W_1 &= \begin{bmatrix} 1 & -1 & -2 \end{bmatrix} && \text{primo prodotto riga per matrice,} \\ \mathrm{ReLU}(y_t W_1) &= \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} && \text{soglia a zero,} \\ \mathrm{FFN}(y_t) &= \begin{bmatrix} 2 & 0 \end{bmatrix} && \text{seconda proiezione,} \\ y_t + \mathrm{FFN}(y_t) &= \begin{bmatrix} 3 & -1 \end{bmatrix} && \text{somma residua prima della normalizzazione.} \end{aligned}$$

Il MLP ha dimensione finale uguale all'ingresso pur cambiando la dimensione interna. La connessione residua aggiunge il contributo diretto, come in reti convoluzionali; non sostituisce la trasformazione né ne evita il calcolo. Il ruolo dell'attention sarà invece quello di mescolare informazioni fra token.

5.5 Layer Normalization: normalizzare le feature del singolo token (slide 26–28)

La **Layer Normalization**, o **LayerNorm**, controlla centro e scala delle attivazioni **all'interno di ogni token**. Su un tensore di forma $N \times T \times D$, qui si aggrega l'ultimo asse D: ogni coppia campione–posizione ha le proprie statistiche.

Per un token $x \in \mathbb{R}^D$, con vettori scritti qui per componenti:

$$\boxed{\begin{aligned} \mu &= \frac{1}{D}\sum_{j=1}^{D} x_j, \\ v &= \frac{1}{D}\sum_{j=1}^{D} (x_j - \mu)^2, \\ \widehat{x}_j &= \frac{x_j - \mu}{\sqrt{v + \epsilon}}, \\ y_j &= \gamma_j \widehat{x}_j + \beta_j. \end{aligned}}$$

μ, v sono scalari calcolati dal token corrente; $\epsilon > 0$ è una costante di stabilità. I vettori $\gamma, \beta \in \mathbb{R}^D$ sono apprendibili e condivisi fra le posizioni per lo stesso modulo. In uscita rimangono D componenti.

5.5.1 Da dove vengono i passaggi (slide 26–27)

La sottrazione della media centra gli scarti:

$$\begin{aligned} \frac{1}{D}\sum_j (x_j - \mu) &= \frac{1}{D}\sum_j x_j - \frac{1}{D}\sum_j \mu && \text{linearità della somma,} \\ &= \mu - \mu = 0 && \text{definizione della media.} \end{aligned}$$

La divisione per la deviazione standard stabilizzata controlla la scala. Poiché $\widehat{x}$ ha media zero:

$$\begin{aligned}\operatorname{var}(\widehat{x}) &= \frac{1}{D}\sum_j \frac{(x_j-\mu)^2}{v+\epsilon},\\ &= \frac{v}{v+\epsilon}.\end{aligned}$$

Questa varianza è vicina a 1 quando v è grande rispetto a ϵ, non identicamente 1. Se tutte le componenti coincidono, $v=0$ e gli scarti normalizzati sono zero; ϵ evita la divisione per zero. Un esempio con componenti tutte uguali a 5 produce quindi $\widehat{x}=0$ e $y=\beta$.

Precisazione slide 26–27 — Deviazione standard e stabilizzatore

Il denominatore della slide è $\sigma+\epsilon$: σ va letto come **deviazione standard**, non varianza né covarianza. Qui si esplicita la convenzione comune $\sqrt{v+\epsilon}$. Le due collocazioni di ϵ sono varianti numeriche diverse: non si sostituisce l'una all'altra come identità algebrica. Dividere direttamente per v non realizza la standardizzazione descritta.

5.5.2 Gamma e beta: scala e traslazione apprendibili (slide 26–27)

La normalizzazione è seguita da una trasformazione affine per feature. γ_j scala la componente j; β_j la trasla. Una scala zero può annullare il contributo di una feature normalizzata, ma γ **non è un gate sigmoidale**: non è vincolato fra 0 e 1, può amplificare o cambiare segno. Non è una maschera nuova scelta separatamente per ciascun token.

I parametri apprendibili sono D scale e D traslazioni:

$$\boxed{P_{\mathrm{LN}} = 2D.}$$

Per $D=4$ sono 8 scalari. Media e varianza del token non sono parametri ottimizzati. Con valori generali di γ,β, l'uscita finale non deve mantenere media zero o varianza unitaria.

5.5.3 Esempio numerico completo (slide 26–27)

Per un token $x=(1,3,5,7)$:

$$\begin{aligned}\mu &= (1+3+5+7)/4 = 4,\\ v &= [(-3)^2+(-1)^2+1^2+3^2]/4 = 20/4 = 5,\\ \sqrt{v+\epsilon} &= \sqrt{5.00001} \approx 2.23607 \qquad (\epsilon = 10^{-5}),\\ \widehat{x} &\approx (-1.34164, -0.44721, 0.44721, 1.34164).\end{aligned}$$

Con $\gamma=(2,2,2,2)$ e $\beta=(1,1,1,1)$ si ottiene

$$\boxed{y \approx (-1.68328, 0.10557, 1.89443, 3.68328).}$$

Qui la media finale è 1 e la varianza circa 4. Il modulo non impone una scala rigida finale: fornisce una base normalizzata e lascia apprendere la trasformazione successiva.

Se una feature è molto grande rispetto alle altre, per esempio $x=(1000,0,0,0)$, con ϵ trascurabile si hanno $\mu=250$, $v=187500$ e

$$\widehat{x} = (\sqrt{3}, -1/\sqrt{3}, -1/\sqrt{3}, -1/\sqrt{3}).$$

L'esempio chiarisce perché la normalizzazione ridimensiona il valore estremo rispetto alla scala del token; l'eventuale γ permette poi una scala diversa per ciascuna coordinata. Non è necessario supporre che il grande valore rimanga invariato dopo aver diviso per la deviazione standard.

5.5.4 LayerNorm e BatchNorm a confronto (slide 27)

Aspetto	LayerNorm del Transformer	BatchNorm convoluzionale di reti convoluzionali
Statistiche	Fra le feature dello stesso token	Per canale, aggregando batch e posizioni spaziali
Dipendenza dagli altri esempi	Non serve il resto del batch	Presente durante il training
Inferenza	Ricalcola le statistiche del token	Nella configurazione standard usa statistiche mobili memorizzate
Parametri affini	Una scala e un bias per feature	Una scala e un bias per canale

La LayerNorm del caso descritto non richiede di passare a statistiche mobili per l'inferenza. Questo non rende identico in train ed eval l'intero Transformer, che può includere altri moduli con comportamenti differenti.

Controllare la scala non significa assegnare a tutti i token uguale importanza semantica, uguali gradienti o una probabilità identica di riceverli. Queste dipendenze sono determinate dal grafo, dalla loss e dai parametri; non sono una conseguenza garantita della normalizzazione. La LayerNorm non rende inoltre gaussiane le feature e non ne annulla automaticamente le correlazioni.

5.6 Attention: ricostruire ogni token dal contesto (slide 29–32)

Il meccanismo di attention costruisce una nuova rappresentazione per ogni posizione combinando informazioni dalla sequenza. I coefficienti non sono uguali per tutte le posizioni: la prima può attribuire molto peso a certi elementi, la seconda ad altri. La scelta dipende dal contenuto dell'ingresso corrente.

Nella forma introduttiva della slide 30:

$$\boxed{z_t = \sum_{i=1}^{T} \alpha_{t,i} x_i.}$$

$x_i \in \mathbb{R}^D$ è un vettore della sequenza; $z_t \in \mathbb{R}^D$ è il nuovo vettore alla posizione t; $\alpha_{t,i}$ **è uno scalare** che pesa l'intero vettore x_i. Non è una seconda rappresentazione multidimensionale. La somma di vettori pesati conserva la loro dimensione.

Se $x_1 = (2, 0)$ e $x_2 = (0, 4)$ e per una posizione si assegnano pesi 0.75 e 0.25, allora

$$z_t = 0.75(2, 0) + 0.25(0, 4) = (1.5, 1).$$

Con pesi 0.25 e 0.75, un'altra posizione produce $(0.5, 3)$. Cambiano i coefficienti e quindi la combinazione. L'attention concreta delle slide successive applicherà questi pesi ai **value proiettati**, non necessariamente agli embedding grezzi x_i.

5.6.1 Dalle similarità ai coefficienti (slide 30)

Per ogni posizione t si calcola un insieme di score $s_{t,1}, \ldots, s_{t,T}$ e si applica la softmax lungo l'indice i:

$$\boxed{\alpha_{t,i} = \frac{e^{s_{t,i}}}{\sum_{j=1}^{T} e^{s_{t,j}}}, \qquad \sum_{i=1}^{T} \alpha_{t,i} = 1.}$$

La normalizzazione deriva dividendo ogni esponenziale per la somma degli esponenziali della **stessa riga**. La somma dei coefficienti è 1 perché numeratore sommato e denominatore coincidono. Con score $(\ln 3, 0)$, gli esponenziali sono $(3, 1)$ e i pesi diventano $(3/4, 1/4)$, come nell'esempio precedente.

Ogni riga può essere letta come una distribuzione sui token a cui si attinge. Non è una probabilità che una parola sia corretta, né una probabilità sulle classi del compito. Con score finiti e senza mascheramento i pesi sono positivi: «non selezionare» un elemento significa spesso attribuirgli peso molto piccolo, non necessariamente zero esatto.

I coefficienti sono calcolati durante il forward mediante funzioni parametriche apprese. Non sono un insieme di parametri statici $\alpha_{t,i}$ memorizzati una volta per tutte per ciascuna posizione. A parità di modello, cambiando la frase possono cambiare le relazioni.

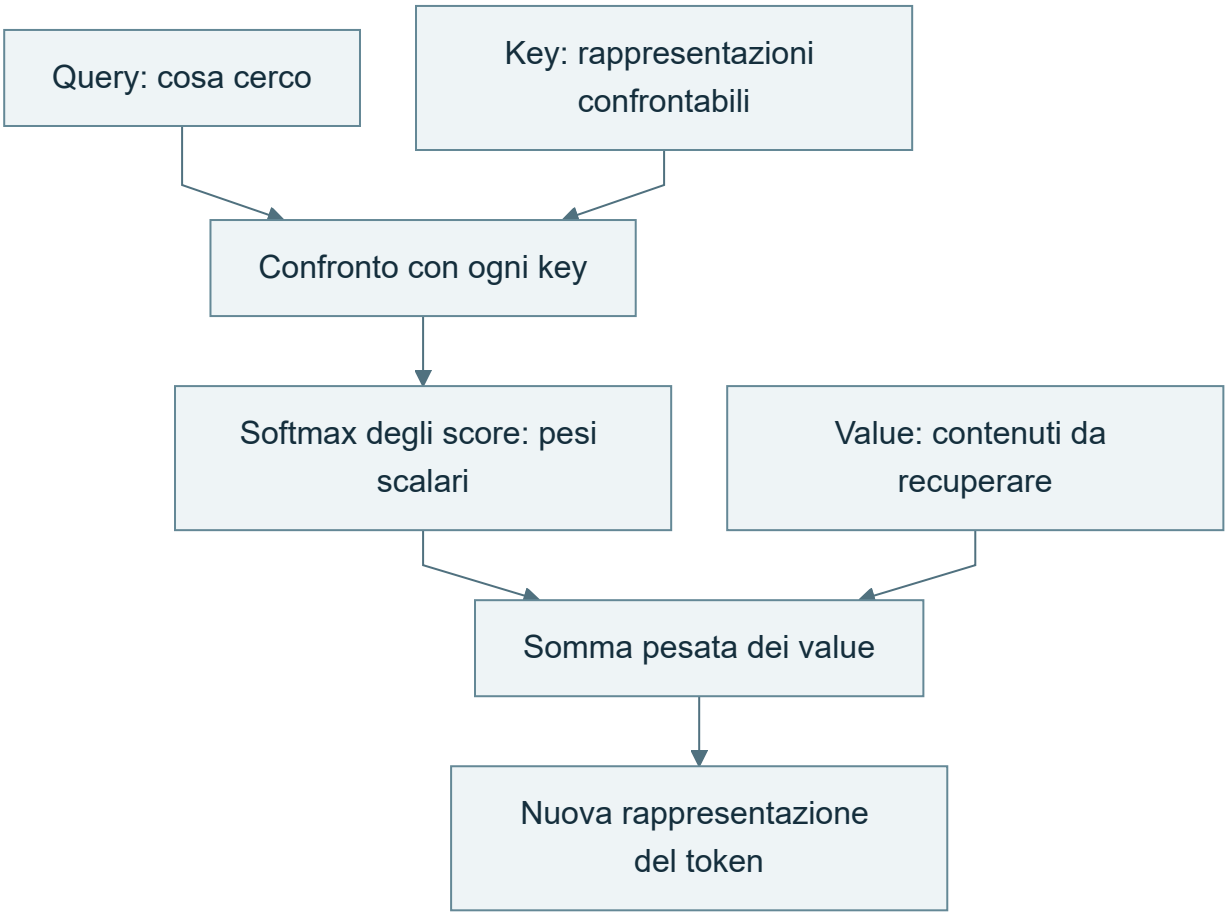
Anche CNN e RNN producono risposte dipendenti dall'ingresso; non trattano ogni contenuto come semanticamente uguale. La novità specifica dell'attention descritta è calcolare esplicitamente **coefficienti di interazione fra coppie di posizioni**.

5.6.2 Analogia con una memoria interrogabile (slide 31–32)

L'attention si può leggere come un accesso a memoria guidato dal contenuto:

1. La **query** descrive ciò che si cerca per costruire una certa uscita.
2. Le **key** rappresentano il contenuto in una forma confrontabile con la query.
3. Il confronto produce score e coefficienti normalizzati.
4. I **value** contengono le informazioni da combinare con quei coefficienti.

La query interroga la stessa collezione di coppie key–value per ogni posizione. Il richiamo a tag e indici delle cache dei calcolatori aiuta a separare **ciò che consente il confronto** da **ciò che viene recuperato**. Le key sono però vettori appresi, non indirizzi interi né necessariamente identificativi univoci.



L'analogia riguarda la selezione e la combinazione di contenuti, non dimostra una capacità generale di ragionamento. L'apprendimento sceglie come rappresentare query, key e value affinché l'uscita serva al compito.

5.7 Query, key e value: tre proiezioni dello stesso ingresso (slide 33)

Le tre rappresentazioni si ottengono mediante trasformazioni lineari. Con una riga per token:

$$Q = XW^Q, \qquad K = XW^K, \qquad V = XW^V.$$

Quantità	Forma	Significato
X	$T \times D$	Sequenza di ingresso, con $D = d_{\text{model}}$
W^Q	$D \times d_k$	Proiezione verso le query
W^K	$D \times d_k$	Proiezione verso le key
W^V	$D \times d_v$	Proiezione verso i value
Q, K	$T \times d_k$	Una query e una key per token
V	$T \times d_v$	Un value per token

Le slide usano $d_v = d_k$; la condizione essenziale per il prodotto scalare è che **query e key abbiano la stessa dimensione**. La dimensione dei value può, in generale, essere distinta. Nell'esempio successivo manteniamo la scelta della slide: $D = 4$ e $d_k = d_v = 3$.

Il prodotto XW^Q applica la stessa trasformazione a ogni riga:

$$q_{t,j} = \sum_{a=1}^{D} X_{t,a} W^Q_{a,j},$$
$$q_t = x_t W^Q \in \mathbb{R}^{1 \times d_k}.$$

Si ripete il procedimento con matrici distinte per K e V. Proiettare permette di apprendere spazi adatti al confronto e al contenuto da recuperare: l'embedding iniziale non deve già essere la rappresentazione migliore per tutti e tre i ruoli.

Poiché query, key e value derivano dalla **stessa sequenza**, si parla di **self-attention**. Nel richiamo alla cross-attention la query può provenire da una sequenza e key–value da un'altra; non viene sviluppato qui il relativo modulo.

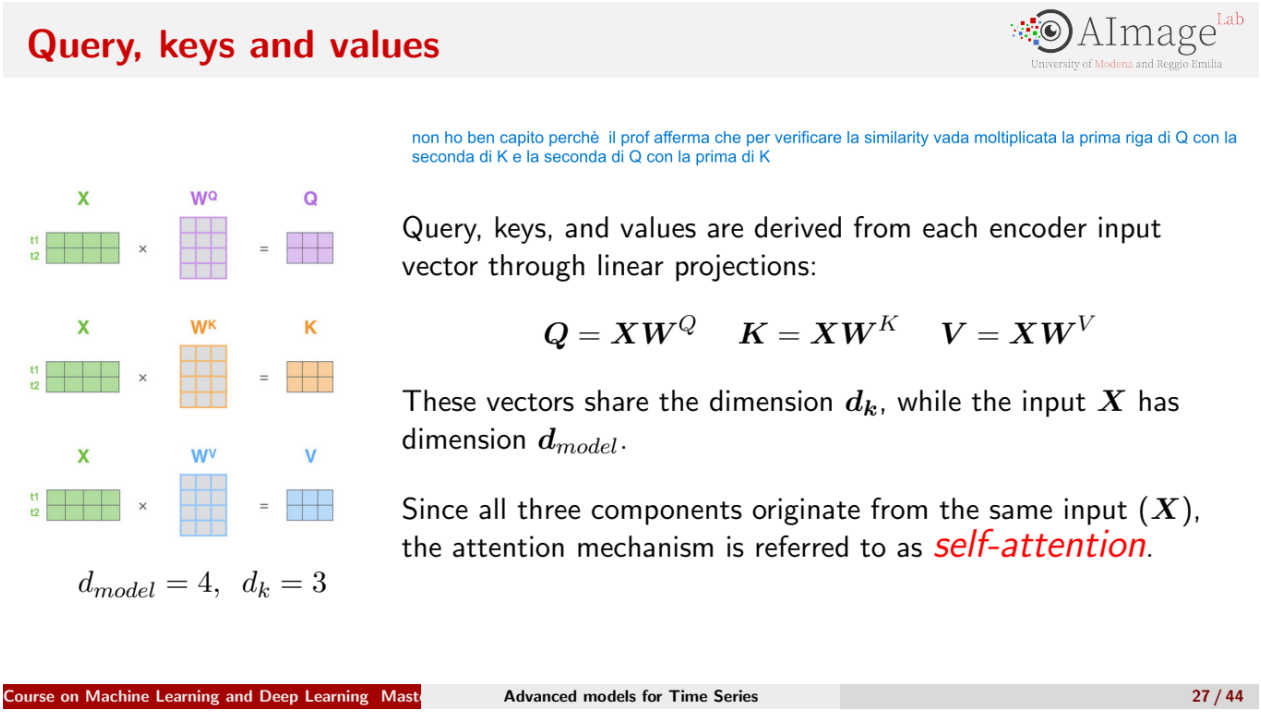


Figura 5.3: Slide 33 — Proiezioni e annotazione sul confronto fra righe di Q e K

5.7.1 Esempio delle tre proiezioni, con dimensioni esplicite (slide 33)

Consideriamo due token di quattro feature:

$$X = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}.$$

Le seguenti matrici 4×3 sono un piccolo esempio numerico costruito per esplicitare i prodotti, non valori leggibili nel disegno della slide:

$$W^Q = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad W^K = \begin{bmatrix} 1 & 0 & 1 \\ 2 & 2 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad W^V = \begin{bmatrix} 1 & 2 & 0 \\ 3 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

La prima riga di X, $(1, 0, 1, 0)$, combina prima e terza riga di ciascuna matrice di proiezione; la seconda combina seconda e quarta. Poiché terza e quarta sono qui nulle:

$$\boxed{Q = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 & 1 \\ 2 & 2 & 0 \end{bmatrix}, \quad V = \begin{bmatrix} 1 & 2 & 0 \\ 3 & 0 & 1 \end{bmatrix}.}$$

Ogni risultato ha ancora **due righe**, una per token; la dimensione delle feature è passata da 4 a 3. Il valore d_k conta le componenti di query e key, non il numero dei token.

5.8 Single-head attention: dal confronto alla somma pesata (slide 34)

La **scaled dot-product attention** esegue in forma matriciale tutti i confronti e tutte le combinazioni:

$$\boxed{Z = \text{Attention}(Q, K, V) = \text{softmax}_{\text{righe}}\left(\frac{QK^T}{\sqrt{d_k}}\right) V.}$$

La lettura va fatta da dentro verso fuori: prodotti scalari query–key, riscalamento, softmax per riga e prodotto finale con i value.

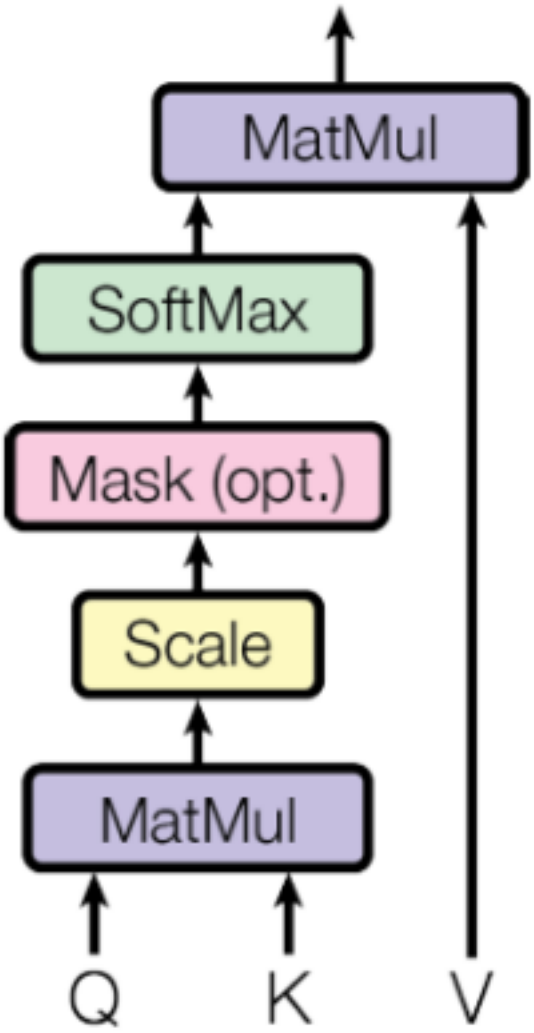


Figura 5.4: Slide 34 — Flusso dello scaled dot-product attention

5.8.1 Perché K viene trasposta e quali righe si confrontano (slide 33–34)

Q e K hanno forma $T \times d_k$. Per confrontare ogni riga di Q con ogni riga di K occorre il prodotto

$$\boxed{S_0 = QK^T \in \mathbb{R}^{T\times T}, \qquad (S_0)_{t,i} = \sum_{a=1}^{d_k} Q_{t,a}K_{i,a} = q_t \cdot k_i.}$$

La trasposizione porta le righe delle key sulle colonne della matrice moltiplicata. Per due token si ottiene:

$$QK^T = \begin{bmatrix} q_1 \cdot k_1 & q_1 \cdot k_2 \\ q_2 \cdot k_1 & q_2 \cdot k_2 \end{bmatrix}.$$

Questa è la risposta al dubbio annotato nella slide 33: **non si moltiplicano soltanto la prima riga di Q con la seconda di K e la seconda di Q con la prima di K**. Quelli sono due confronti incrociati; si calcolano anche i confronti sulla diagonale. Per ogni query si considerano tutte le key ammesse.

Nel nostro esempio:

$$\begin{aligned} q_1 \cdot k_1 &= 1\cdot 1 + 0\cdot 0 + 0\cdot 1 = 1, \\ q_1 \cdot k_2 &= 1\cdot 2 + 0\cdot 2 + 0\cdot 0 = 2, \\ q_2 \cdot k_1 &= 0\cdot 1 + 1\cdot 0 + 0\cdot 1 = 0, \\ q_2 \cdot k_2 &= 0\cdot 2 + 1\cdot 2 + 0\cdot 0 = 2, \\ QK^T &= \begin{bmatrix} 1 & 2 \\ 0 & 2 \end{bmatrix}. \end{aligned}$$

Il prodotto è una misura di compatibilità appresa, non necessariamente una correlazione statistica normalizzata. Poiché W^Q e W^K sono diversi, q_t e k_t non sono in generale lo stesso vettore. Perciò la matrice non deve essere simmetrica e la diagonale **non deve essere la massima**. Nell'esempio la prima query ha score 2 con la seconda key, maggiore del proprio score diagonale 1. La conservazione dell'ingresso è favorita anche dalla connessione residua esterna, non garantita da una diagonale dominante.

5.8.2 Perché dividere per la radice della dimensione (slide 34)

Il prodotto scalare somma d_k contributi. All'aumentare della dimensione, la sua scala può crescere e produrre score molto separati; la softmax può allora diventare estremamente concentrata, con gradienti piccoli.

Per chiarire il riscalamento, sotto l'ipotesi illustrativa di componenti indipendenti, a media zero e varianza unitaria, e query e key indipendenti:

$$\begin{aligned} s &= \sum_{a=1}^{d_k} q_a k_a, \\ \mathbb{E}[q_a k_a] &= \mathbb{E}[q_a]\mathbb{E}[k_a] = 0 && \text{indipendenza e medie nulle,} \\ \operatorname{var}(q_a k_a) &= \mathbb{E}[q_a^2]\mathbb{E}[k_a^2] = 1 && \text{indipendenza e secondi momenti unitari,} \\ \operatorname{var}(s) &= \sum_{a} 1 = d_k && \text{somma di contributi indipendenti,} \\ \operatorname{var}\left(\frac{s}{\sqrt{d_k}}\right) &= \frac{d_k}{d_k} = 1 && \text{varianza di una quantità riscalata.} \end{aligned}$$

Non è una garanzia che query e key apprese soddisfino queste ipotesi; è la motivazione di scala. Per $d_k = 64$, il denominatore è **8**, non 64. La normalizzazione degli score e la LayerNorm hanno posizioni e scopi collegati alla stabilità, ma sono operazioni diverse.

Precisazione slide 34 — Matrici e fattore di scala

Nella formula, Q, K, V sono matrici che raccolgono vettori per token, non tre singoli vettori di lunghezza 64. $d_k = 64$ è un esempio di dimensione delle query e delle key; il fattore di scala è $1/\sqrt{d_k}$. Il numero di righe resta il numero di token e non viene sostituito da d_k.

5.8.3 Softmax per riga: esempio completo (slide 34)

Con $d_k = 3$, si riscalano gli score:

$$S = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 & 2 \\ 0 & 2 \end{bmatrix} \approx \begin{bmatrix} 0.577350 & 1.154701 \\ 0 & 1.154701 \end{bmatrix}.$$

Per la prima riga:

$$\alpha_{1,1} = \frac{e^{1/\sqrt{3}}}{e^{1/\sqrt{3}} + e^{2/\sqrt{3}}} = \frac{1}{1 + e^{1/\sqrt{3}}} \approx 0.359543,$$
$$\alpha_{1,2} = 1 - \alpha_{1,1} \approx 0.640457.$$

Si ripete sulla seconda riga, separatamente:

$$\alpha_{2,1} = \frac{1}{1 + e^{2/\sqrt{3}}} \approx 0.239632,$$
$$\alpha_{2,2} = 1 - \alpha_{2,1} \approx 0.760368.$$

La matrice dei coefficienti è quindi

$$\boxed{A \approx \begin{bmatrix} 0.359543 & 0.640457 \\ 0.239632 & 0.760368 \end{bmatrix}.}$$

Ogni riga somma a 1; le colonne non devono farlo. La prima riga risponde alla domanda «quanto pesa ciascun value per costruire l'uscita della prima query?». La seconda risponde alla stessa domanda per la seconda query. Applicare la softmax per colonne normalizzerebbe una domanda diversa e non realizzerebbe la formula descritta.

5.8.4 Il prodotto finale con V ricostruisce i token (slide 34)

Il prodotto

$$\boxed{Z = AV \in \mathbb{R}^{T \times d_v}, \qquad z_t = \sum_{i=1}^{T} A_{t,i} v_i}$$

usa i pesi per combinare i value. Con $V = \begin{bmatrix} 1 & 2 & 0 \\ 3 & 0 & 1 \end{bmatrix}$:

$$
\begin{aligned}
z_1 &\approx 0.359543(1,2,0) + 0.640457(3,0,1) \\
&\approx (0.359543 + 1.921372,\ 0.719085,\ 0.640457) \\
&\approx (2.280915, 0.719085, 0.640457), \\
z_2 &\approx 0.239632(1,2,0) + 0.760368(3,0,1) \\
&\approx (0.239632 + 2.281105,\ 0.479263,\ 0.760368) \\
&\approx (2.520737, 0.479263, 0.760368).
\end{aligned}
$$

I conti usano i coefficienti non arrotondati fino al risultato finale. Si ottiene ancora una sequenza di due elementi, ora con tre feature ciascuno. Ogni uscita combina entrambe le righe di V, con pesi diversi.

Il passaggio AV è lineare rispetto a V **a coefficienti fissati**. L'intero meccanismo è però non lineare rispetto all'ingresso, perché anche A dipende da X attraverso proiezioni, prodotti e softmax. Non è quindi uno strato lineare con una matrice di pesi costante per tutte le sequenze.

5.9 Multi-Head Attention: più confronti in parallelo (slide 35–36)

Una sola testa costruisce un insieme di relazioni mediante le sue proiezioni. La **Multi-Head Attention**, o **MHA**, ripete il procedimento con proiezioni distinte, così che rappresentazioni differenti possano concorrere alla stessa uscita.

Per H teste:

$$
\boxed{
\begin{aligned}
Z_h &= \text{Attention}(XW_h^Q, XW_h^K, XW_h^V), \qquad h = 1, \dots, H, \\
\text{MHA}(X) &= \text{Concat}_{\text{feature}}(Z_1, \dots, Z_H) W^O.
\end{aligned}
}
$$

La concatenazione avviene sulle **feature dello stesso token**, non sull'asse della sequenza. Se ogni Z_h ha forma $T \times d_v$, la concatenazione ha forma $T \times Hd_v$ e

$$
\boxed{W^O \in \mathbb{R}^{Hd_v \times D}, \qquad \text{MHA}(X) \in \mathbb{R}^{T \times D}.}
$$

Prima si tengono le uscite delle teste affiancate, poi si apprende come combinarle con la proiezione W^O. Si torna così alla dimensione richiesta dalla somma residua.

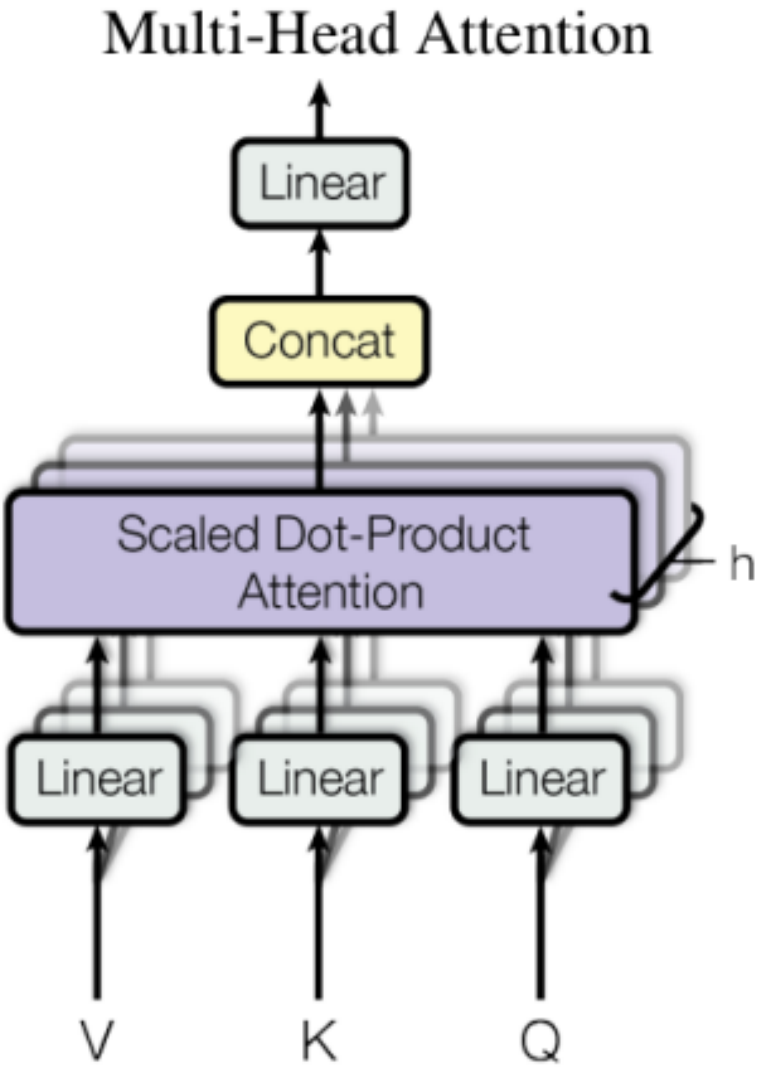


Figura 5.5: Slide 35 — Struttura della multi-head attention

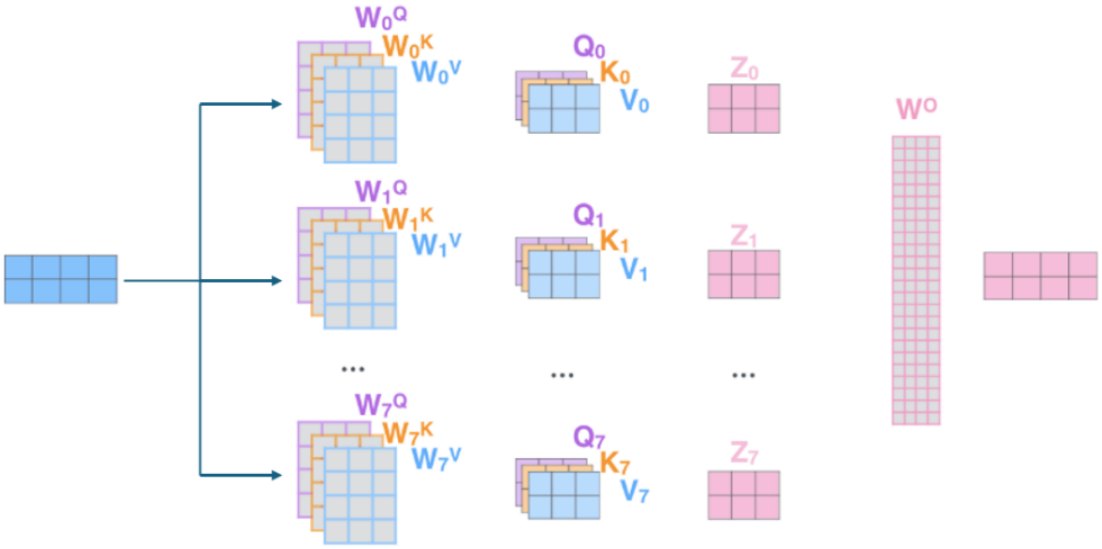


Figura 5.6: Slide 36 — Proiezioni distinte, teste e ricombinazione delle feature

5.9.1 Esempio di concatenazione e proiezione (slide 35–36)

Con $T = 2$, $H = 2$ e $d_v = 1$, supponiamo che due teste abbiano prodotto:

$$Z_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \qquad Z_2 = \begin{bmatrix} 3 \\ 4 \end{bmatrix}.$$

La concatenazione per feature è

$$C = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix},$$

non una sequenza di quattro righe. Se $D = 2$ e

$$W^O = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix},$$

allora

$$CW^O = \begin{bmatrix} 1+3 & 1-3 \\ 2+4 & 2-4 \end{bmatrix} = \boxed{\begin{bmatrix} 4 & -2 \\ 6 & -2 \end{bmatrix}}.$$

Rimangono due token e la proiezione combina le due informazioni di testa per ciascuno. È l'analogia con più filtri convoluzionali: si apprendono più trasformazioni parallele, senza assegnare manualmente a ogni testa un significato prestabilito.

Somma o media delle teste sarebbero altre regole di aggregazione, ma la formula della MHA qui studiata usa **concatenazione seguita da proiezione**. Nella configurazione frequente $D = Hd_k$ e $d_v = d_k$, il totale delle feature concatenate è già D, ma W^O rimane una trasformazione apprendibile.

Precisazione slide 35–36 — Le teste non moltiplicano la lunghezza

Ogni testa produce T righe; le H sequenze sono risultati intermedi paralleli. La concatenazione finale della MHA unisce le feature allineate alla stessa posizione, dando $T \times Hd_v$, non HT token successivamente compressi lungo il tempo. Una proiezione lineare applicata per token cambia le feature, non il numero dei token.

All'interno dell'attention le teste possono cambiare dimensione delle feature; il modulo completo deve restituire D per la somma residua. Quando si dice che il blocco conserva la forma, si parla della sua interfaccia, non dell'assenza di proiezioni interne.

5.10 Dalla sequenza di rappresentazioni alla classificazione (slide 37–38)

Dopo una pila di encoder rimane una matrice $Z \in \mathbb{R}^{T \times D}$, non un unico vettore globale. Ogni riga è una rappresentazione **contestualizzata** della posizione: può aver ricevuto informazioni dagli altri token, ma rimane una riga della sequenza.

Per classificare l'intera frase o serie si può costruire un vettore globale mediante **average pooling** oppure tramite un **token di classificazione**. È l'interfaccia fra estrattore e classificatore, analoga a quella discussa per le CNN in reti convoluzionali.

5.10.1 Average pooling sui token (slide 37)

La media viene calcolata lungo l'asse della sequenza, separatamente per ogni feature:

$$\boxed{g = \frac{1}{T}\sum_{t=1}^{T} z_t \in \mathbb{R}^{1 \times D}, \qquad g_j = \frac{1}{T}\sum_{t=1}^{T} Z_{t,j}.}$$

Si sommano le righe e si divide per il loro numero. Non si fa la media di tutte le feature in un singolo scalare. Per esempio:

$$Z = \begin{bmatrix} 1 & 3 \\ 3 & 1 \end{bmatrix} \quad \Longrightarrow \quad g = \left(\frac{1+3}{2}, \frac{3+1}{2} \right) = (2, 2).$$

Una testa lineare per C classi produce

$$\boxed{s = gW_c + b_c \in \mathbb{R}^{1 \times C}, \qquad W_c \in \mathbb{R}^{D \times C}, \quad b_c \in \mathbb{R}^{1 \times C}.}$$

Con $C = 3$, $W_c = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 0.5 & 1 \end{bmatrix}$ e bias nullo:

$$s = (2, 2)W_c = (2, 1, 0).$$

Sono tre score di classe. La softmax darebbe circa $(0.66524, 0.24473, 0.09003)$, mentre una loss di classificazione può lavorare direttamente sugli score secondo la propria definizione, come spiegato in Multilayer Perceptron. I parametri della testa sono $DC + C = C(D + 1)$: qui $3(2 + 1) = 9$ con bias incluso, indipendentemente da T.

Il pooling finale riduce la sequenza; non contraddice il fatto che i singoli blocchi encoder conservino la lunghezza. Somma e media sono operazioni distinte: la media richiede il divisore T, importante quando si confrontano lunghezze diverse.

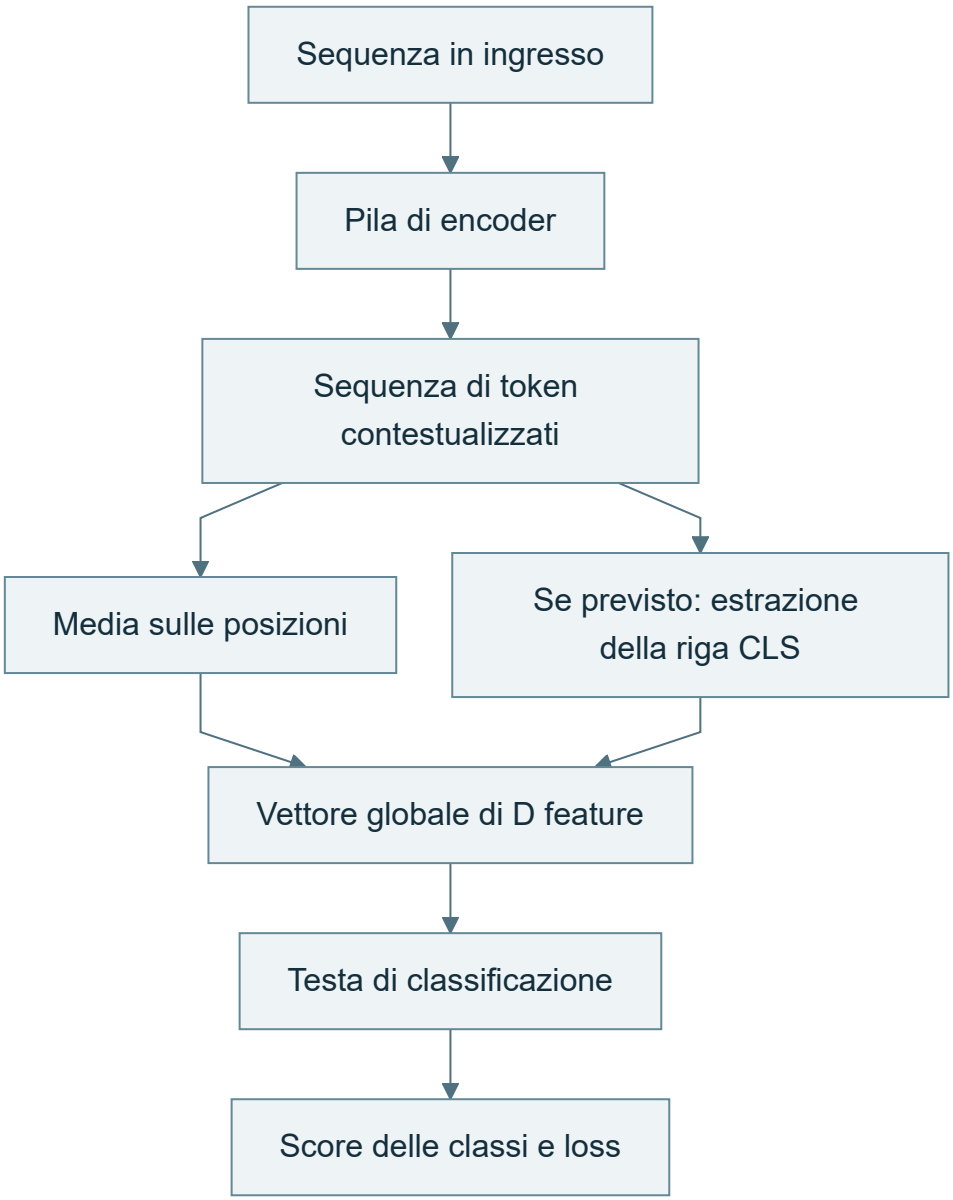
5.10.2 Token [CLS]: una posizione dedicata al riassunto (slide 38)

In alternativa si antepone un token speciale all'ingresso. Se gli elementi originari sono T, la sequenza elaborata ha forma $(T + 1) \times D$. Il token partecipa all'attention; alla fine si preleva la sua rappresentazione:

$$\boxed{g = z_{[\mathrm{CLS}]} \in \mathbb{R}^{1\times D}.}$$

Per esempio, tre token ordinari più [CLS] danno quattro righe; dopo i blocchi rimangono quattro righe e soltanto quella corrispondente a [CLS] viene data alla testa di classificazione. Il token speciale non è di per sé l'etichetta della classe: è una posizione a cui il modello può imparare a trasferire informazioni utili.

La selezione finale della riga non è una somma manuale: il riassunto è stato costruito dalle interazioni nei blocchi. Il modello deve essere addestrato o adattato a utilizzare quella posizione; inserire un token arbitrario in una rete preaddestrata non garantisce che il relativo vettore sia già un buon riassunto.



Il pooling medio può essere calcolato su rappresentazioni già disponibili, ma la sua efficacia va valutata rispetto al compito e all'addestramento del modello. Non è intercambiabile per definizione con una rappresentazione [CLS] appresa.

5.11 TCN e Transformer: confronto, modalità e costi (slide 39–41)

Le due famiglie organizzano in modo diverso il contesto disponibile:

Aspetto	TCN descritta	Transformer encoder descritto
Interazioni	Filtri causali locali con dilatazioni fissate	Coefficienti query–key calcolati dal contenuto
Contesto di un'uscita	Campo recettivo determinato da kernel, dilatazioni e profondità	Tutti i token ammessi dall'attention
Asse temporale	Mappe indicizzate per posizione	Token indicizzati per posizione
Parallelismo	Posizioni e filtri nello stesso livello	Proiezioni, teste, confronti e MLP per token
Trasformazioni	Convoluzioni e blocchi residui	MHA, MLP, residui e LayerNorm
Memoria	Attivazioni delle mappe convoluzionali	Attivazioni dei token e, nell'implementazione diretta, matrici di attention

«Campo recettivo non fisso» nei Transformer indica che cambiano **le rilevanze attribuite alle posizioni**. Nell'encoder con attention piena le posizioni potenzialmente accessibili sono già tutte quelle del contesto fornito. Non vengono creati accessi a una sequenza infinita o a informazioni non presenti nell'ingresso.

Il vincolo causale non nasce dal semplice uso della parola Transformer: l'encoder qui studiato può usare entrambi i lati della sequenza. Se il compito richiede predizioni online, bisogna impedire l'accesso al futuro mediante una configurazione appropriata. I dettagli del masking restano fuori dal perimetro sviluppato.

5.11.1 Stesso meccanismo, ingressi di modalità diverse (slide 39–41)

L'attention opera su vettori. Se testo, musica, misure numeriche o porzioni di immagini vengono rappresentati in uno spazio di feature compatibile, lo stesso tipo di blocco può elaborarli.

Per un'immagine, l'intuizione è dividerla in piccoli quadrati, rappresentare ciascuna **patch** con un vettore e organizzare i vettori in una sequenza. L'attention può allora collegare parti diverse dell'immagine. È il richiamo ai Vision Transformer, senza sviluppare un'architettura visiva completa.

L'analogia delle patch come «parole visive» riguarda il formato a token. Non significa che un tokenizer testuale possa elaborare direttamente pixel grezzi o che basti rendere uguali le dimensioni per ottenere comprensione fra modalità. Servono proiezioni compatibili e un addestramento che dia significato alle relazioni. L'architettura comune rende possibile la fusione, non la realizza automaticamente.

5.11.2 Dove sono i parametri: proiezioni e MLP (slide 39–41)

MHA richiede proiezioni per query, key e value di ogni testa, oltre alla proiezione di uscita. L'MLP aggiunge due matrici, e ogni blocco della pila ha i propri pesi. La condivisione del MLP fra posizioni non implica condivisione fra tutti i blocchi.

Con H teste, dimensioni $d_k = d_v$ e bias omessi:

$$
\begin{aligned}
P_Q + P_K + P_V &= 3HDd_k && \text{tre proiezioni per testa,} \\
P_O &= Hd_kD && \text{ricombinazione delle feature,} \\
\boxed{P_{\text{MHA}}} &= \boxed{4HDd_k.}
\end{aligned}
$$

Se si mantiene $D = Hd_k$, il conteggio diventa $4D^2$. A D fisso, aumentare H riducendo d_k proporzionalmente non moltiplica automaticamente i pesi della MHA. Se invece si mantiene d_k fisso e si aumentano le teste, il conteggio cresce con H. Occorre sempre dichiarare quale dimensione è mantenuta costante.

Per il feed-forward con dimensione interna F:

$$
\begin{aligned}
P_{W_1} = DF, \qquad P_{W_2} &= FD, \\
\boxed{P_{\text{FFN}}} &= \boxed{2DF} \quad \text{senza bias,} \\
P_{\text{FFN,con bias}} &= 2DF + F + D.
\end{aligned}
$$

Con $D = 4, H = 2, d_k = 2, F = 8$, la MHA ha $4 \cdot 2 \cdot 4 \cdot 2 = 64$ pesi e il feed-forward $2 \cdot 4 \cdot 8 = 64$ pesi; includendo i bias del solo FFN, quest'ultimo arriva a 76. Sono conteggi delle trasformazioni, non di tutto il modello con embedding e testa finale.

Per una configurazione illustrativa più grande, $D = 512$ e $F = 4D = 2048$, si hanno $1\,048\,576$ pesi di MHA e $2\,097\,152$ pesi di FFN. Fra queste due componenti il FFN pesa $2/3$, circa il 66.7%. Può essere la parte predominante, ma una quota dell'80% non è una proprietà universale: dipende dalle dimensioni interne e da quali componenti si includono nel totale. La stessa cautela nel distinguere estrattore e parte densa era necessaria per VGG in reti convoluzionali.

Il numero dei parametri dei blocchi descritti non cresce direttamente con T: le proiezioni e il MLP vengono riutilizzati sulle posizioni. Ciò che cresce con T è il **calcolo e la memoria delle attivazioni**.

5.11.3 Lunghezza del contesto e matrici di attention (slide 39–41)

Una sequenza più lunga richiede più righe di rappresentazioni in ogni blocco. Per una testa con attention piena, inoltre, QK^T ha T^2 coefficienti:

$$
\boxed{QK^T \in \mathbb{R}^{T \times T} \quad \Longrightarrow \quad T^2 \text{ score per testa.}}
$$

Con 100 token sono 10 000 score; con 200 token sono 40 000. Raddoppiare la lunghezza quadruplica i confronti della matrice piena. Con otto teste, nell'implementazione diretta si avrebbero rispettivamente 80 000 e 320 000 score, prima di considerare batch e numero di blocchi.

Questa crescita si aggiunge ai vettori di feature, alle attivazioni dei MLP e allo stato necessario al training. Non significa che ogni implementazione debba materializzare sempre tutte le matrici allo stesso modo; chiarisce il costo della formula diretta studiata.

L'esempio di un libro molto lungo rende intuitiva la differenza rispetto a comprimere tutto in un singolo stato ricorrente: preservare una rappresentazione per ogni elemento è costoso. Parole e token non coincidono necessariamente, e ogni modello concreto ha limiti di contesto e memoria: non si assume che una sequenza arbitrariamente lunga entri interamente nel modello.

5.11.4 Cenno conclusivo: cache di key e value nella generazione (slide 39–41, estensione orale)

Nella generazione progressiva, una procedura ingenua potrebbe ricalcolare a ogni nuovo token tutte le rappresentazioni del prefisso precedente. Questo ripeterebbe molto lavoro. La **KV cache** conserva le key e i value già calcolati, per riutilizzarli quando arriva il nuovo elemento.

Per il singolo livello e la singola testa, se il prefisso contiene T elementi, la cache contiene T righe di key e T righe di value. Dopo un nuovo elemento le righe diventano $T + 1$: si aggiungono le sue rappresentazioni e la nuova query può confrontarsi con le key conservate. Il nuovo token deve comunque interagire con il contesto: la cache riduce ricalcoli, non rende nullo il costo dell'attention e occupa memoria crescente.

Il riuso così descritto riguarda una generazione **causale**: aggiungere un token futuro non deve cambiare le rappresentazioni già calcolate del passato. Non si trasferisce automaticamente all'encoder bidirezionale pieno qui studiato, dove un token precedente potrebbe cambiare rappresentazione vedendo il nuovo elemento. La cache viene mantenuta nei livelli e nelle teste interessate, non è soltanto una lista di token grezzi. I dettagli implementativi non sono sviluppati.

5.12 Confine della spiegazione e materiale rinviato (slide 42–60)

La spiegazione si conclude dopo il confronto delle architetture e il cenno alla KV cache. L'ultima slide di contenuto corrispondente è il riepilogo stampato **32/44**, completato alla **pagina PDF 41**. La pagina PDF 42 apre invece «Loss Function for Regression», numerata 33/44.

Slide 42–60 — Materiale nel PDF non ancora spiegato

La parte successiva contiene loss di regressione, MSE e MAE, discretizzazione del target in intervalli, limiti delle penalizzazioni simmetriche, quantili e quantile loss. La trascrizione rinvia esplicitamente questi argomenti alla spiegazione successiva. Non vengono integrati come contenuti già trattati, né si aggiungono qui derivazioni o esempi di queste loss. Le pagine 61–63 contengono riferimenti e ringraziamenti.

La causalità delle TCN, il riassunto delle RNN e la differenza fra parametri e attivazioni si collegano a reti ricorrenti; filtri, canali, campo recettivo, pooling, residui e BatchNorm a reti convoluzionali; MLP, ReLU, softmax e loss di classificazione a Multilayer Perceptron.

5.13 Riepilogo delle forme e delle formule (slide 5–41)

Operazione	Formula o forma	Da ricordare
Conv1D multicanale	$Y_{t,o} = b_o + \sum_{i,c} X_{t+i,c} K_{i,c,o}$	Filtro condiviso lungo un solo asse
Parametri Conv1D	$C_{\text{out}}(kC_{\text{in}} + 1)$	Con bias; non dipendono da T
Dilatazione causale	$y_t = \sum_{i=0}^{k-1} x_{t-di}\omega_i$	Punti distanziati di d
Ampiezza del kernel dilatato	$1 + (k-1)d$	Ampiezza coperta, non numero di pesi
Padding causale	$P_{\text{sinistra}} = (k-1)d$	Stride 1, stessa lunghezza in uscita
Campo recettivo	$r_\ell = r_{\ell-1} + (k_\ell - 1)d_\ell j_{\ell-1}$	$j_\ell = s_\ell j_{\ell-1}$
Sequenza Transformer	$X : T \times D$	Una riga per token
Feed-forward	$\text{ReLU}(y_t W_1 + b_1)W_2 + b_2$	Stessi pesi sulle posizioni; $D \to F \to D$
LayerNorm	$y_j = \gamma_j(x_j - \mu)/\sqrt{v + \epsilon} + \beta_j$	Statistiche sulle feature del singolo token
Proiezioni	$Q = XW^Q$, $K = XW^K$, $V = XW^V$	Q e K: $T \times d_k$; V: $T \times d_v$
Score	$QK^T : T \times T$	Riga = query; colonna = key
Scala	$S = QK^T/\sqrt{d_k}$	Con $d_k = 64$ si divide per 8
Pesi	$A = \text{softmax}_{\text{righe}}(S)$	Ogni riga somma a 1
Attention	$Z = AV : T \times d_v$	Somma pesata dei value per query

Operazione	Formula o forma	Da ricordare
Multi-head	$\text{Concat}(Z_1, \ldots, Z_H)W^O$	Concatenazione sulle feature, uscita $T \times D$
Residui e norm	$Y = \text{LN}_1(X + \text{MHA}(X))$; $B = \text{LN}_2(Y + \text{FFN}(Y))$	Forma del blocco conservata
Average pooling	$g = T^{-1} \sum_t z_t$	Da $T \times D$ a un vettore di D feature
Token [CLS]	Ingresso $(T + 1) \times D$, poi estrazione di una riga	Il riassunto va appreso dal modello

Percorso essenziale da saper ricostruire

Le TCN combinano localmente una sequenza usando filtri condivisi, dilatati e causali. Il Transformer rende invece dinamici i coefficienti delle interazioni: proietta i token in query, key e value; confronta ogni query con le key; normalizza gli score per riga; combina i value. Più teste offrono proiezioni diverse, poi ricombinate sulle feature. Residui, LayerNorm e MLP completano il blocco, mantenendone la forma. Per classificare l'intera sequenza occorre infine un riassunto, per esempio media dei token o rappresentazione [CLS].

5.14 Errata e precisazioni principali (slide 5–41)

Riferimento	Precisazione
Slide 5–6	LeNet-5 non è la prima comparsa assoluta delle idee convoluzionali. 1D descrive l'asse di scorrimento, non l'assenza di canali.
Slide 7–8	La formula a ritardi e l'esempio a finestre usano convenzioni di orientamento diverse; esplicitare indici e ribaltamento del kernel. I cinque risultati numerici della slide 8 sono verificati.
Slide 9–11	Dilatazione 2 su tre punti dà offset 0, 2, 4. La dilatazione è distinta dallo stride; il campo recettivo cresce anche a stride 1.
Slide 10	Causalità significa non accedere a ingressi futuri; non dimostra causalità fisica e non sostituisce l'allineamento corretto dei target.
Slide 11	Tre convoluzioni con dilatazioni 1, 2, 4 danno campo recettivo 15; tre blocchi con due convoluzioni ciascuno danno 29 alle stesse dilatazioni.
Slide 12–13	Aggiungere feature aumenta i canali e i pesi; accuracy e F1 possono ordinare diversamente i modelli. F1 casuale 0.5 non è una baseline universale.
Slide 14	TCN non garantisce assenza di vanishing/exploding gradient; usa la backpropagation. Anche RNN può essere studiata con metodi di attribuzione.
Slide 19–20	Encoder–decoder non serve soltanto a cambiare modalità. Sei livelli sono una configurazione di riferimento, non una regola generale.
Slide 21	Tokenizzazione ed embedding sono distinti; non c'è sempre un token per parola e l'embedding può essere addestrato end-to-end.
Slide 22–25	Il MLP è applicato separatamente alle posizioni ma con pesi condivisi; la forma finale uguale non esclude dimensioni interne diverse.

Riferimento	Precisazione
Slide 26–27	Si divide per una deviazione standard stabilizzata, non per la varianza. γ è una scala apprendibile non vincolata, non un gate sigmoidale.
Slide 26–27	LayerNorm non rende tutti i token ugualmente rilevanti e non garantisce identici gradienti; le statistiche sono per token, non fra tutti i token della sequenza.
Slide 30	Ogni $\alpha_{t,i}$ è uno scalare; la softmax normalizza l'intero insieme degli score della riga. Nell'attenzione concreta si combinano i value proiettati.
Slide 33	Si confrontano tutte le coppie query–key, comprese quelle sulla diagonale. Q e K devono avere la stessa dimensione; V può in generale averne una diversa.
Slide 34	Q, K, V raccolgono vettori in matrici. Il fattore di scala usa $\sqrt{d_k}$, non d_k. La matrice degli score non è necessariamente simmetrica e la diagonale non deve dominare.
Slide 35–36	Si concatenano le feature delle teste allineate per posizione, non le lunghezze delle sequenze.
Slide 37–38	Il pooling finale riduce la sequenza; i blocchi interni la conservano. Un token [CLS] arbitrario non è automaticamente un riassunto utile.
Slide 39–41	L'80% di parametri nel MLP non è una quota universale; contano le dimensioni. Più token aumentano attivazioni e calcolo, non direttamente i pesi condivisi.
Slide 39–41	La cache KV descritta richiede la stabilità delle rappresentazioni passate nella generazione causale; non si trasferisce invariata all'encoder bidirezionale.

Le forme trascritte come «deletion», «casual», «GCN/DCN/TCM», «MootGate/MootHead attention», «Venus/batteries» e «graph count» sono ricondotte, dove il contesto lo rende univoco, a **dilation**, **causal**, **TCN**, **Multi-Head Attention**, **values** e **Grad-CAM**. Il passaggio dai termini corrotti alle nozioni corrette segue formule e figure, senza introdurre nuovi moduli.